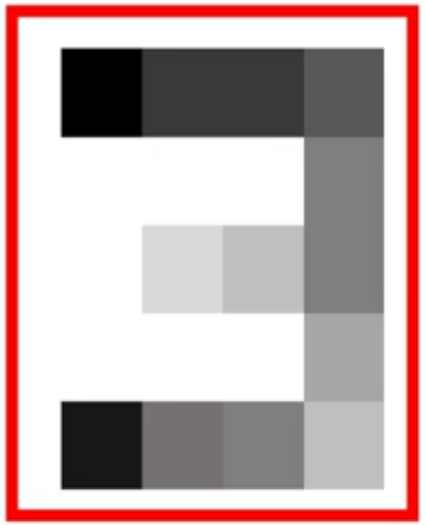
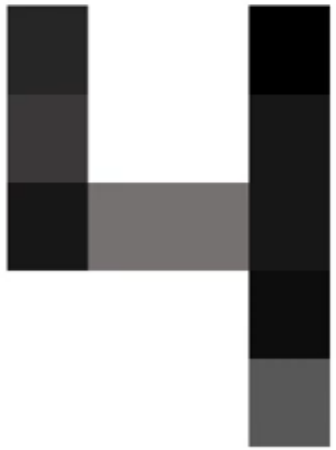
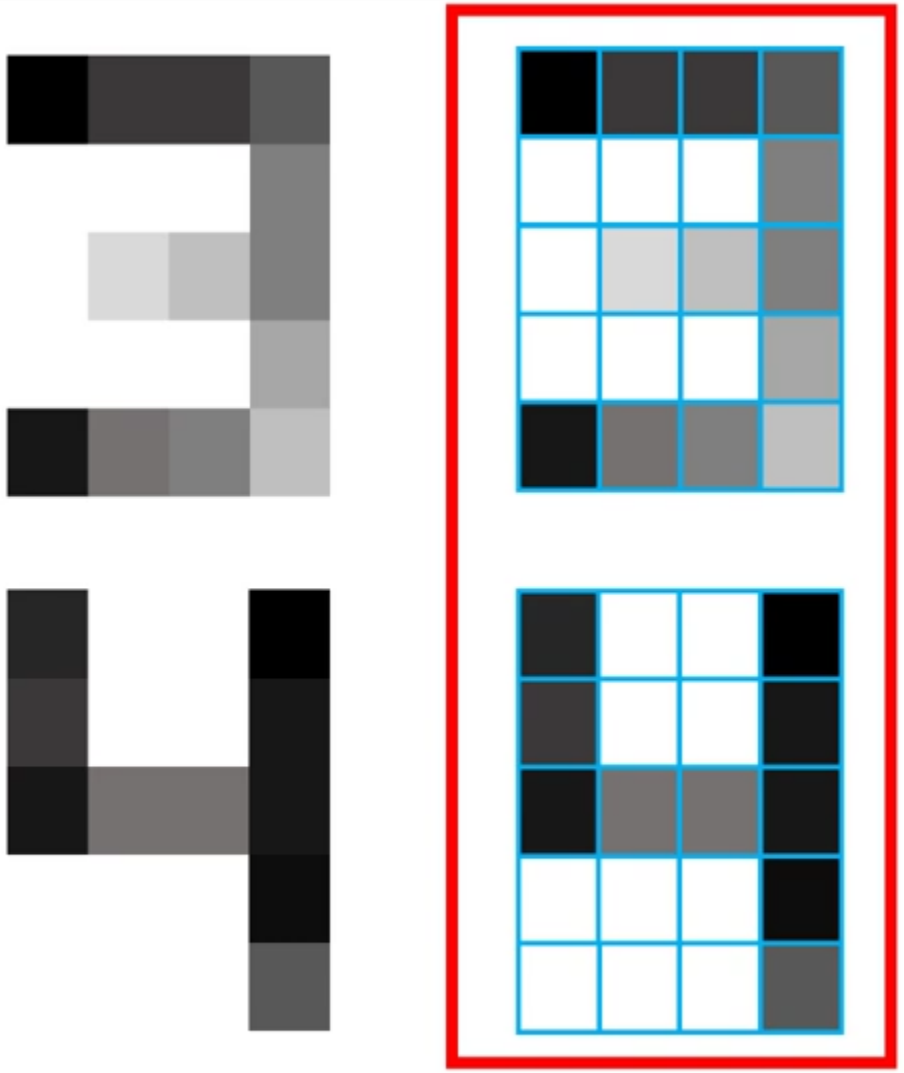
What is an image?





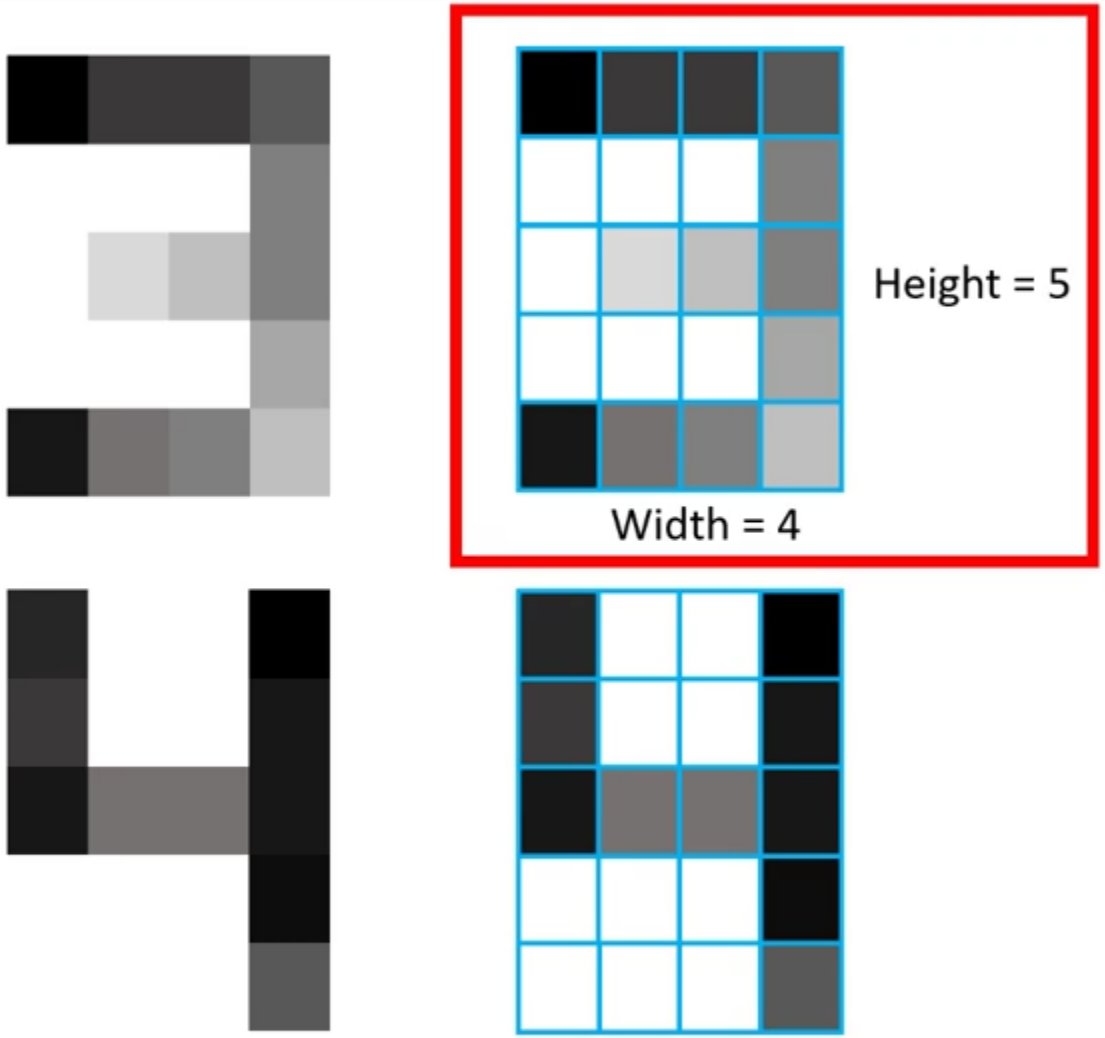
Suppose that we have taken a picture of this digit,

What is an image?



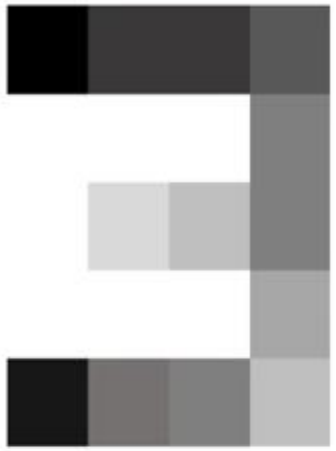
A picture consists of a grid of pixels.

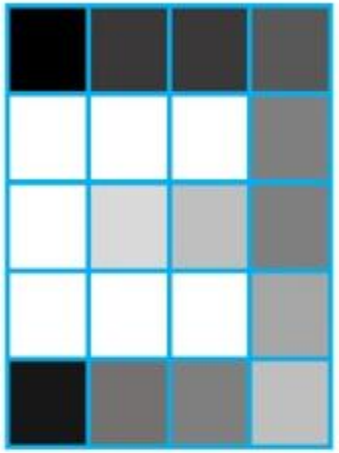
What is an image?



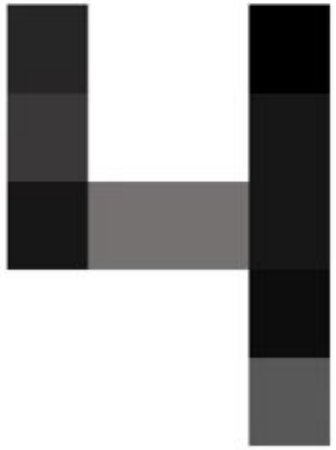
For example, this image has four pixels in width and five pixels in height. In total, the image therefore consists of 20 pixels.

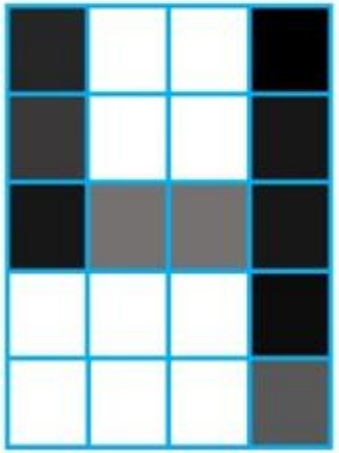
What is an image?





255	200	200	180
0	0	0	120
0	50	80	120
0	0	0	100
250	120	120	80

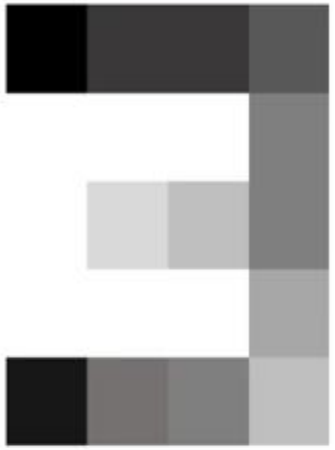


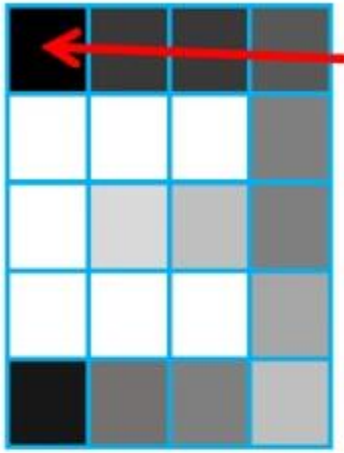


255	0	0	255
230	0	0	255
255	180	180	255
0	0	0	255
0	0	0	200

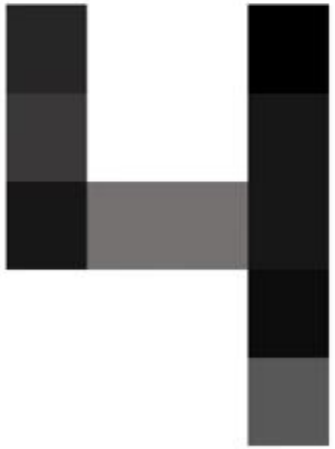
Each pixel has a certain value that corresponds to its brightness. The values usually span between zero and 255.

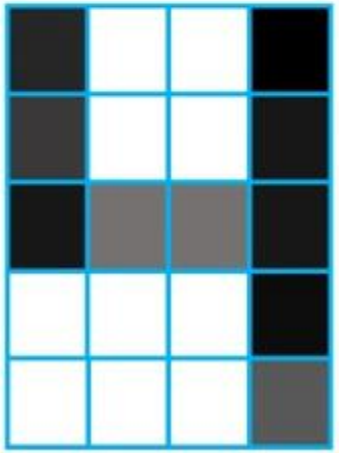
What is an image?





255	200	200	180
0	0	0	120
0	50	80	120
0	0	0	100
250	120	120	80

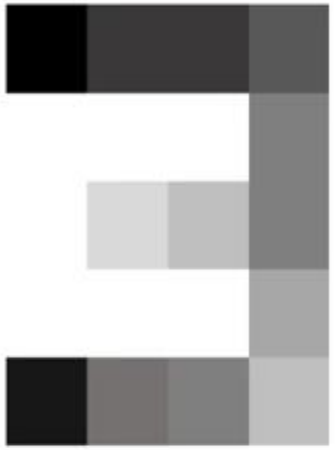


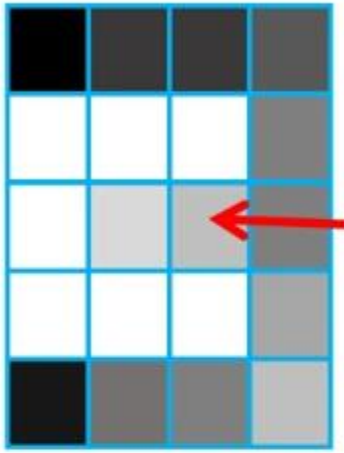


255	0	0	255
230	0	0	255
255	180	180	255
0	0	0	255
0	0	0	200

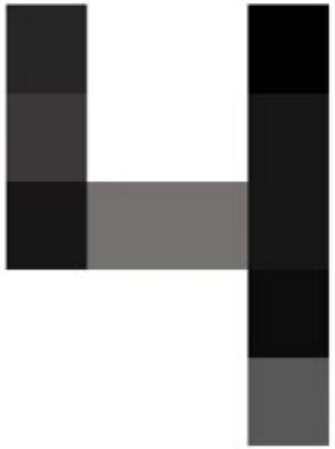
In this example, high numbers represent a black color,

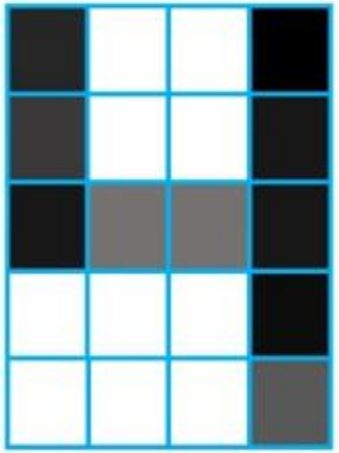
What is an image?





255	200	200	180
0	0	0	120
0	50	80	120
0	0	0	100
250	120	120	80

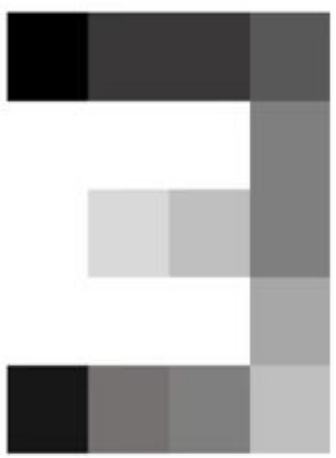


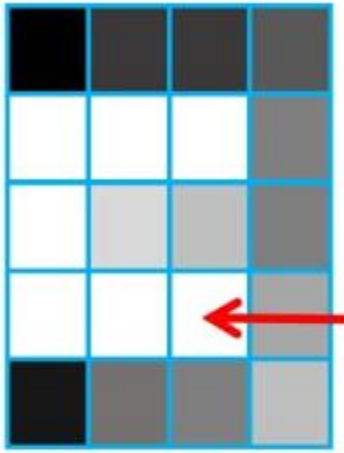


255	0	0	255
230	0	0	255
255	180	180	255
0	0	0	255
0	0	0	200

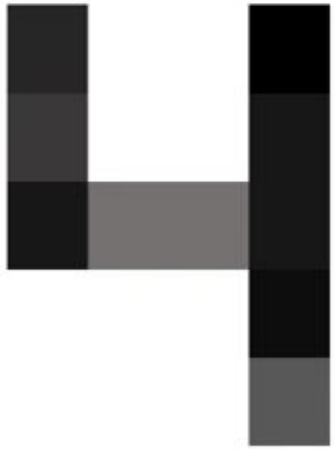
whereas low numbers represent a gray color,

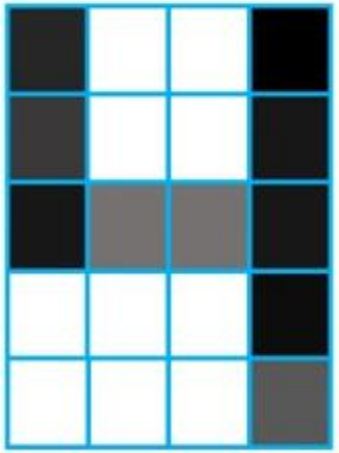
What is an image?





255	200	200	180
0	0	0	120
0	50	80	120
0	0	0	100
250	120	120	80

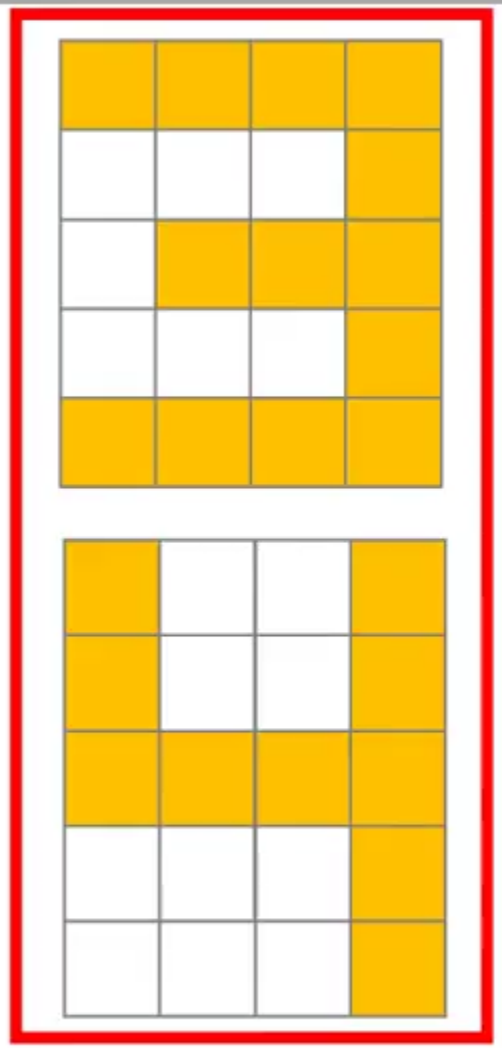




255	0	0	255
230	0	0	255
255	180	180	255
0	0	0	255
0	0	0	200

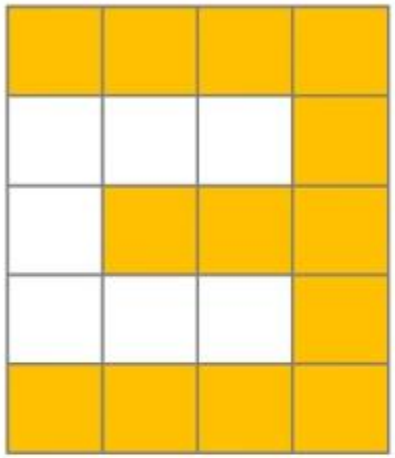
and the white background is represented by zeros.

Example data

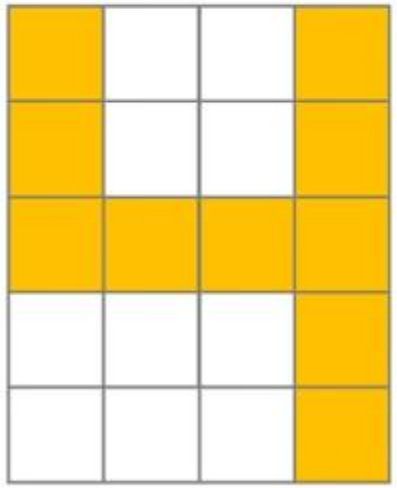


To explain image classification in the simplest possible way, we will here use only two colors for our images.

Example data



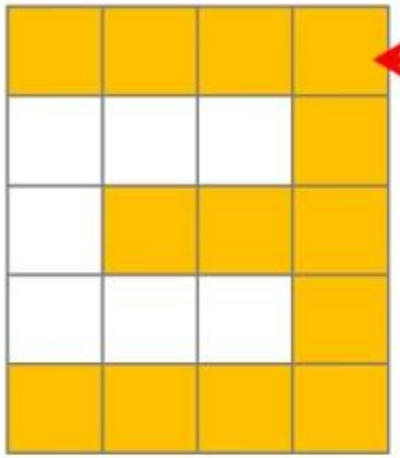
1	1	1	1
0	0	0	1
0	1	1	1
0	0	0	1
1	1	1	1



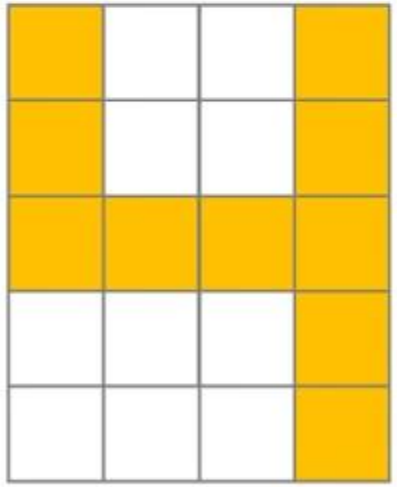
1	0	0	1
1	0	0	1
1	1	1	1
0	0	0	1
0	0	0	1

The corresponding numerical matrices hold normalized values,

Example data



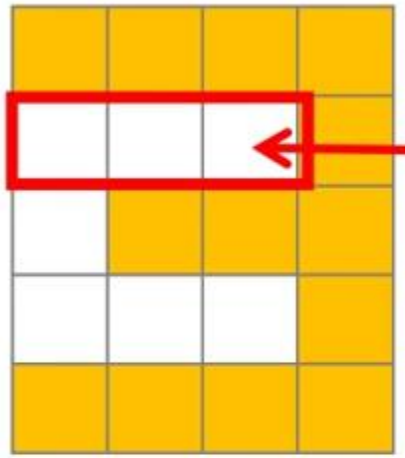
1	1	1	1
0	0	0	1
0	1	1	1
0	0	0	1
1	1	1	1



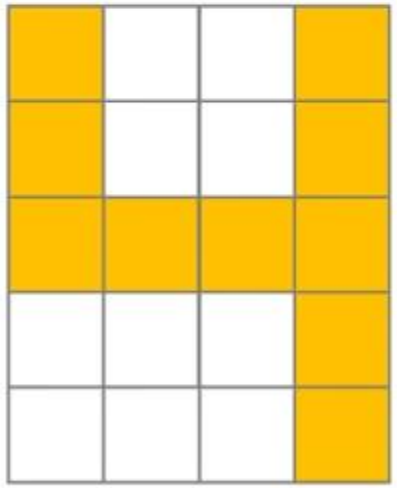
1	0	0	1
1	0	0	1
1	1	1	1
0	0	0	1
0	0	0	1

where the ones represent the orange color,

Example data



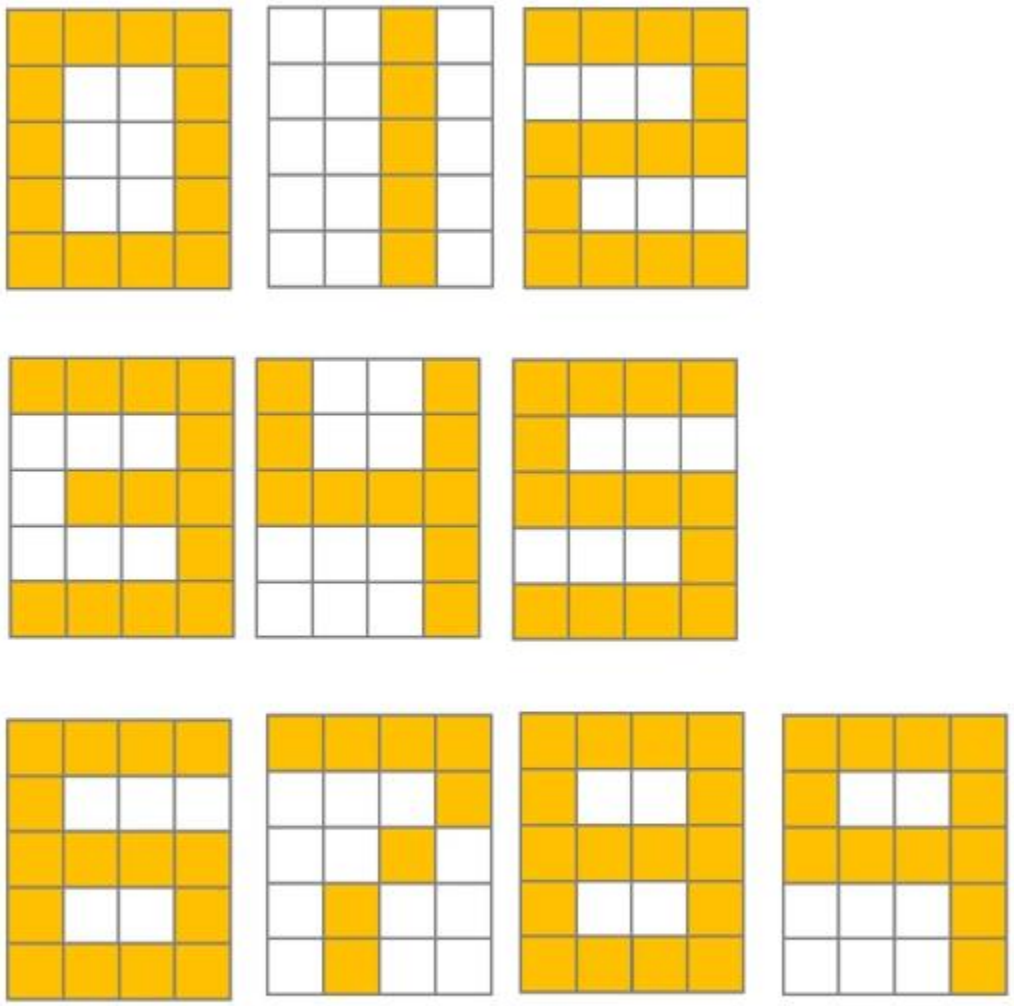
1	1	1	1
0	0	0	1
0	1	1	1
0	0	0	1
1	1	1	1

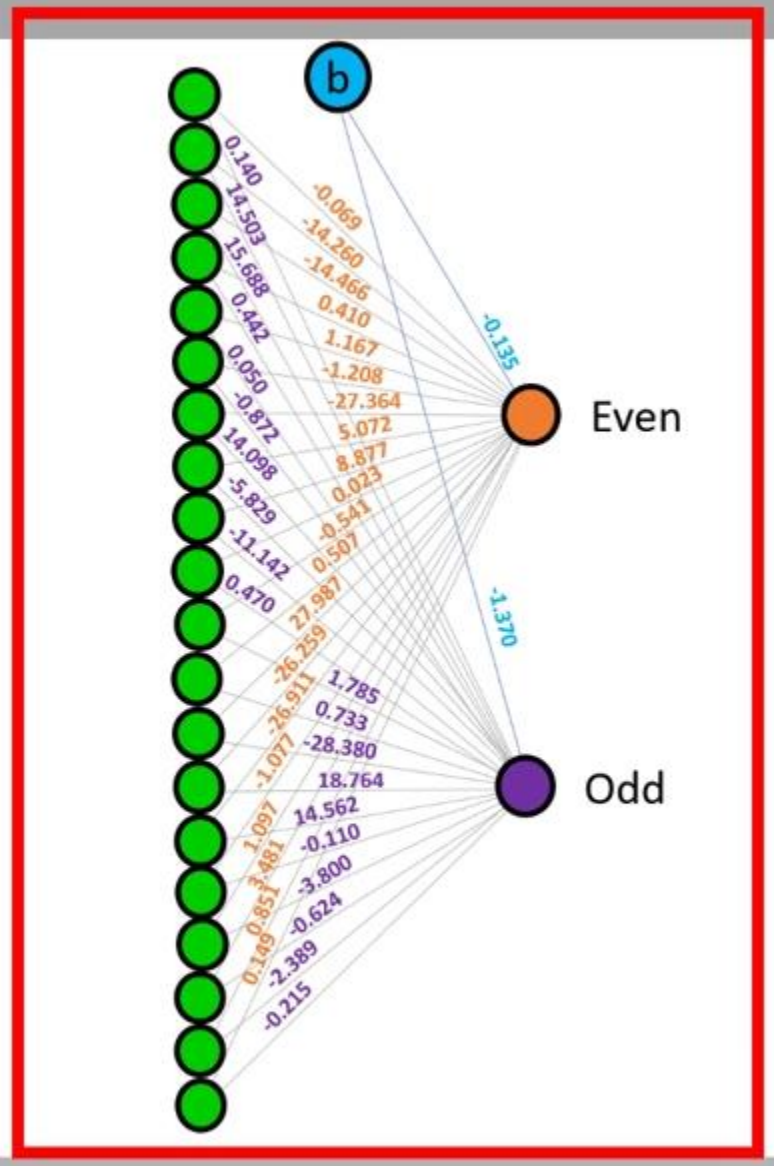


1	0	0	1
1	0	0	1
1	1	1	1
0	0	0	1
0	0	0	1

and the zeros represent the white color.

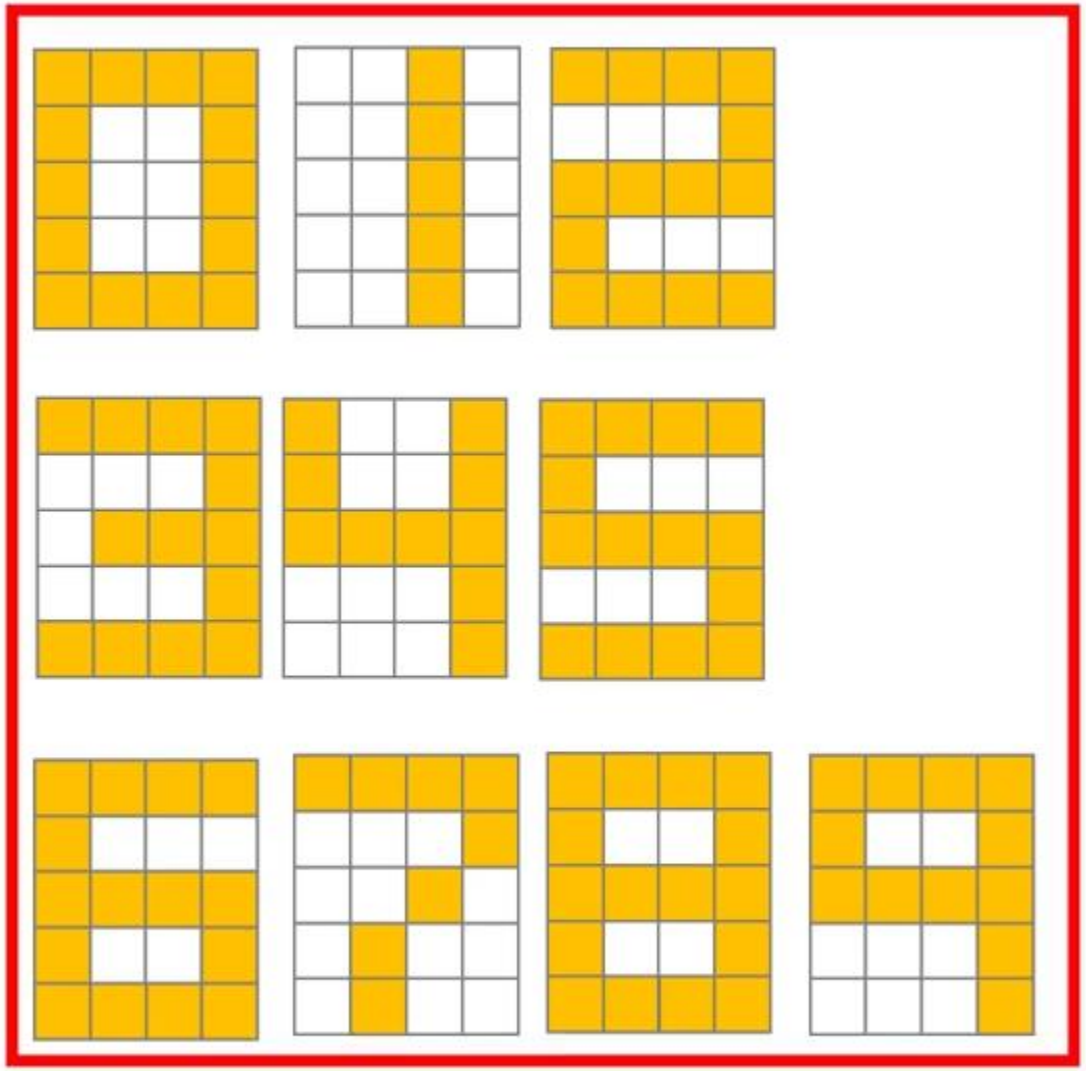
ANN

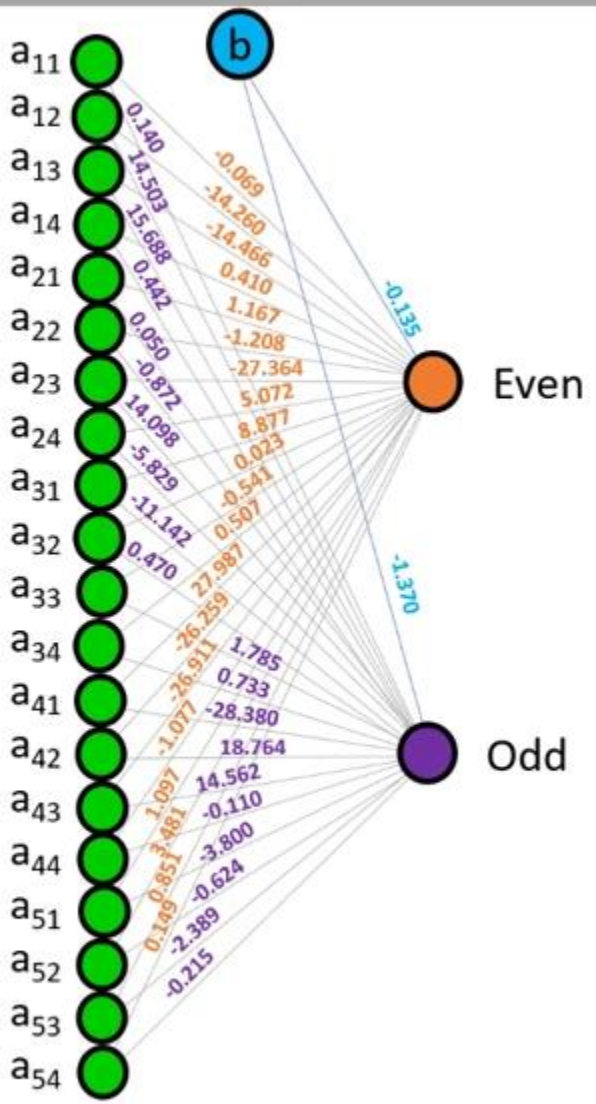




We will now use a simple artificial neural network without any hidden layers,

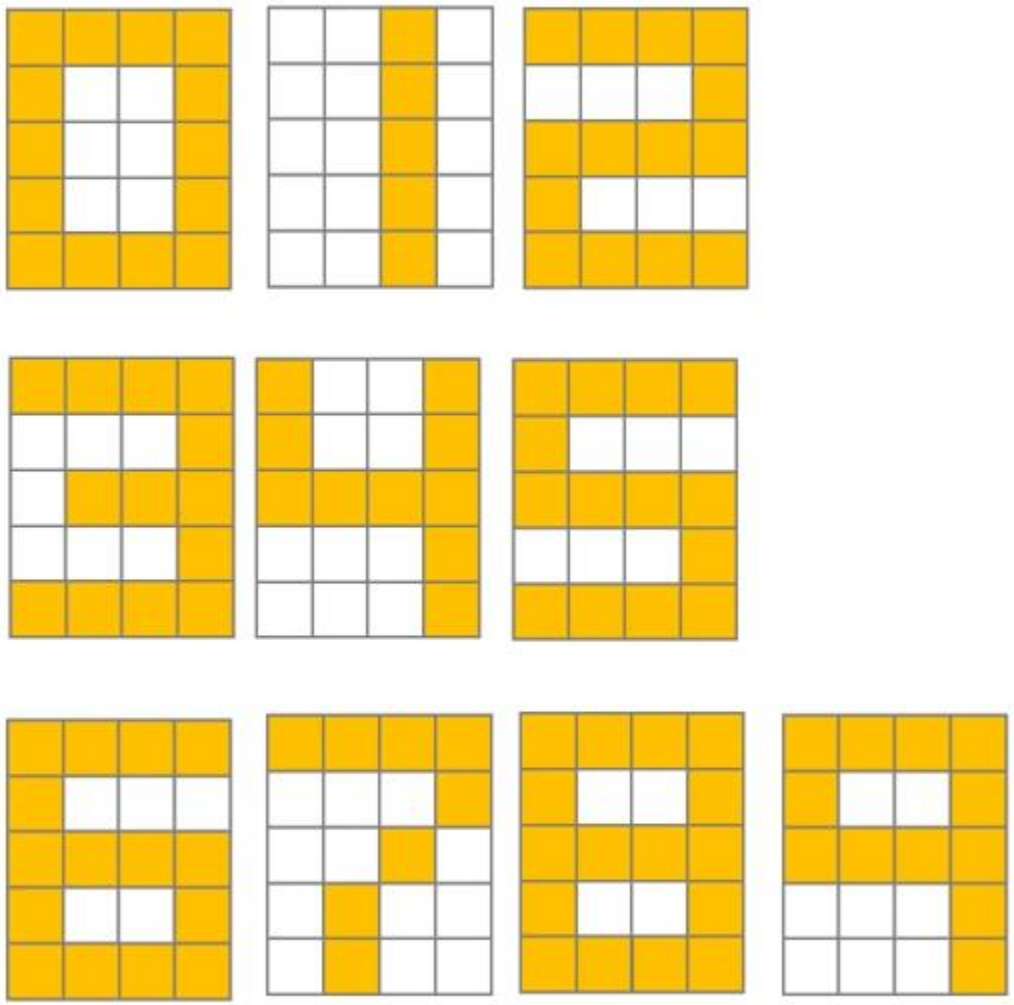
ANN

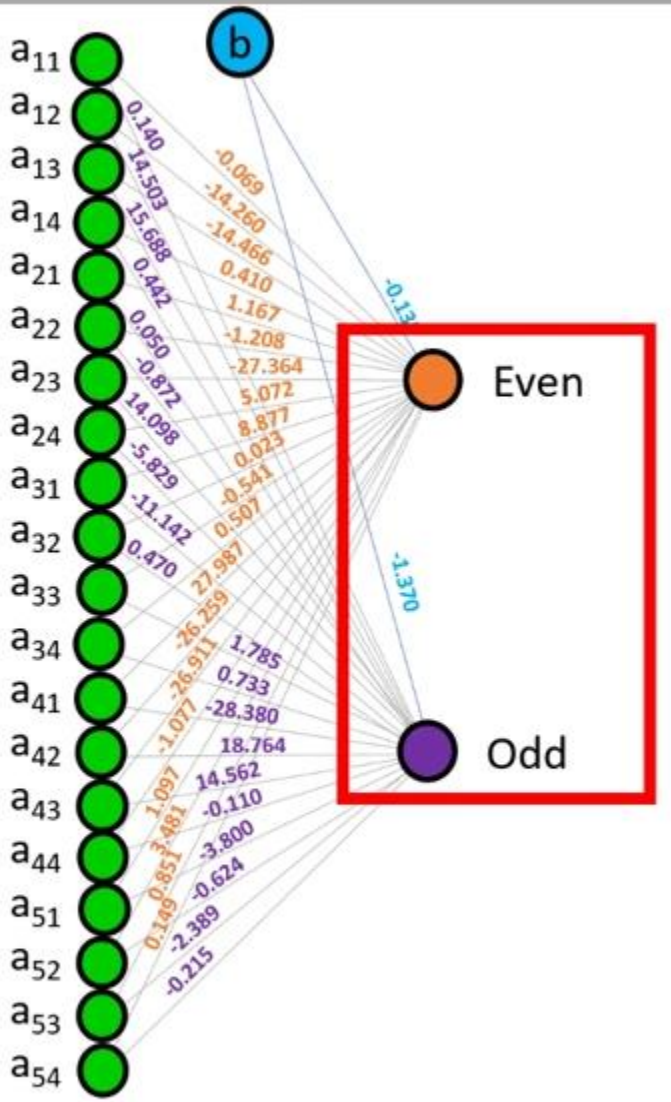




to predict if these images,

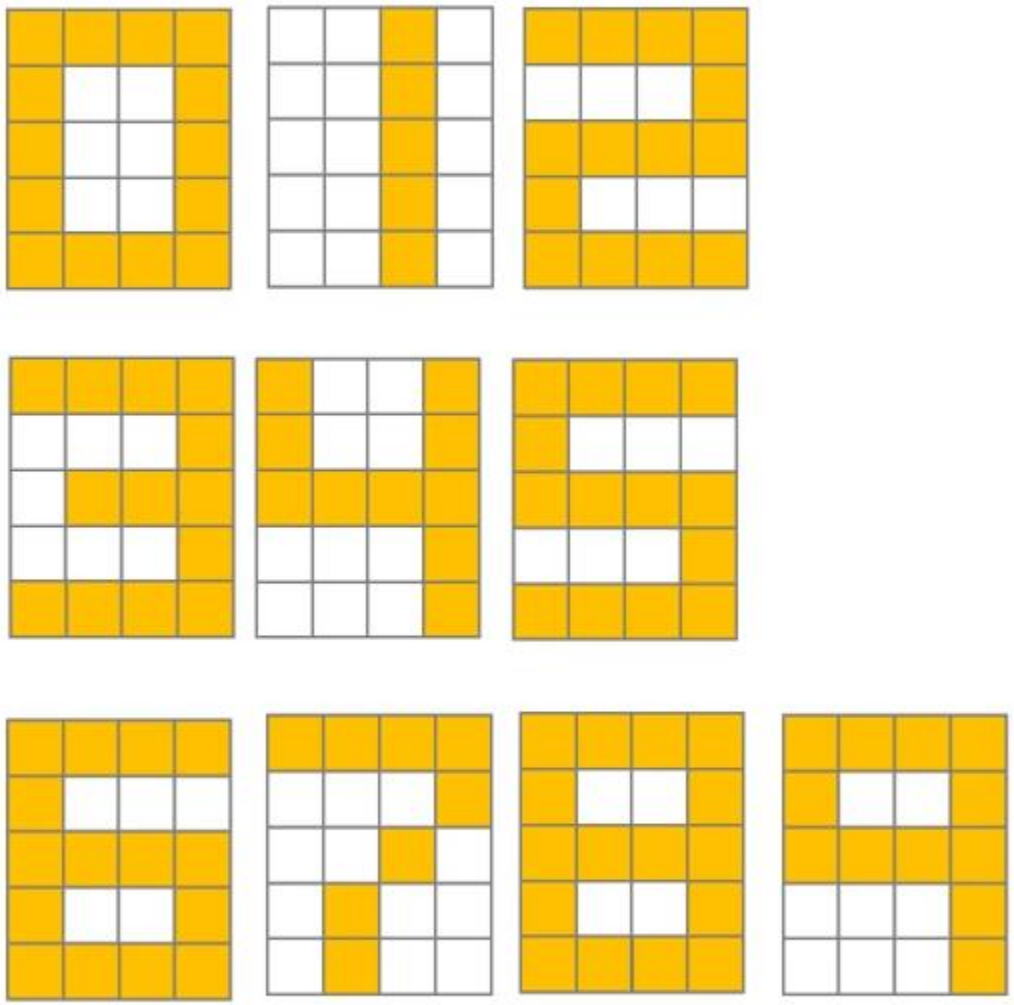
ANN

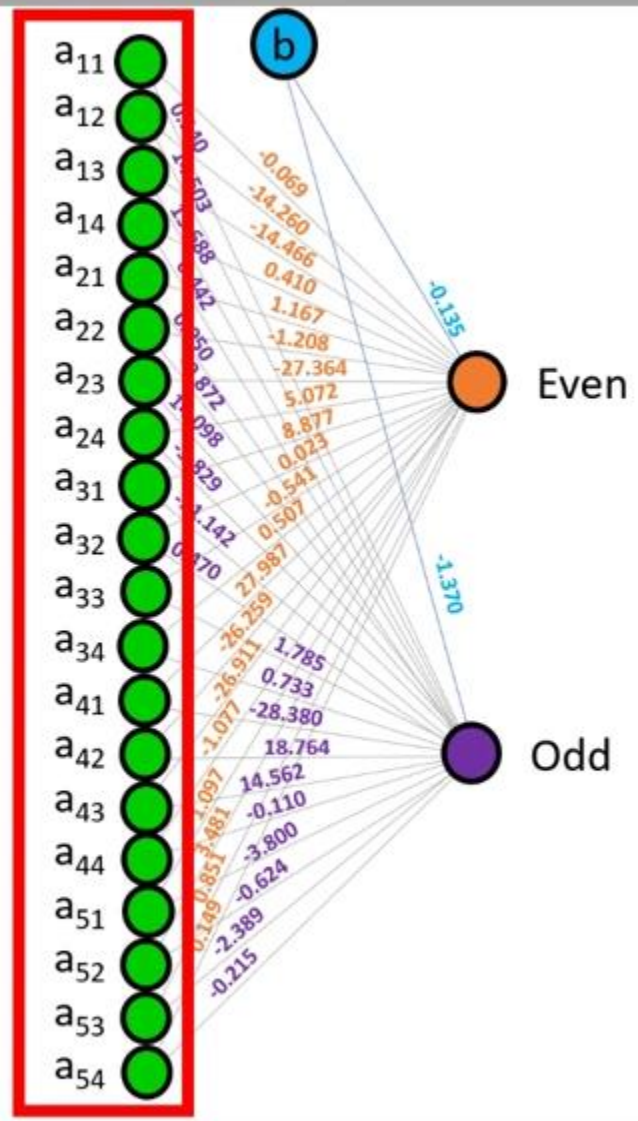




include an even or an odd digit.

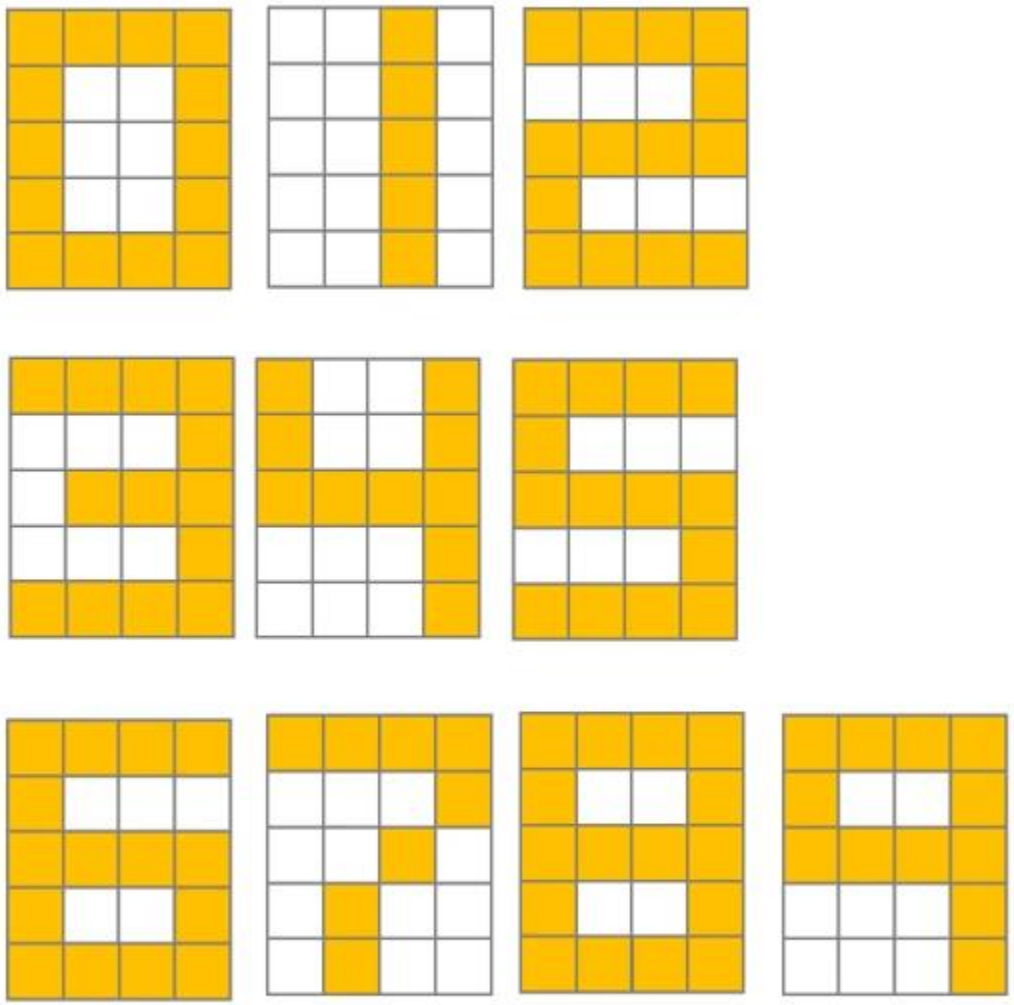
ANN

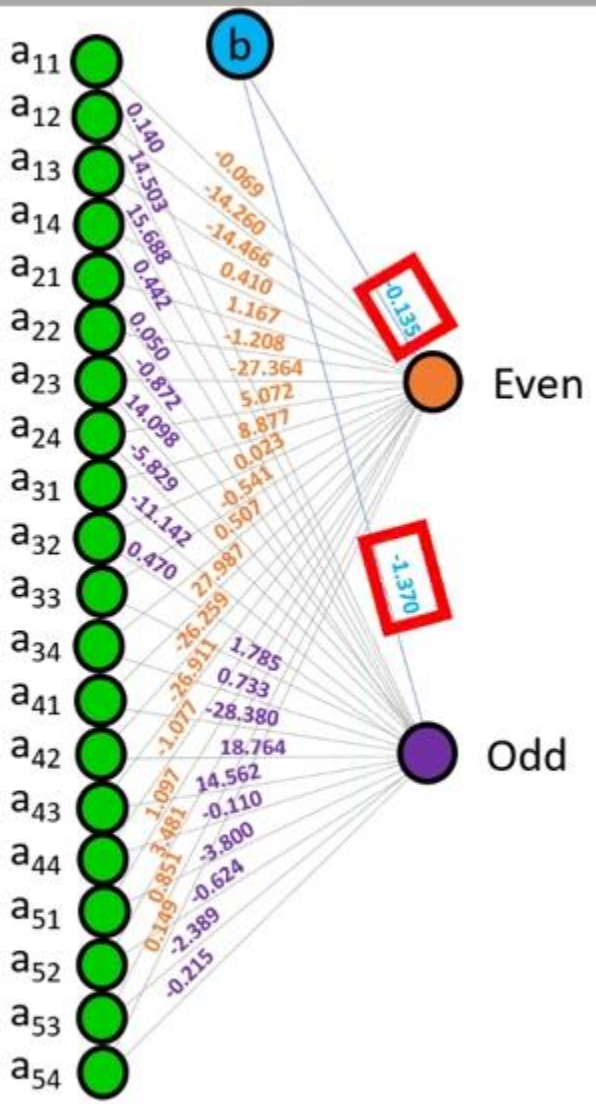




The network has 20 input nodes, because the images have 20 pixels,

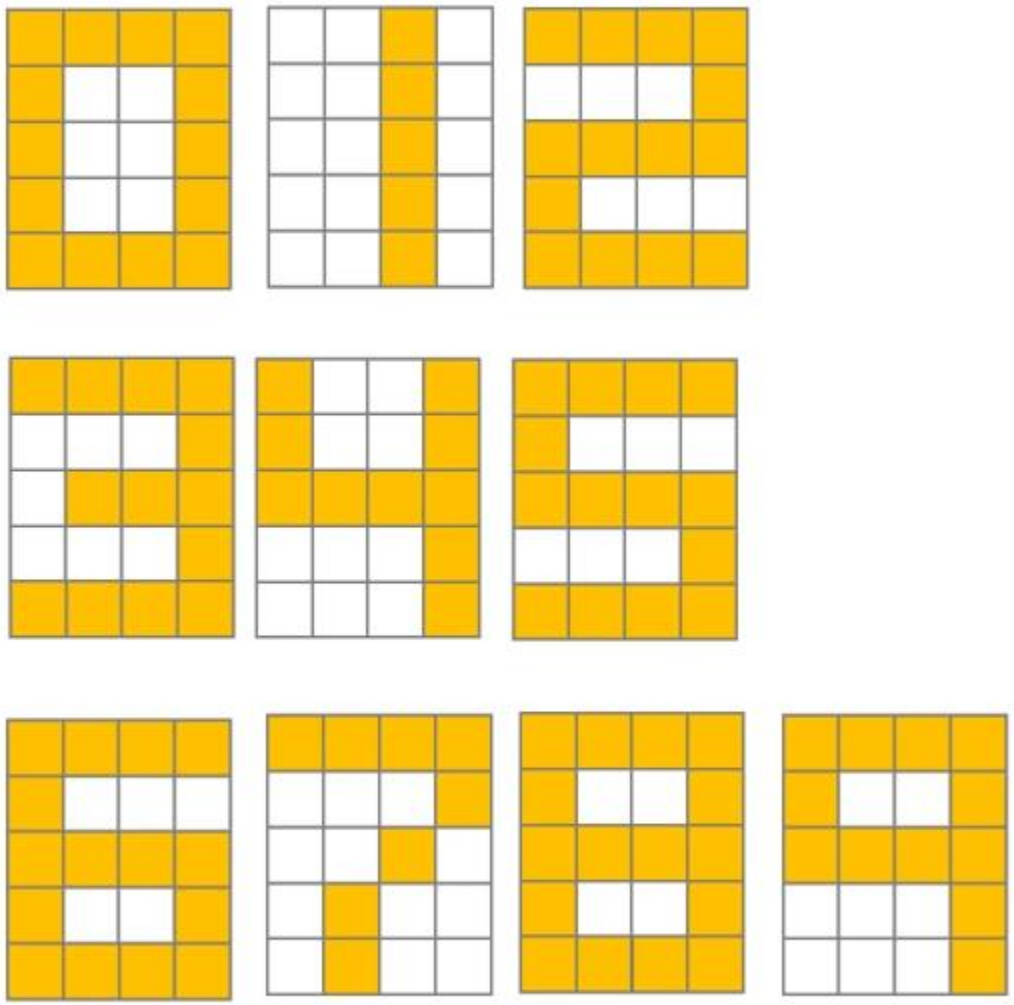
ANN

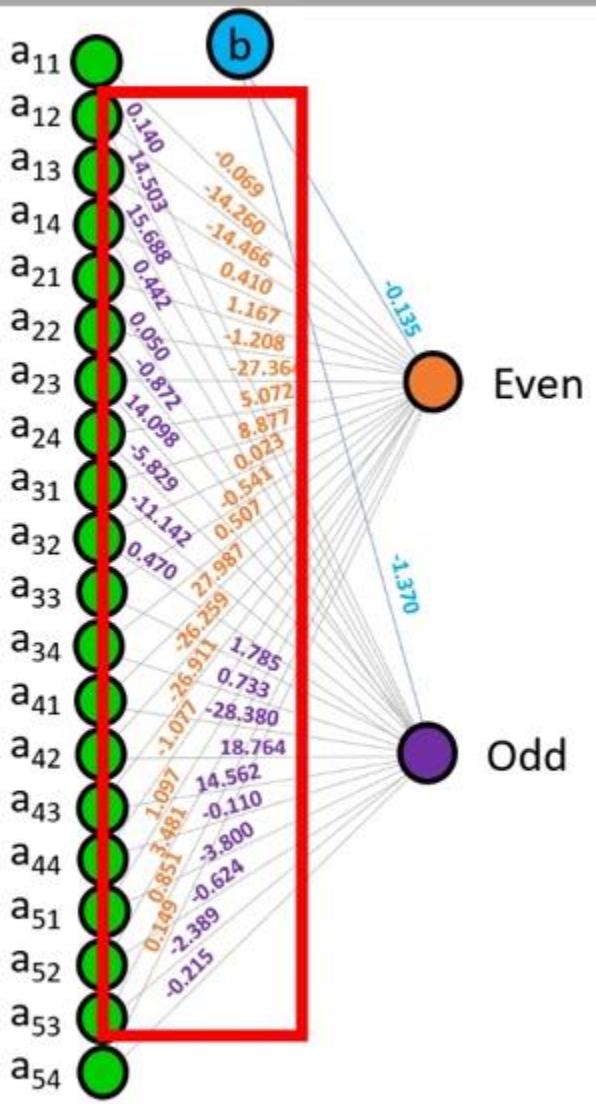




These are the bias weights,

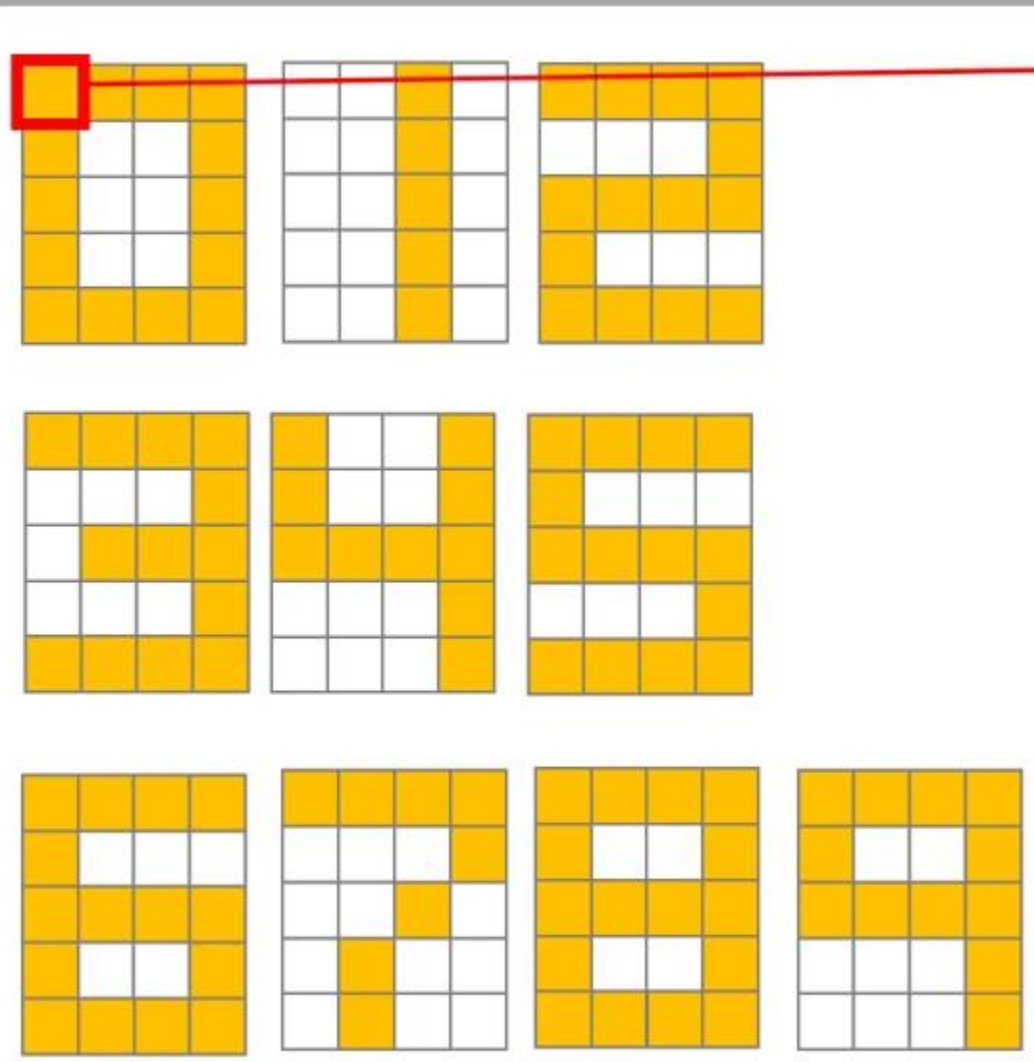
ANN

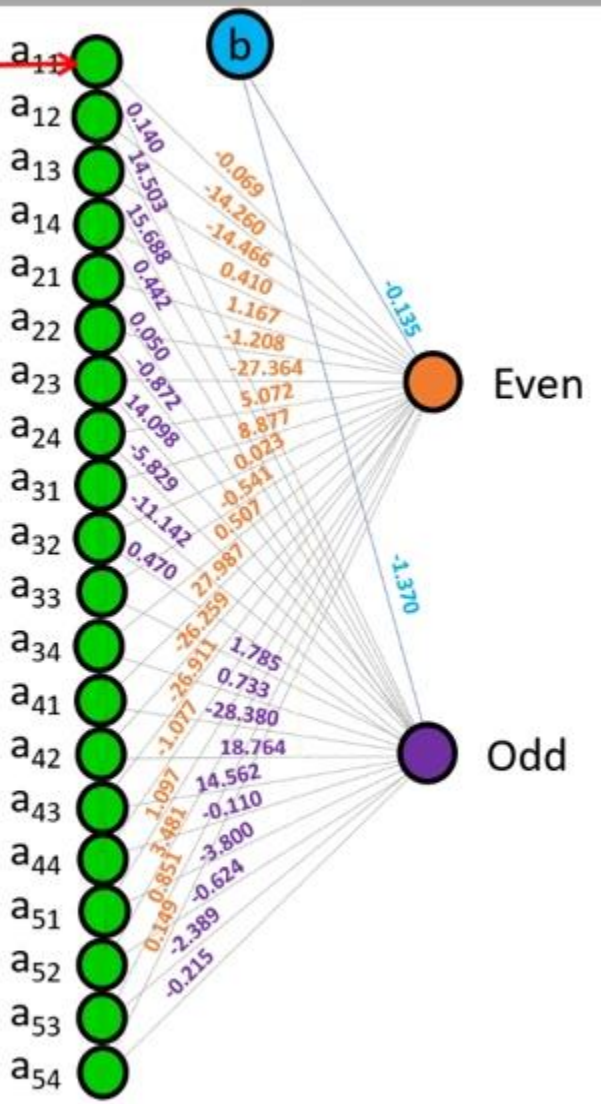




and these are the weights that are multiplied with the input signals to compute the values in the output nodes.

ANN

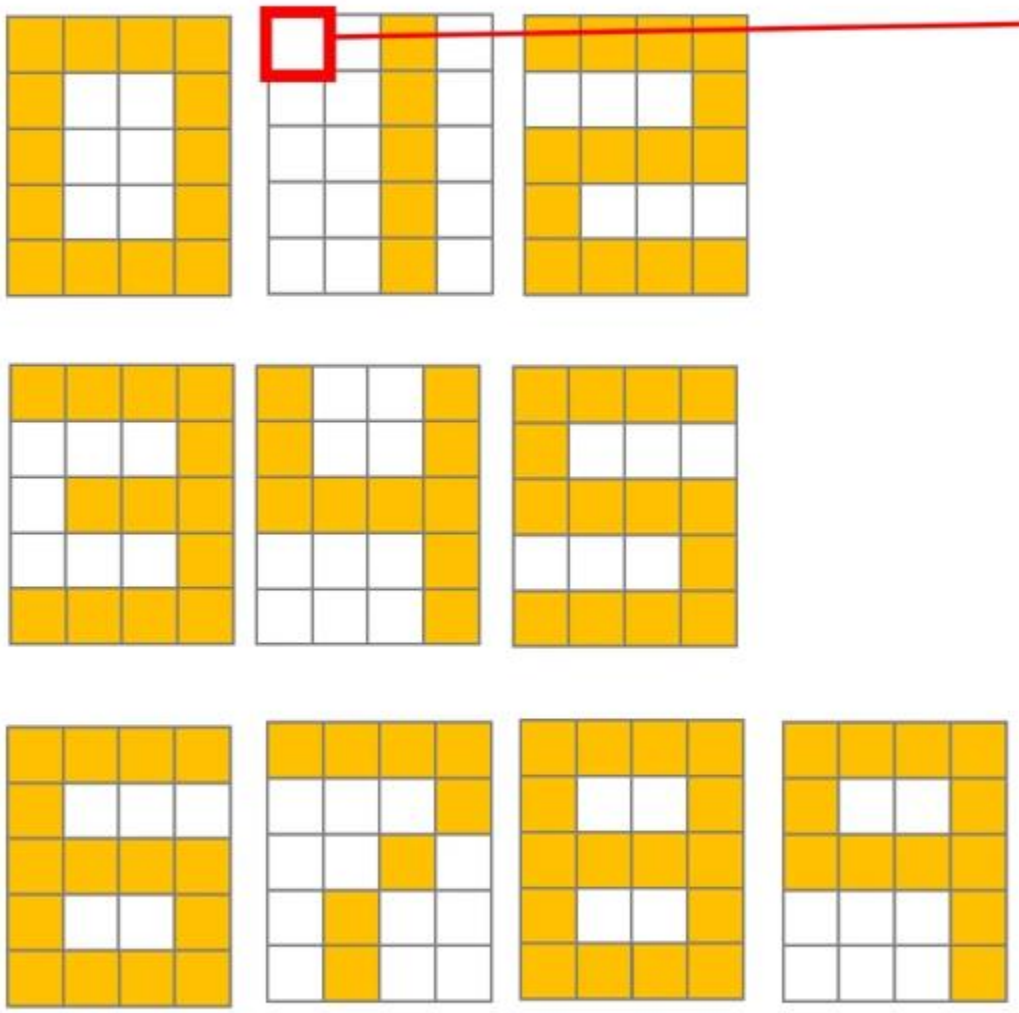


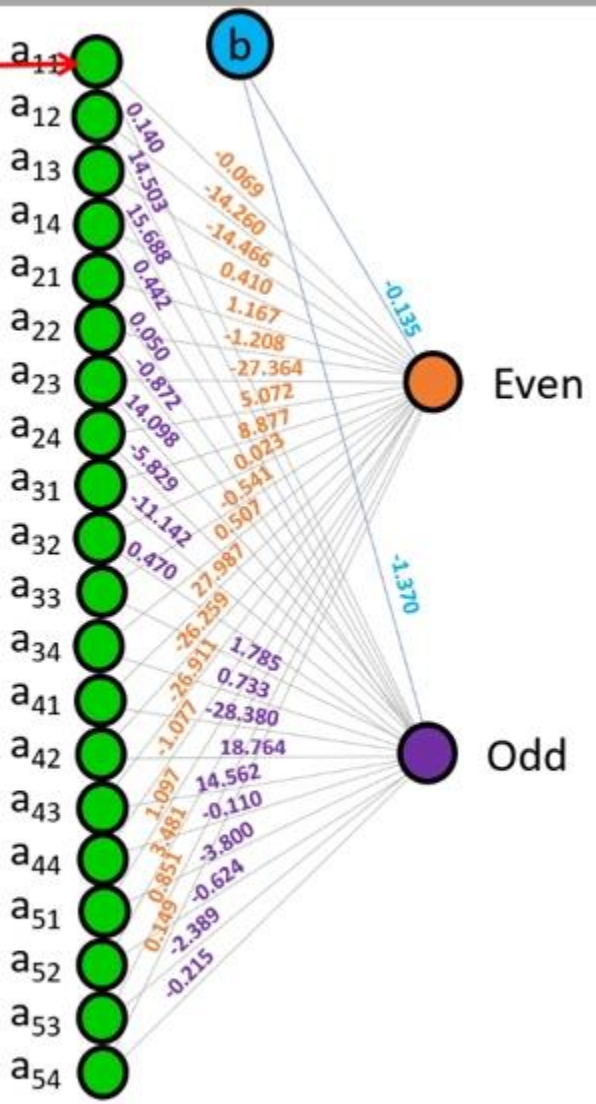


a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

This image holds the value one in this pixel because this pixel has a color,

ANN

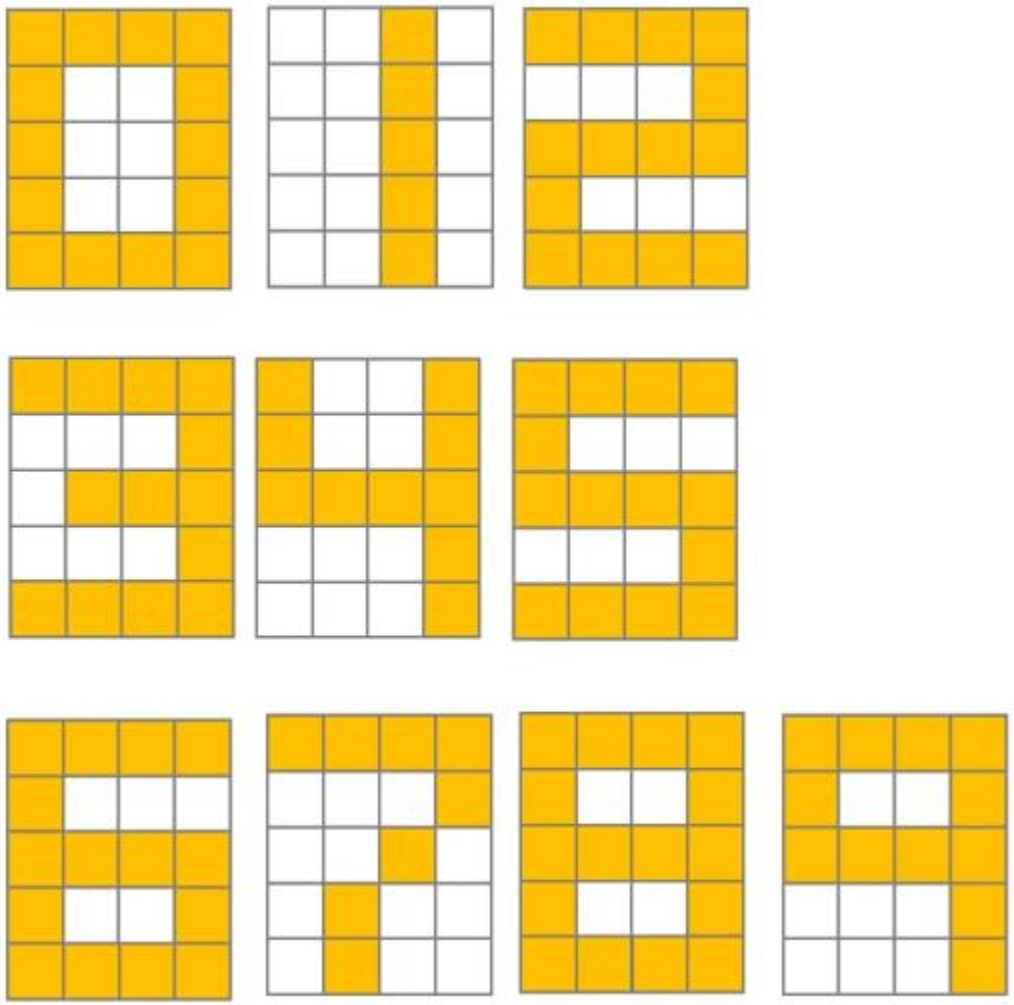


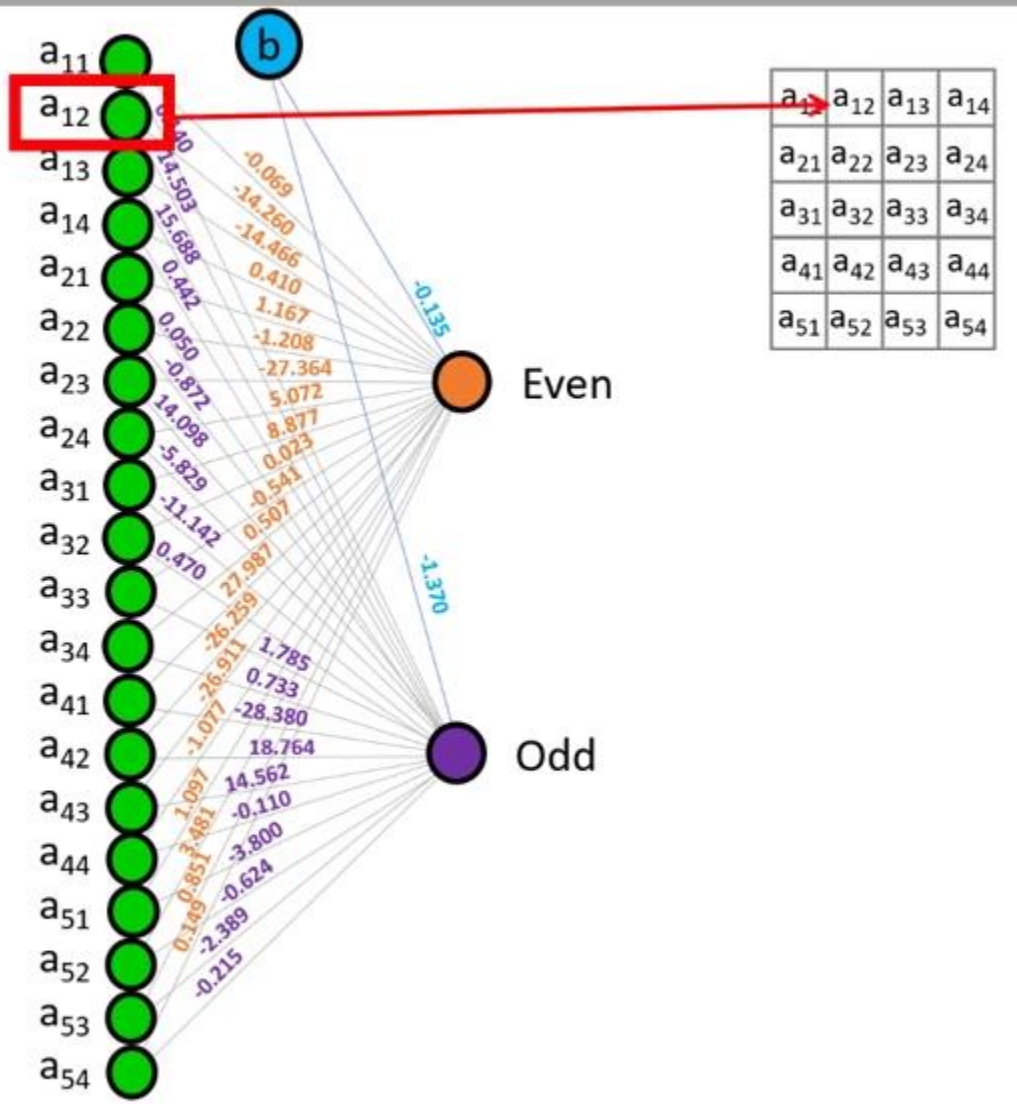


a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

whereas this image holds the value zero because the corresponding pixel in this image has no color.

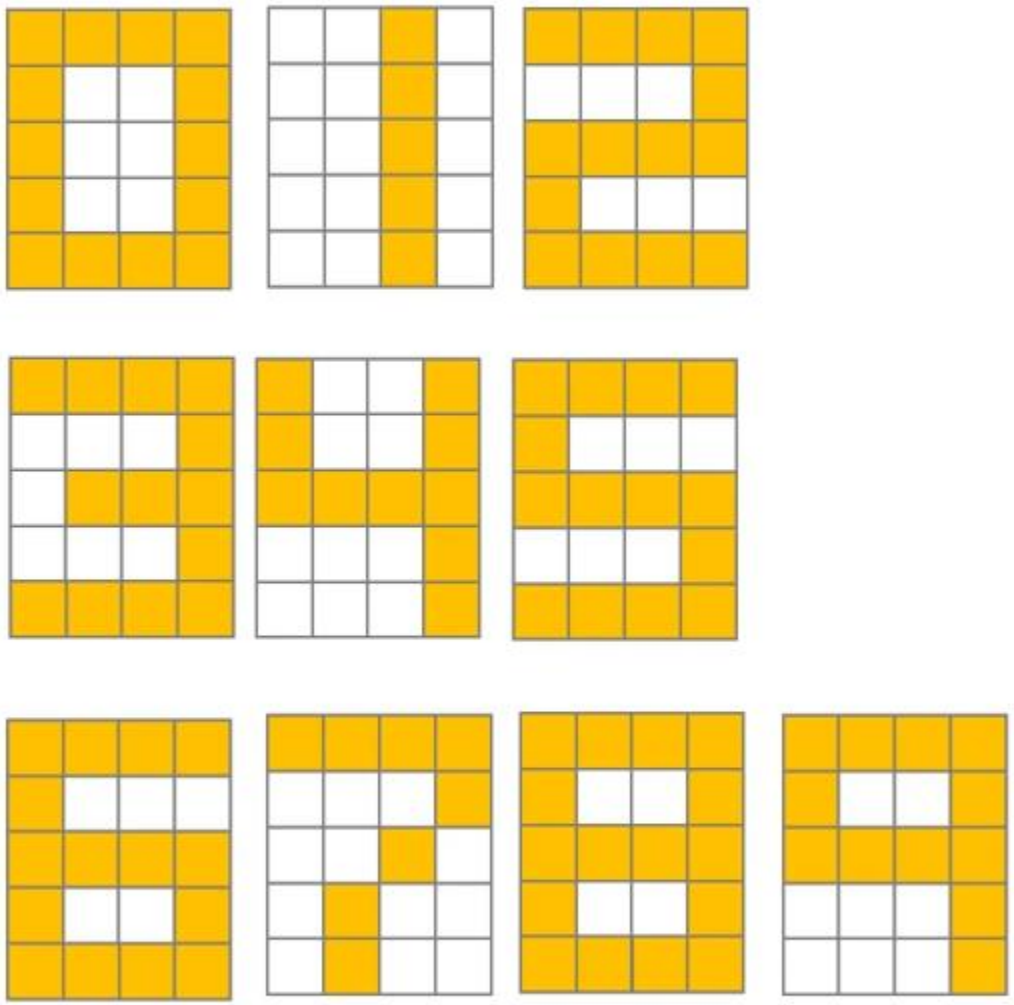
ANN

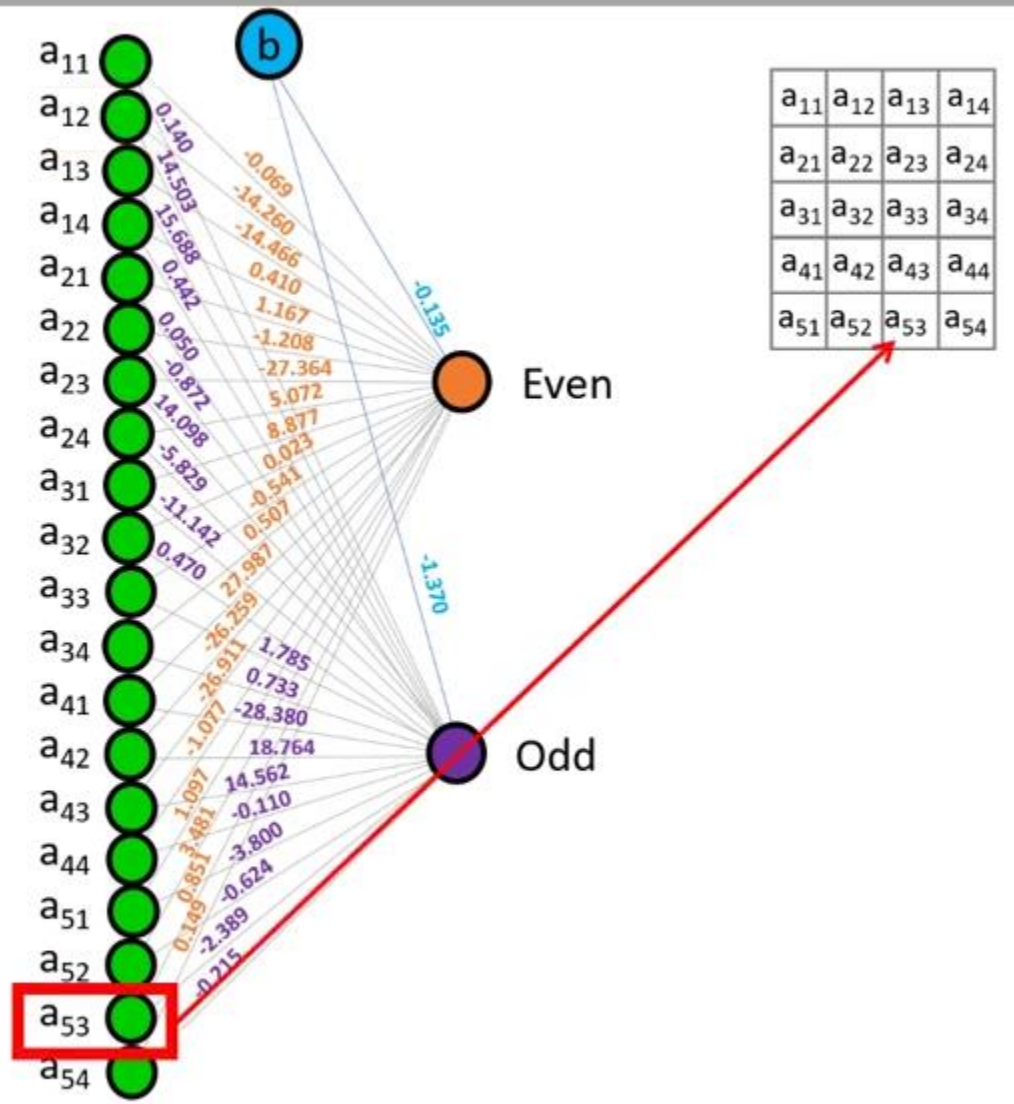




This input node takes the value of the pixel in the image that is located on the first row, second column,

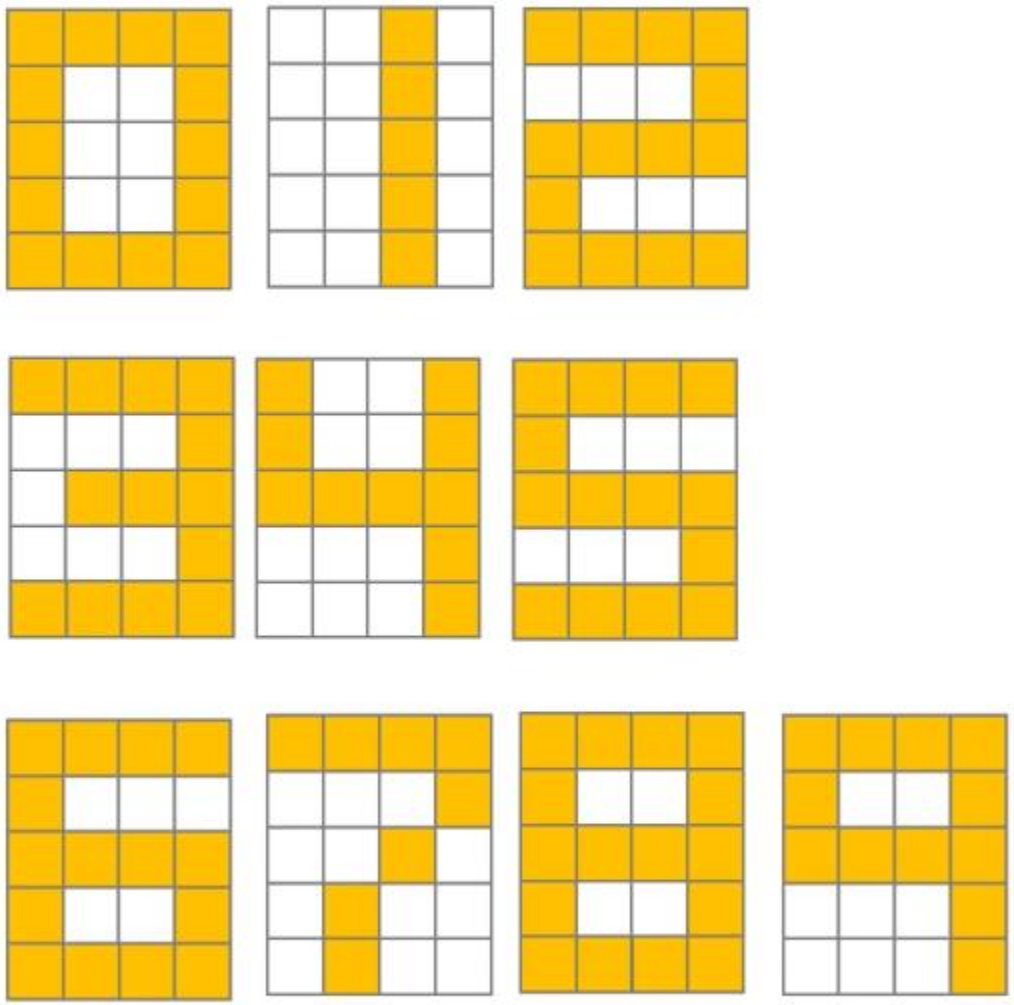
ANN

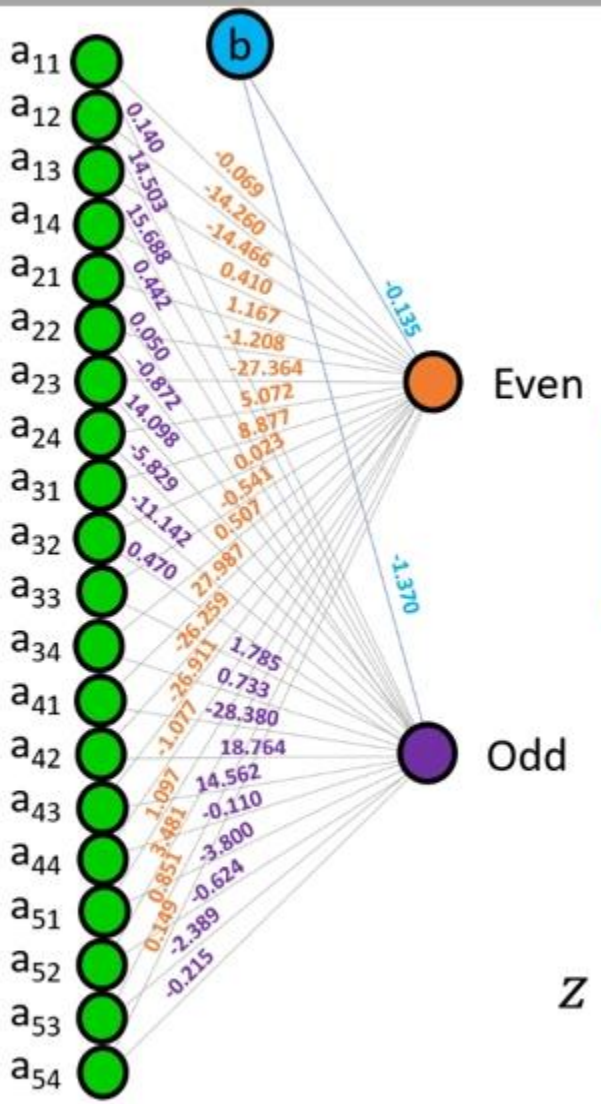




whereas this input node takes the value of the pixel in the image that is located on the fifth row, third column.

ANN





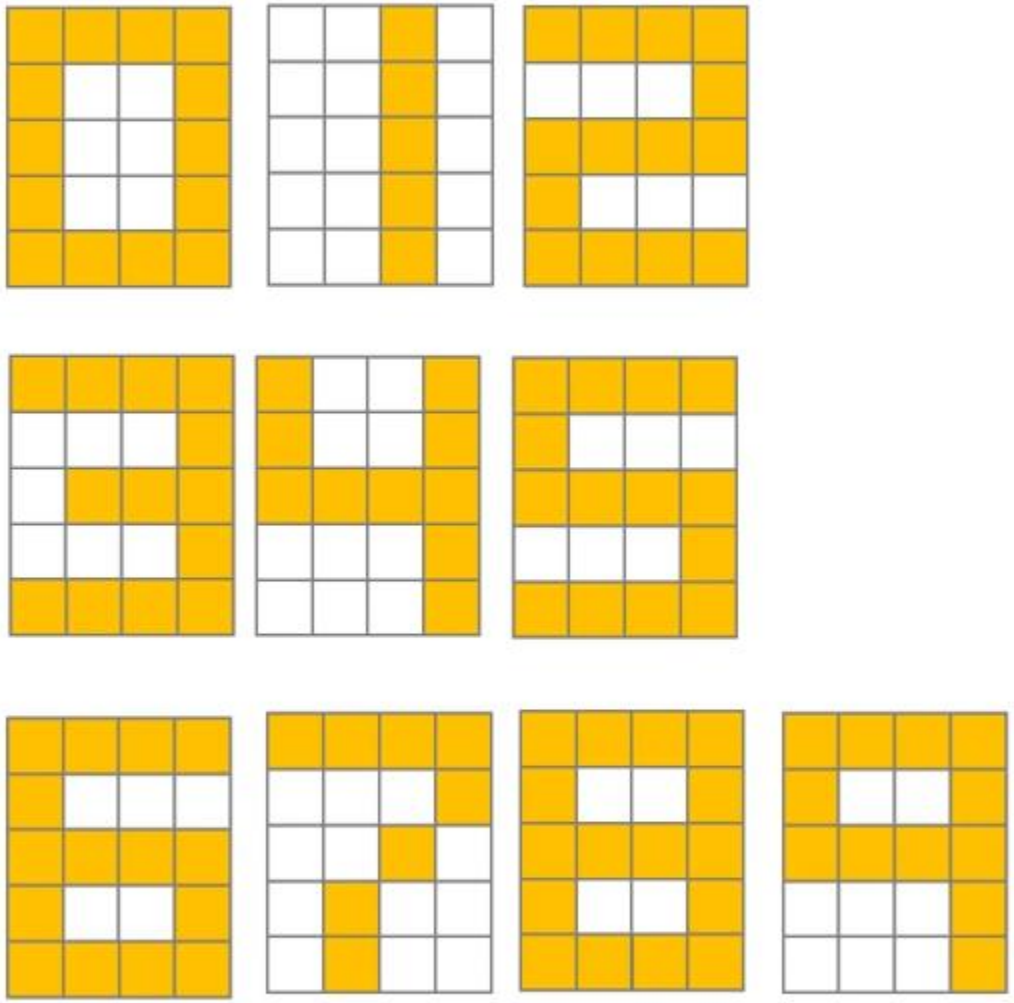
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

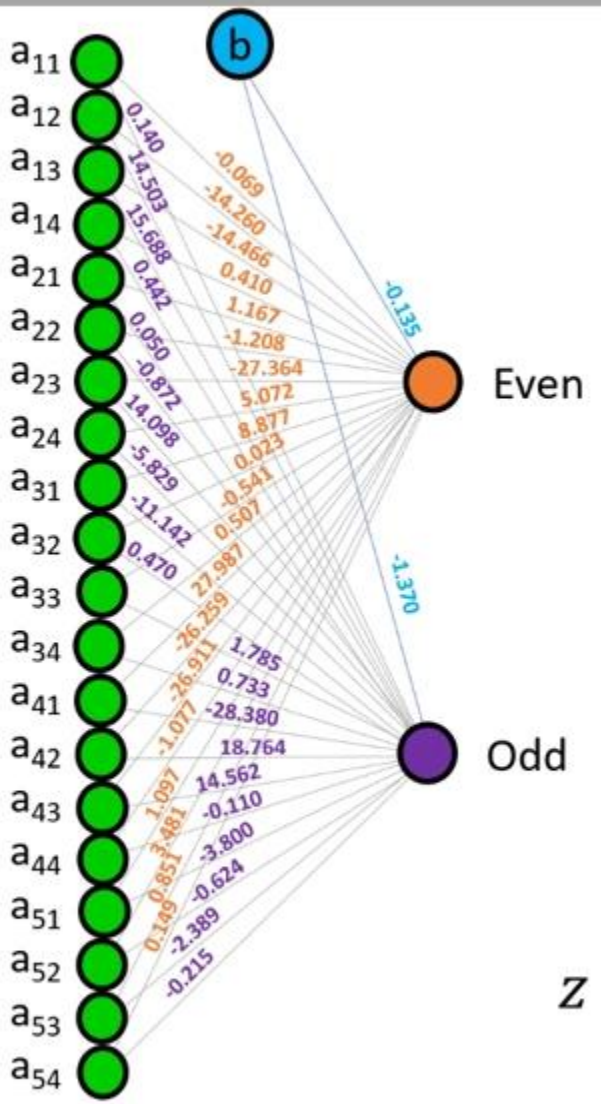
$$f(z) = \frac{1}{1 + e^{-z}}$$

$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

In this example, I will use the logistic activation function to compute the values of the output nodes,

ANN





a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

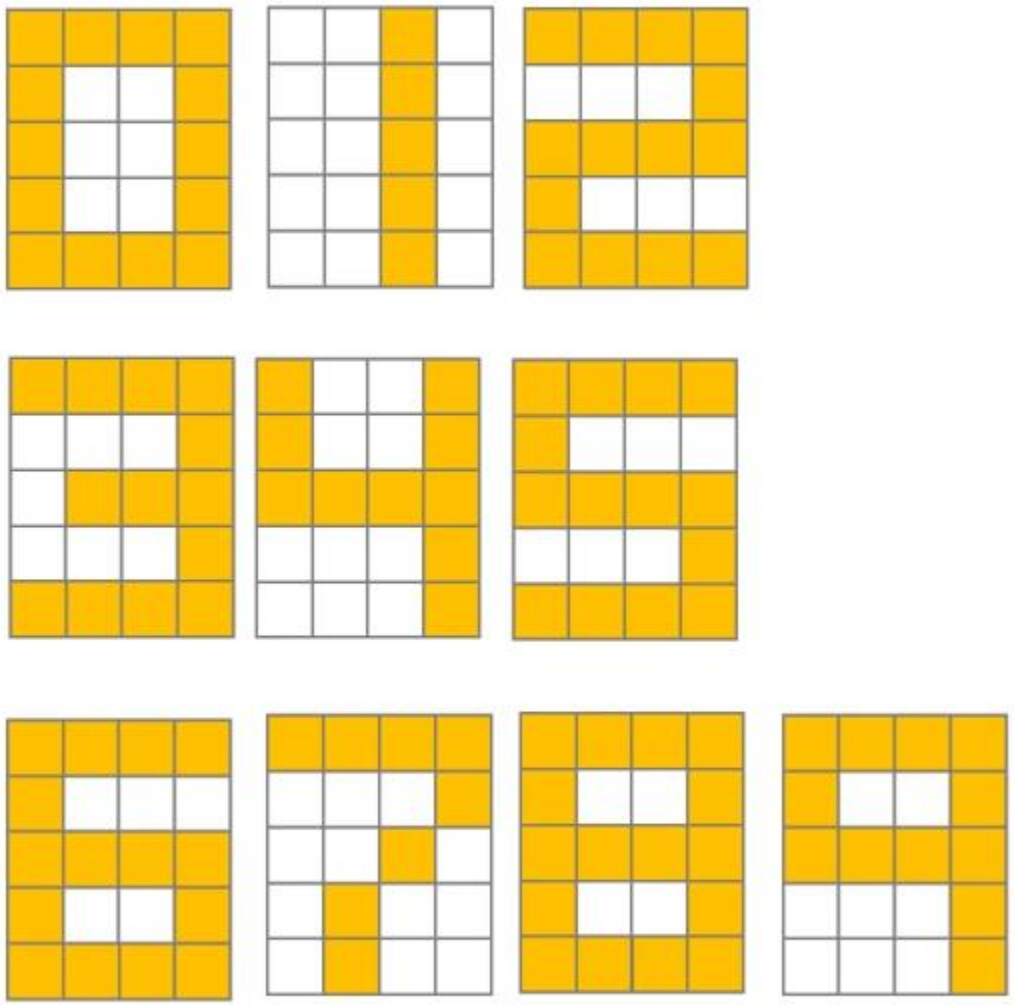
$$f(z) = \frac{1}{1 + e^{-z}}$$

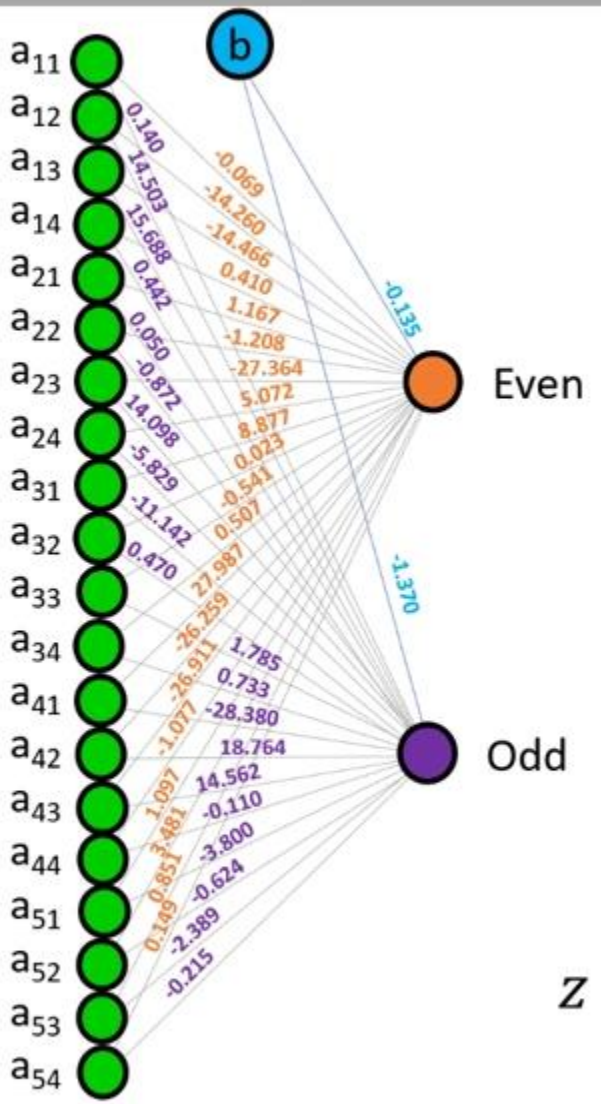
$$e = 2.71828..$$

$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

where e is the Euler's number,

ANN





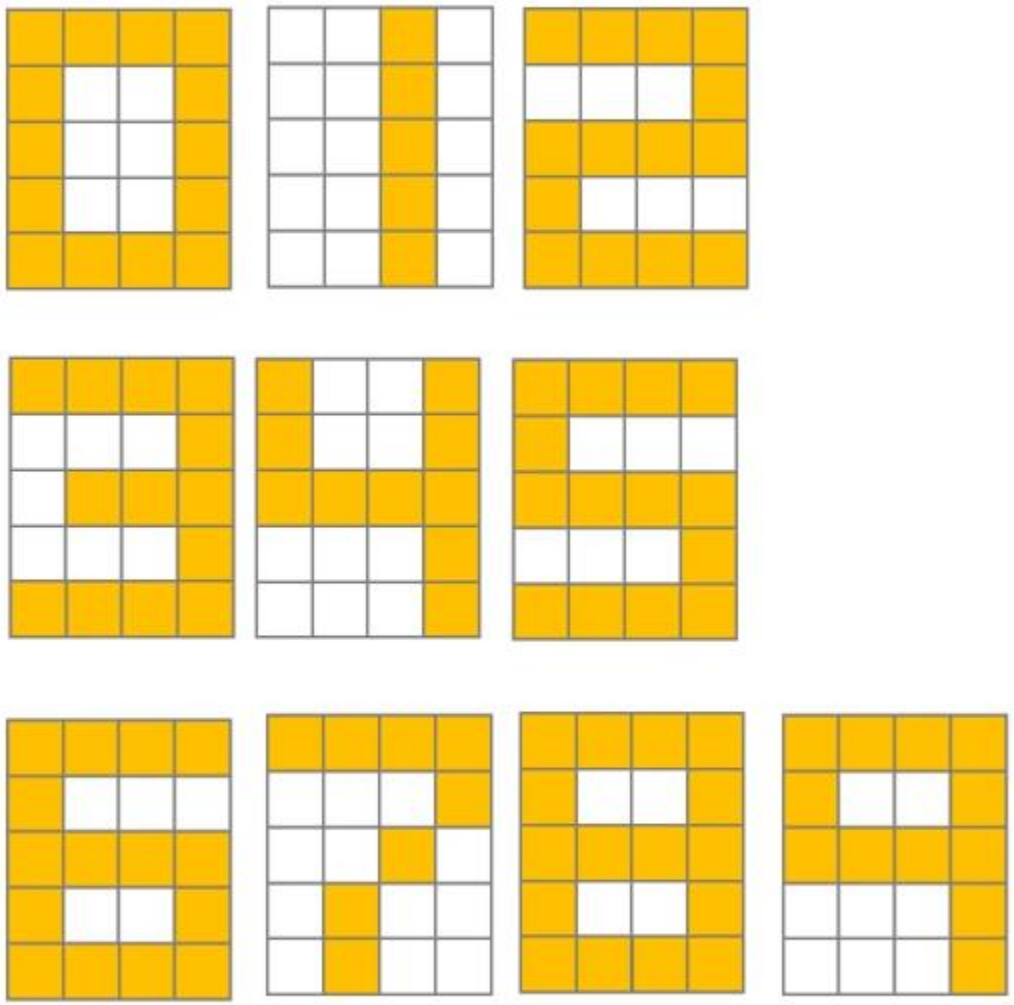
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

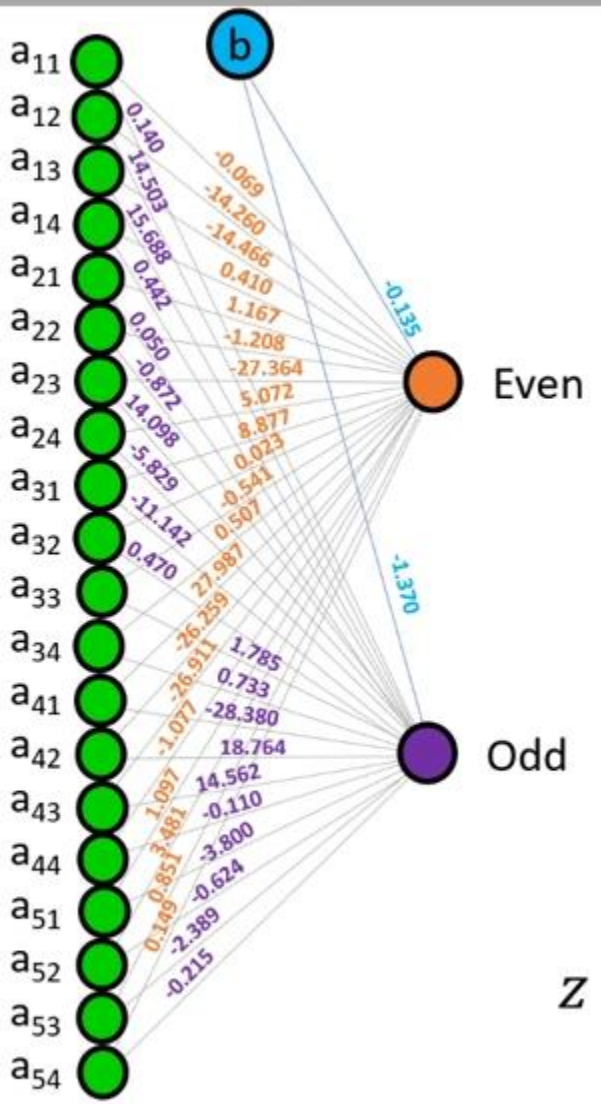
$$f(z) = \frac{1}{1 + e^{-z}}$$

$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

and z represents,

ANN





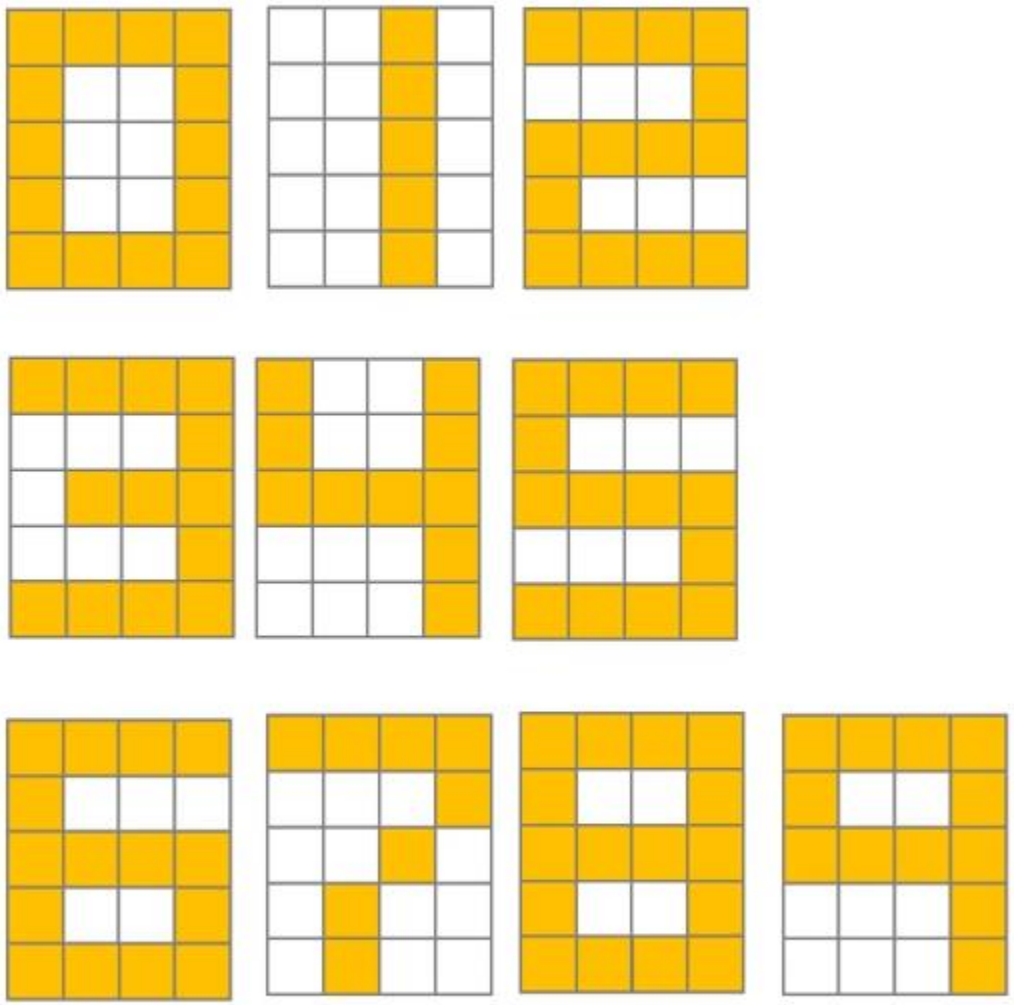
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

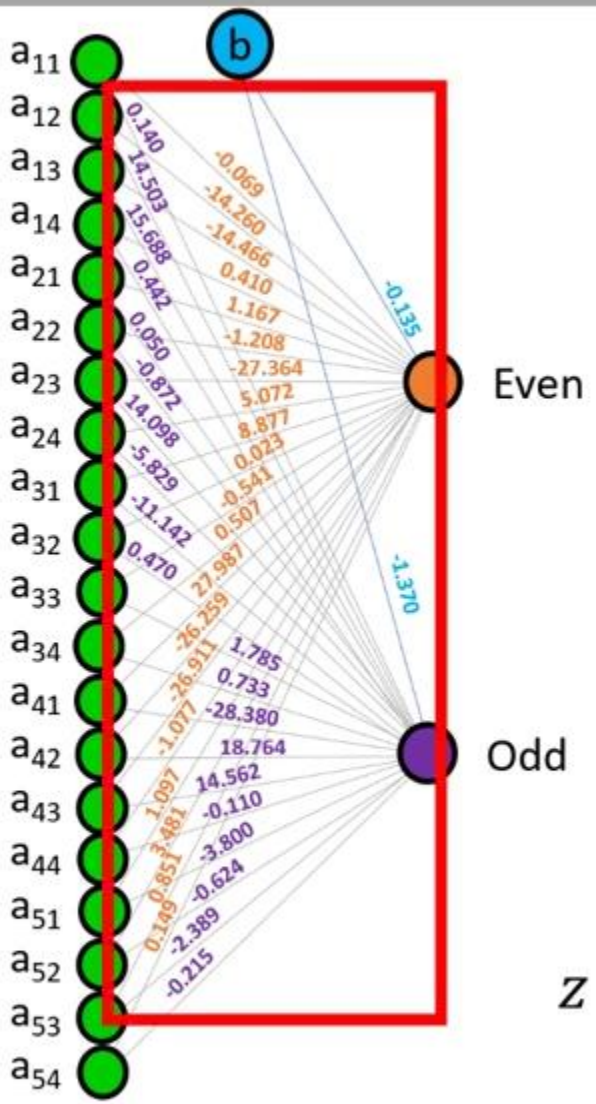
$$f(z) = \frac{1}{1 + e^{-z}}$$

$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

the sum of the products of the weights and the input values.

ANN





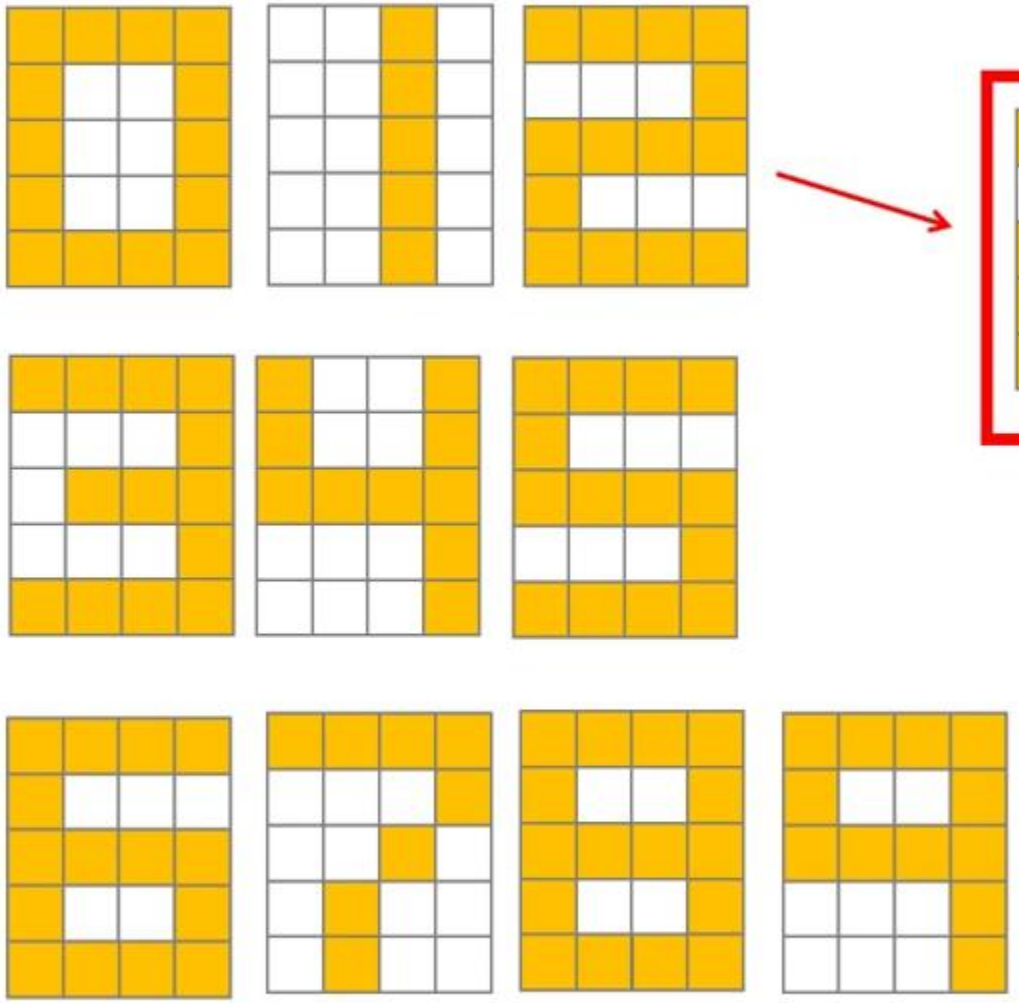
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

$$f(z) = \frac{1}{1 + e^{-z}}$$

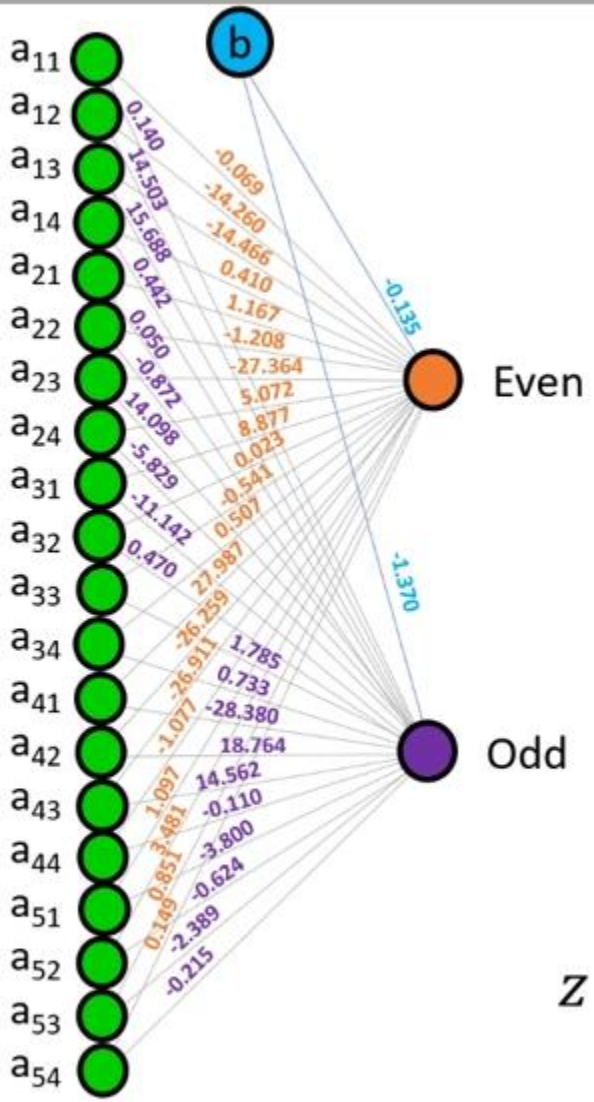
$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

The values of the weights have been optimized so that the network classifies the images as good as possible.

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1



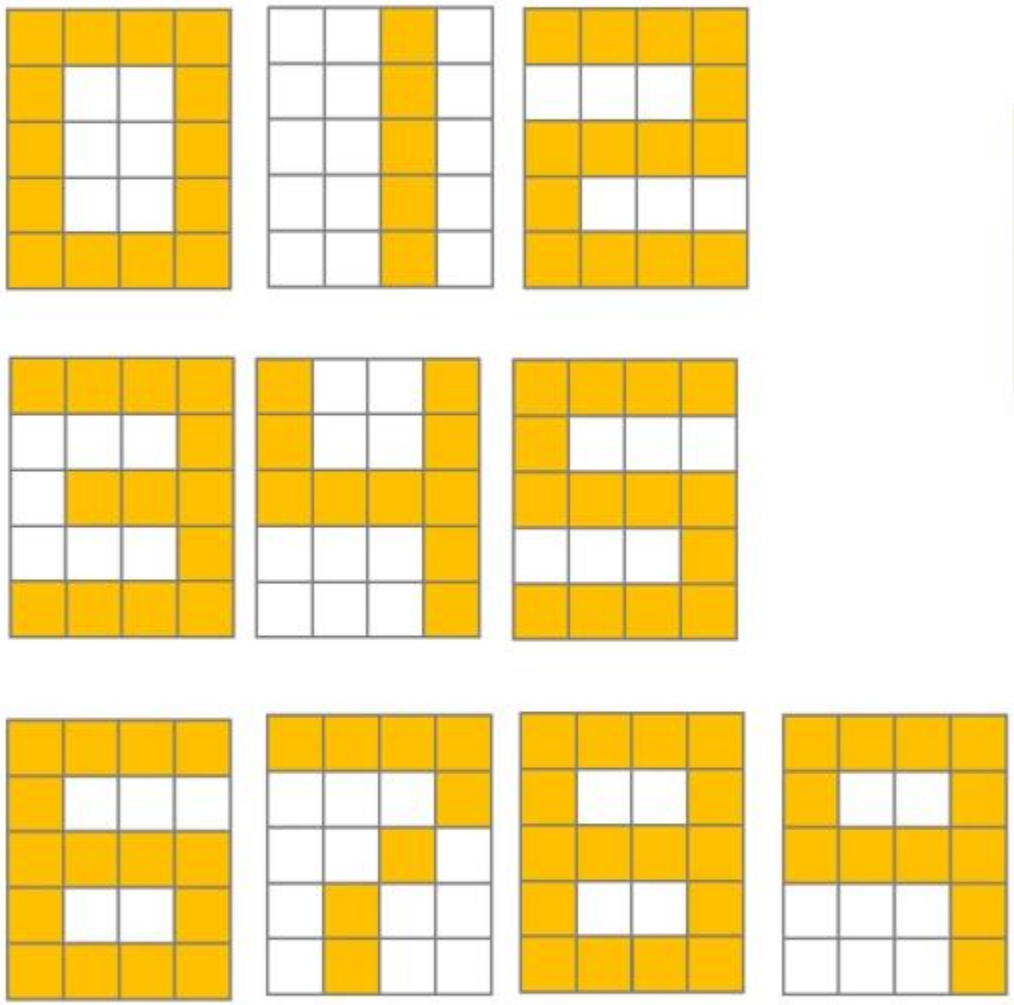
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

$$f(z) = \frac{1}{1 + e^{-z}}$$

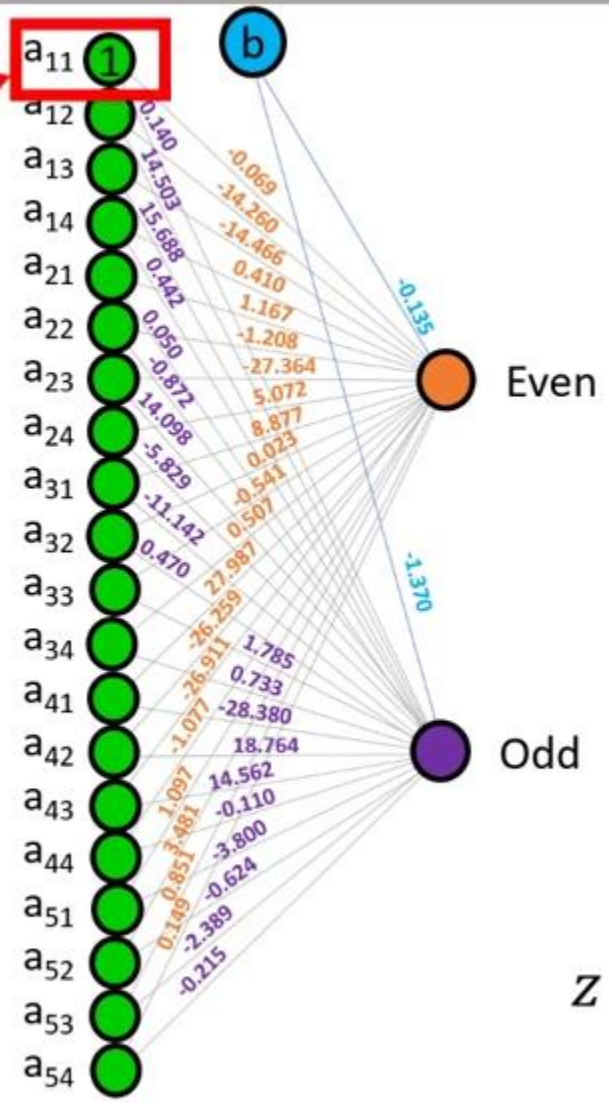
$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

Let's say that we would use the network to predict if this image contains an odd or an even number.

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1



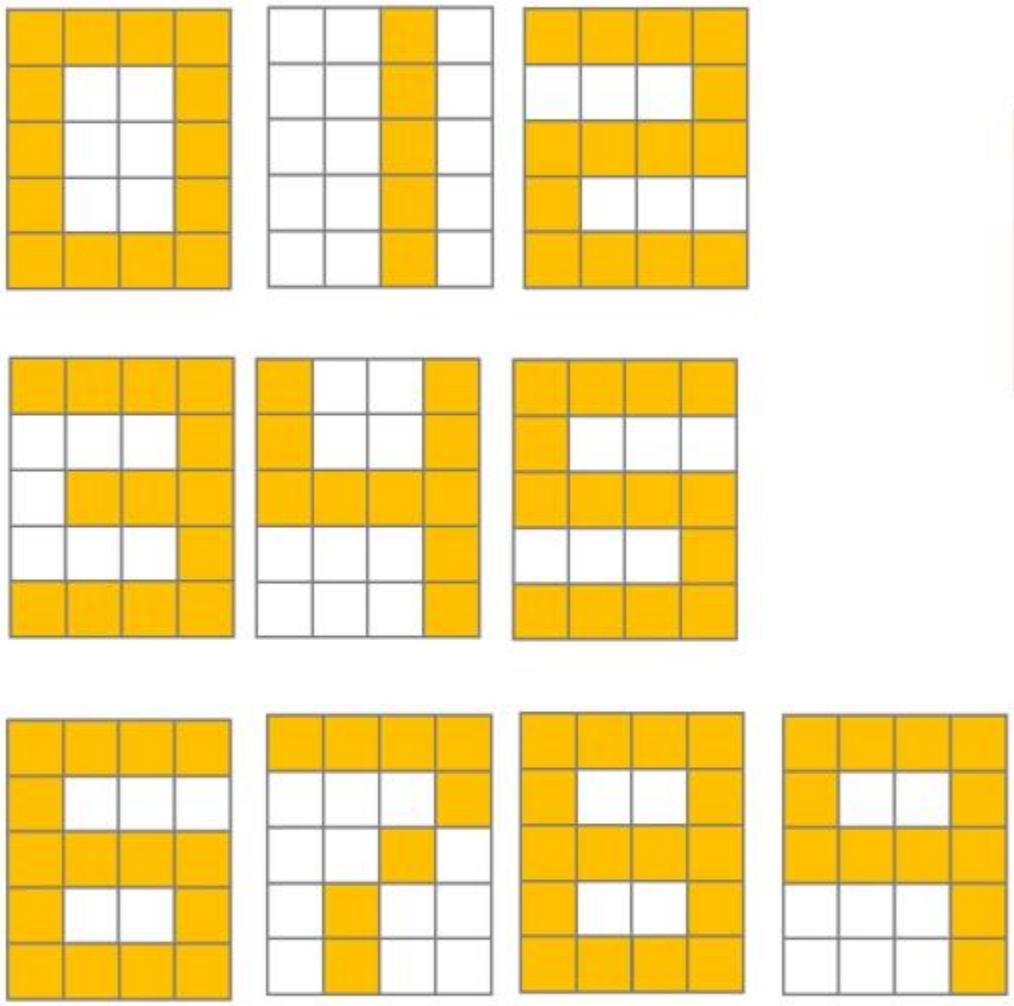
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

$$f(z) = \frac{1}{1 + e^{-z}}$$

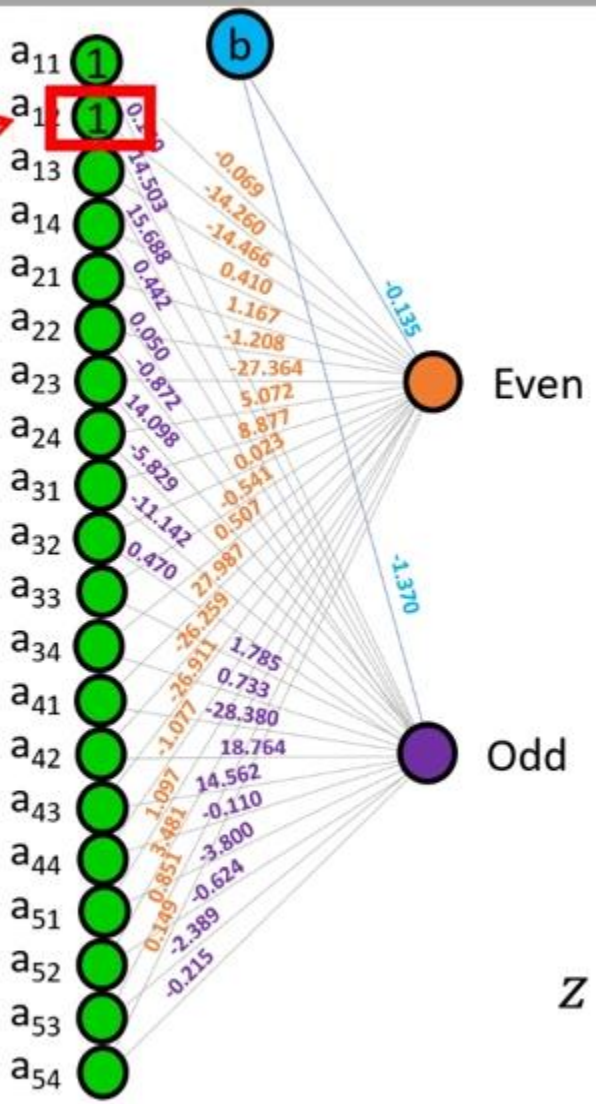
$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

We would then plug in the pixel values in the corresponding input nodes,

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1

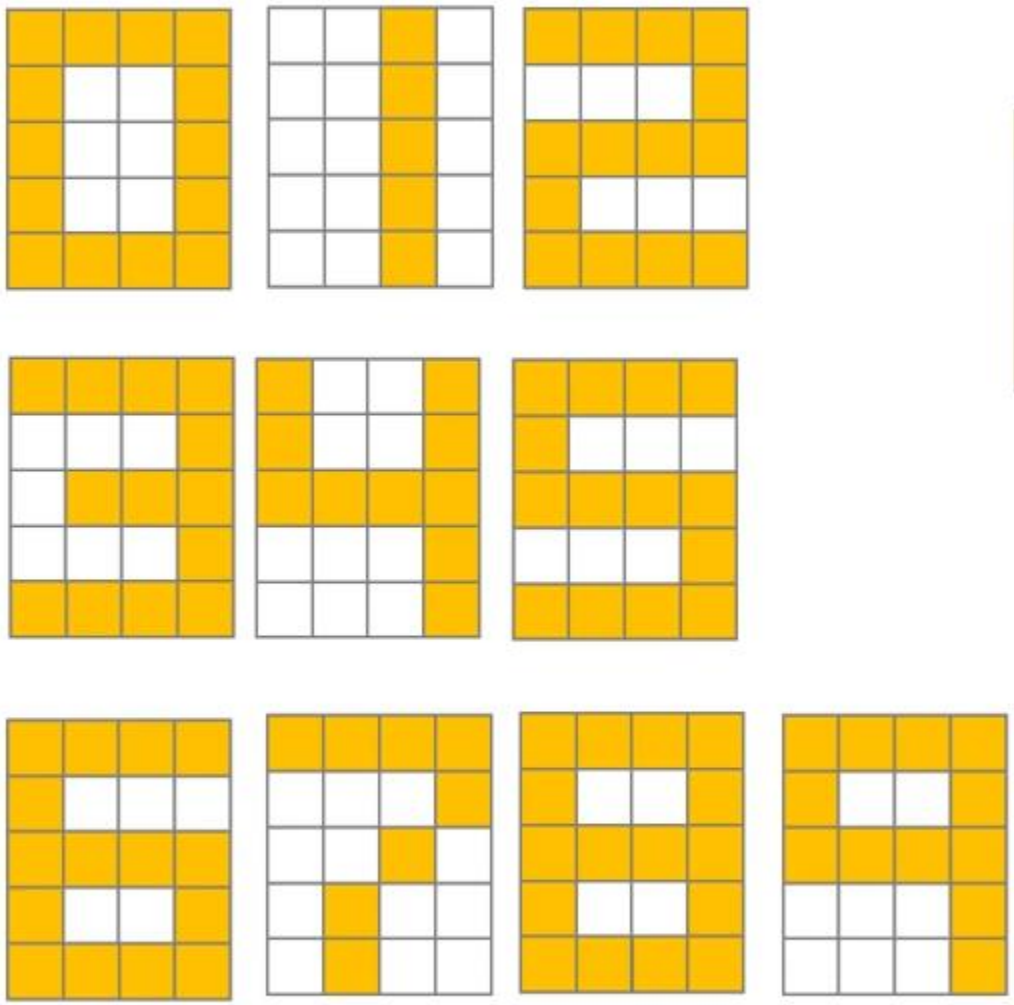


a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

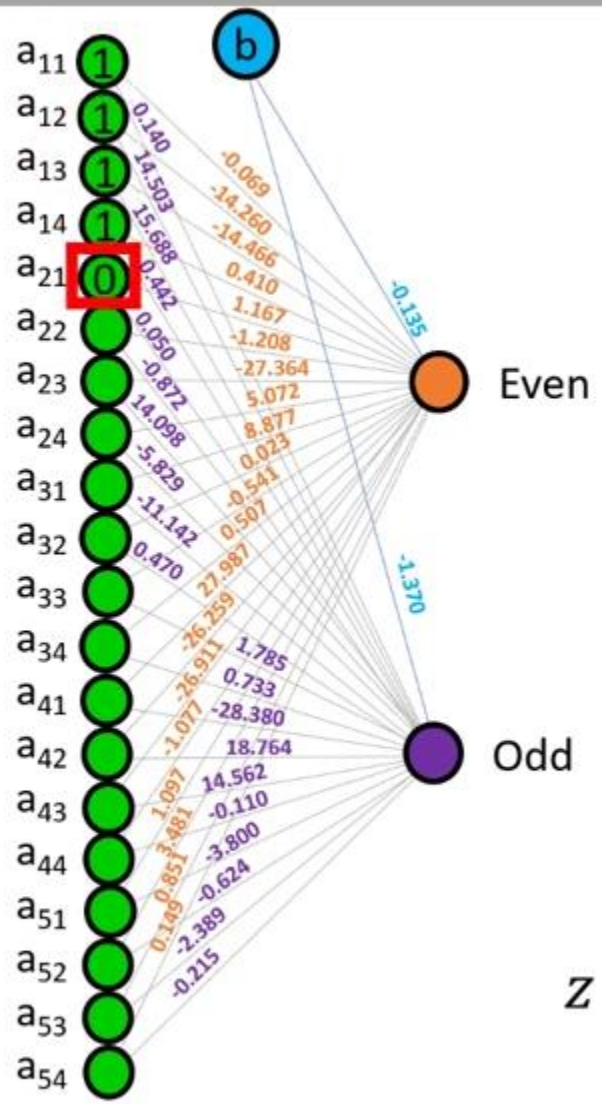
$$f(z) = \frac{1}{1 + e^{-z}}$$

$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1

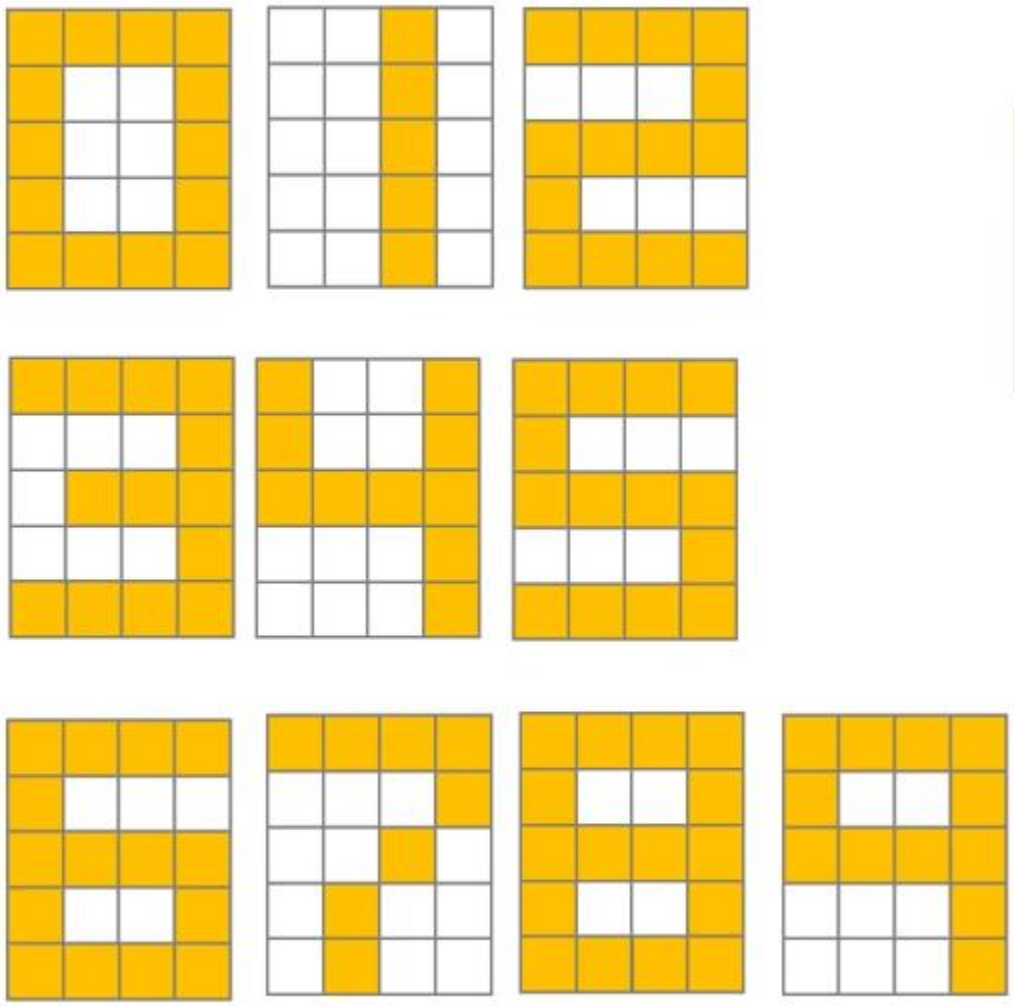


a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

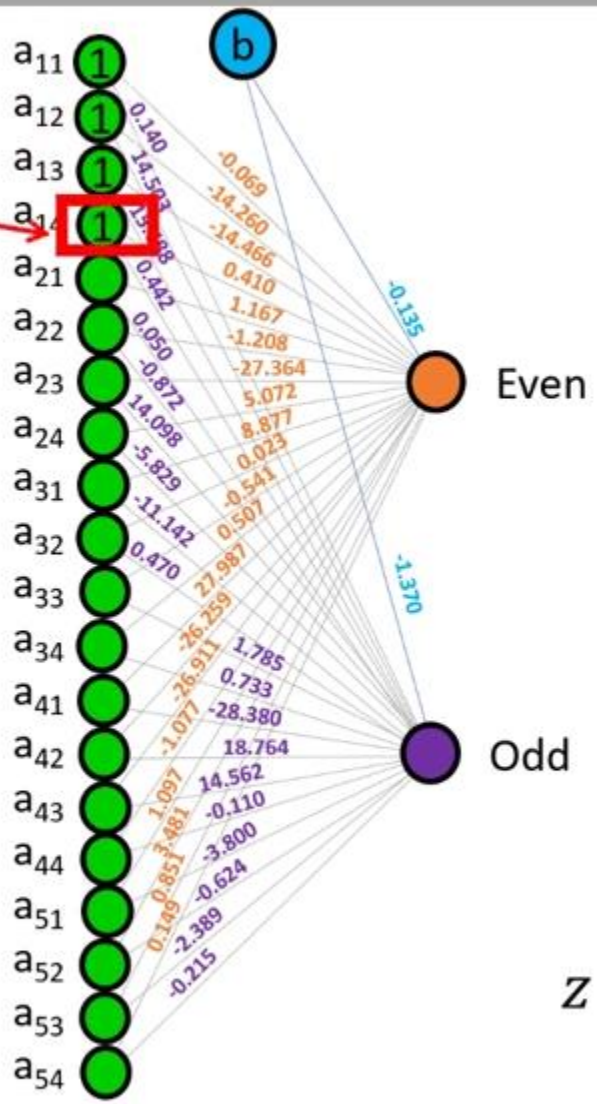
$$f(z) = \frac{1}{1 + e^{-z}}$$

$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1

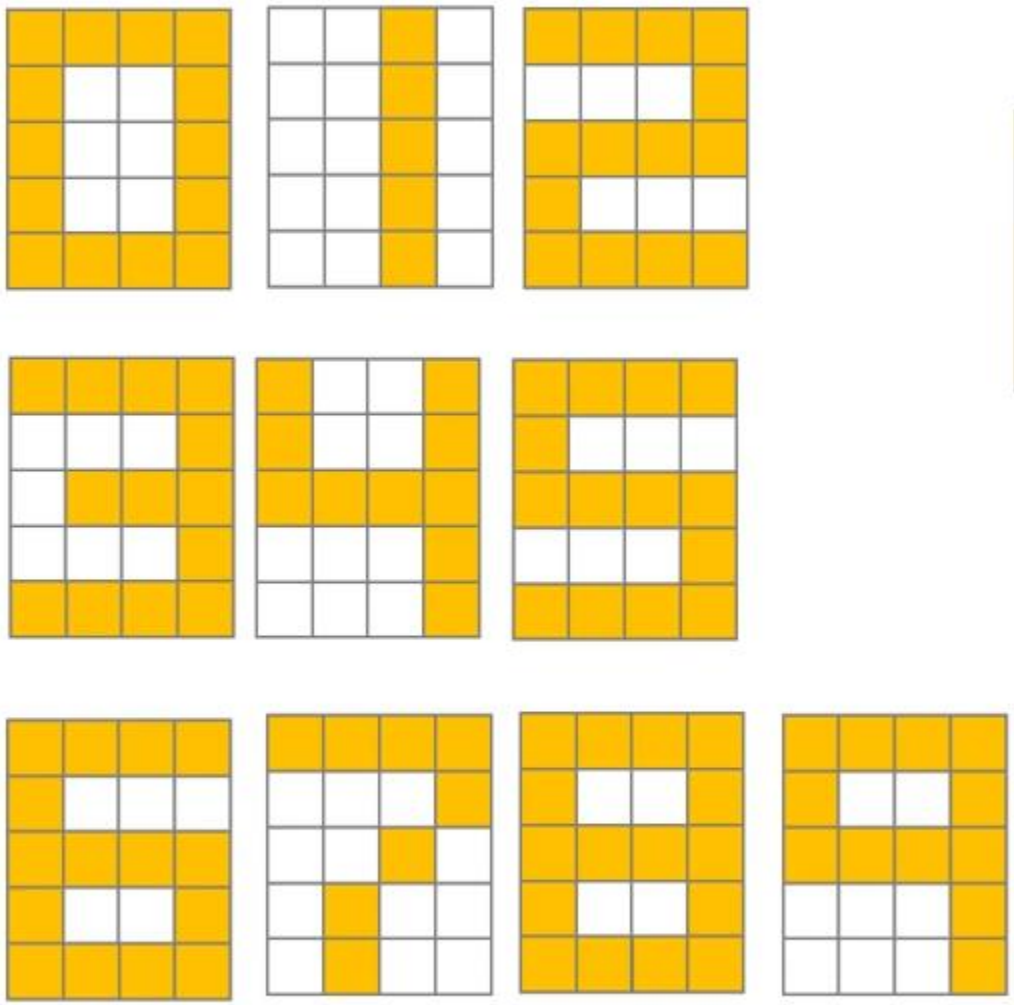


a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

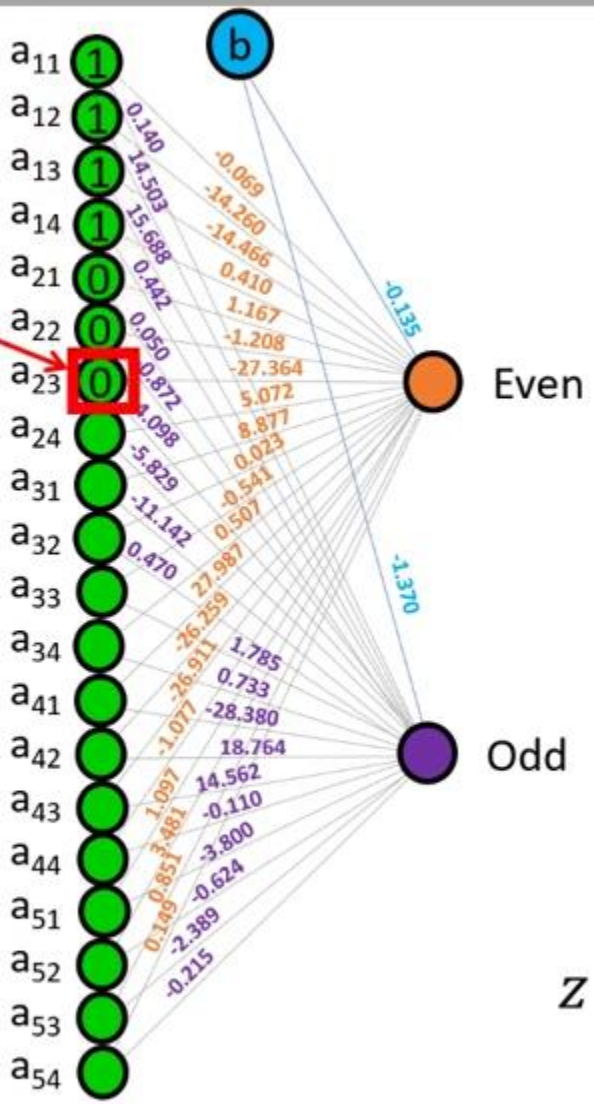
$$f(z) = \frac{1}{1 + e^{-z}}$$

$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1

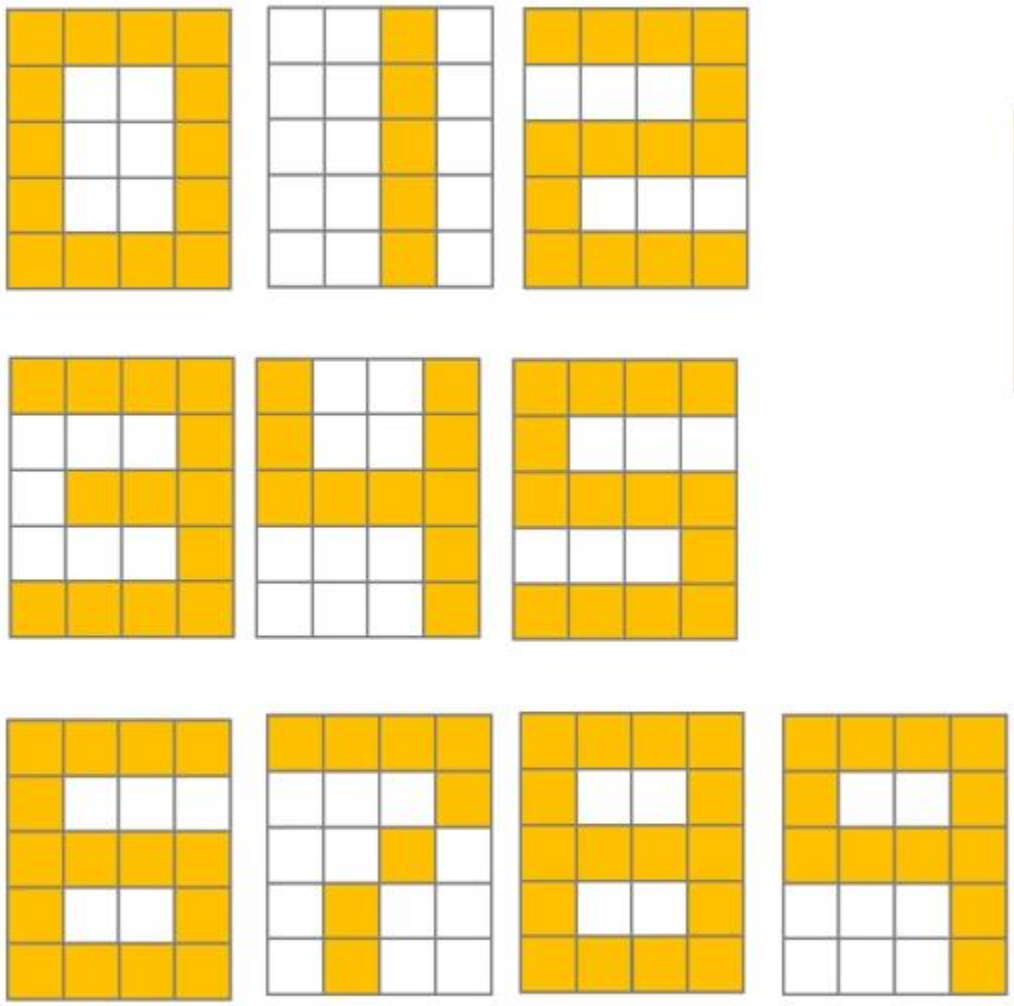


a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

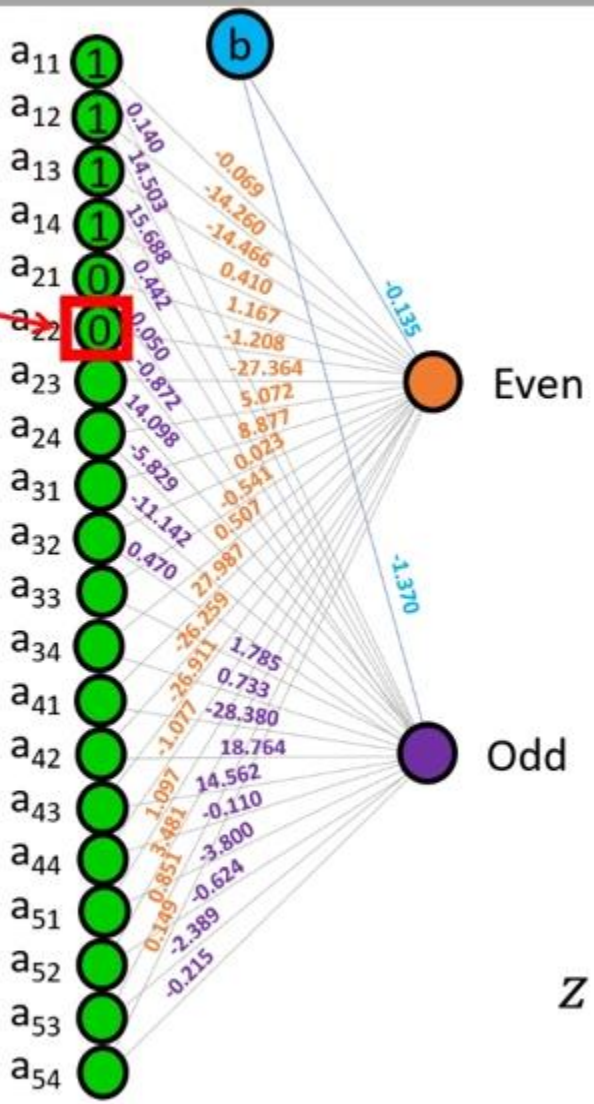
$$f(z) = \frac{1}{1 + e^{-z}}$$

$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1

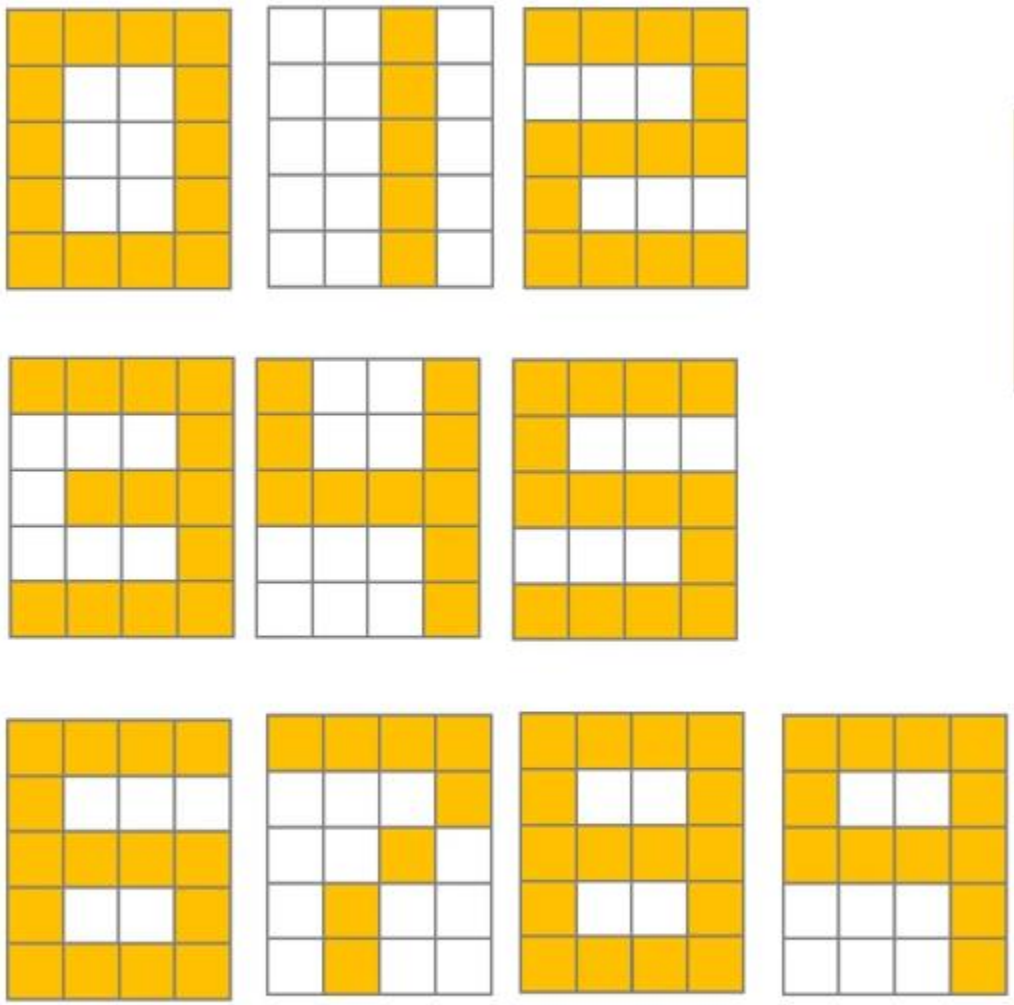


a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

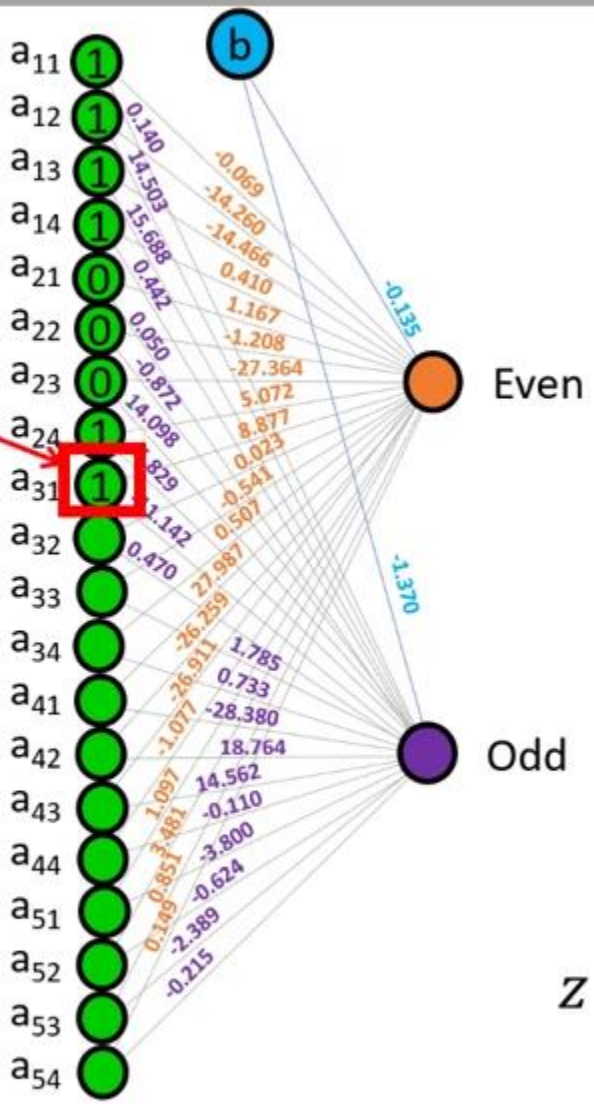
$$f(z) = \frac{1}{1 + e^{-z}}$$

$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1

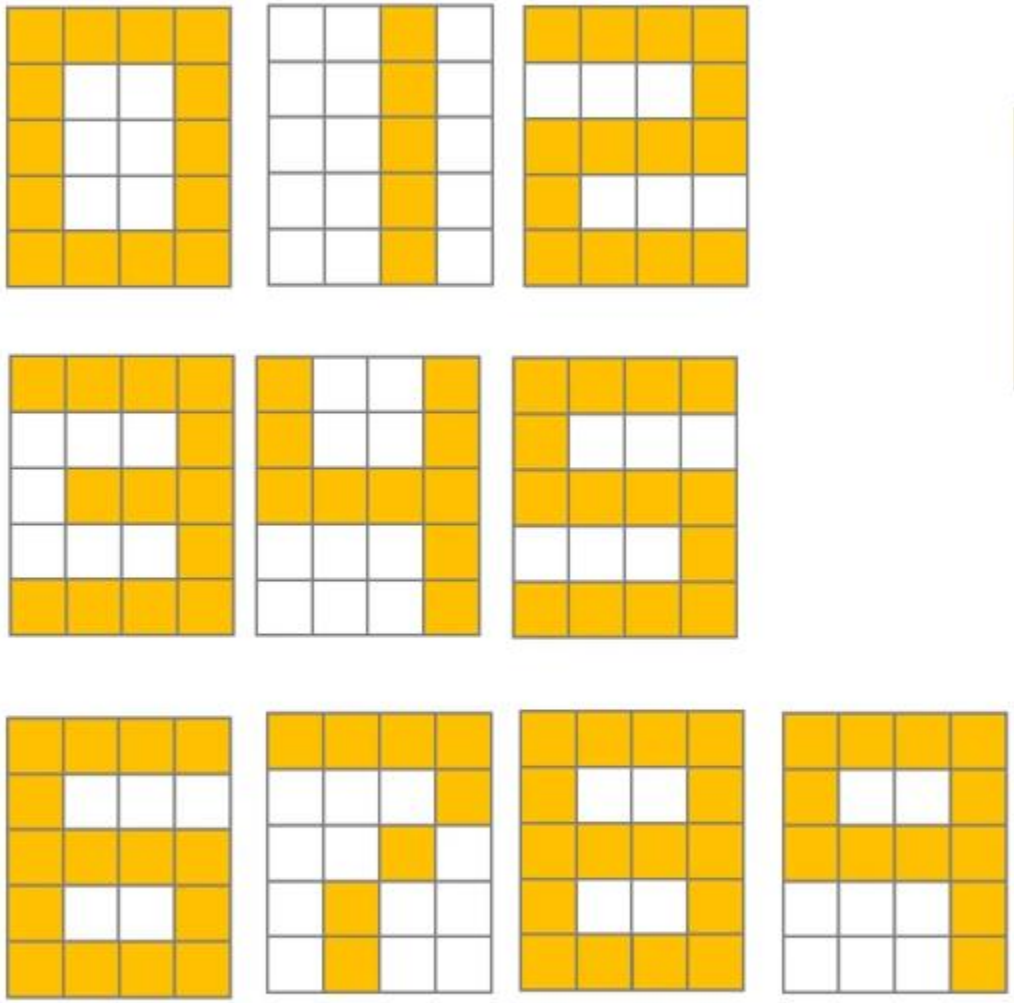


a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

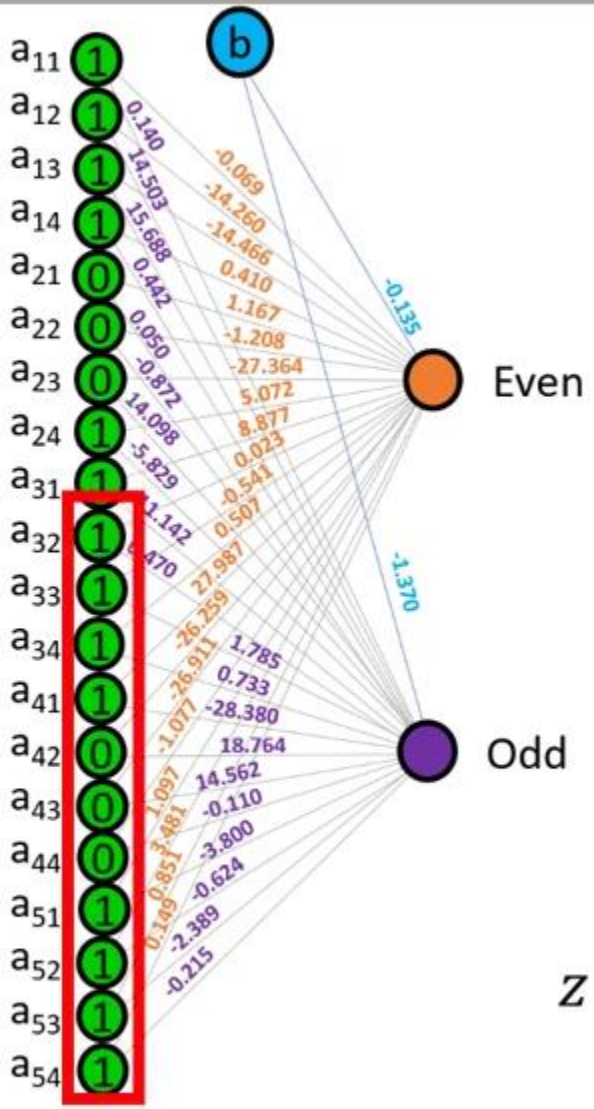
$$f(z) = \frac{1}{1 + e^{-z}}$$

$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1

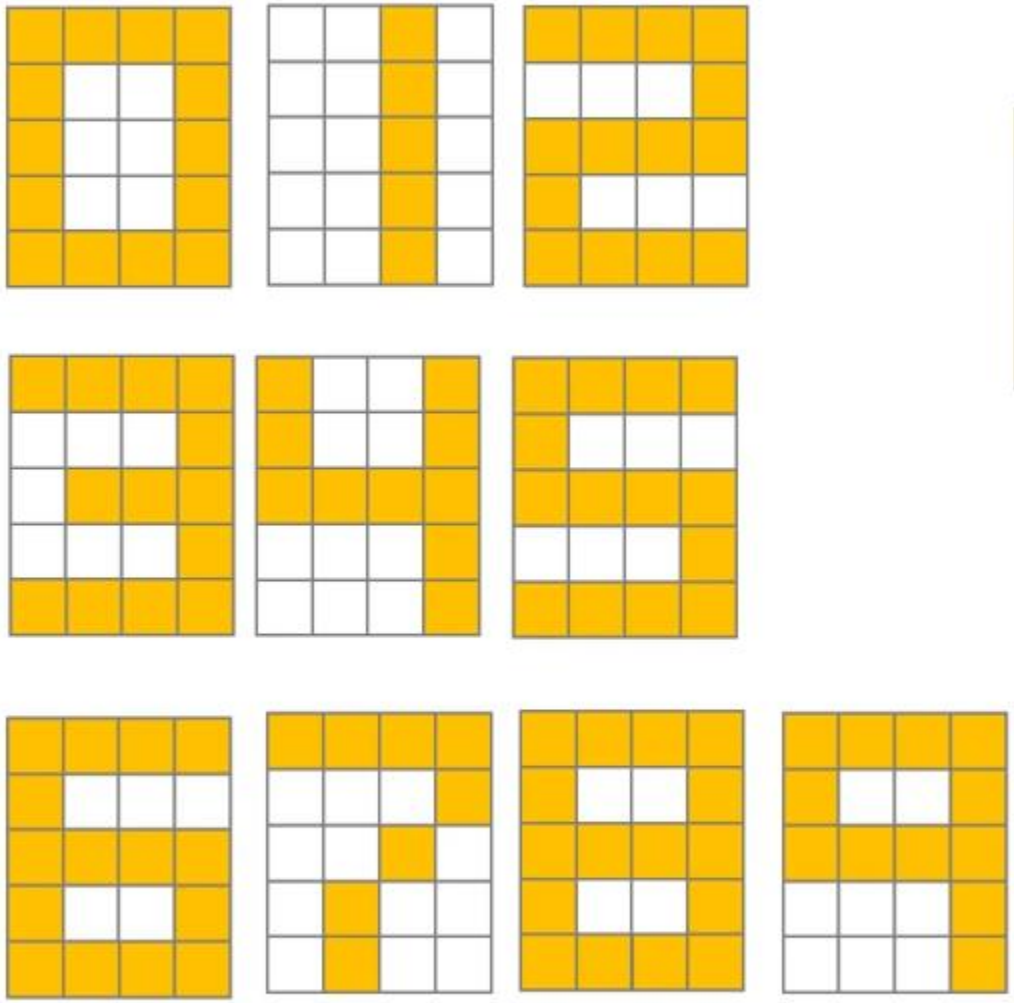


a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

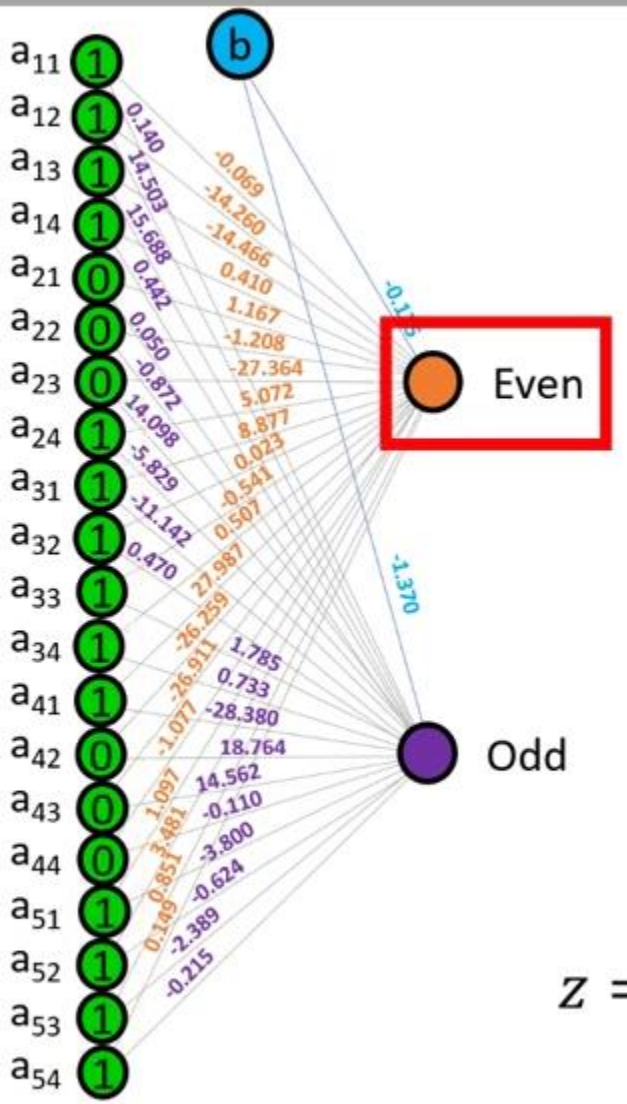
$$f(z) = \frac{1}{1 + e^{-z}}$$

$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1



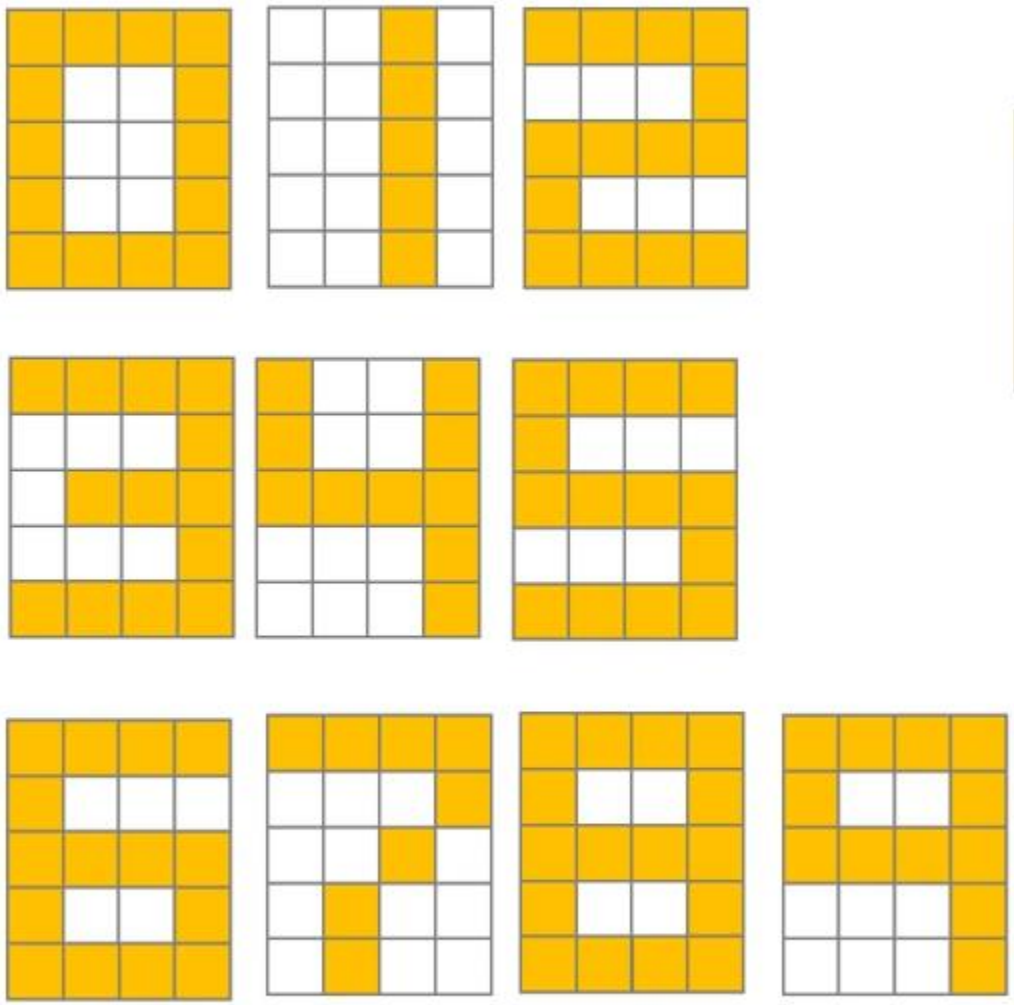
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

$$f(z) = \frac{1}{1 + e^{-z}}$$

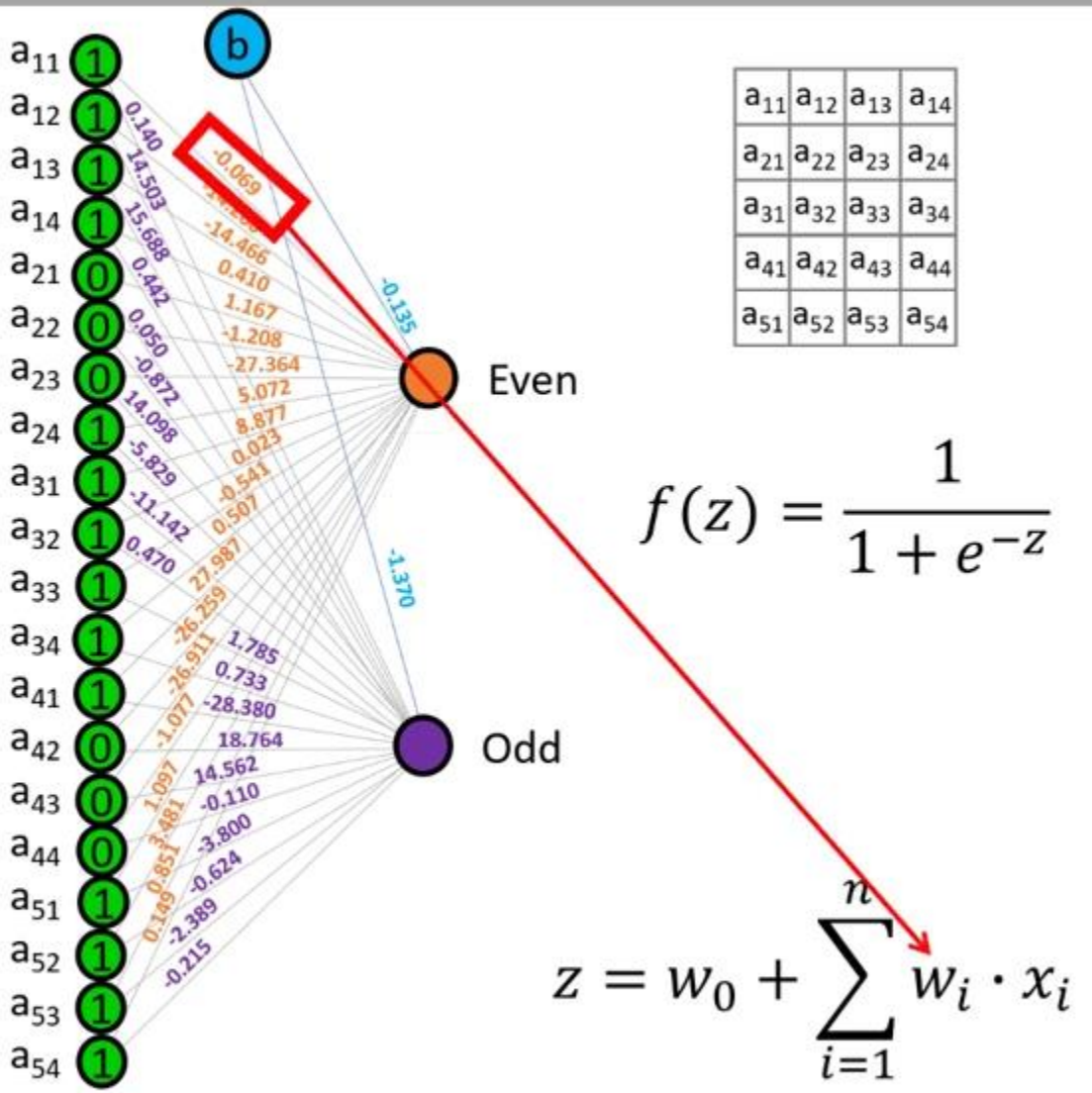
$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

To compute the probability that this image contains an even number,

ANN



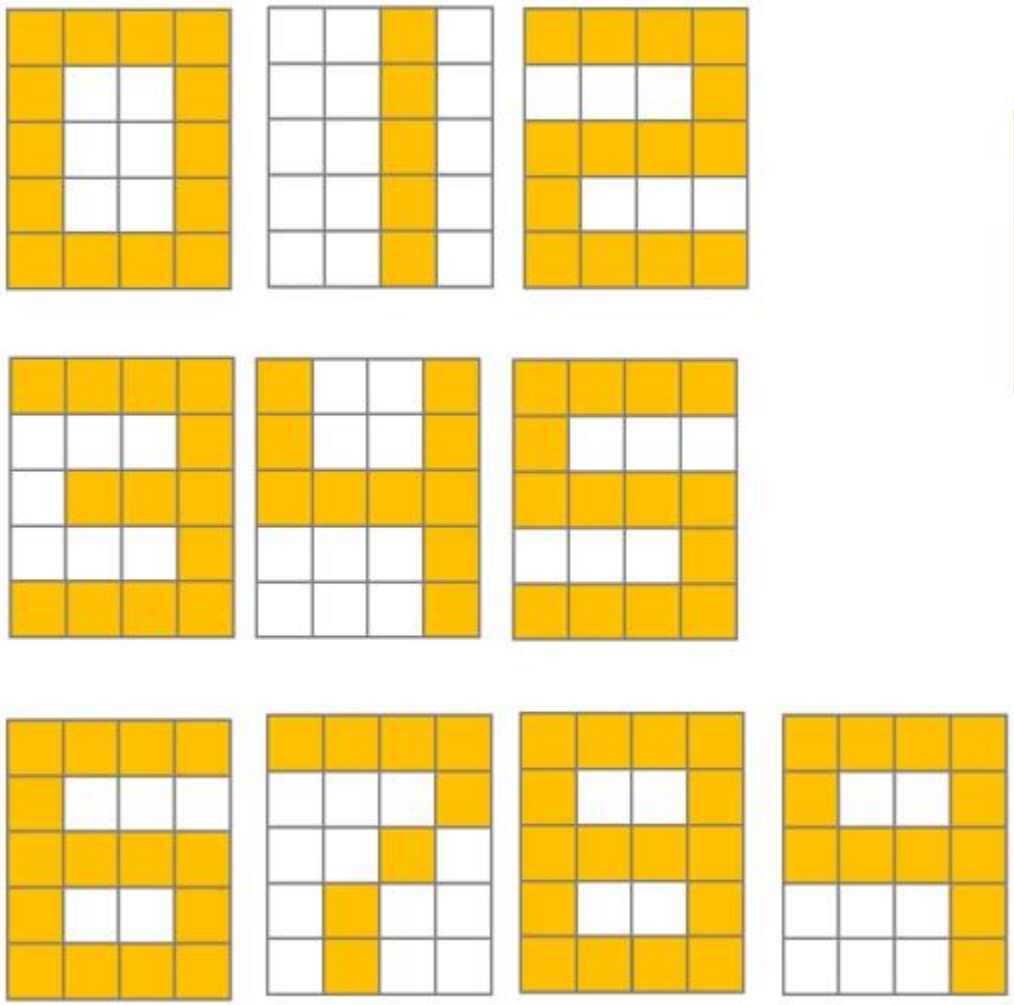
1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1



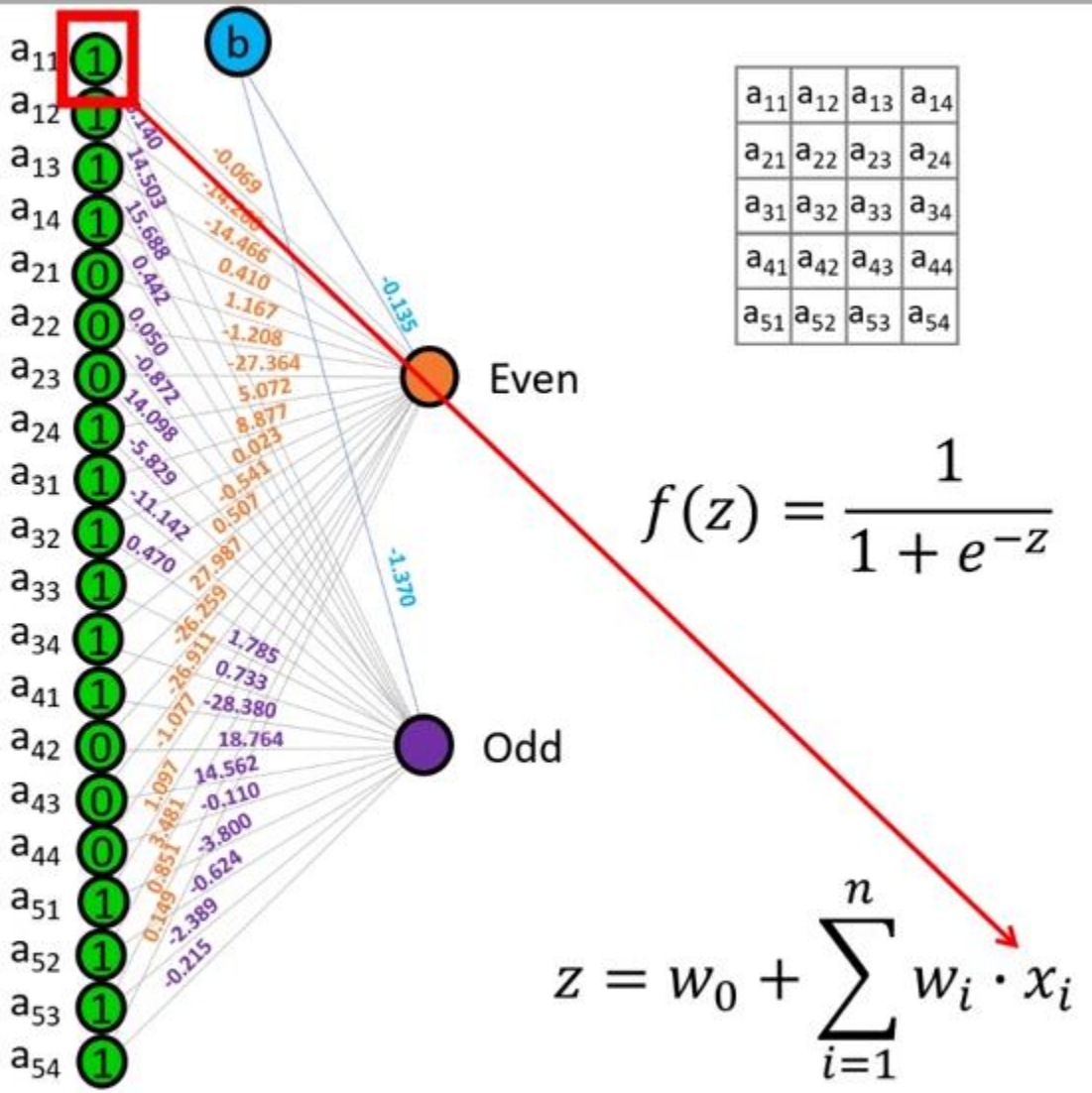
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

we multiply the first weight,

ANN



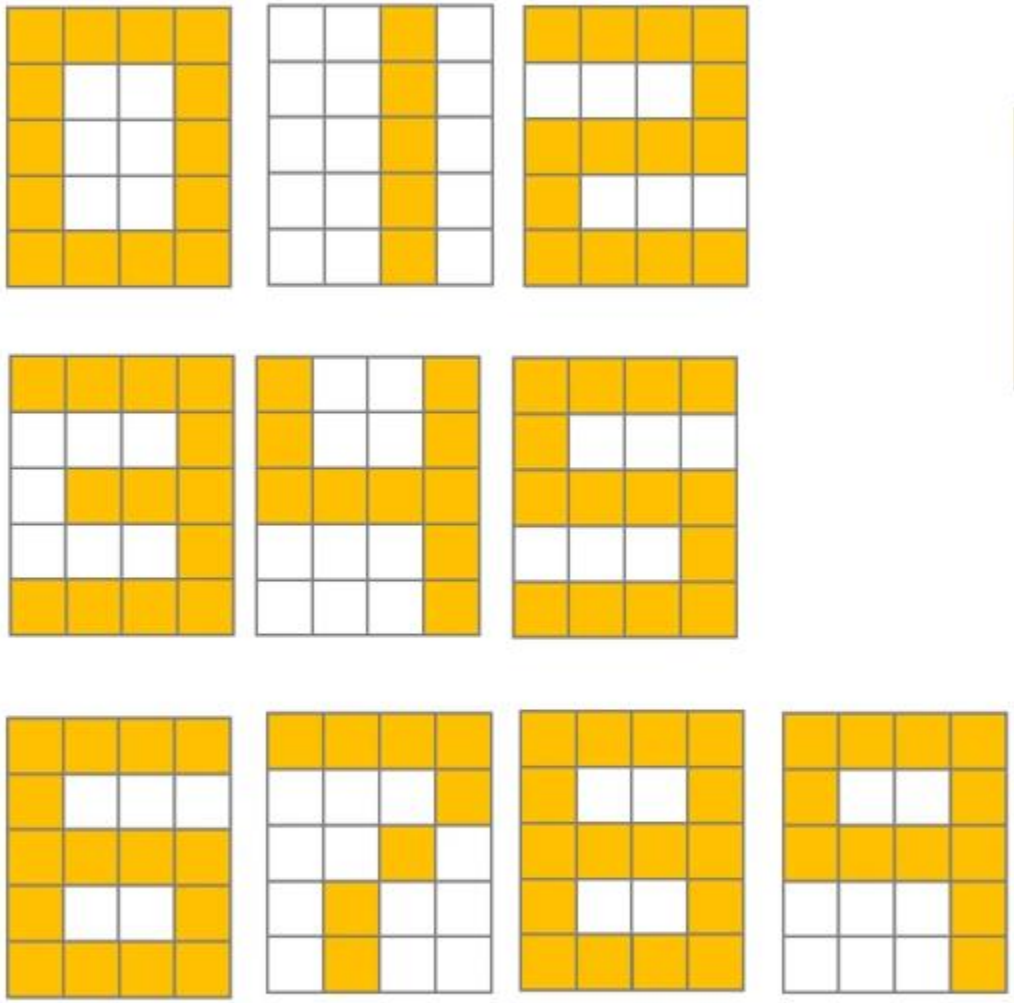
1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1



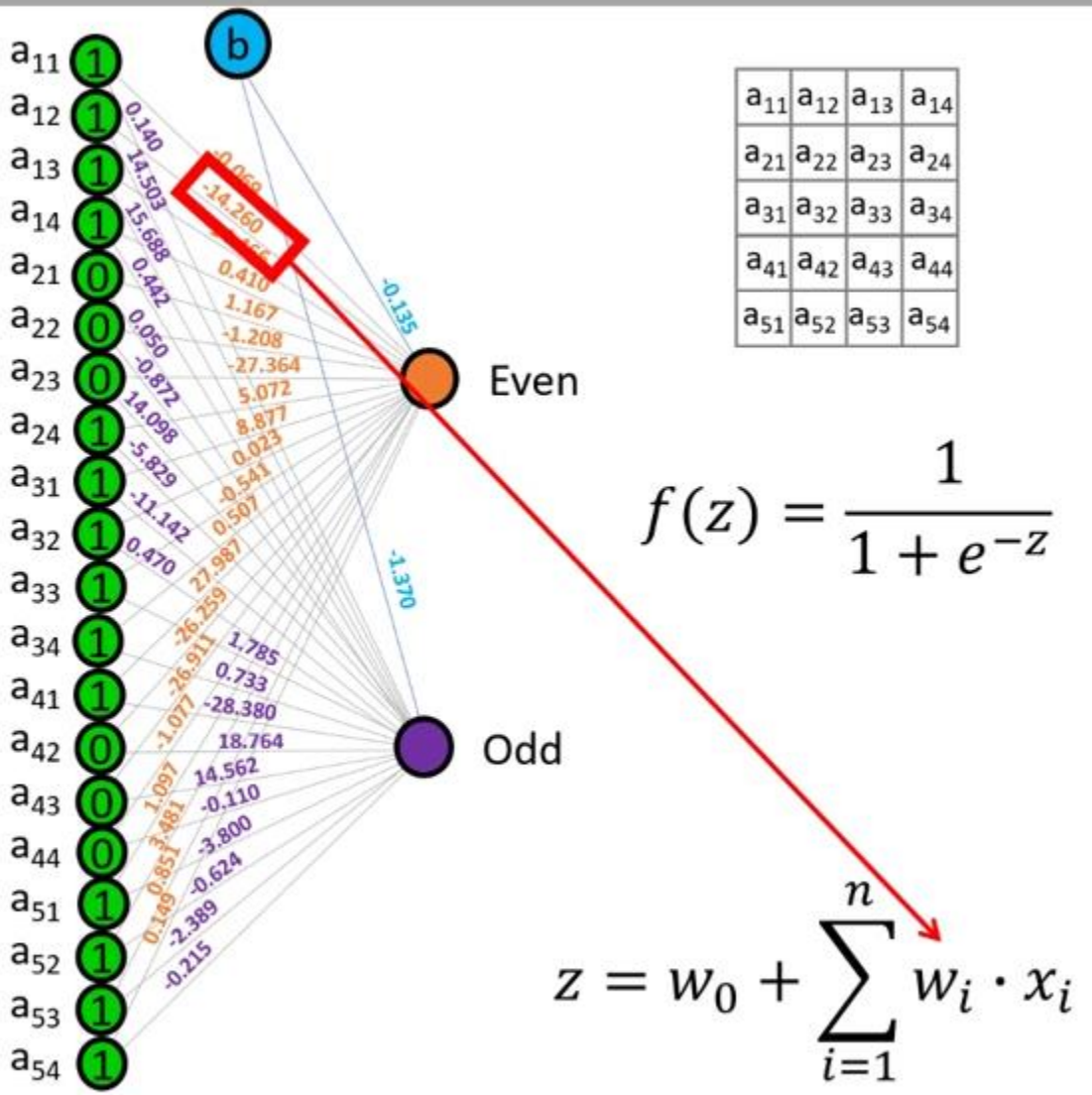
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

by the value of the first input node,

ANN



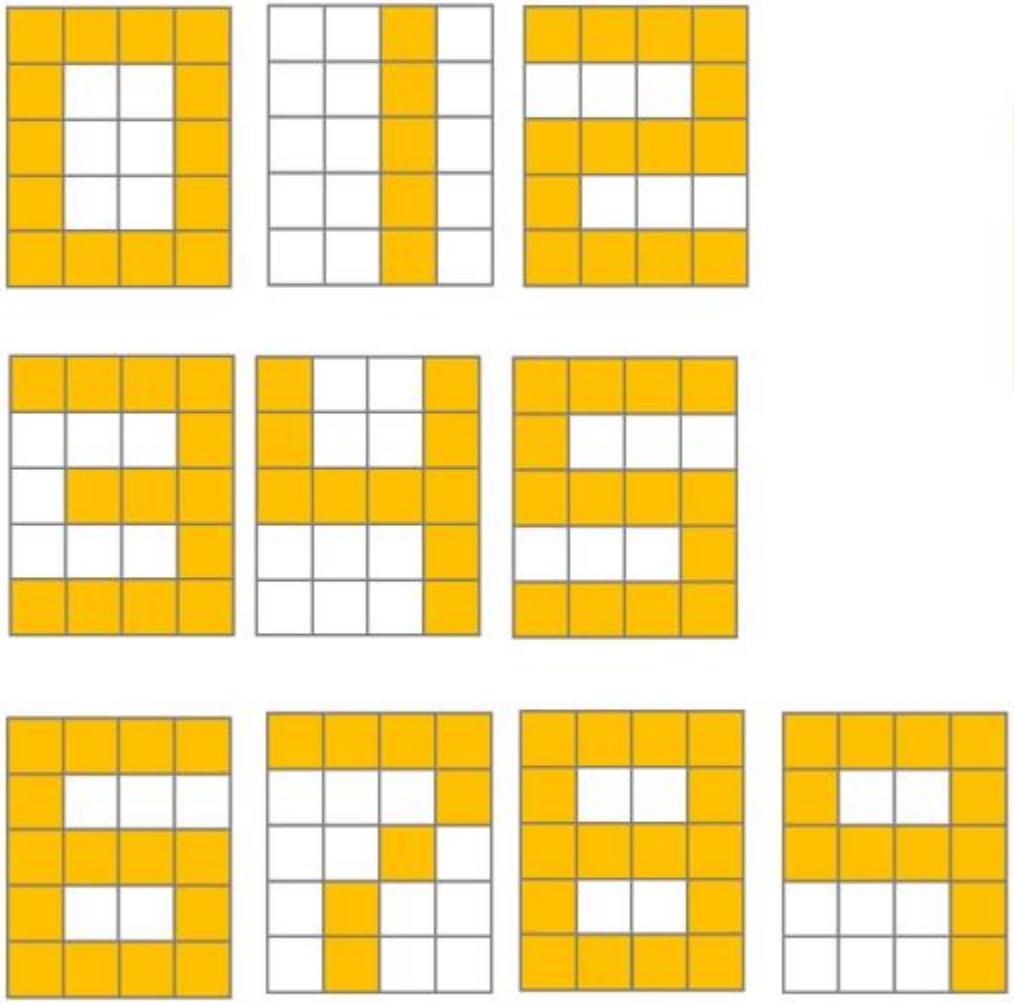
1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1



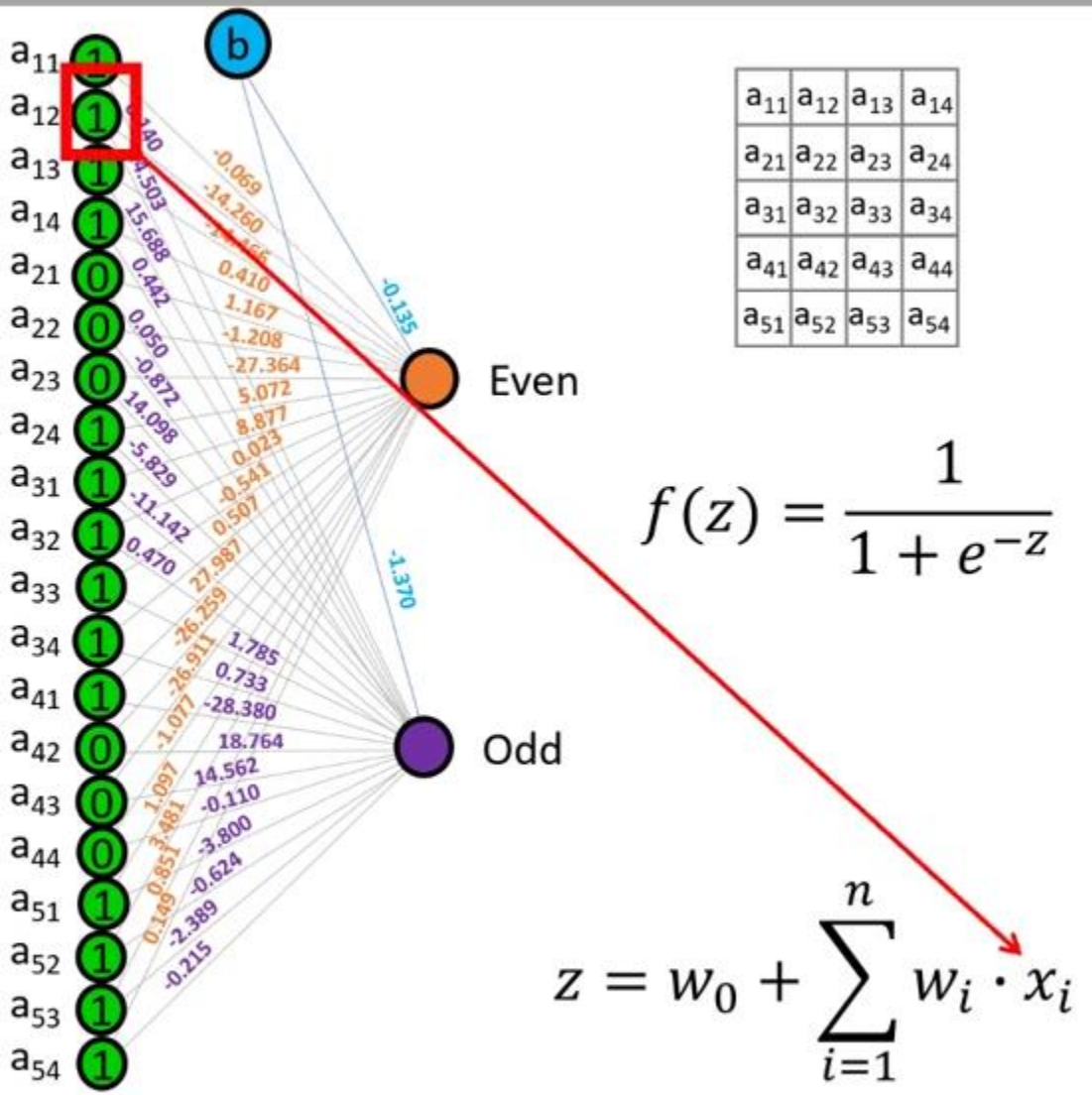
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

and then we multiply the second weight,

ANN



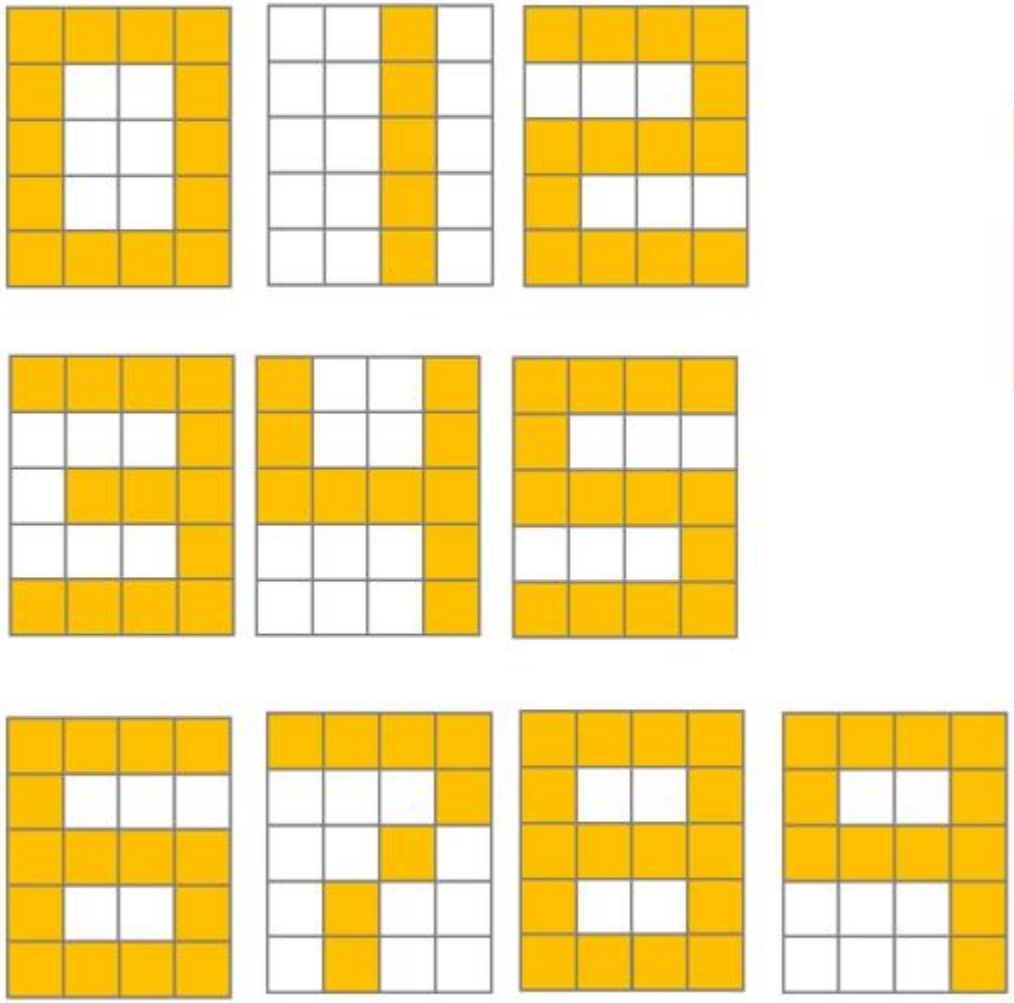
1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1



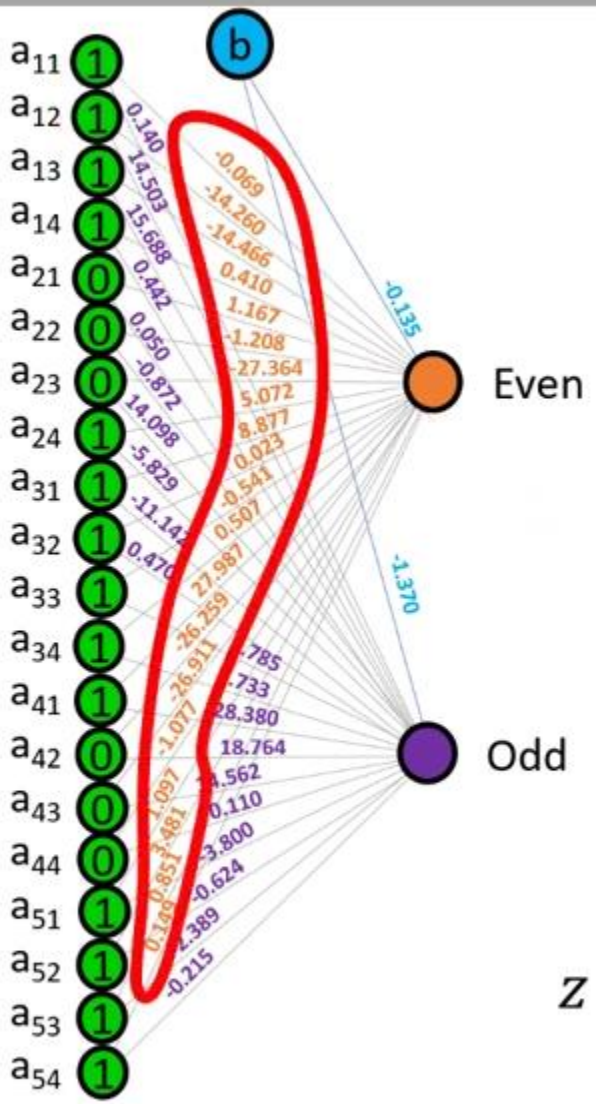
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

by the second input node, and so forth,

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1



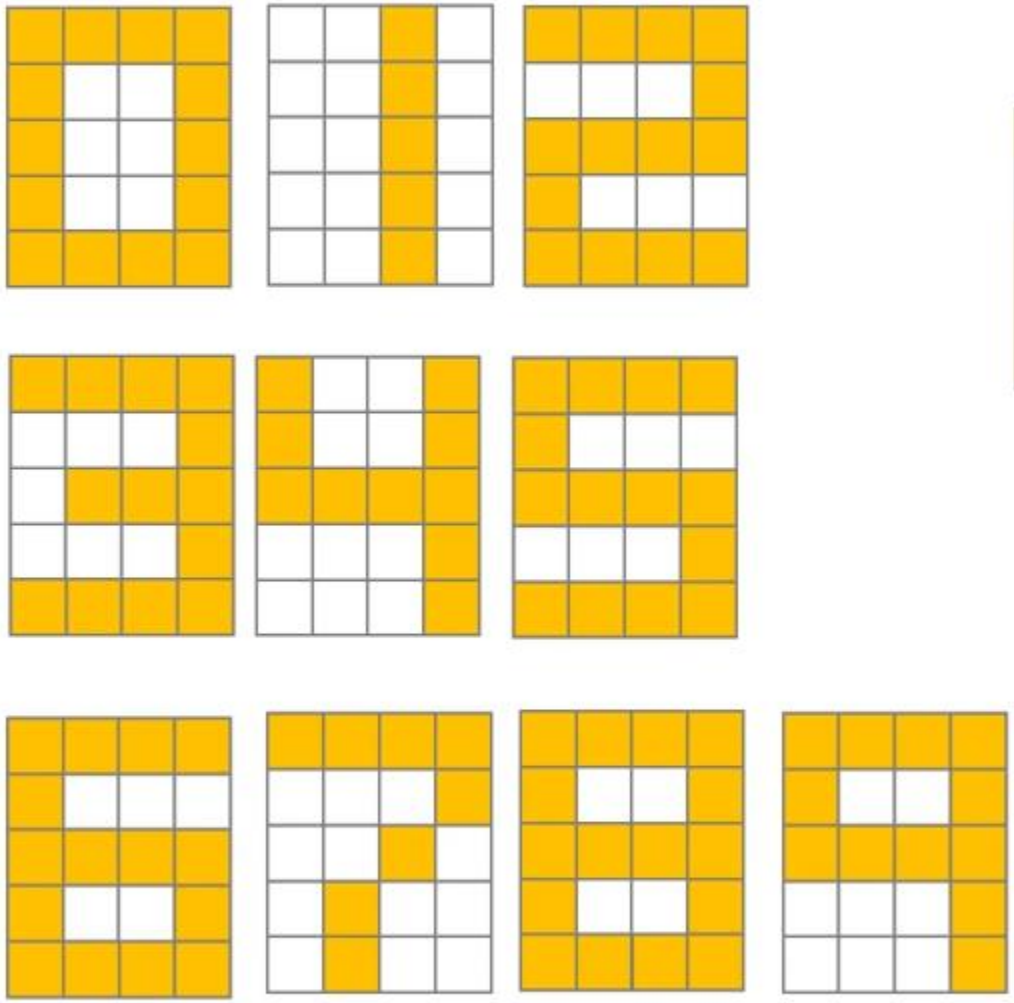
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

$$f(z) = \frac{1}{1 + e^{-z}}$$

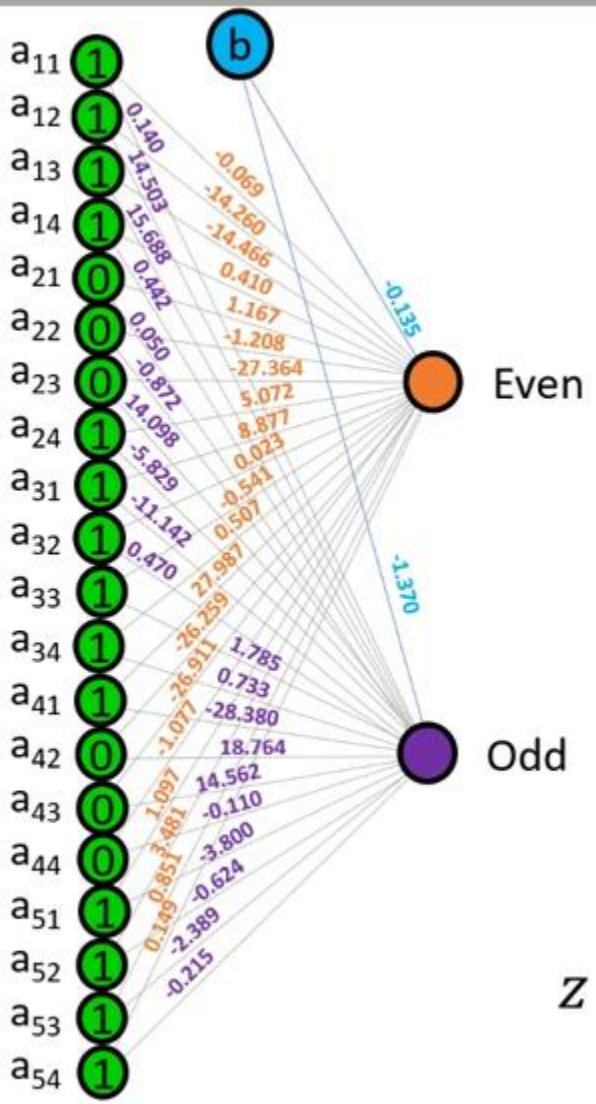
$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

until we have done this for all the weights.

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1



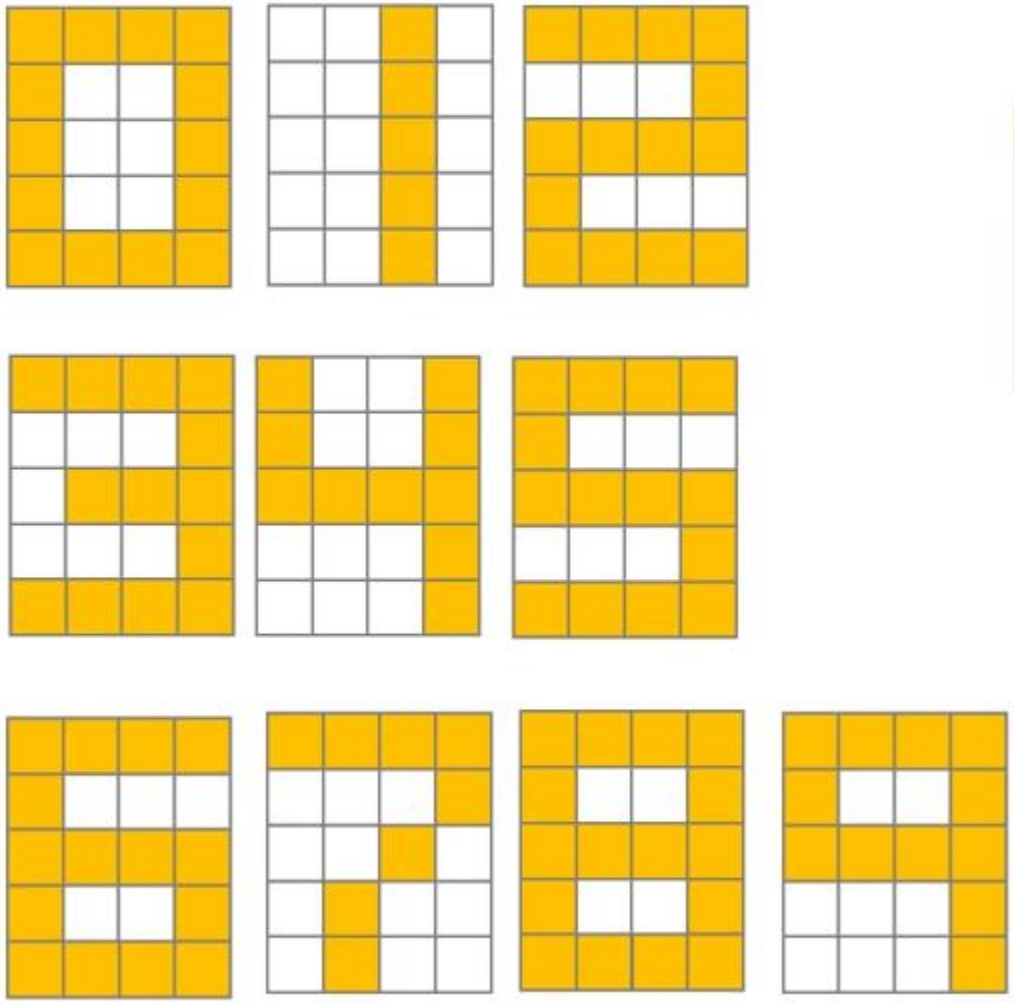
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

$$f(z) = \frac{1}{1 + e^{-z}}$$

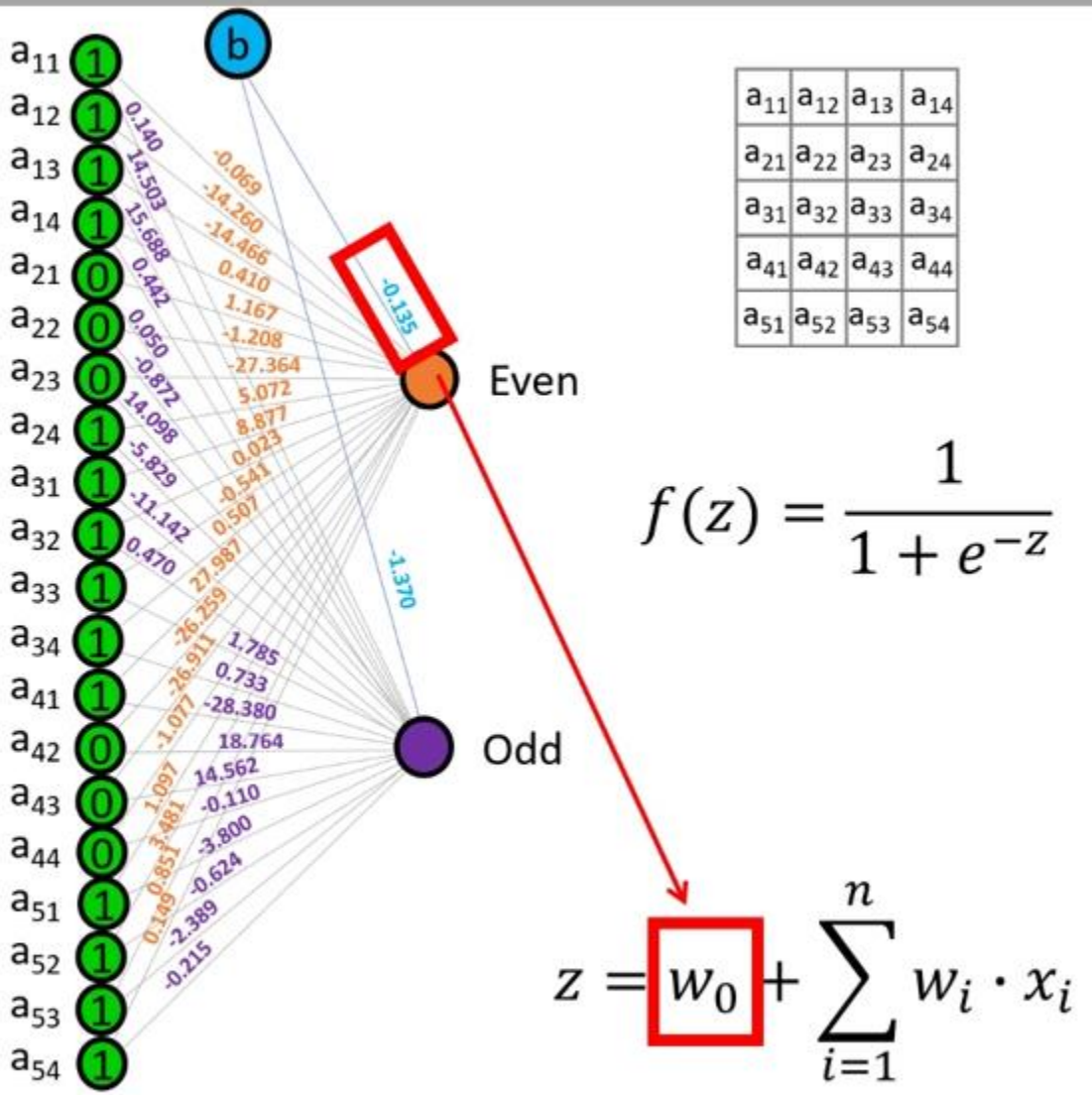
$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

Then we sum all these 20 products,

ANN



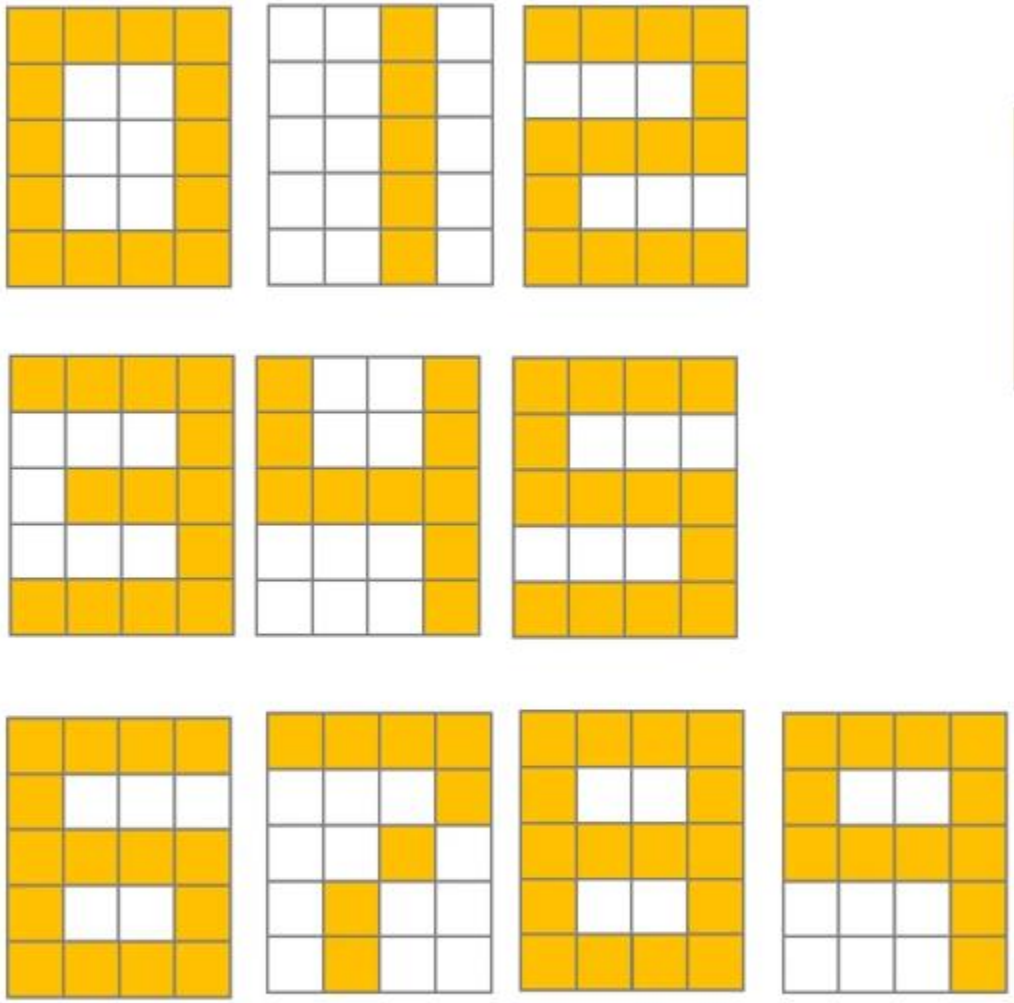
1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1



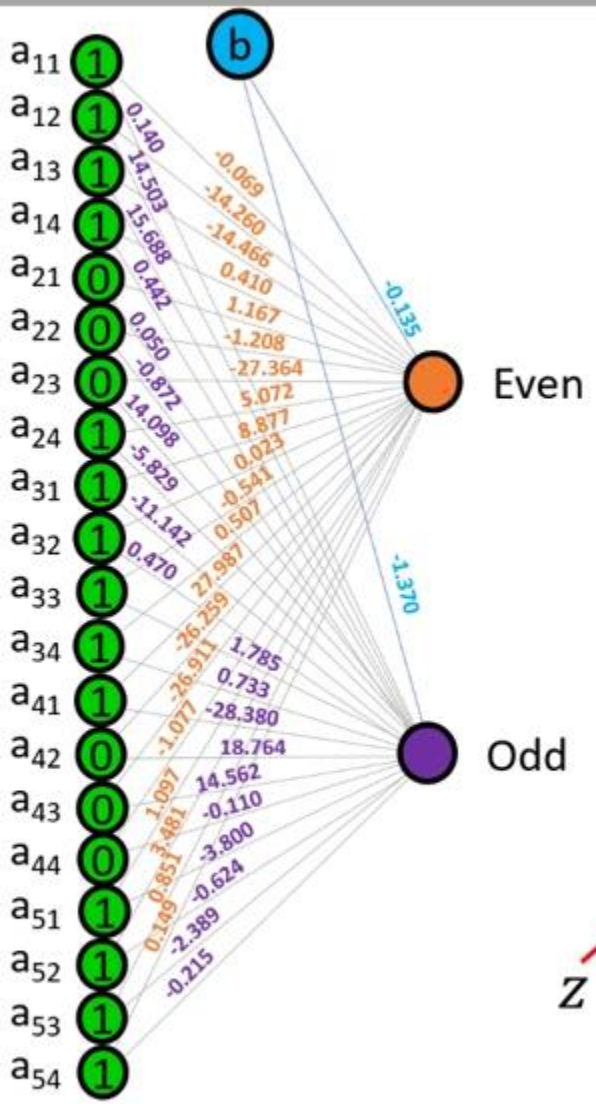
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

and add the bias weight.

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1



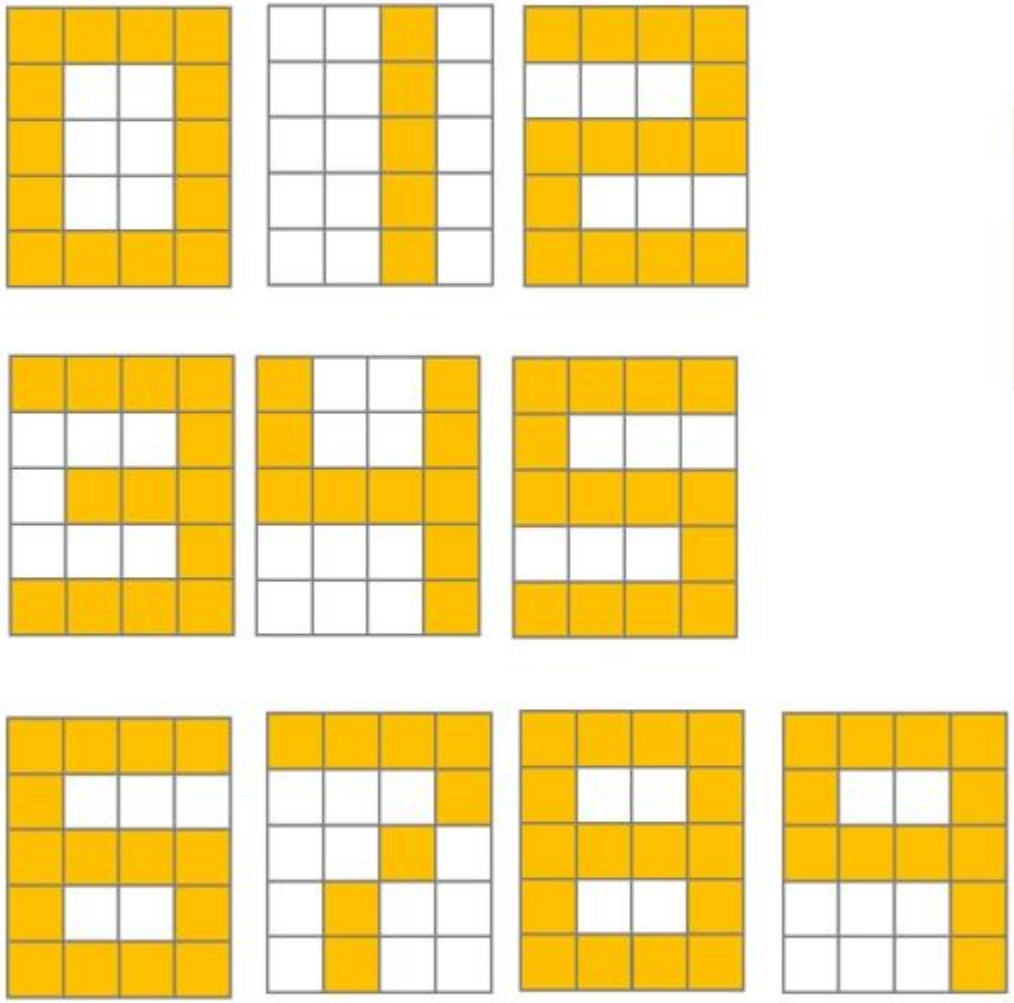
a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}
a_{51}	a_{52}	a_{53}	a_{54}

$$f(z) = \frac{1}{1 + e^{-z}}$$

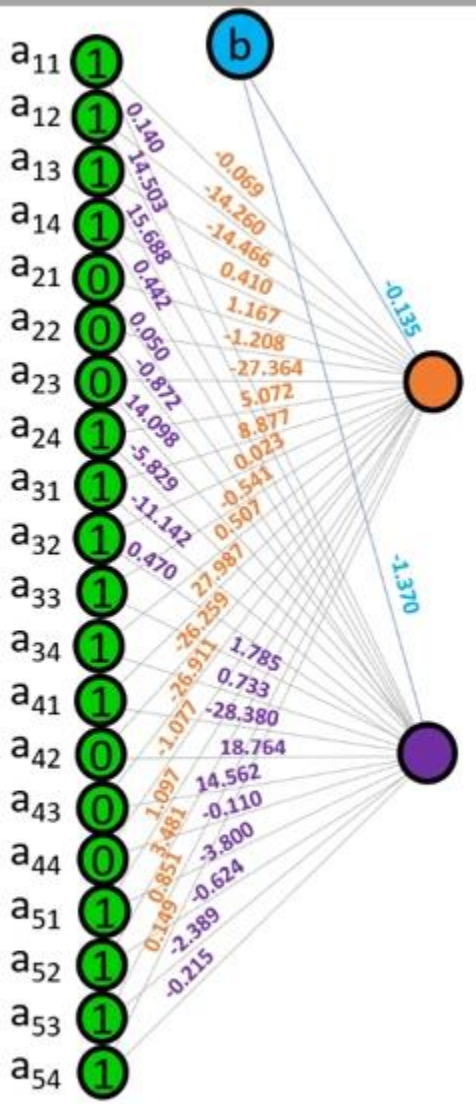
$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

Next, we plug in the computed value in the activation function and do the math,

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1

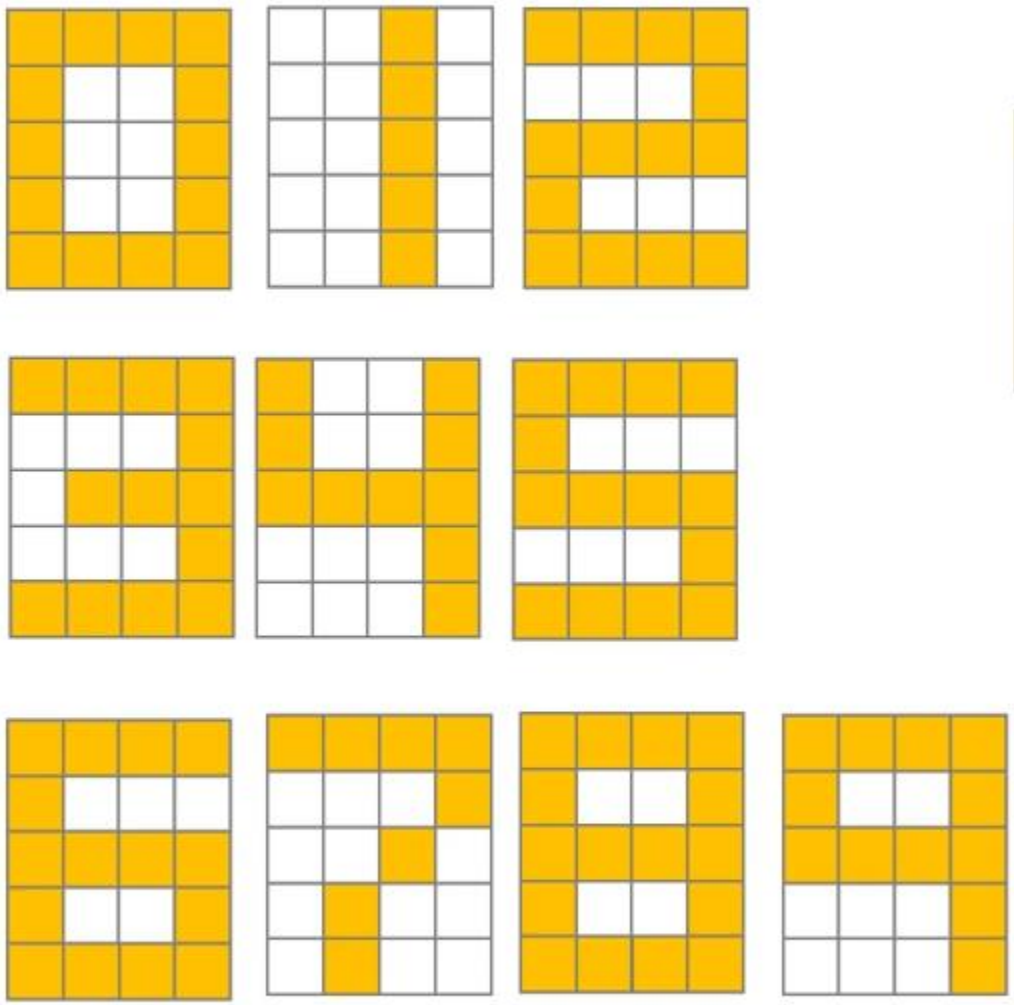


$$f(z) = \frac{1}{1 + e^{-z}}$$

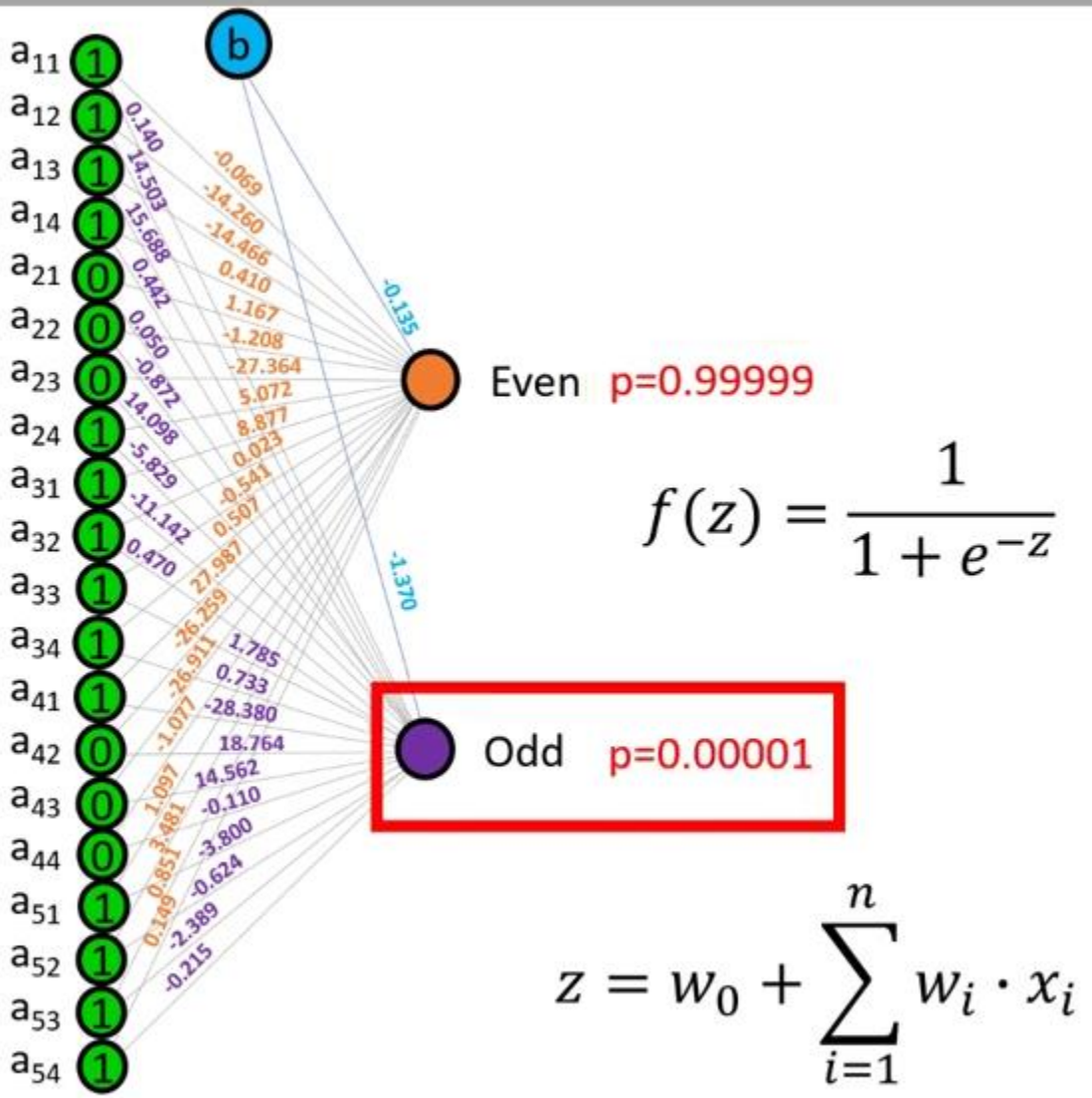
$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

which in this case results in a value that is approximately equal to one.

ANN



1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1

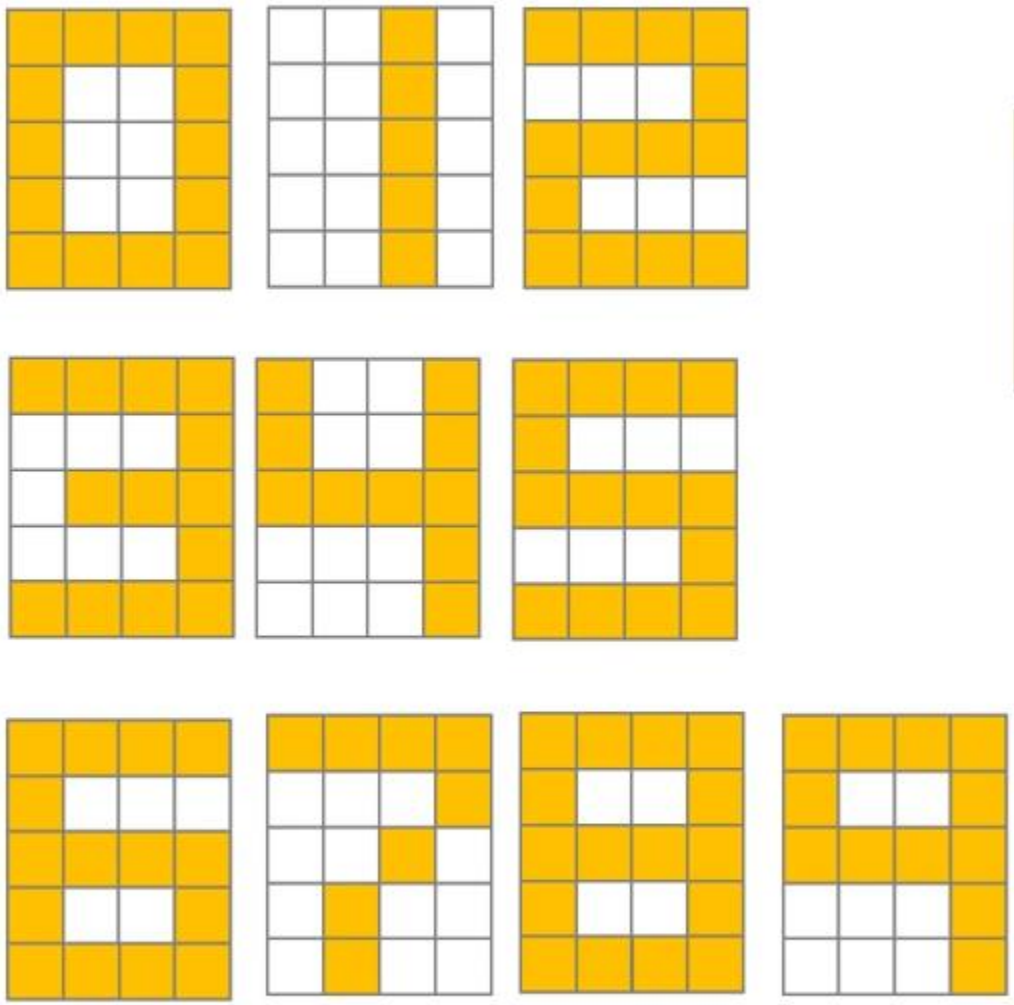


$$f(z) = \frac{1}{1 + e^{-z}}$$

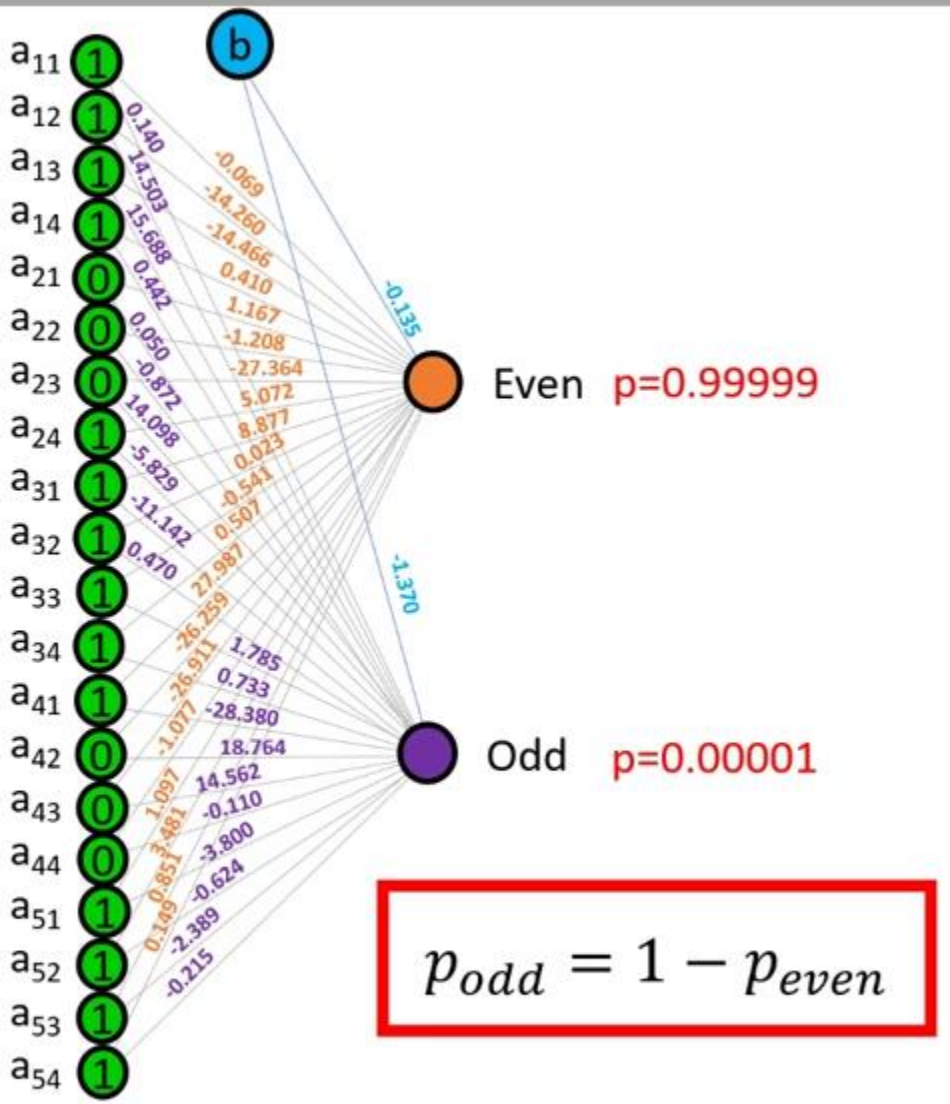
$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

If we would compute the same thing for the odd output, the probability would be equal to approximately zero.

ANN



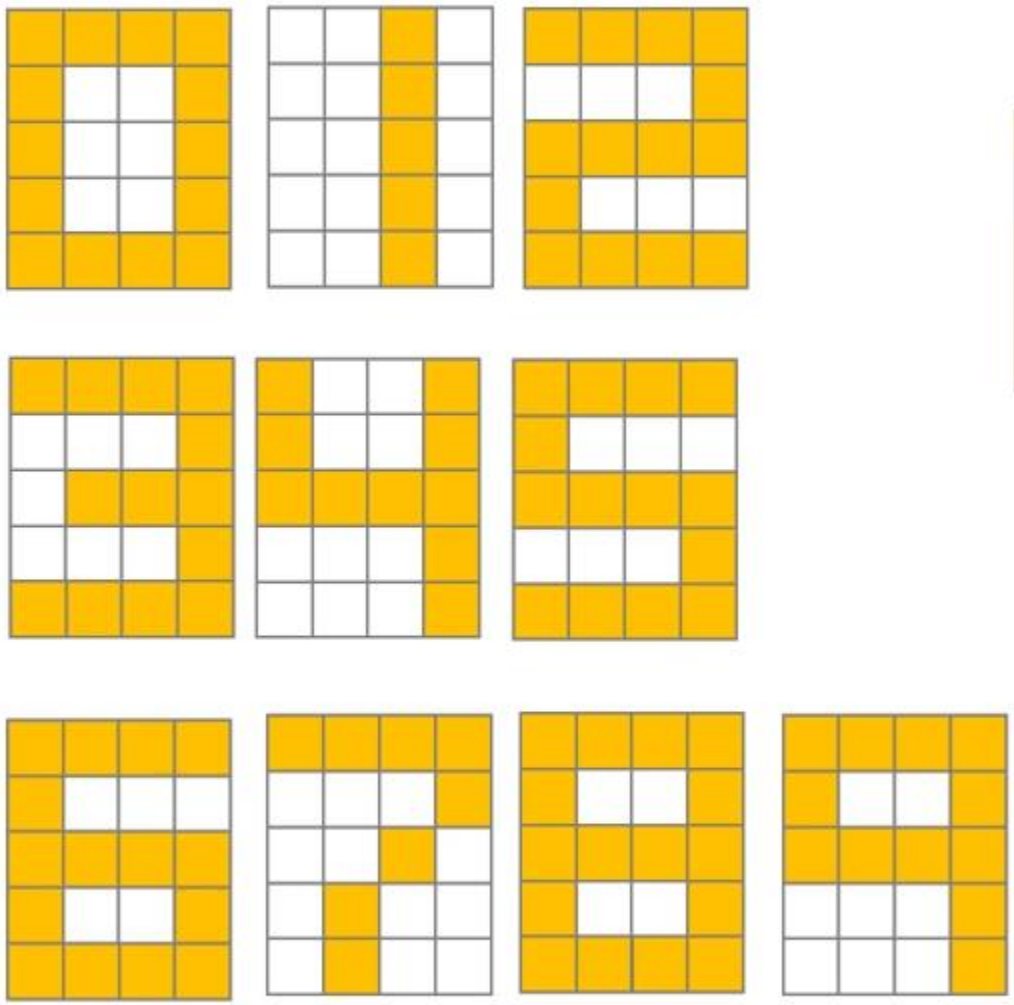
1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1



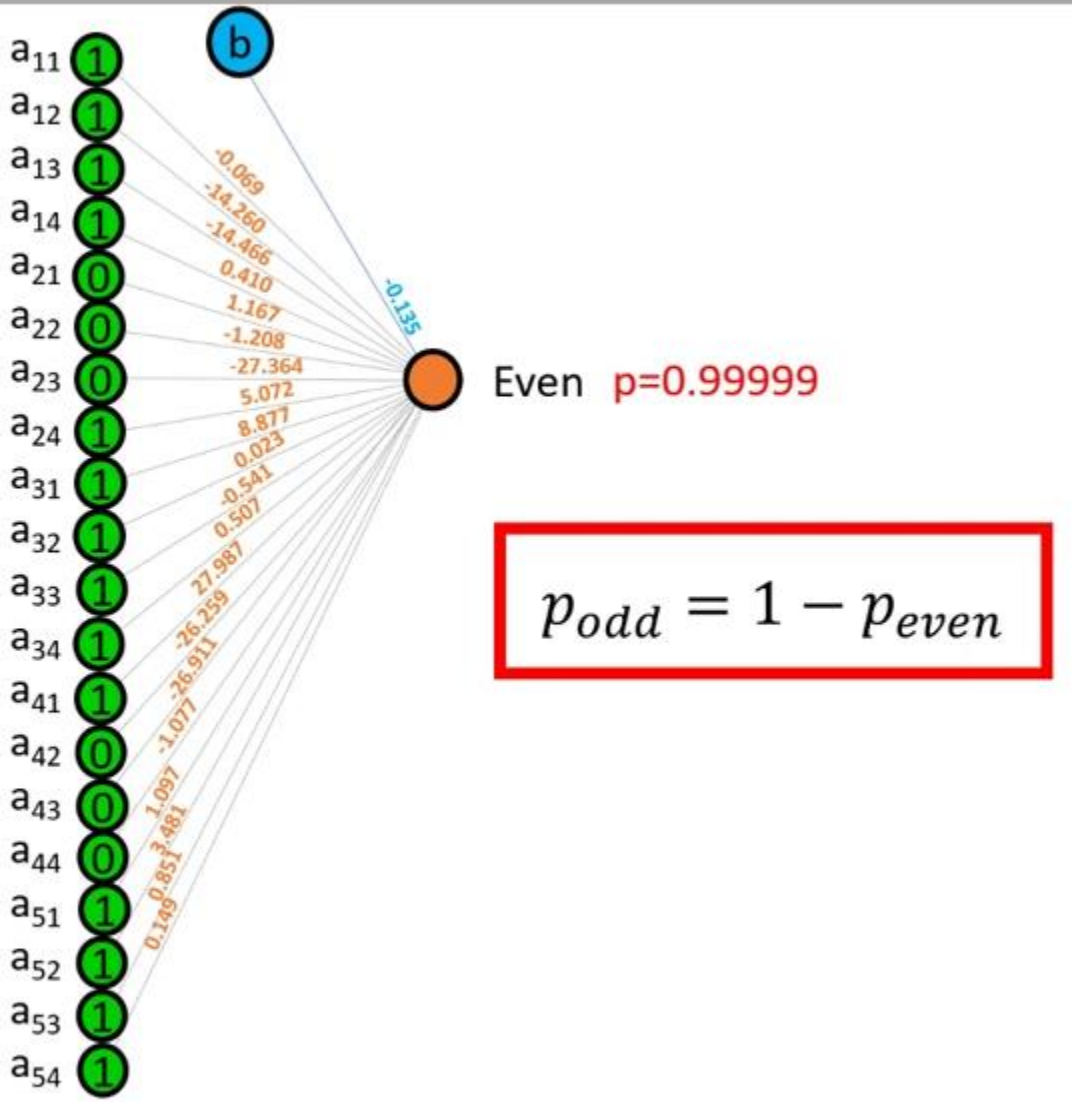
$$p_{odd} = 1 - p_{even}$$

When we have just two outcomes and use the sigmoid activation function, the probability of an odd number is simply equal to one minus the probability of an even number.

ANN



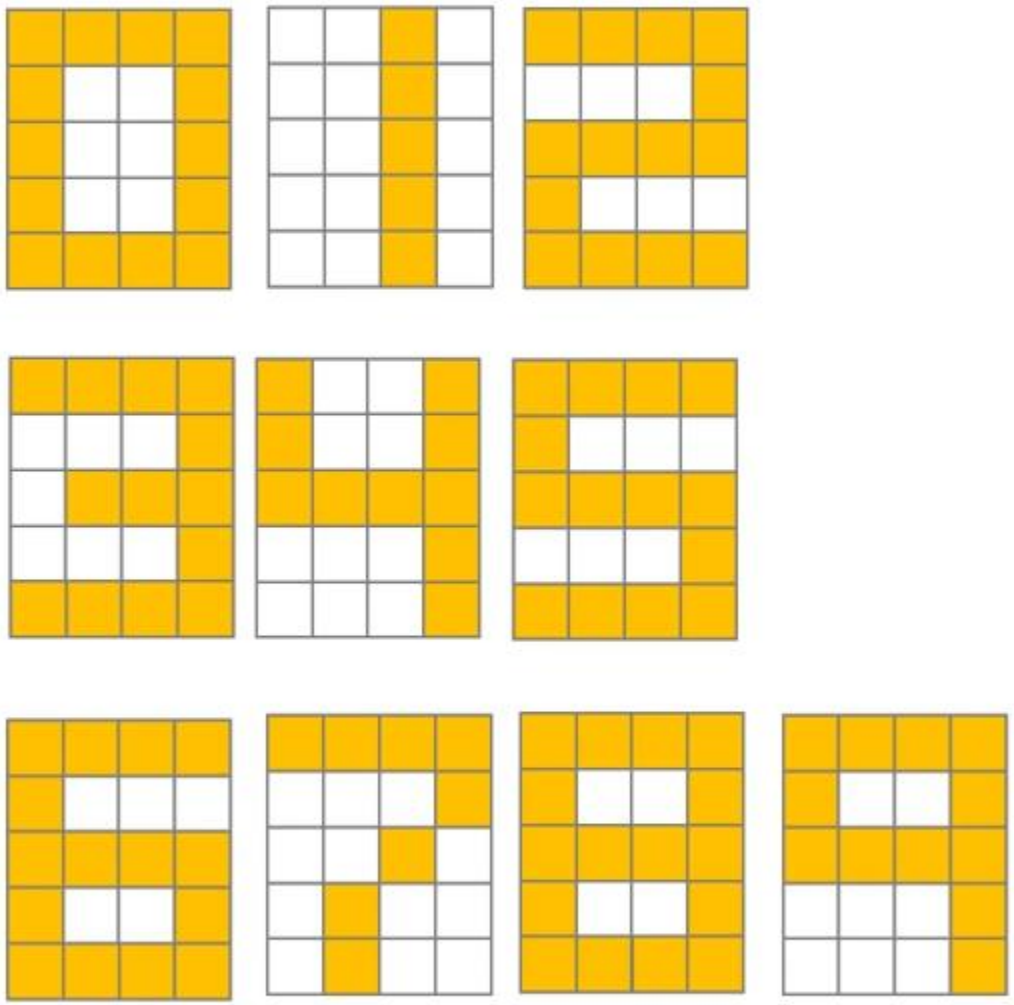
1	1	1	1
0	0	0	1
1	1	1	1
1	0	0	0
1	1	1	1

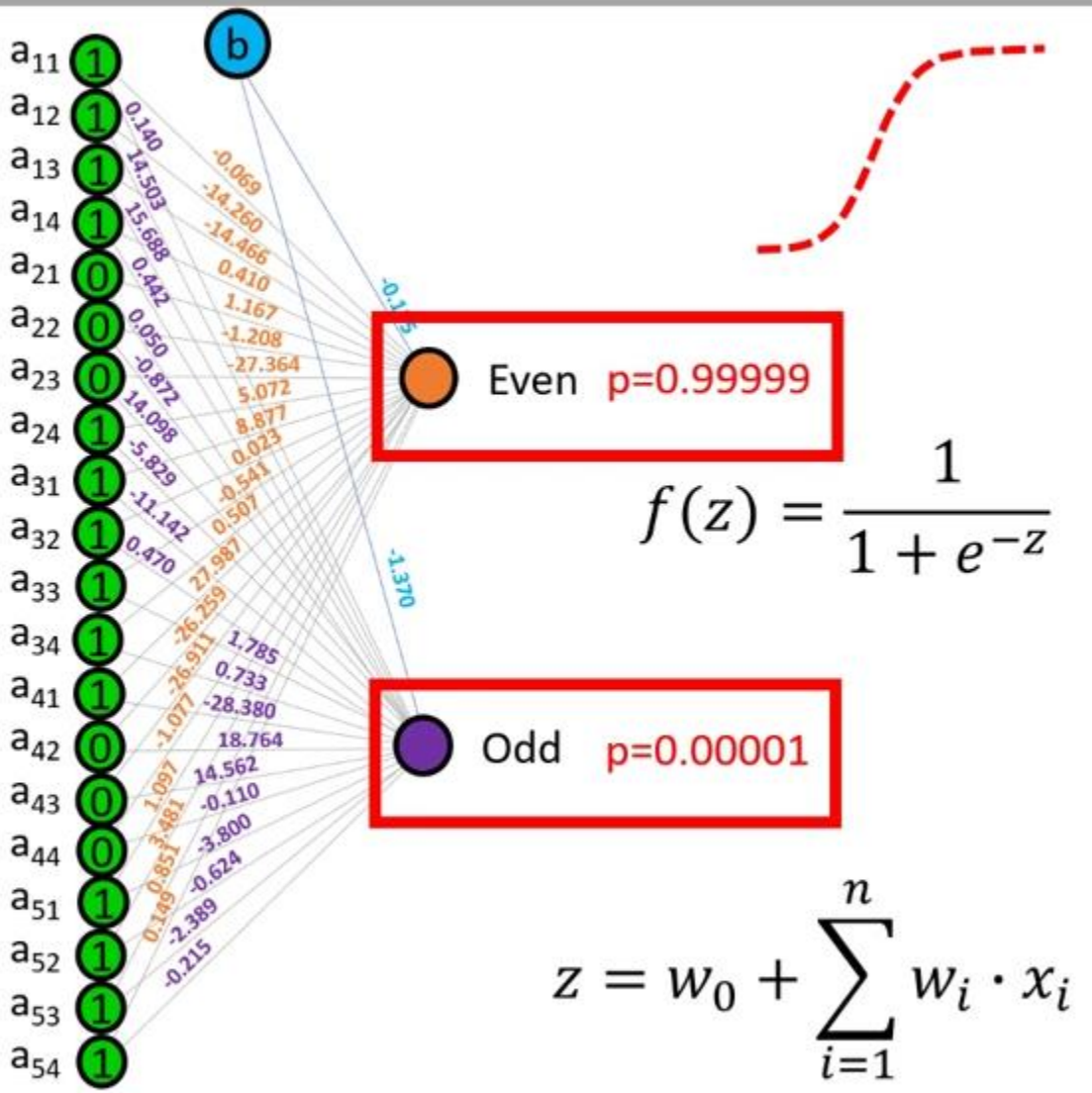


$$p_{odd} = 1 - p_{even}$$

This means that it would be enough to have just one output node for our network.

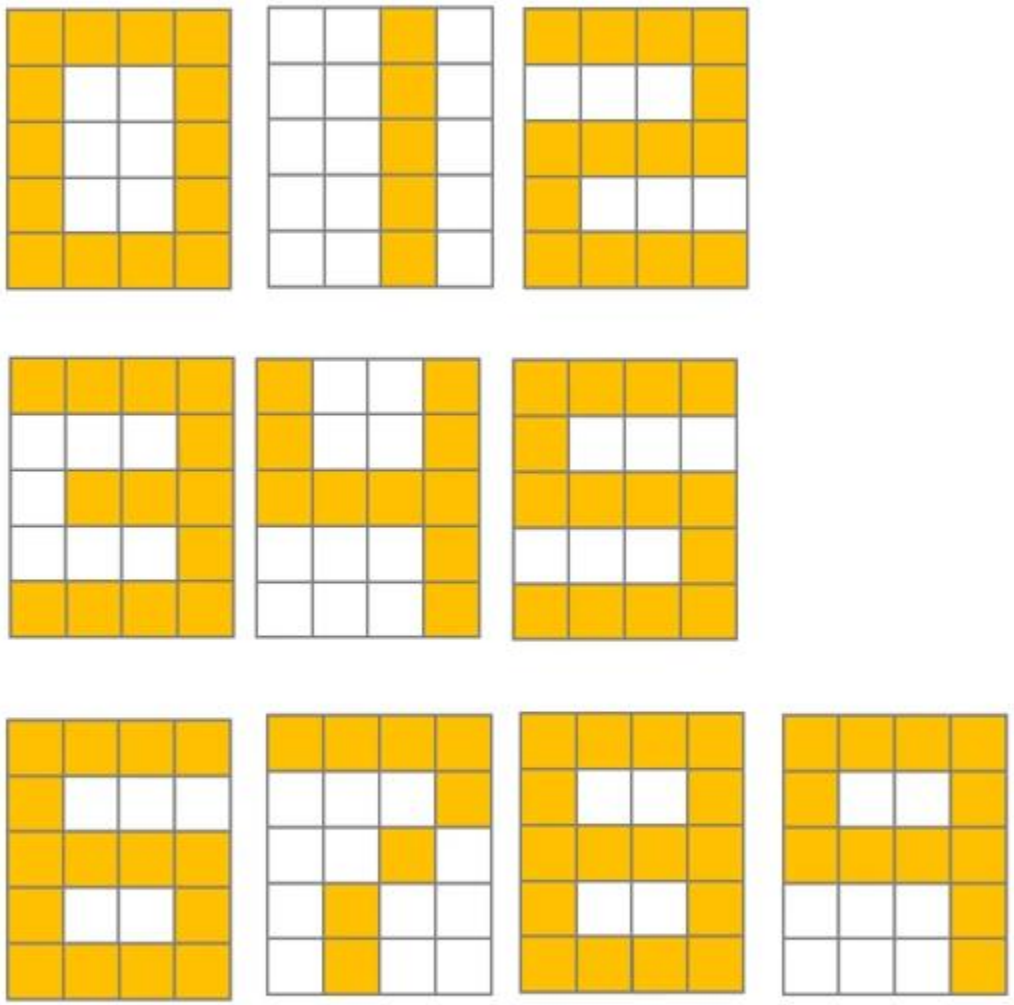
How the weights work

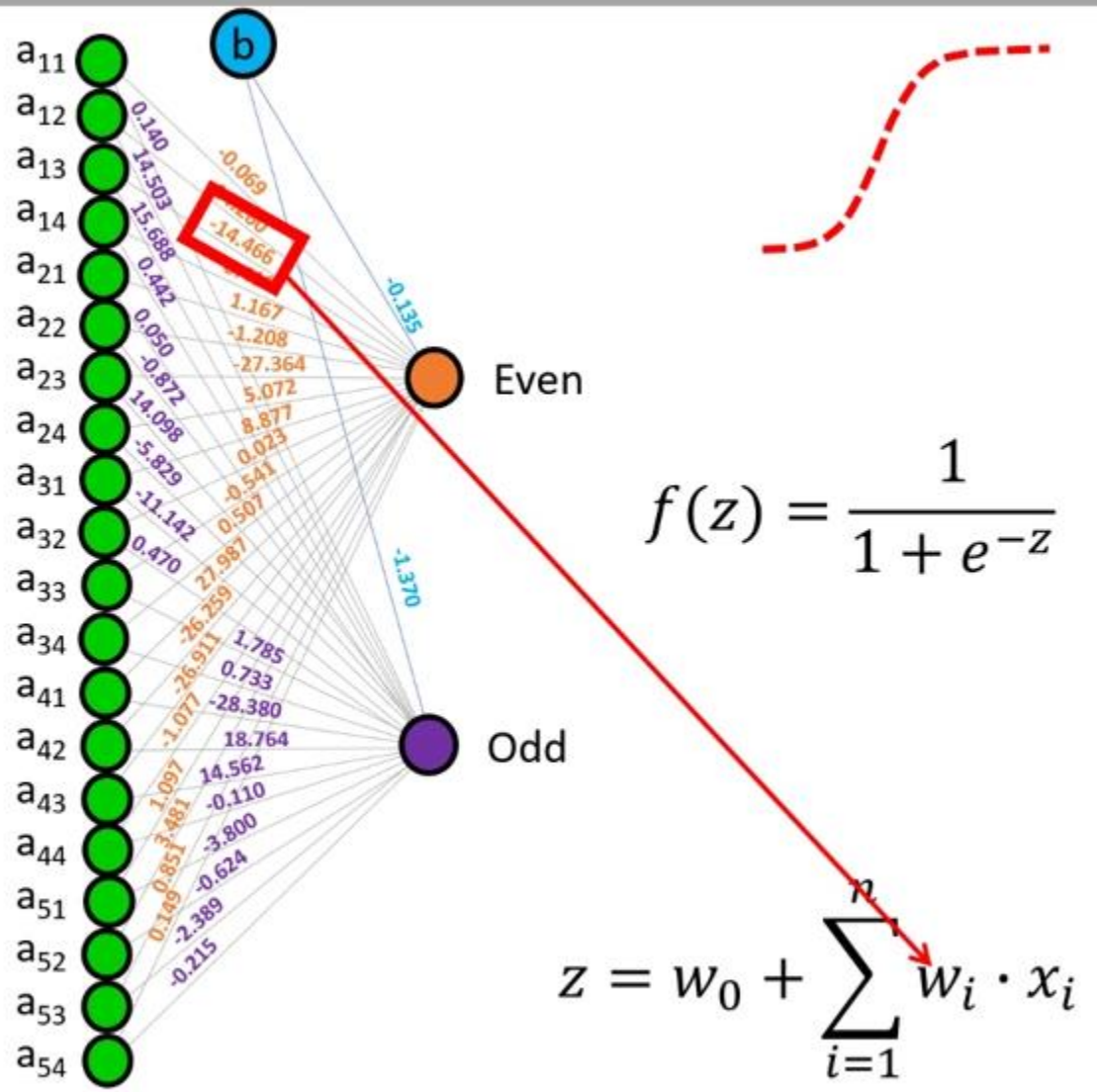




Let's try to understand how the predictions are associated with the values of the weights.

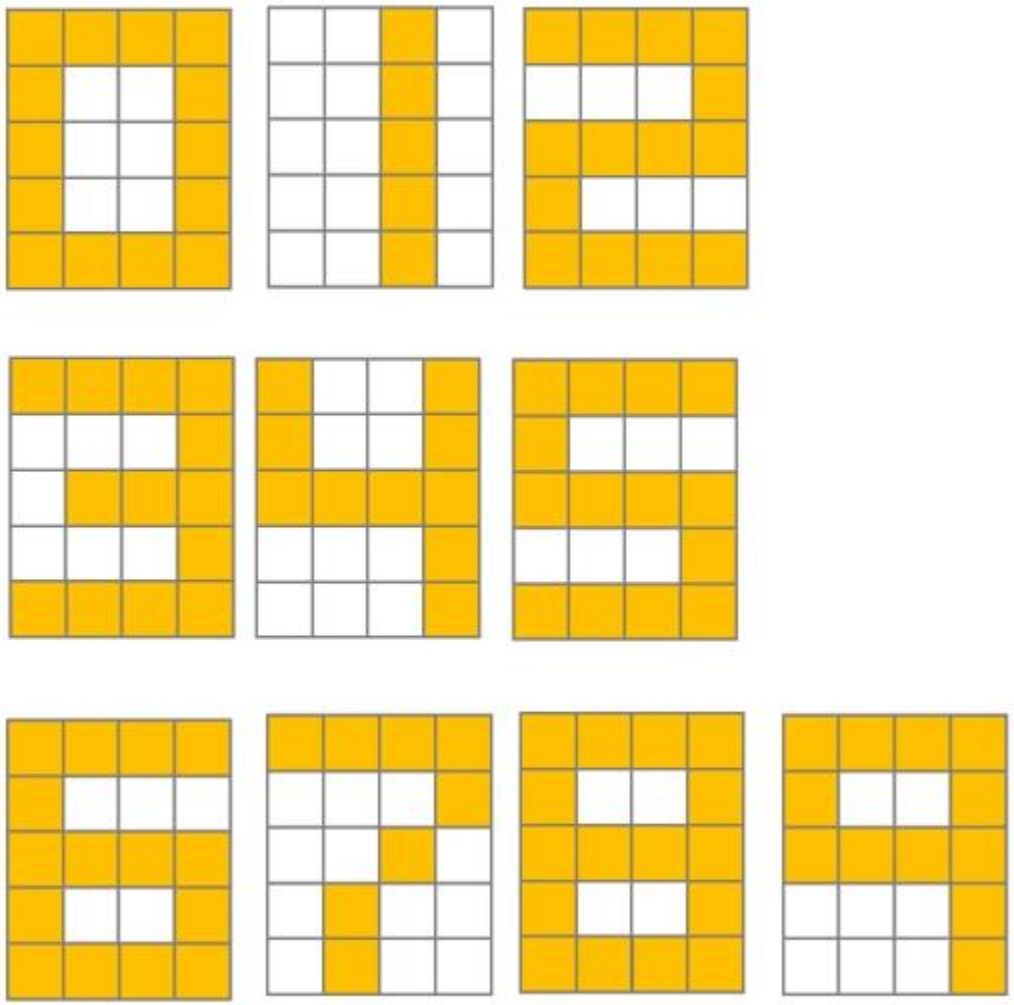
How the weights work

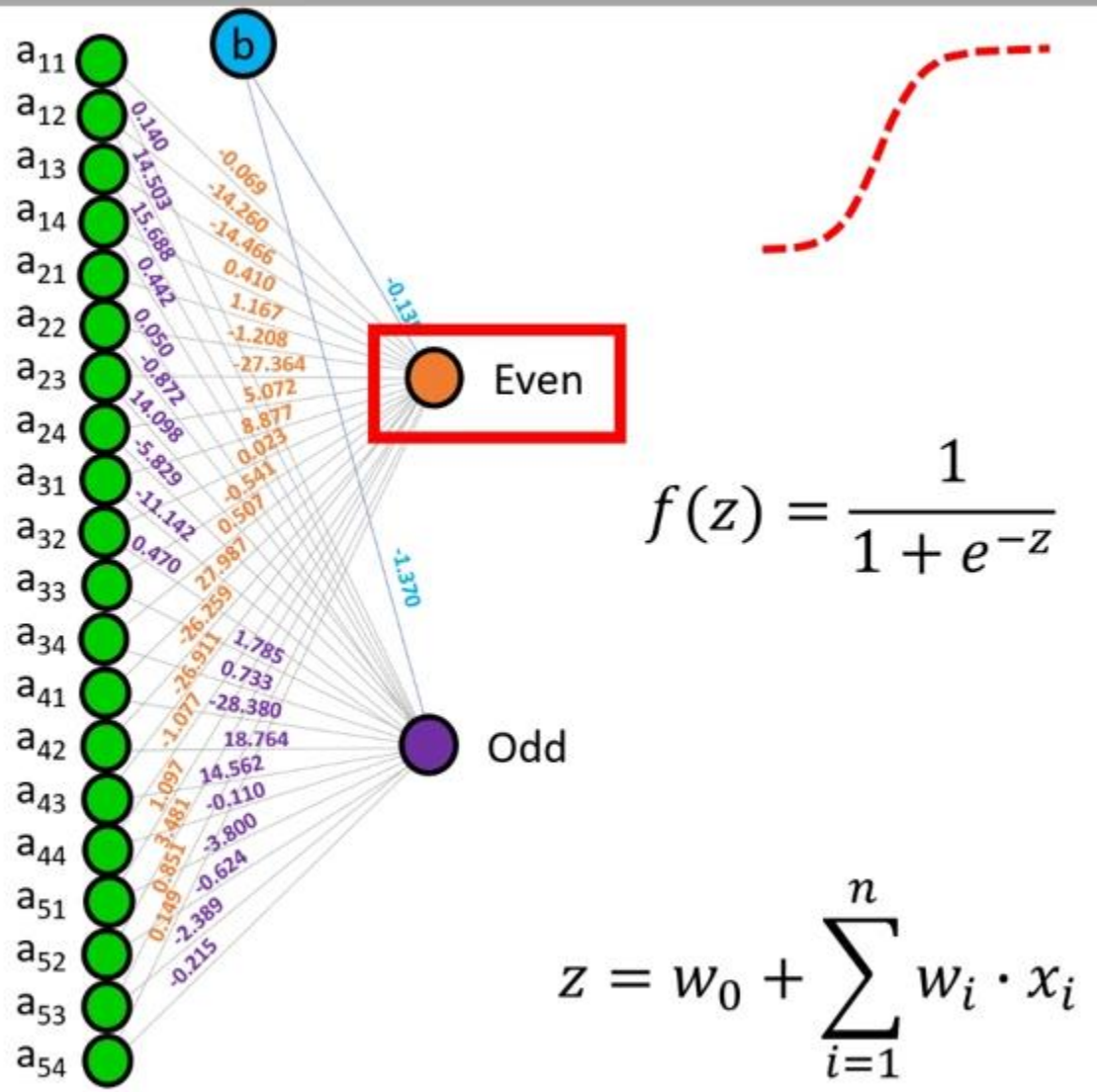




Note that, if a weight has a large negative value,

How the weights work



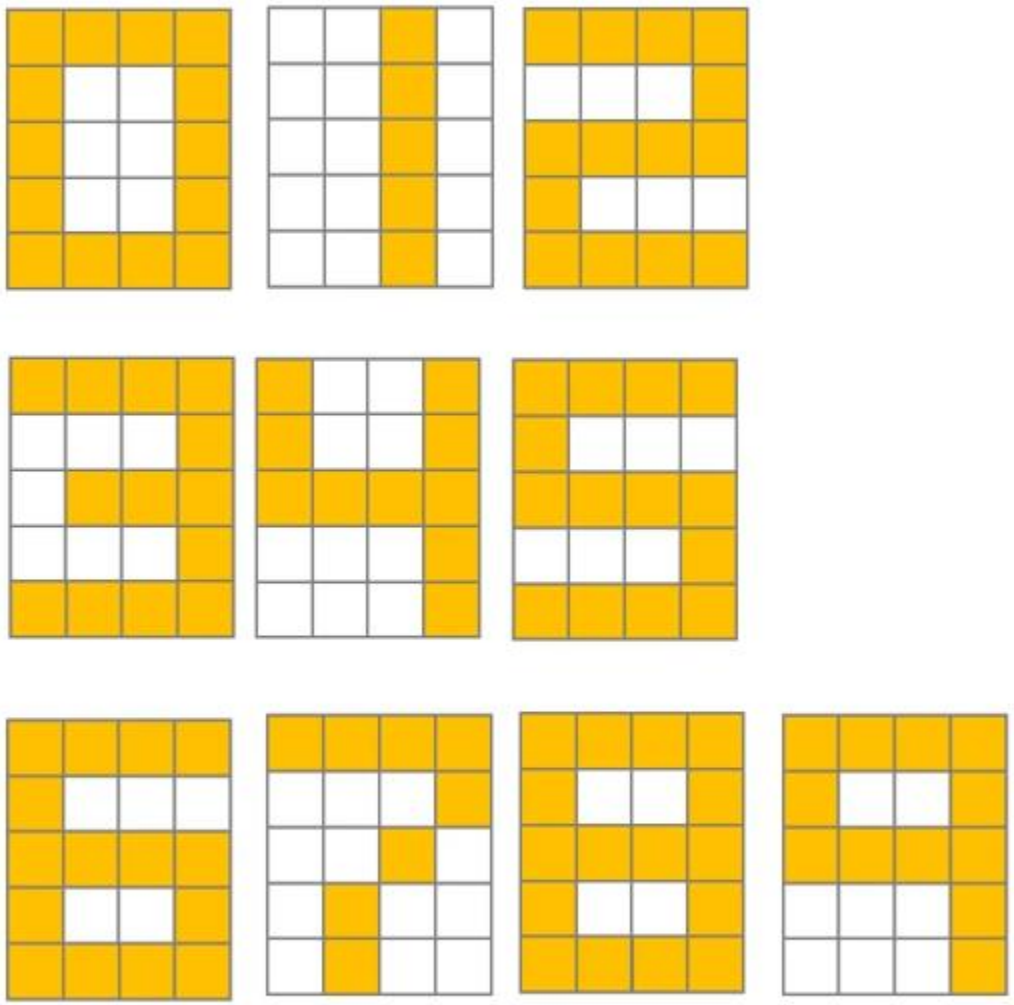


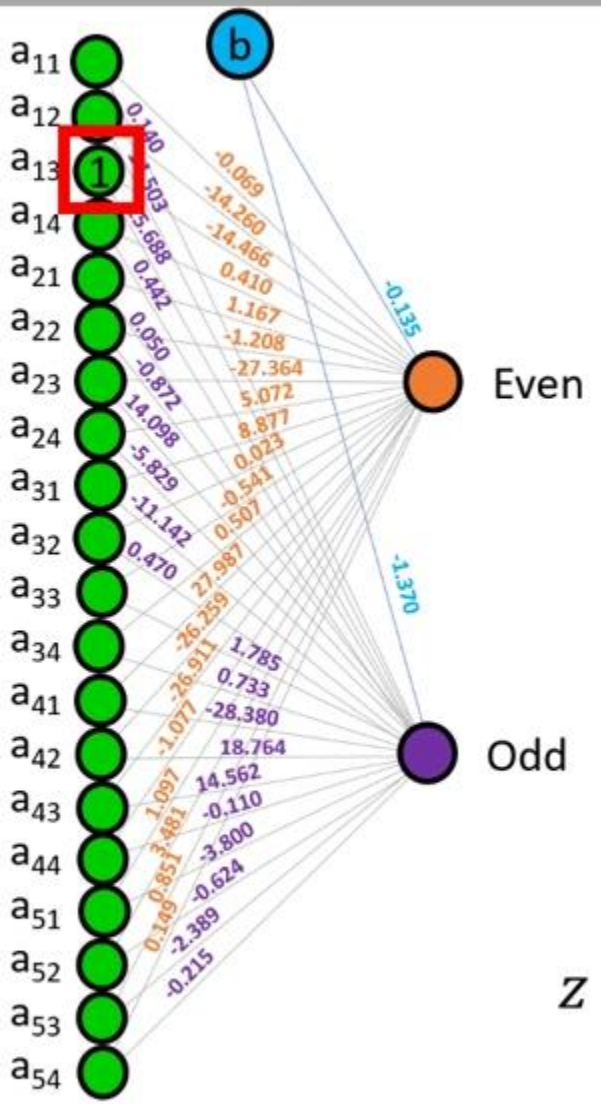
$$f(z) = \frac{1}{1 + e^{-z}}$$

$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

and is associated with an even number,

How the weights work



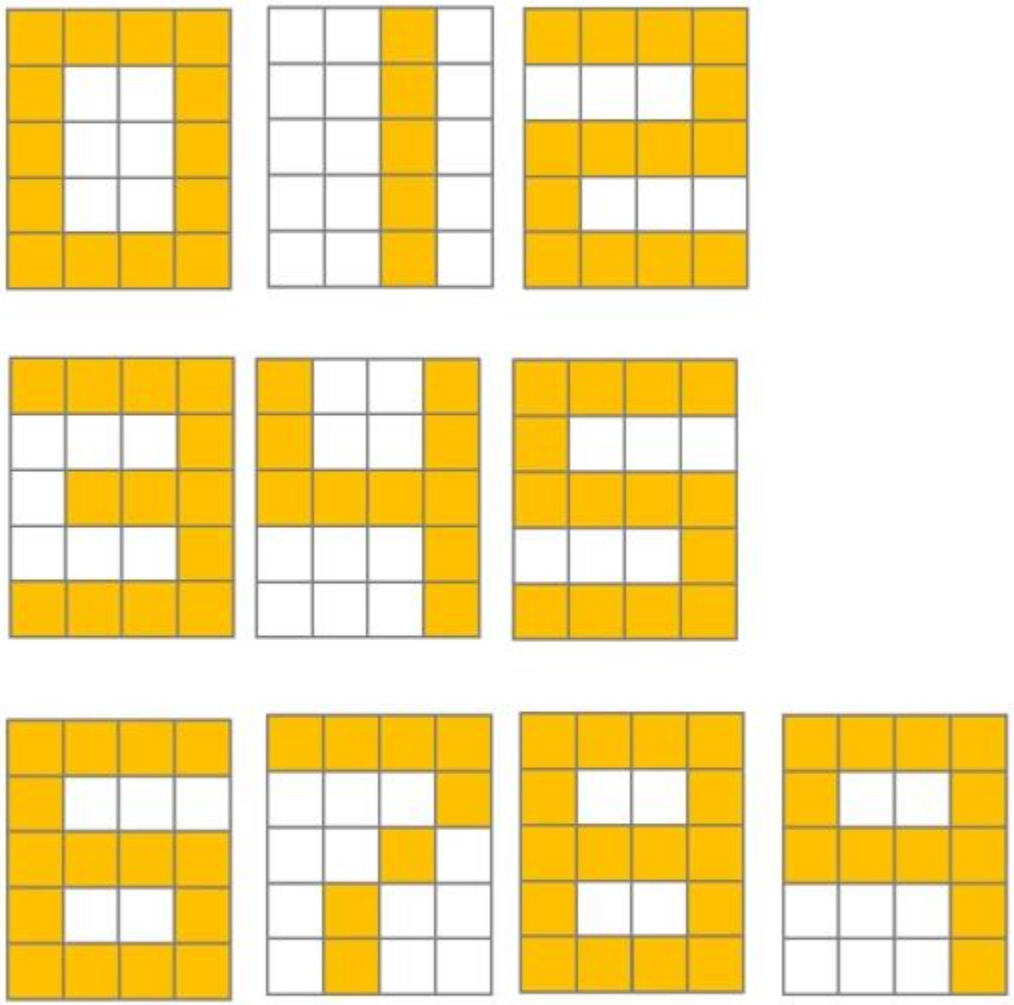


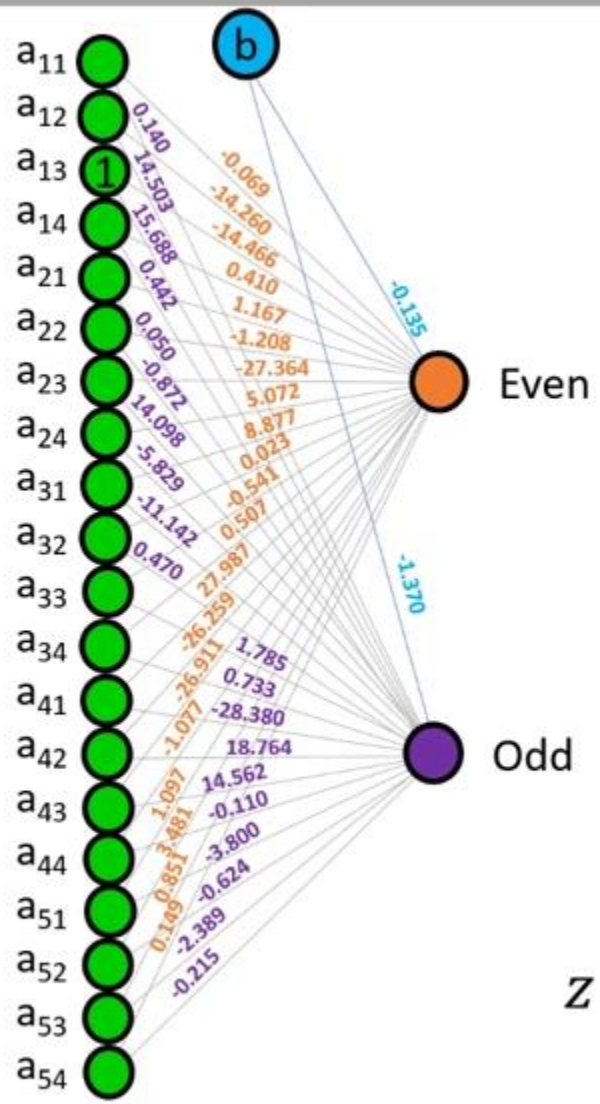
$$f(z) = \frac{1}{1 + e^{-z}}$$

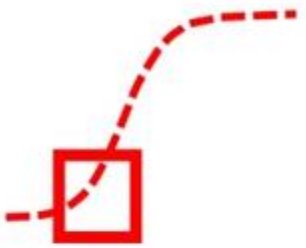
$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

and where the input node is equal to one,

How the weights work





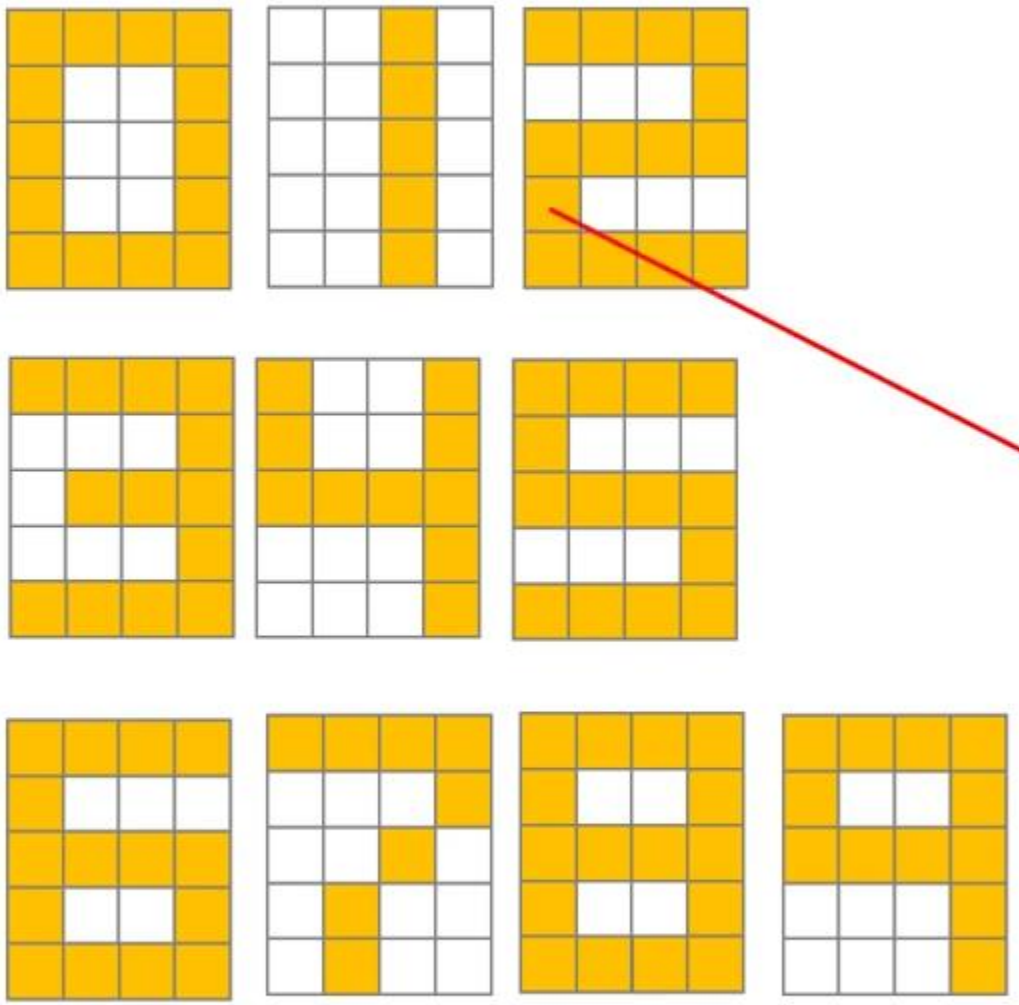


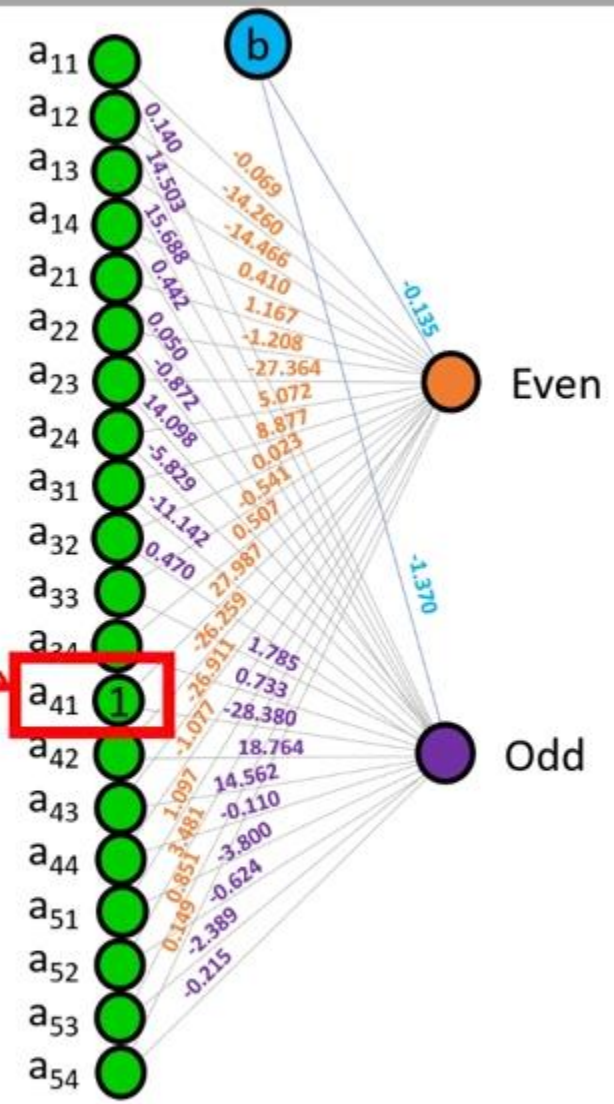
$$f(z) = \frac{1}{1 + e^{-z}}$$

$$z = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

which will reduce the probability that the image contains an even number.

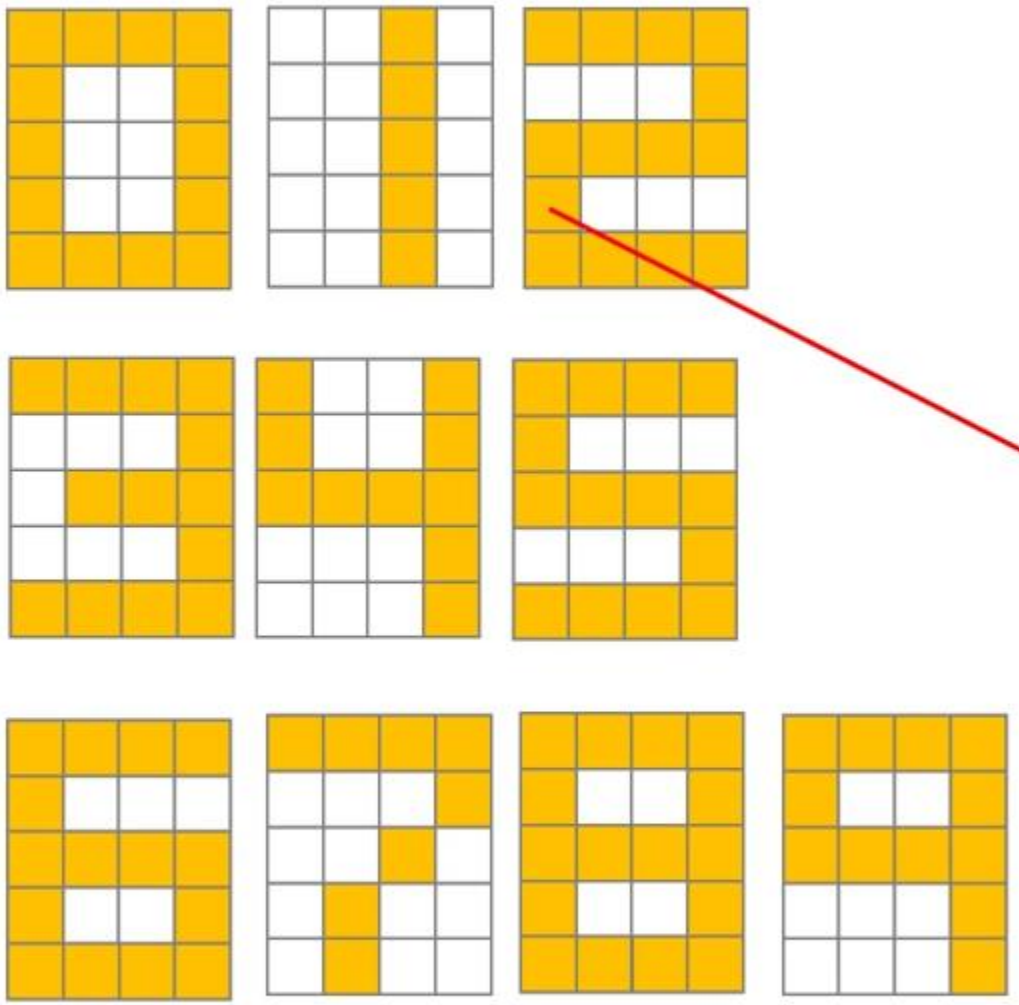
How the weights work

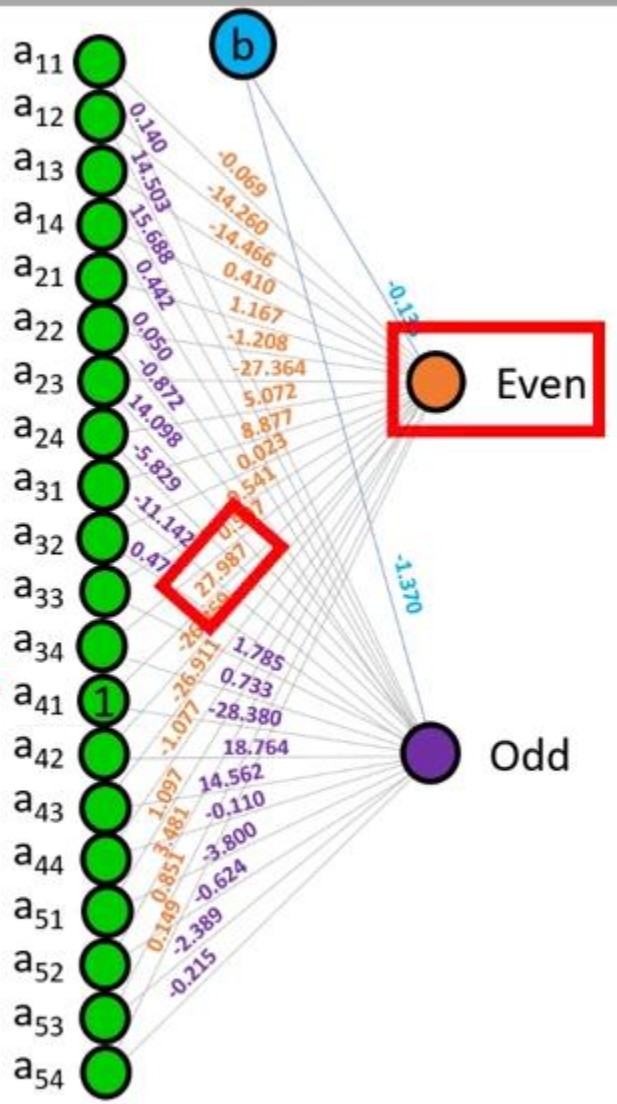




For example, if the pixel in the fourth row, first column, is equal to one,

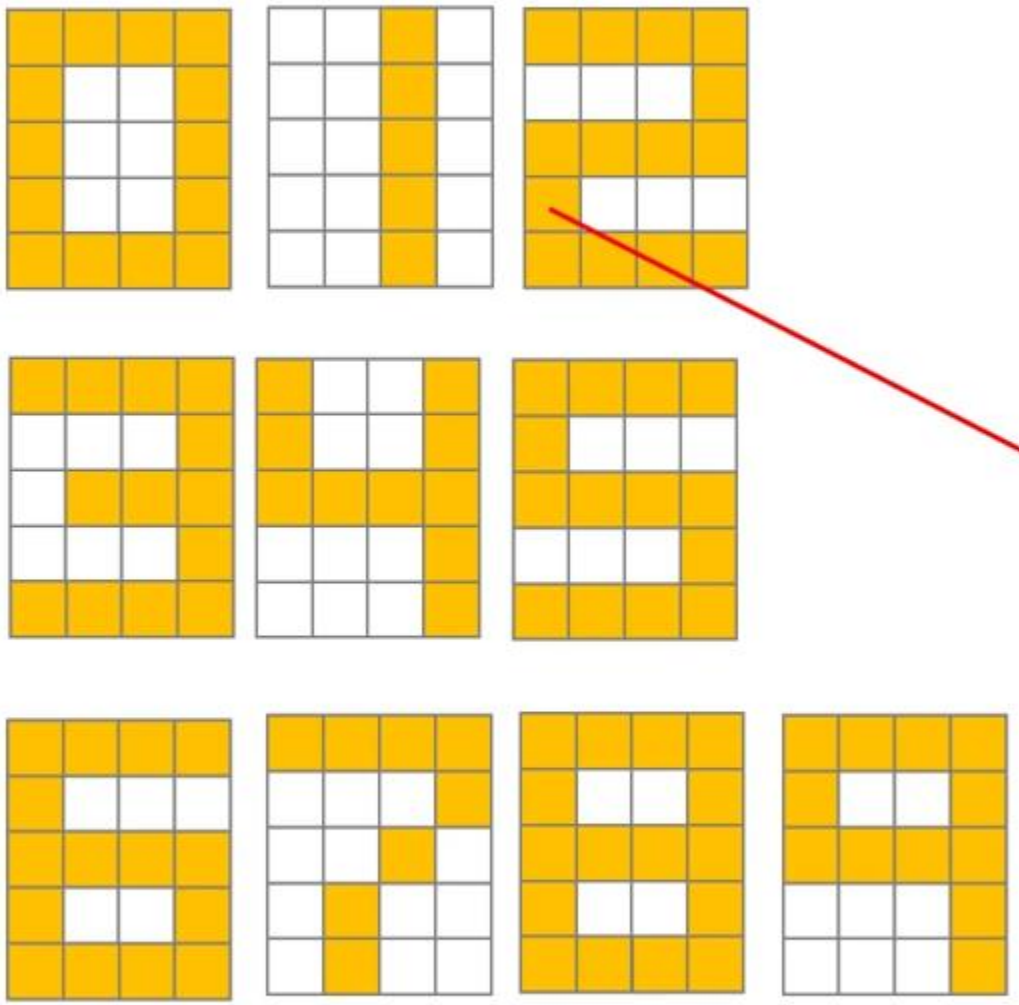
How the weights work

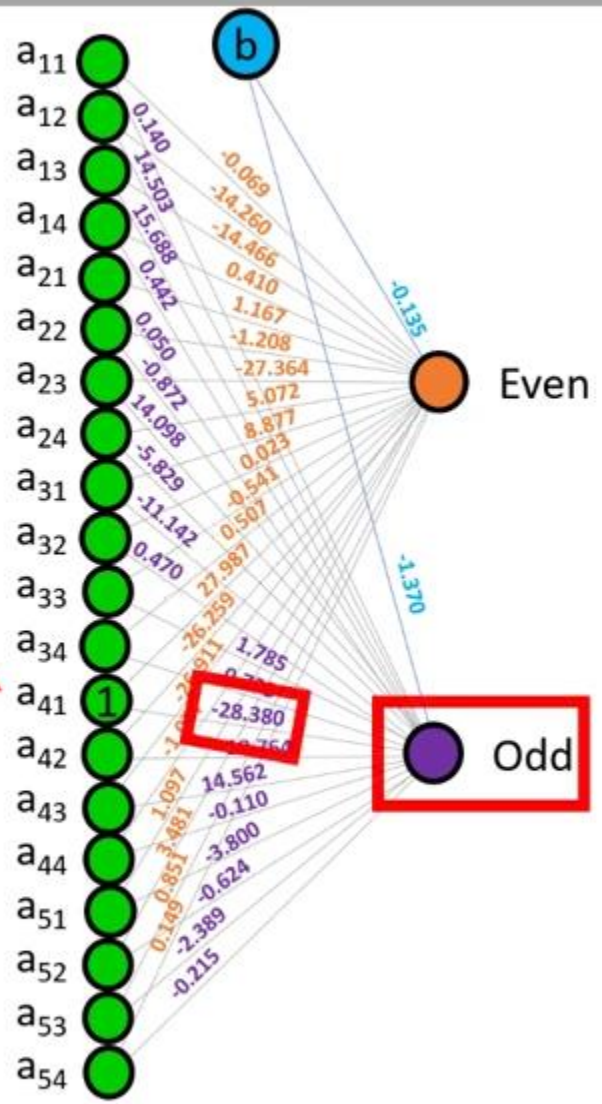




this will increase the probability that it is an even digit, because the associated weight is a large positive number.

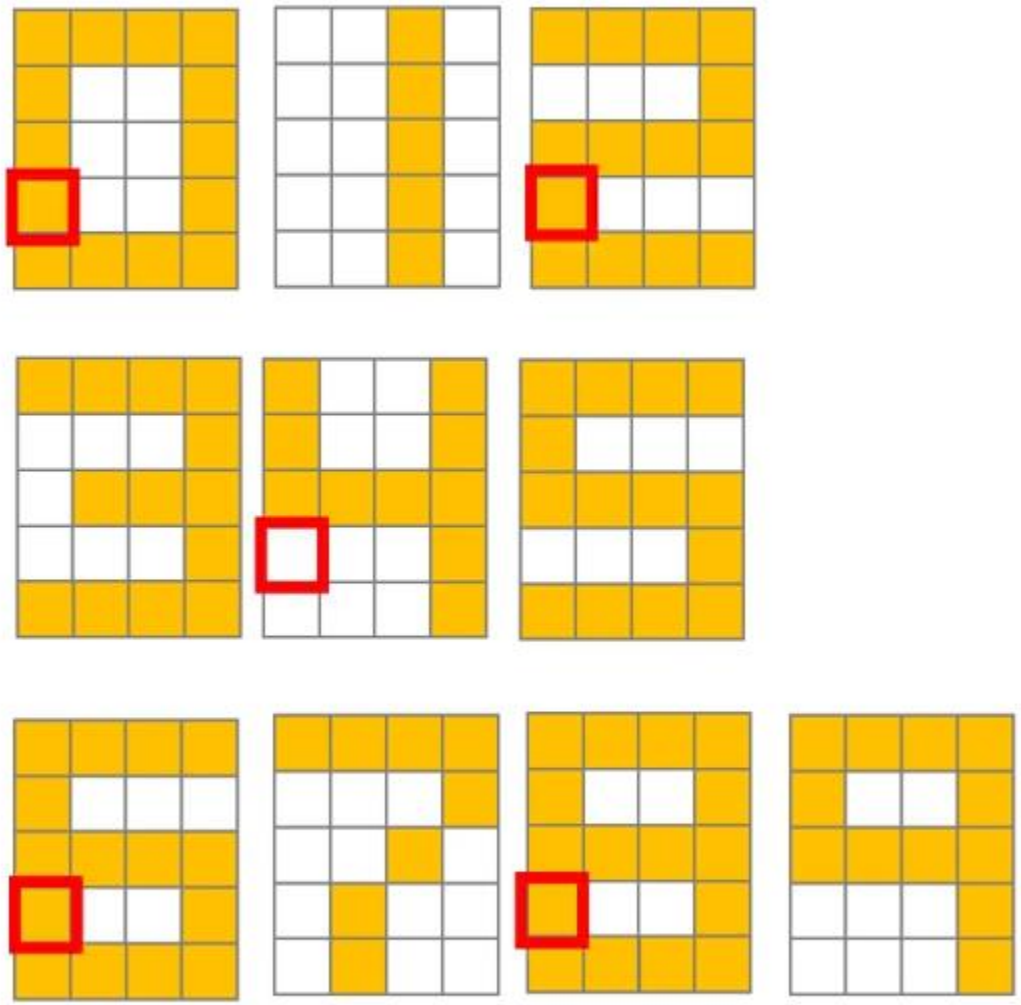
How the weights work

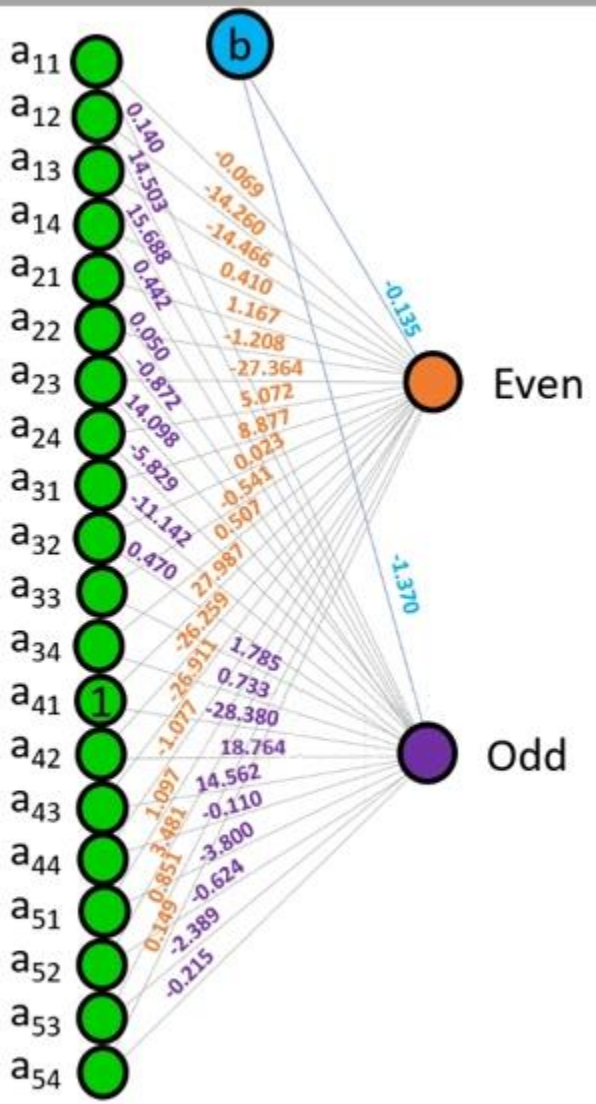




In comparison, the corresponding weight for an odd digit is instead a large negative number, which will reduce the probability that it is an odd digit.

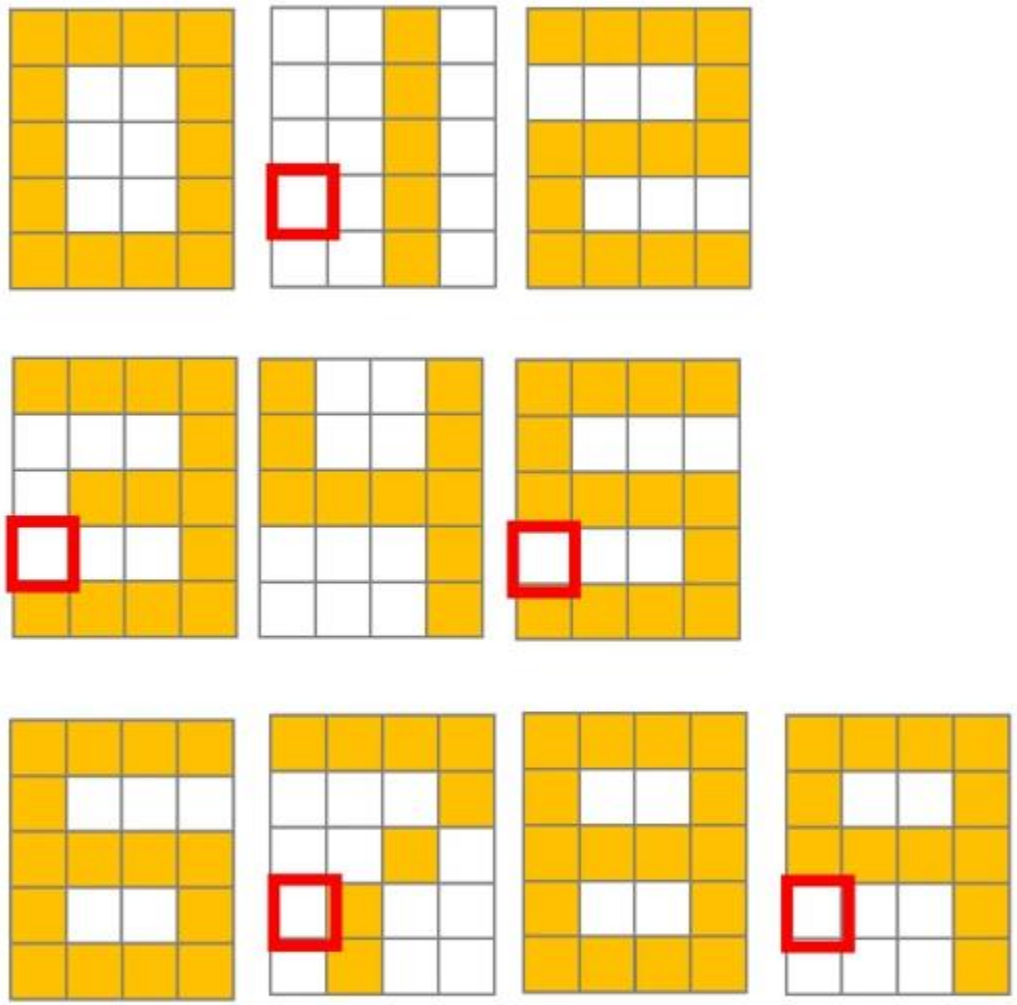
How the weights work

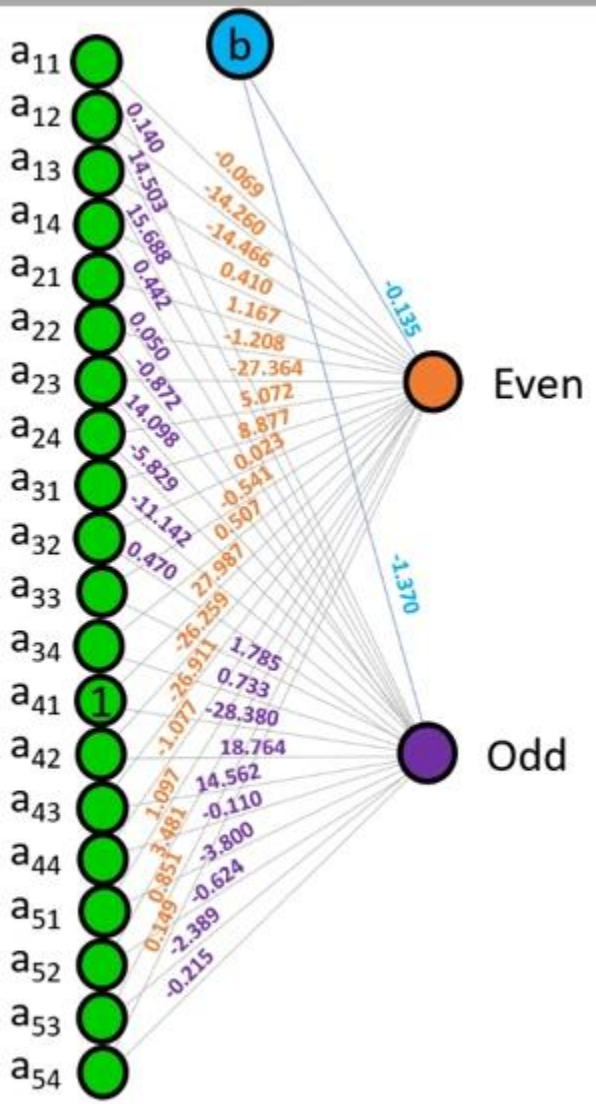




This makes sense because four out of five even digits have a color in this pixel,

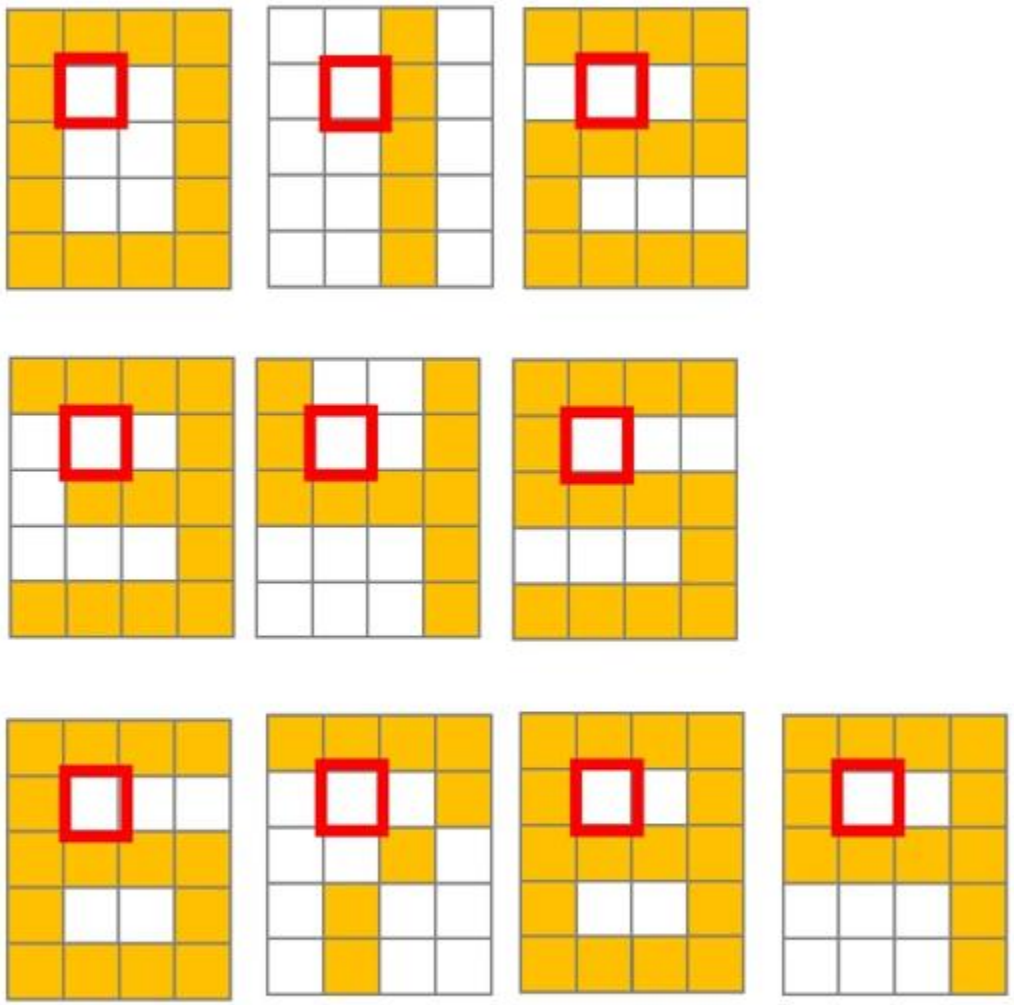
How the weights work

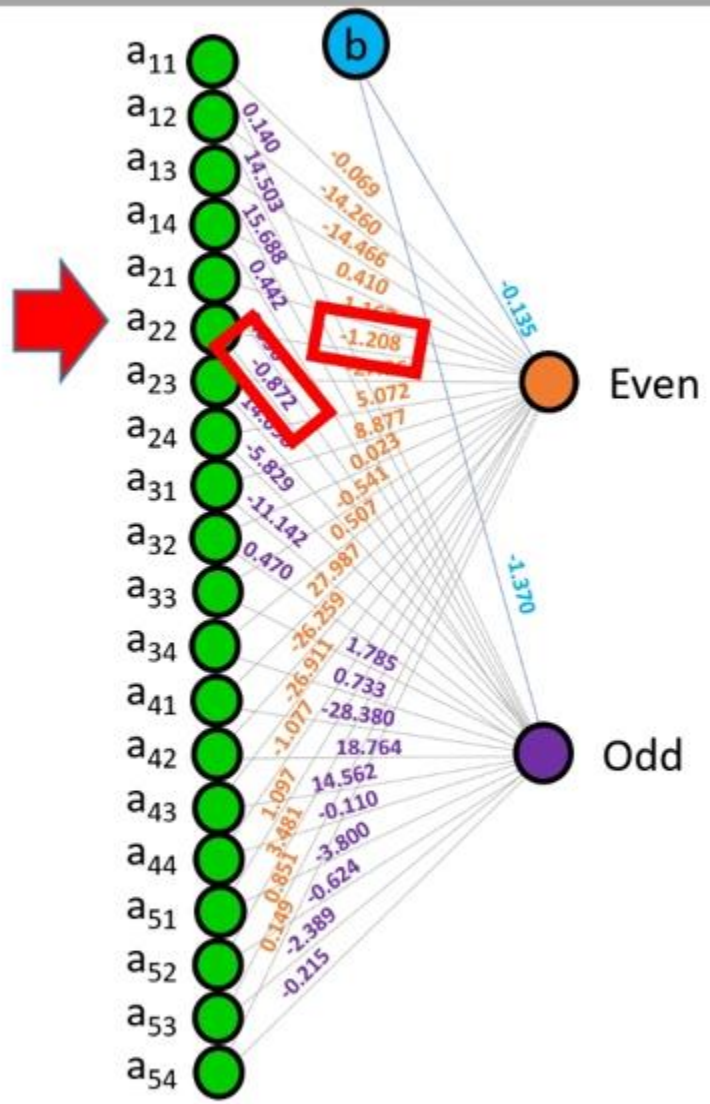




whereas no odd digit has a color in this pixel.

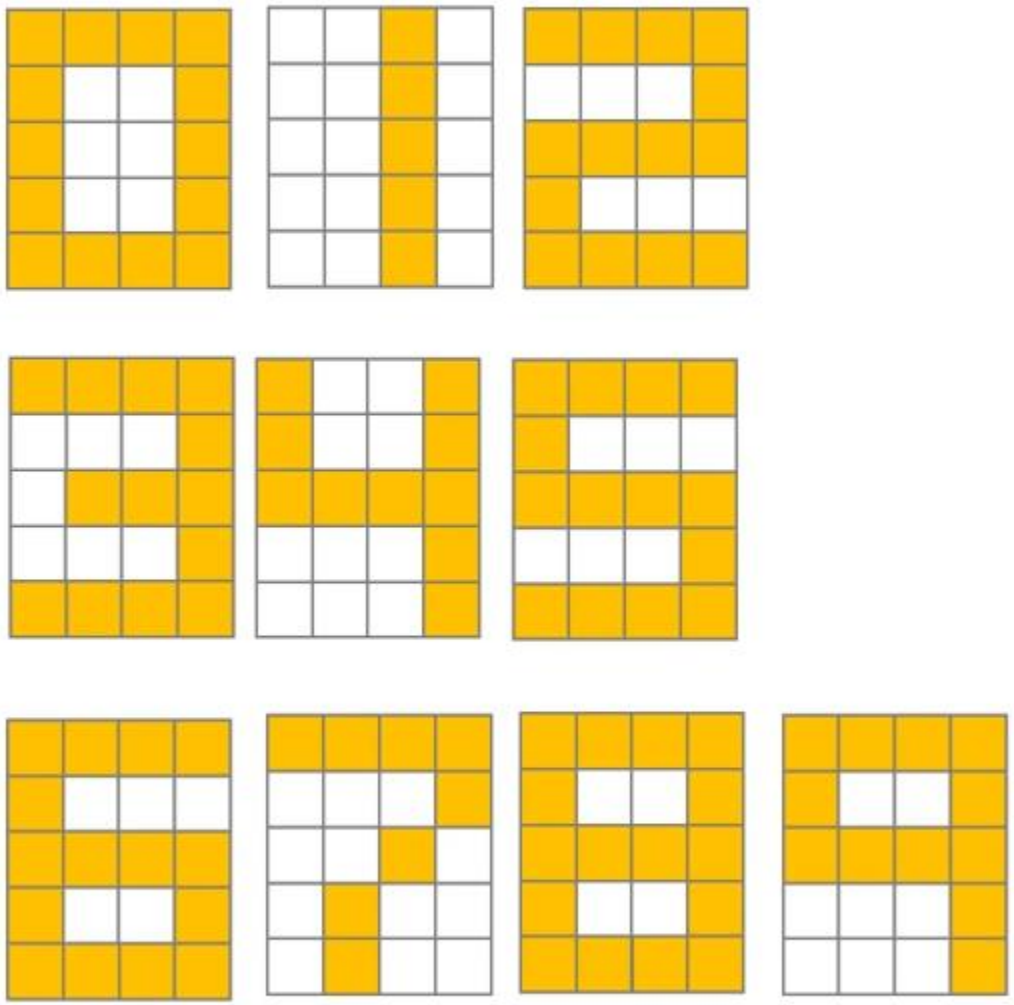
How the weights work

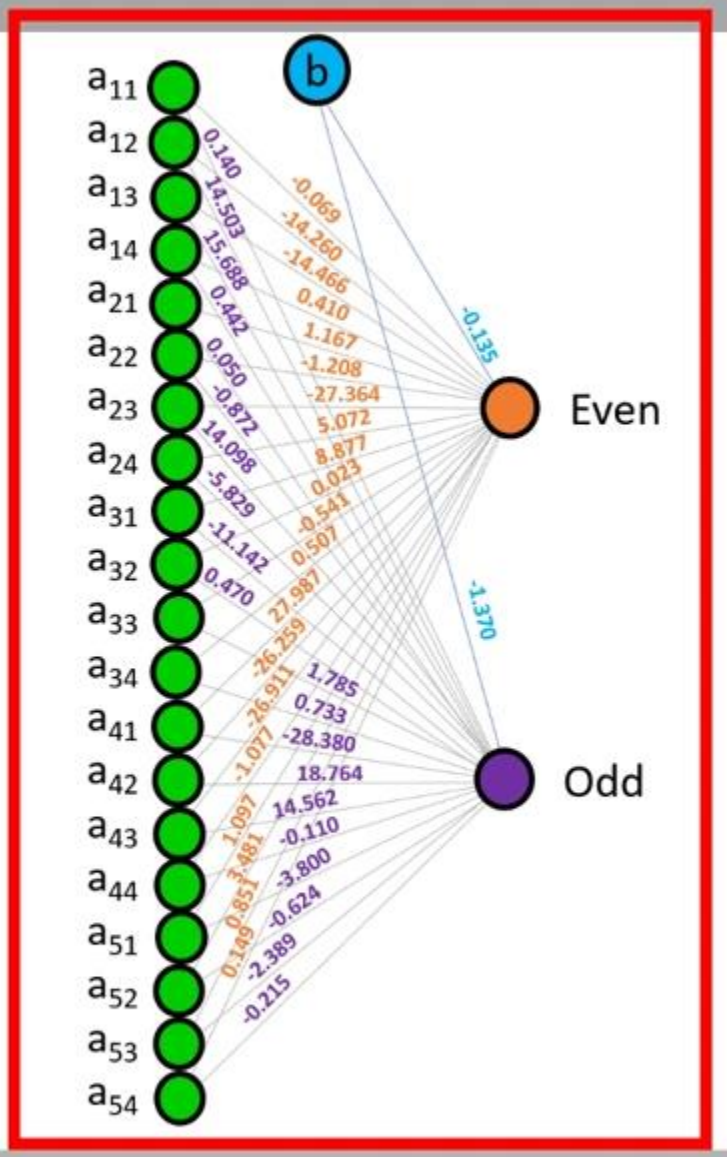




Since none of the images have a color in the pixel in the second row, second column, the associate weights to this input node are close to zero, which means that the value of this pixel will have a low impact on the prediction.

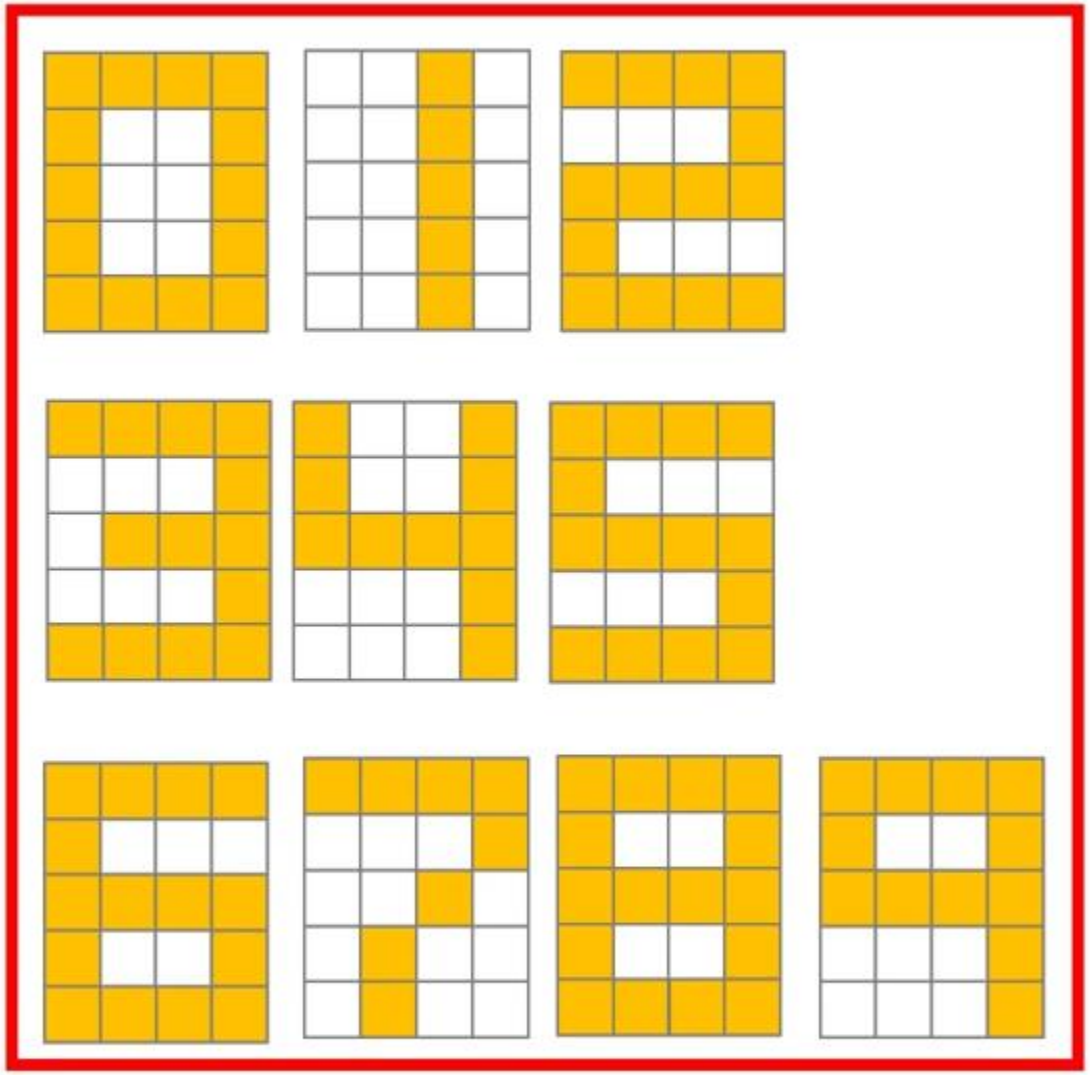
How the network trains

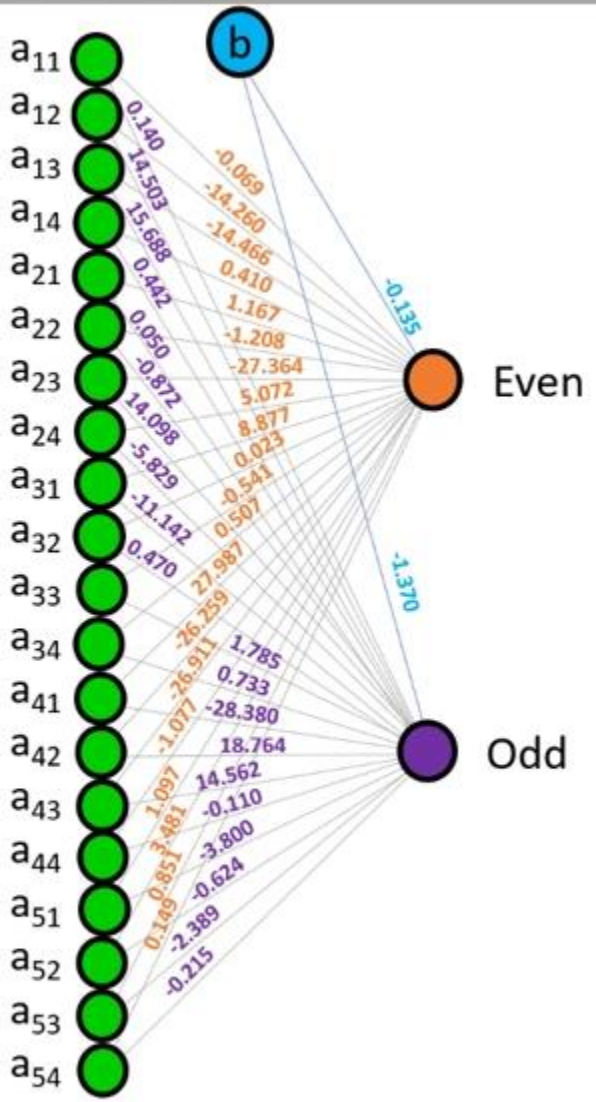




Training a neural network means that we find the optimal values of the weights so that the network predicts as good as possible.

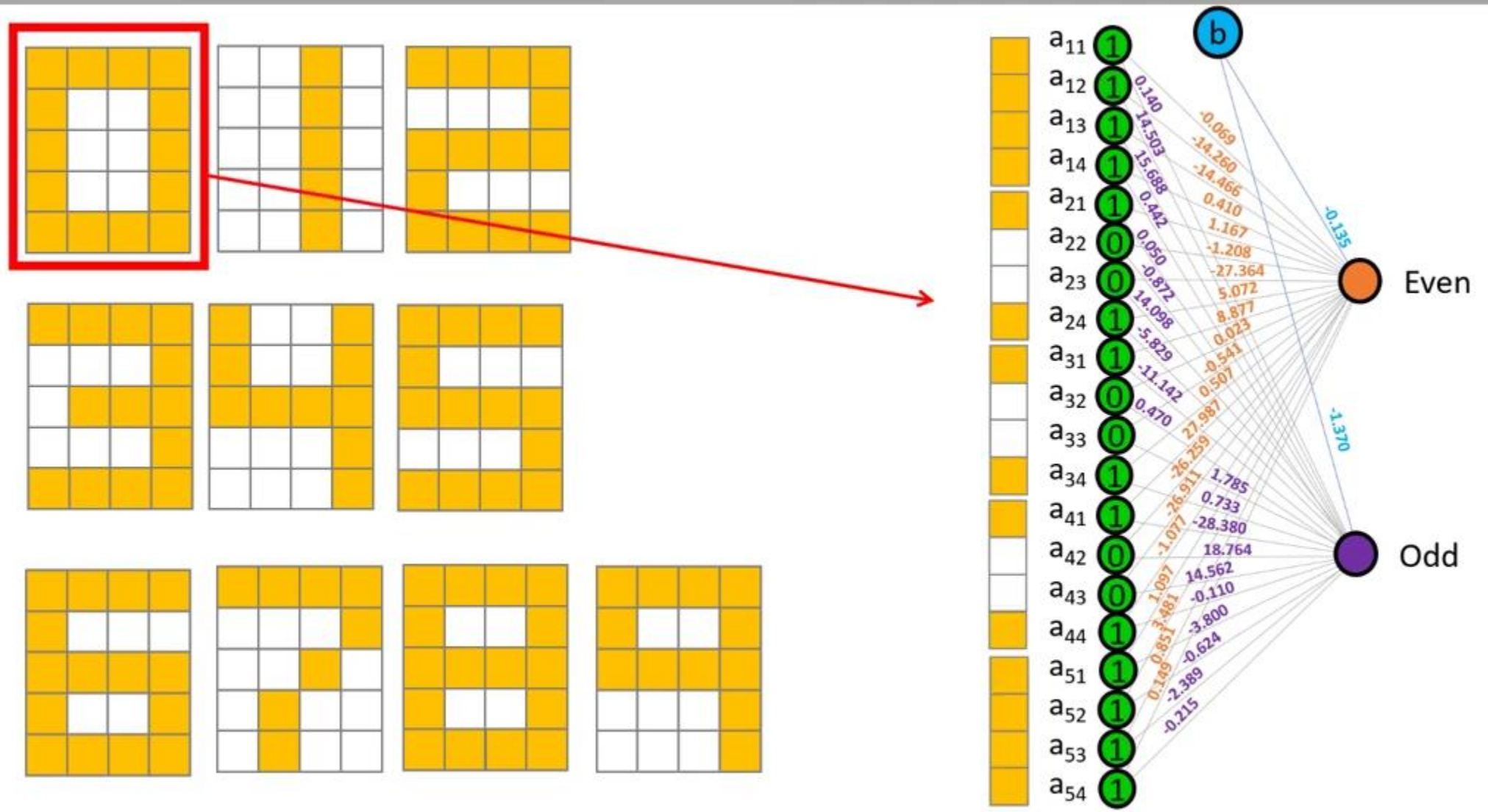
How the network trains





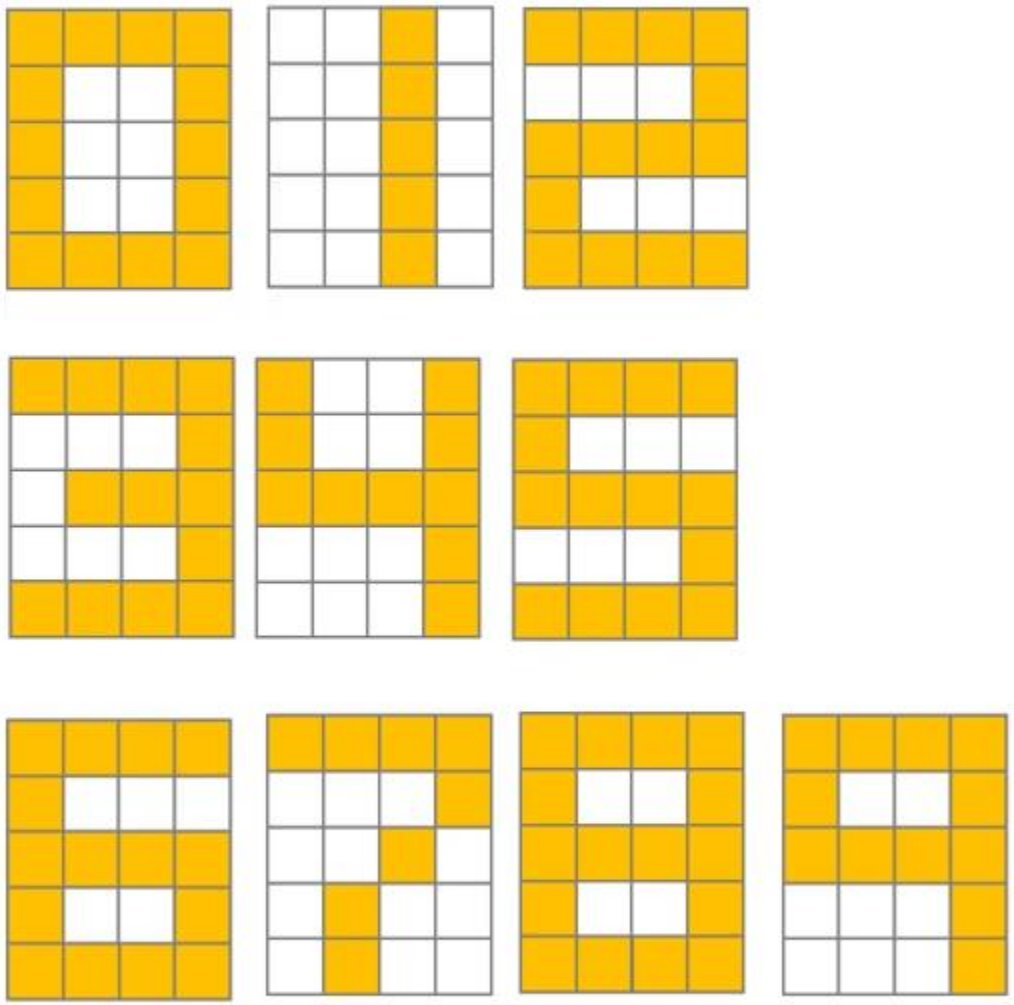
To optimize the values of the weights, we let the network train on this data.

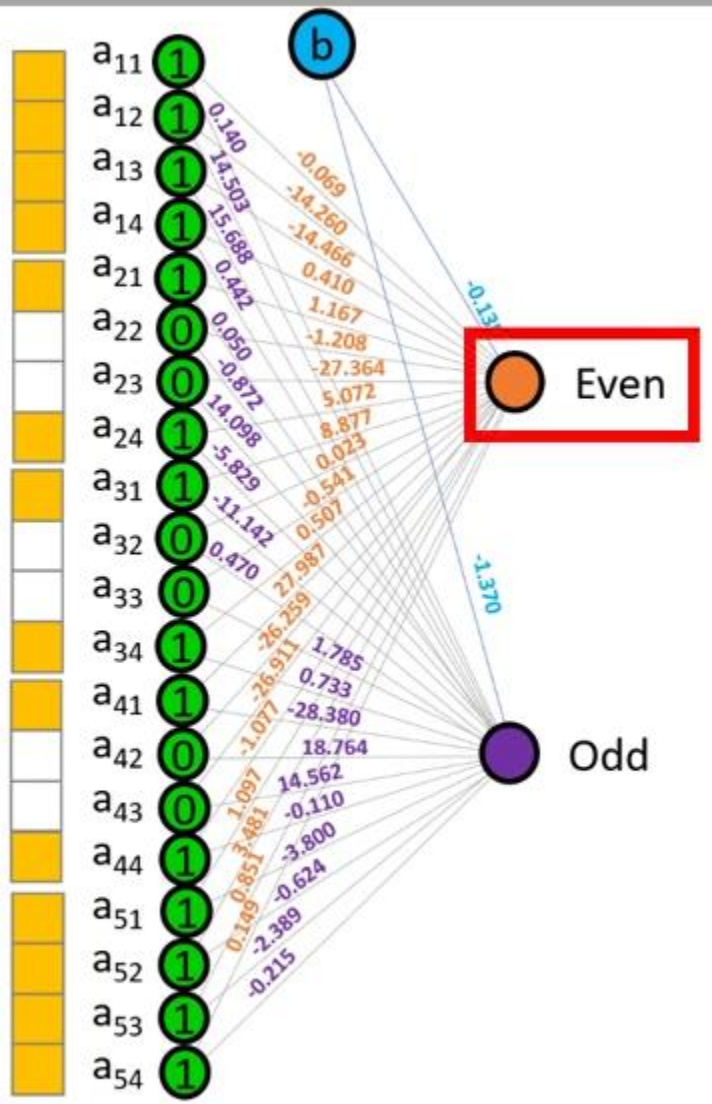
How the network trains



For example, we plug in the values of all the pixels from this image in the input nodes,

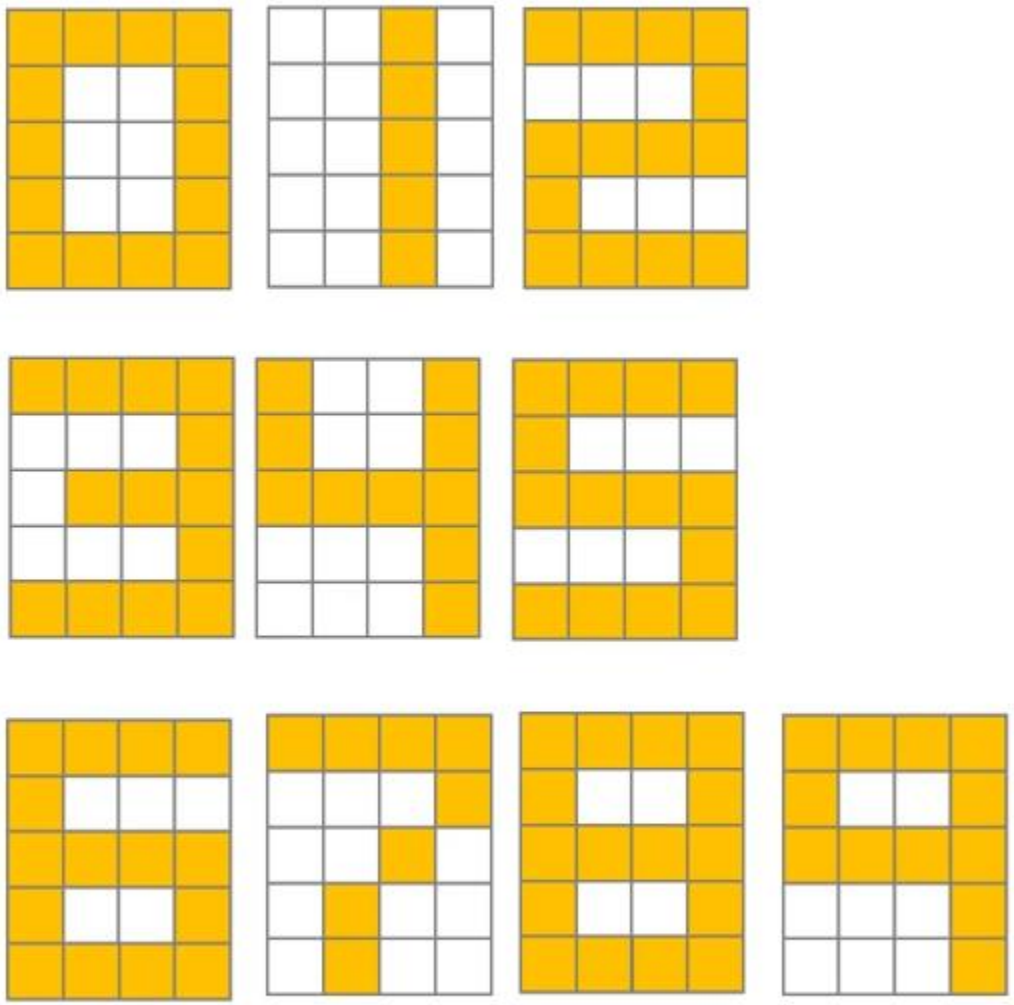
How the network trains

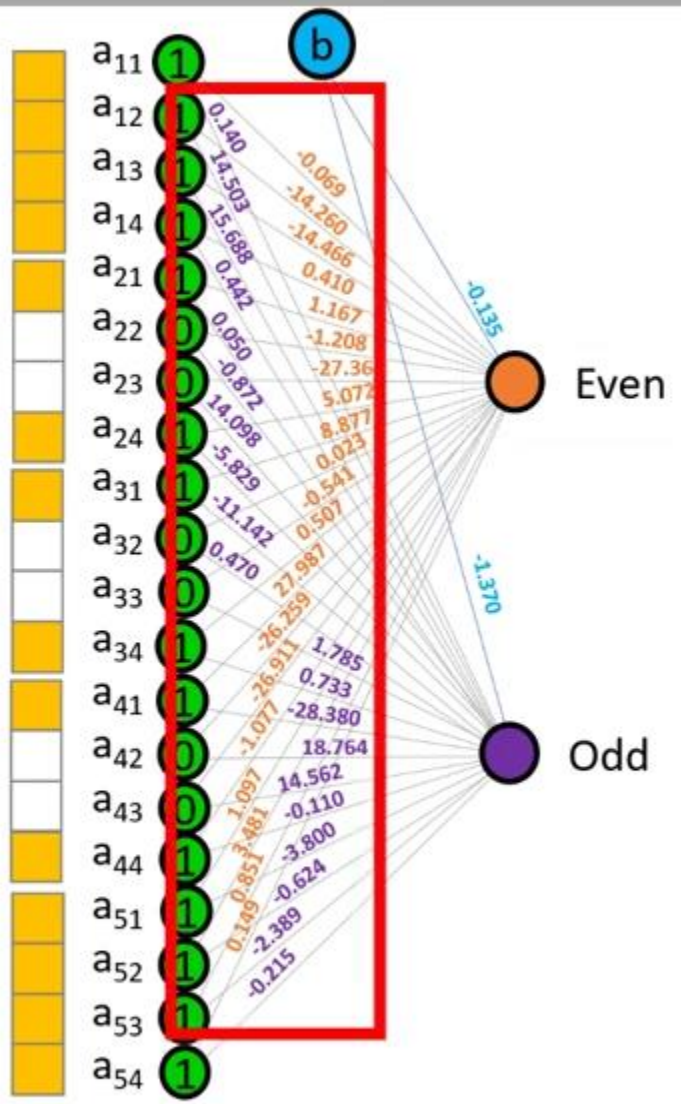




and tell the network that this is an even digit,

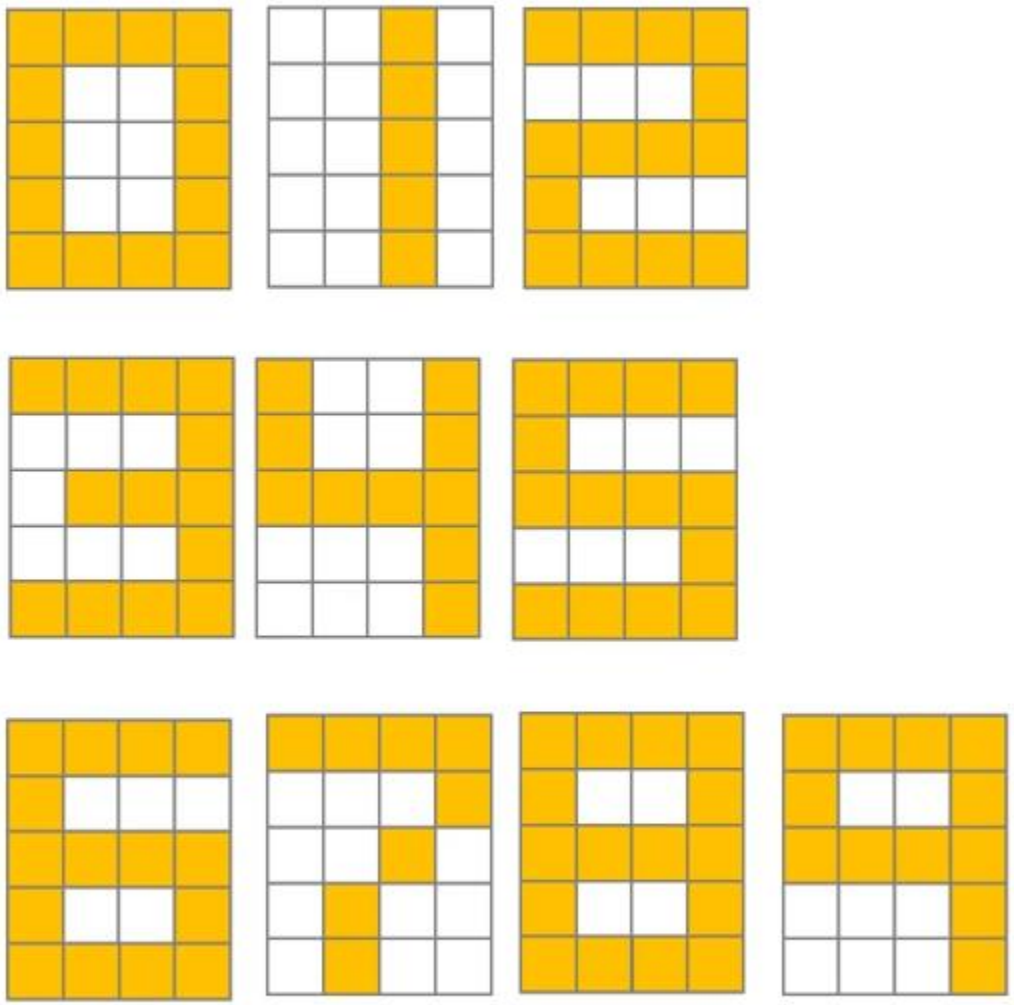
How the network trains

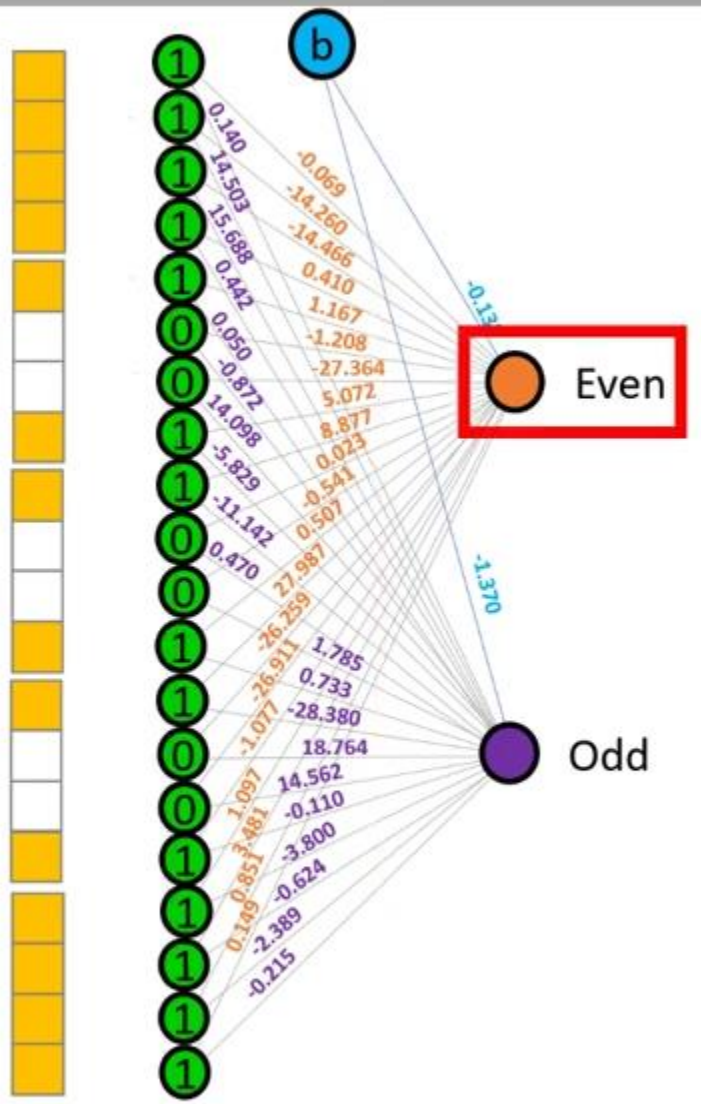




it will then change the weights,

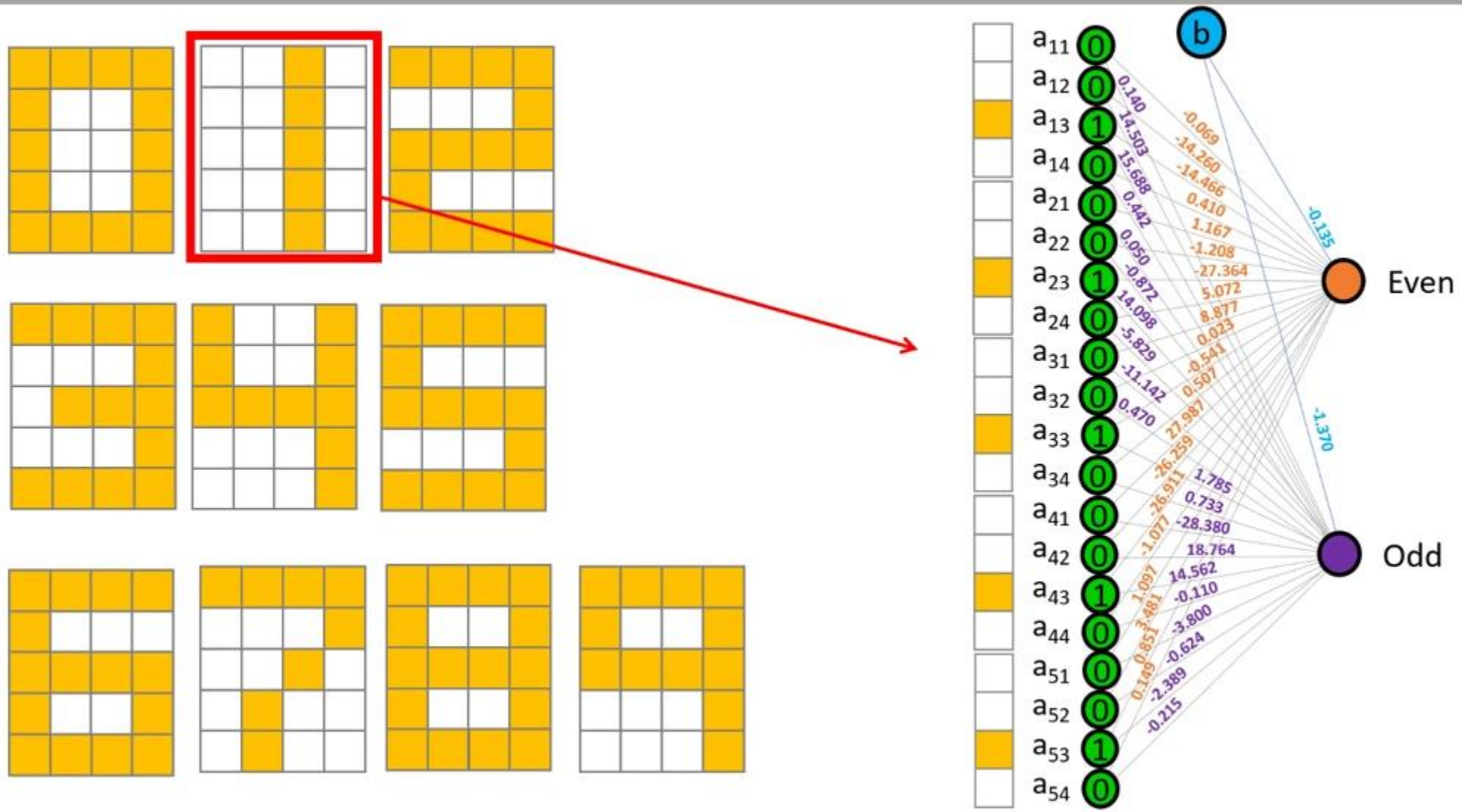
How the network trains





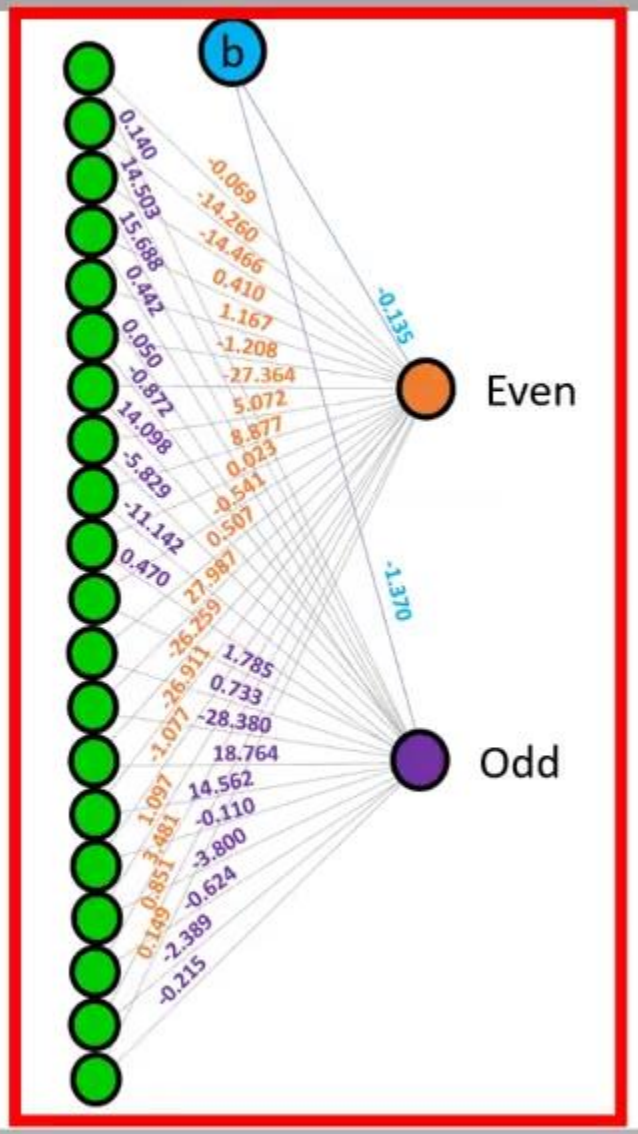
so that it predicts that it is an even digit.

How the network trains



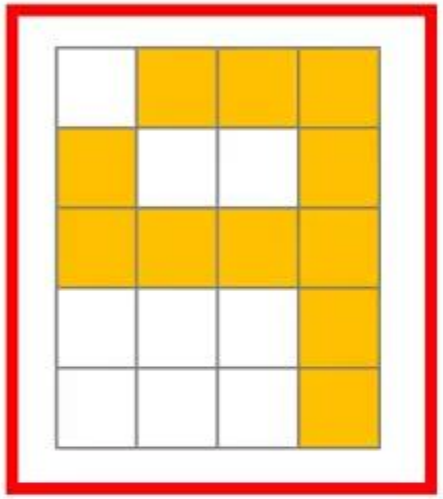
Then we do the same for the next image and tell the network that this is an odd number, and so forth. Watch my basic video about artificial neural networks if you like to understand how the weights are optimized based on a certain loss function.

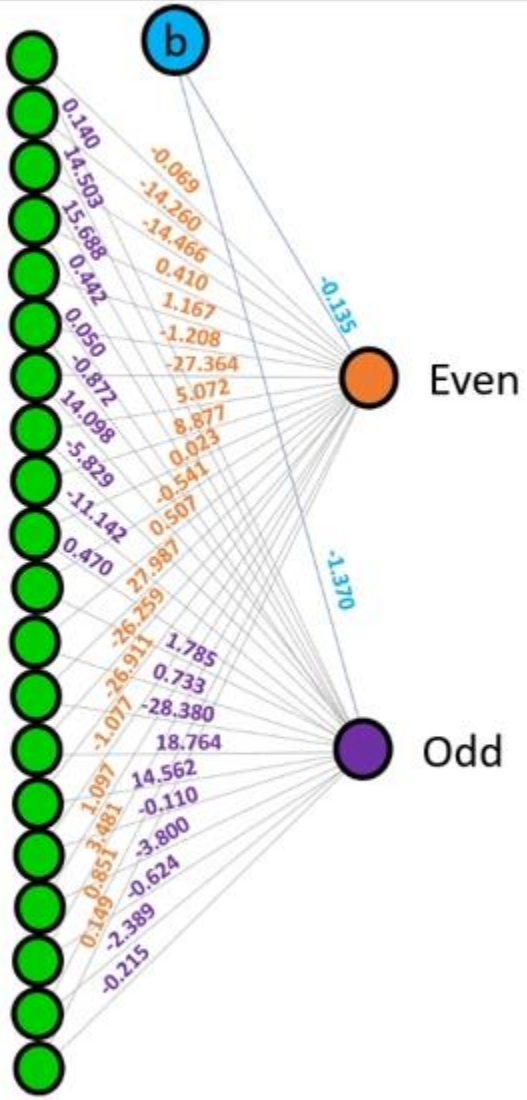
Predict



Once we have trained a neural network on the images with digits, we can use it to predict if an image contains an even or an odd digit.

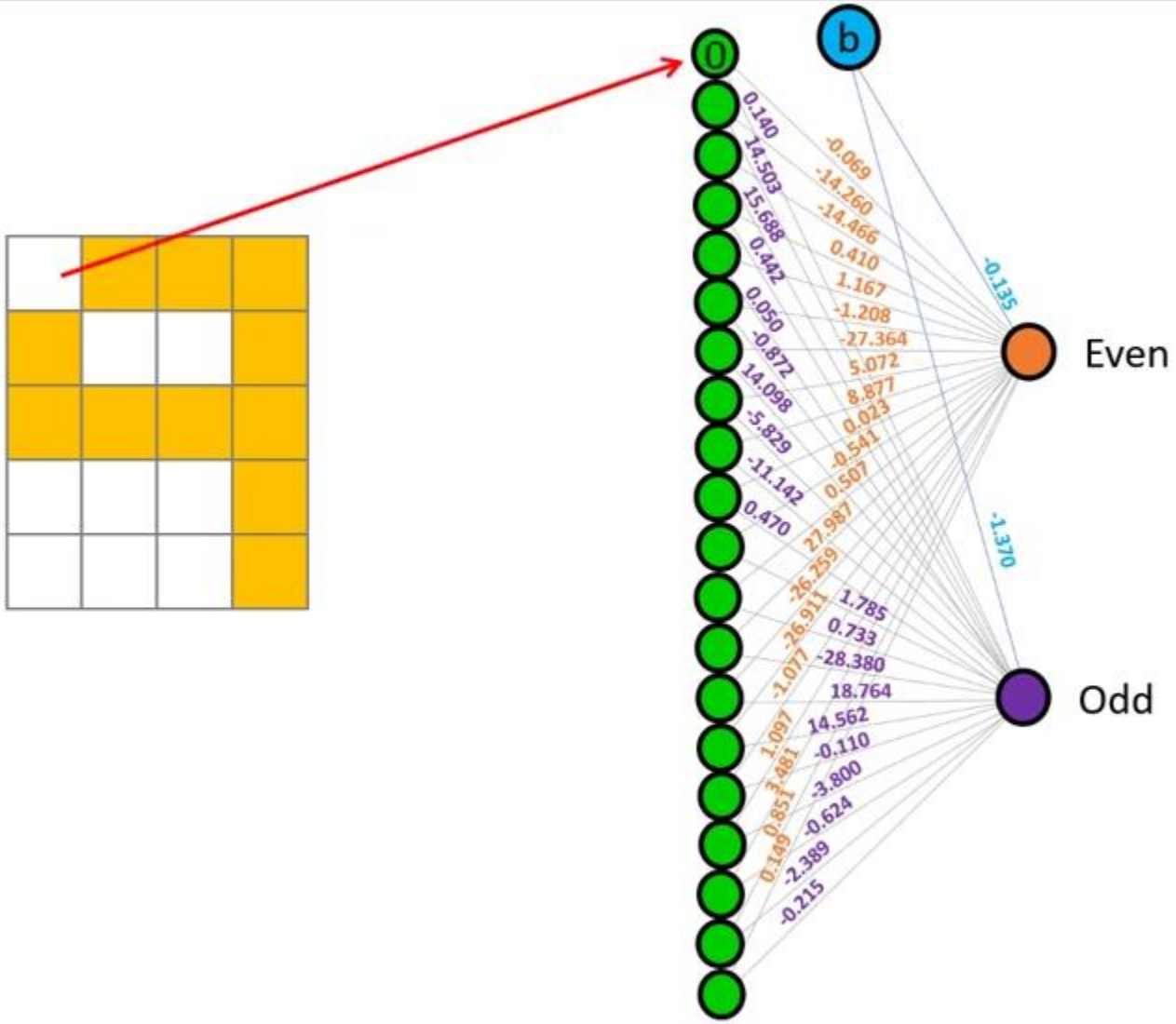
Predict





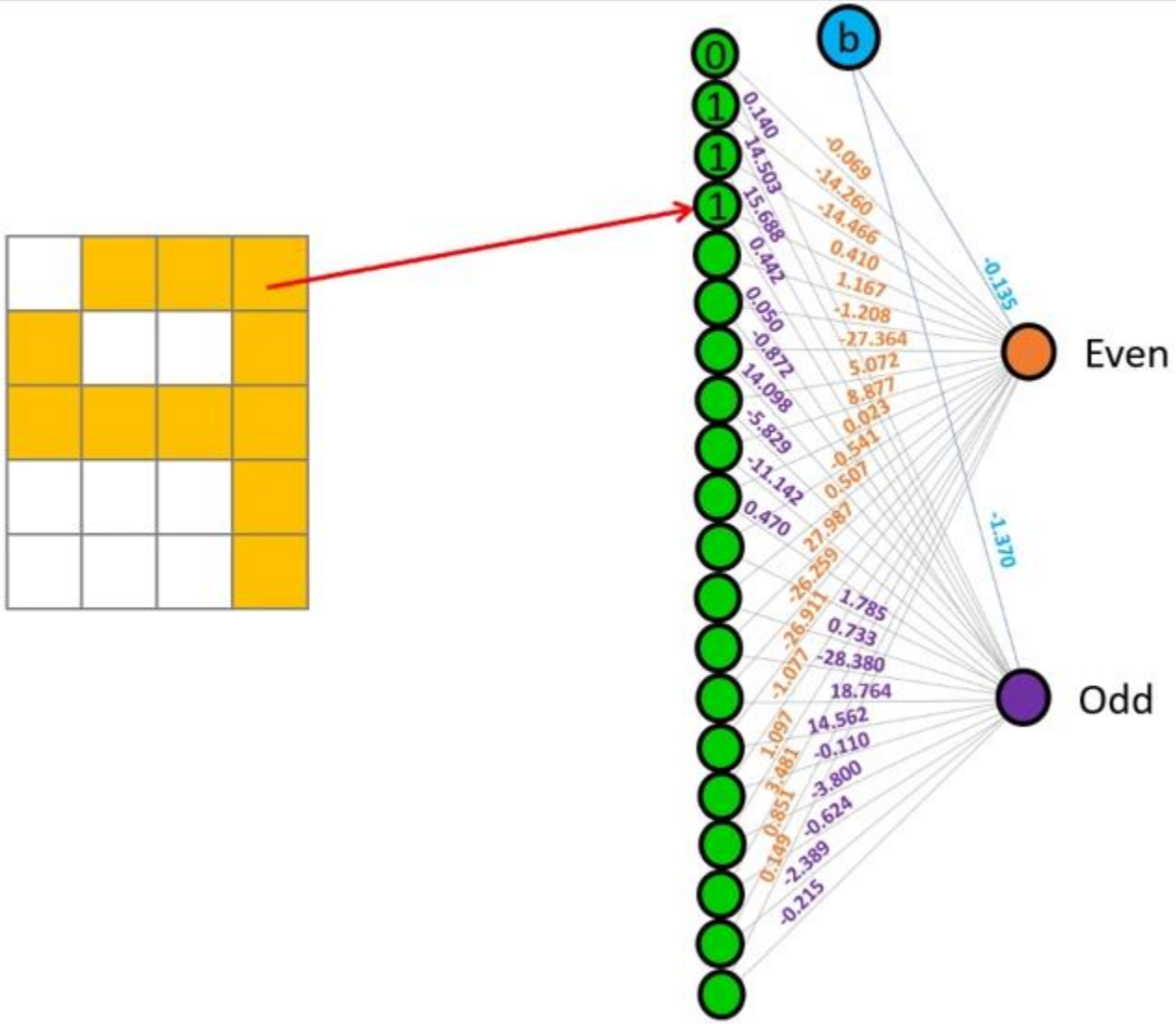
Suppose that we would use the network to predict if this image contains an odd or an even digit.

Predict

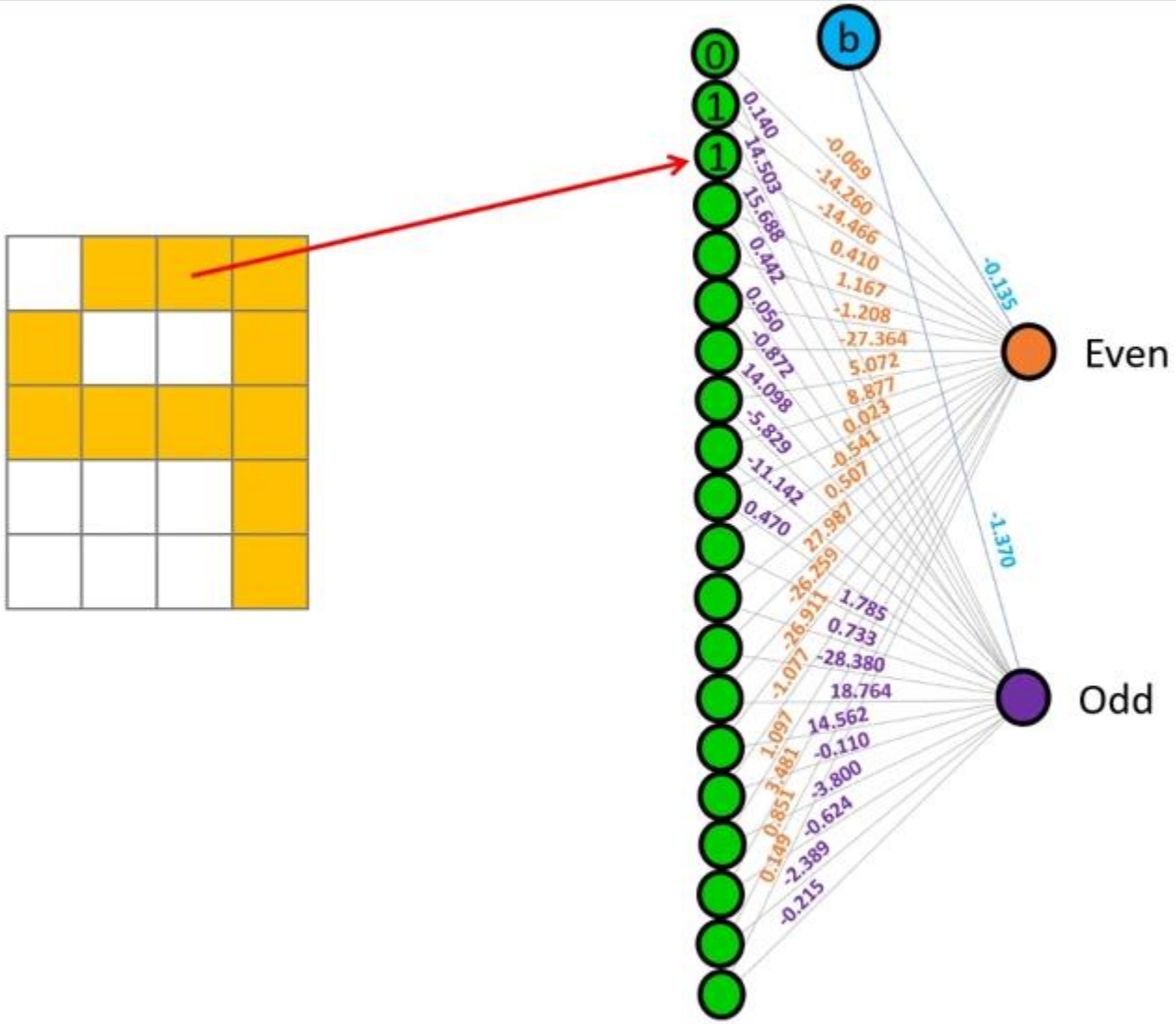


We therefore plug in the image and the corresponding pixels.

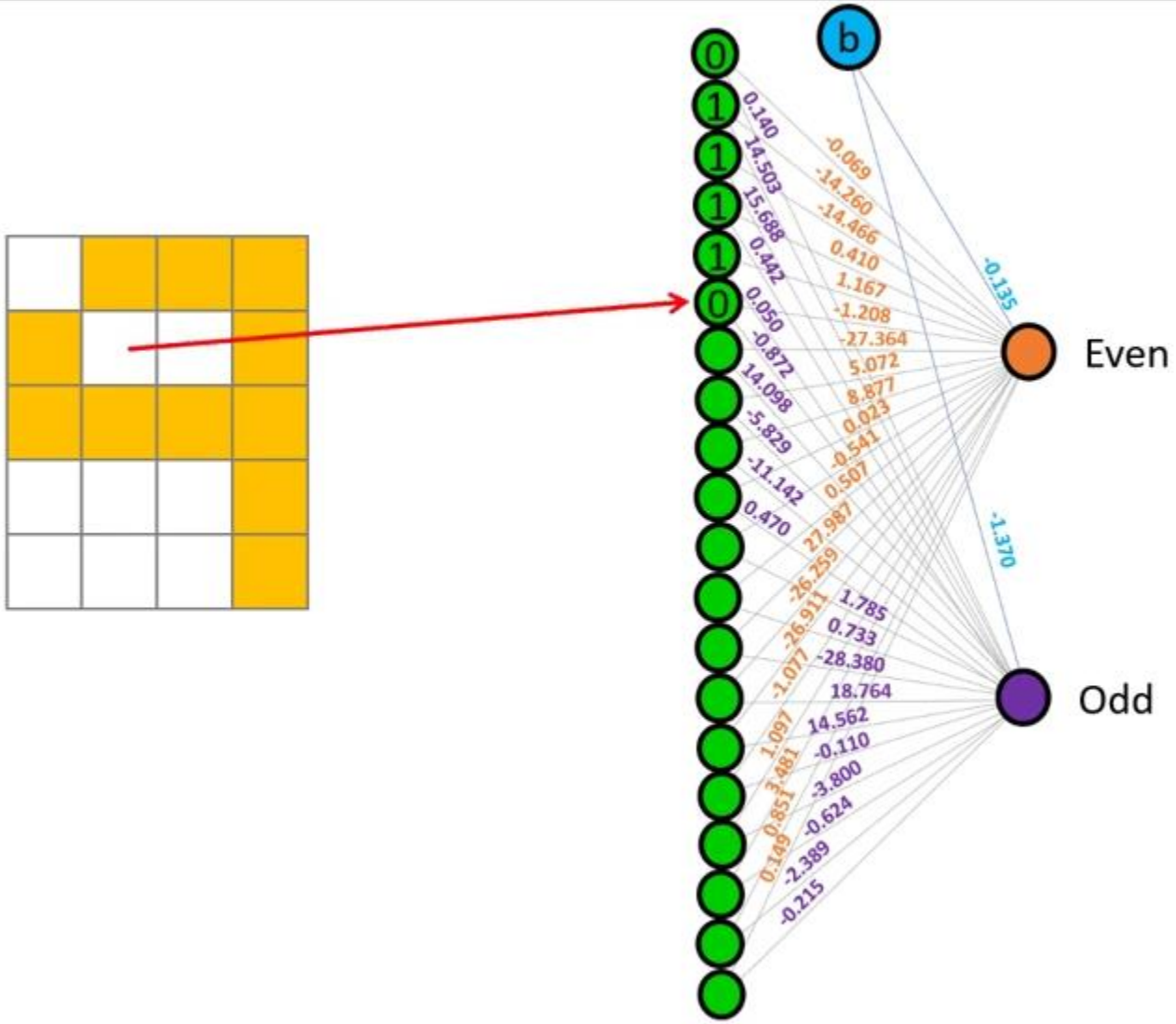
Predict


b
0
1
1
1
0.140
14.503
15.688
0.442
0.050
-0.872
14.098
-5.829
-11.142
0.470
-0.069
-14.260
-14.466
0.410
1.167
-1.208
-27.364
5.072
8.877
0.023
-0.541
0.507
27.987
-26.259
-26.911
-1.077
1.097
3.481
0.851
0.149
1.785
0.733
-28.380
18.764
14.562
-0.110
-3.800
-0.624
-2.389
-0.215
-0.135
-1.370
Even
Odd

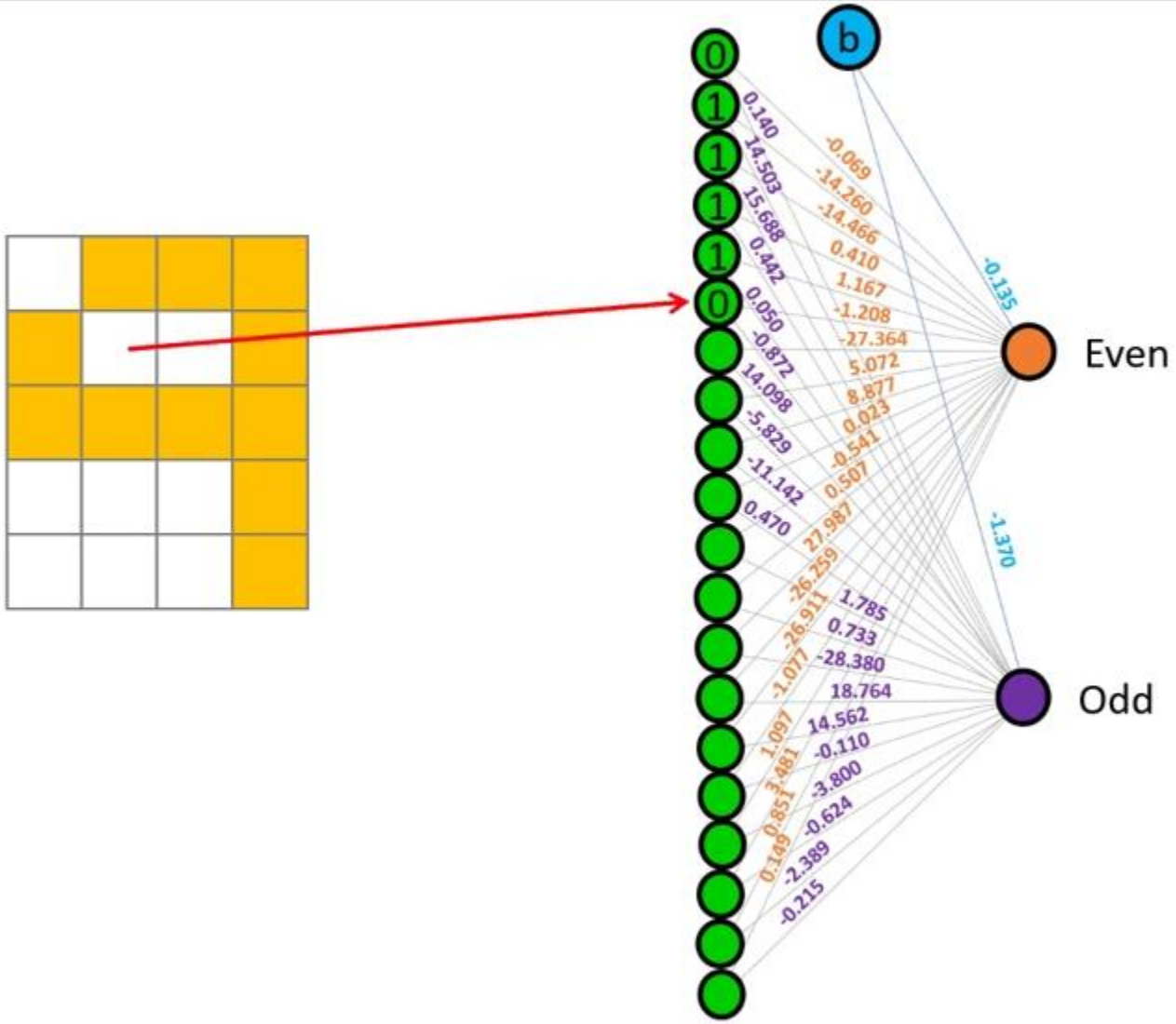
Predict


b
0
1
1
0.140
14.503
15.688
0.442
0.050
-0.872
14.098
-5.829
-11.142
0.470
-0.069
-14.260
-14.466
0.410
1.167
-1.208
-27.364
5.072
8.877
0.023
-0.541
0.507
27.987
-26.259
-26.911
-1.077
1.097
3.481
0.851
0.149
1.785
0.733
-28.380
18.764
14.562
-0.110
-3.800
-0.624
-2.389
-0.215
-0.135
-1.370
Even
Odd

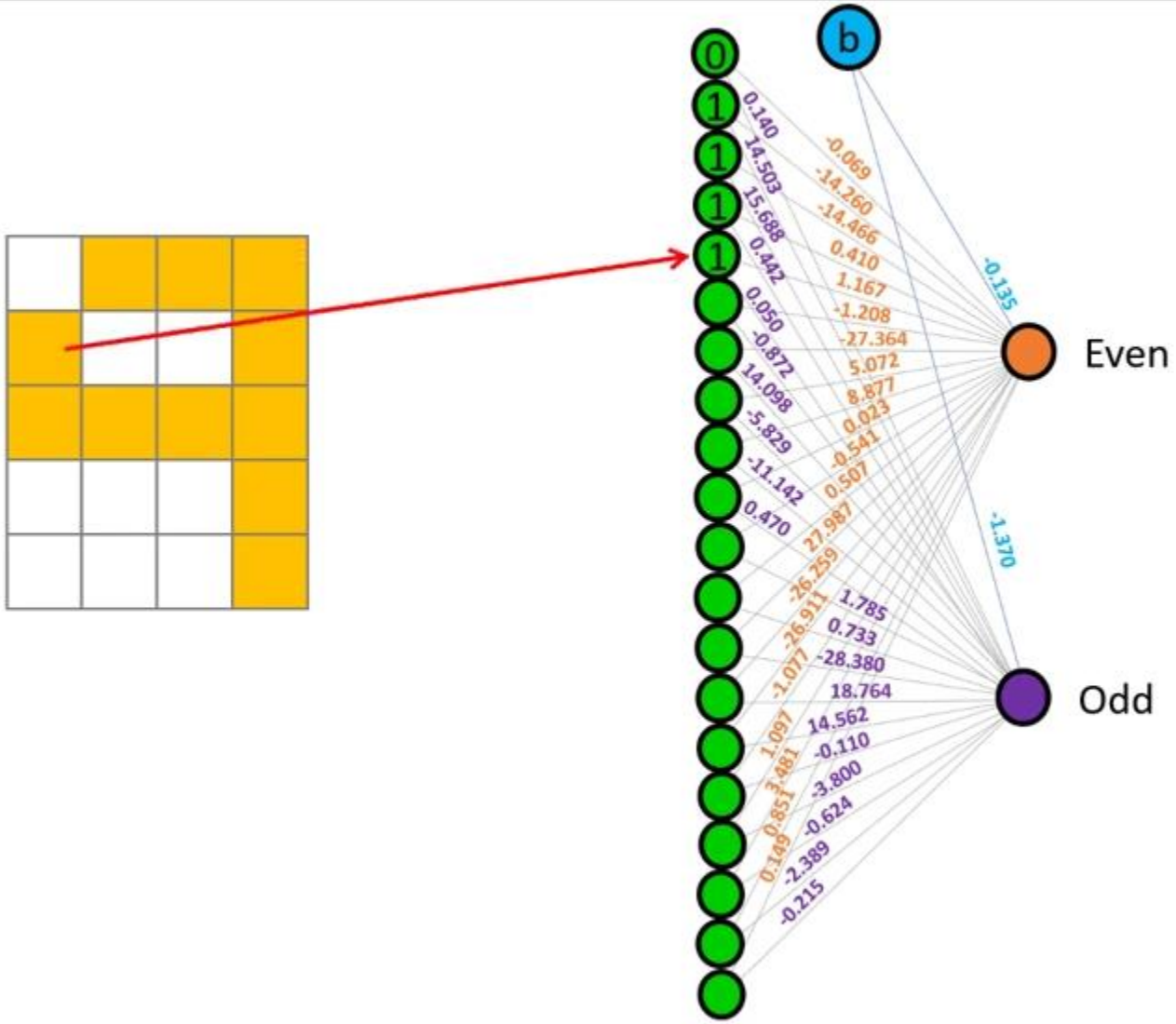
Predict


b
0
1
1
1
1
0
0.140
14.503
15.688
0.442
0.050
-0.872
14.098
-5.829
-11.142
0.470
-0.069
-14.260
-14.466
0.410
1.167
-1.208
-27.364
5.072
8.877
0.023
-0.541
0.507
27.987
-26.259
-26.911
-1.077
1.097
3.481
0.851
0.149
1.785
0.733
-28.380
18.764
14.562
-0.110
-3.800
-0.624
-2.389
-0.215
-0.135
-1.370
Even
Odd

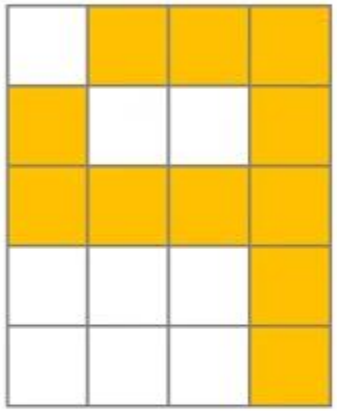
Predict

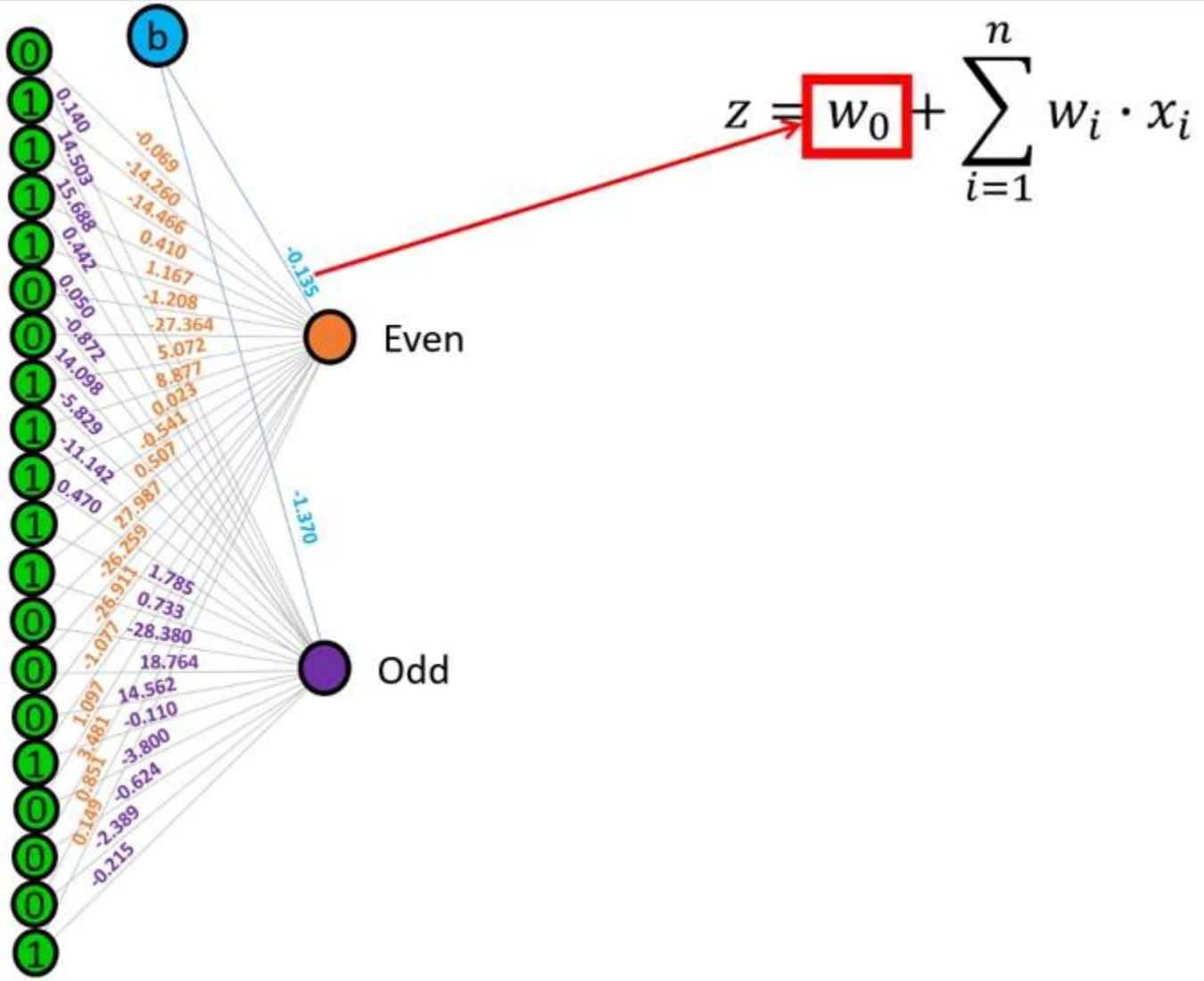

b
0
1
1
1
1
0
0.140
14.503
15.688
0.442
0.050
-0.872
14.098
-5.829
-11.142
0.470
-0.069
-14.260
-14.466
0.410
1.167
-1.208
-27.364
5.072
8.877
0.023
-0.541
0.507
27.987
-26.259
-26.911
-1.077
1.097
3.481
0.851
0.149
-0.135
-1.370
1.785
0.733
-28.380
18.764
14.562
-0.110
-3.800
-0.624
-2.389
-0.215
Even
Odd

Predict


b
0
1
1
1
1
0.140
14.503
15.688
0.442
0.050
-0.872
14.098
-5.829
-11.142
0.470
-0.069
-14.260
-14.466
0.410
1.167
-1.208
-27.364
5.072
8.877
0.023
-0.541
0.507
27.987
-26.259
-26.911
-1.077
1.097
3.481
0.851
0.149
1.785
0.733
-28.380
18.764
14.562
-0.110
-3.800
-0.624
-2.389
-0.215
-0.135
-1.370
Even
Odd

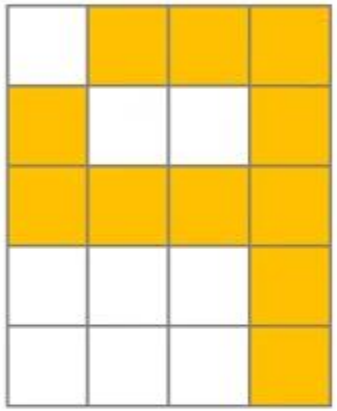
Predict

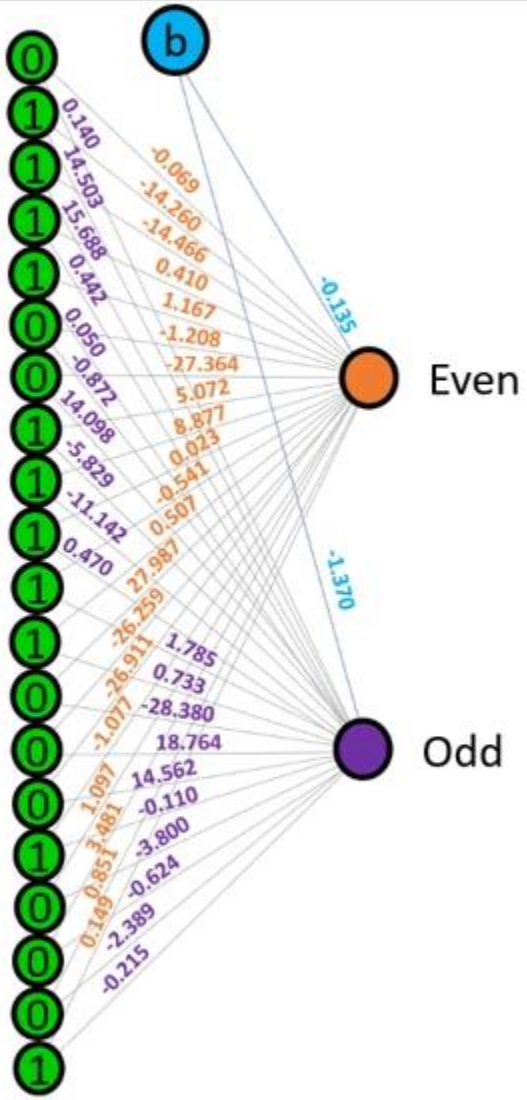




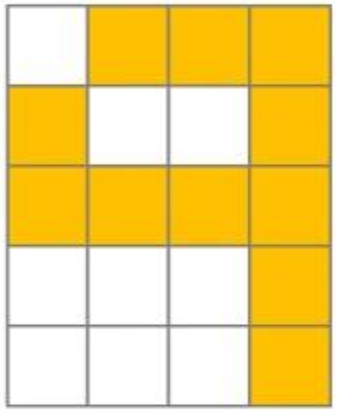
To calculate the probability that this is an even digit, we plug in the bias weight here,

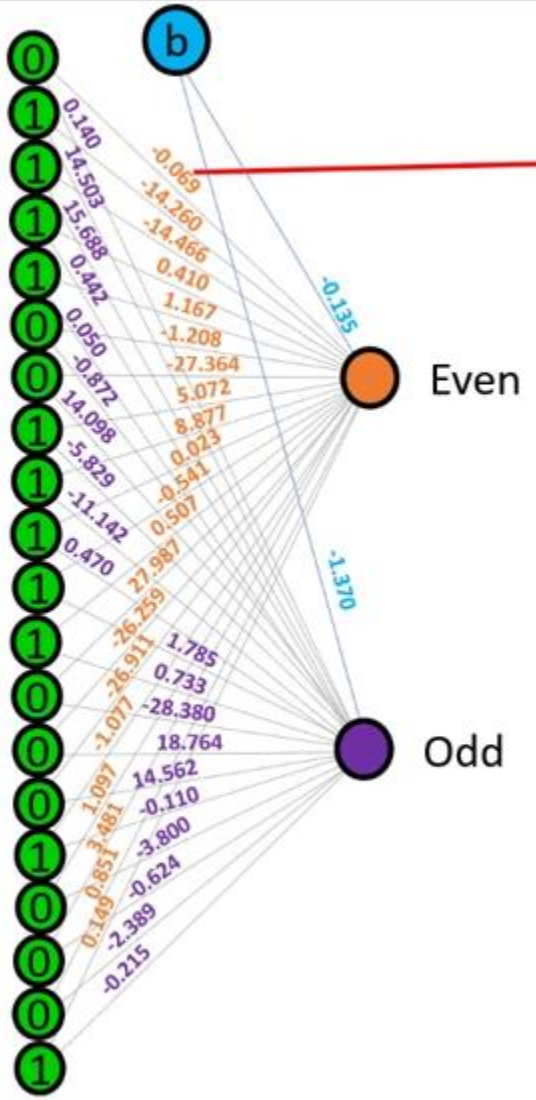
Predict




b
0
1
1
1
1
0
0
1
1
1
1
1
0
0
0
1
0
0
0
1
0.140
14.503
15.688
0.442
0.050
-0.872
14.098
-5.829
-11.142
0.470
-0.069
-14.260
-14.466
0.410
1.167
-1.208
-27.364
5.072
8.877
0.023
-0.541
0.507
27.987
-26.259
-26.911
-1.077
1.097
3.481
0.851
0.149
1.785
0.733
-28.380
18.764
14.562
-0.110
-3.800
-0.624
-2.389
-0.215
-0.135
-1.370
Even
Odd

Predict

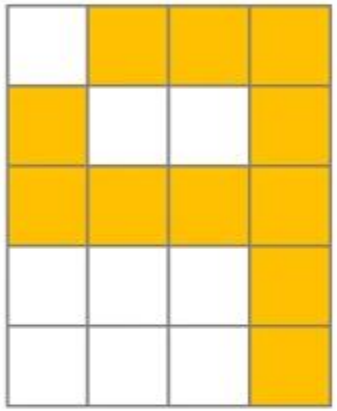


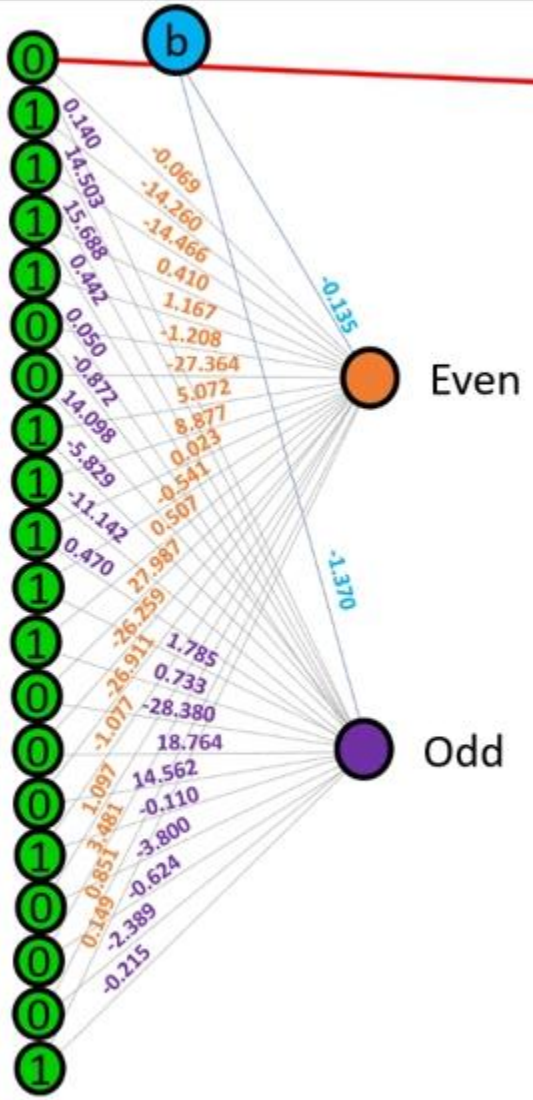


$$z = -0.135 + w_1 \cdot x_1 +$$

the weight associated with the pixel on the first row, first column here,

Predict

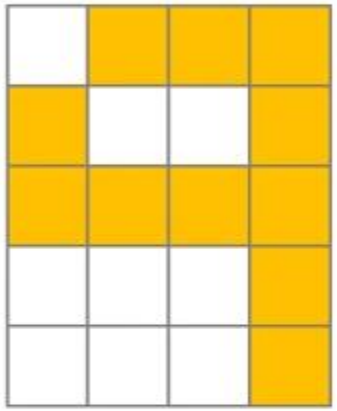


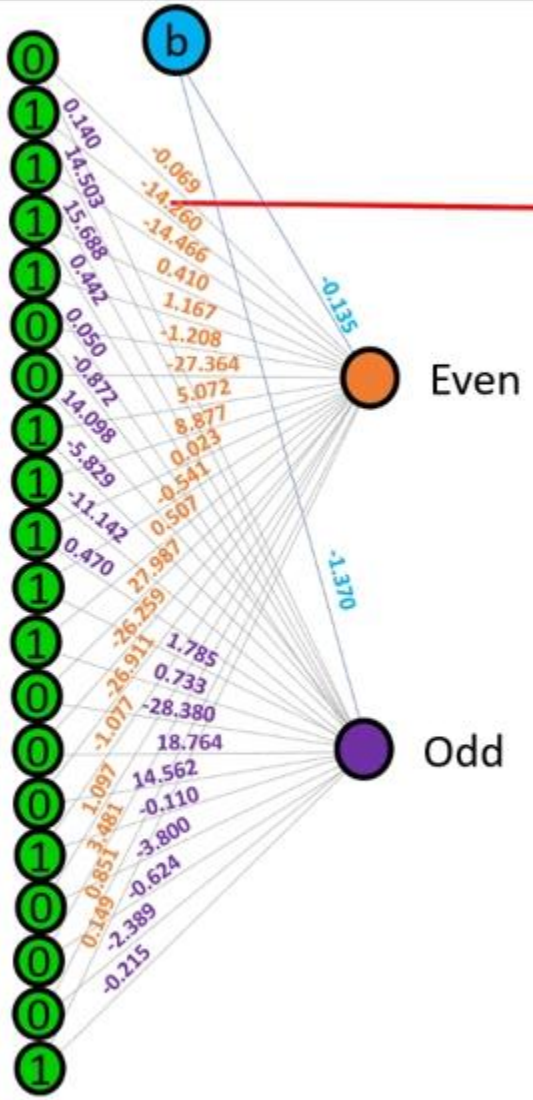


$$z = -0.135 + (-0.069)x_1 +$$

and the corresponding value in this pixel.

Predict

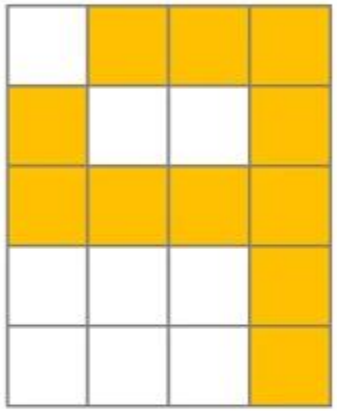


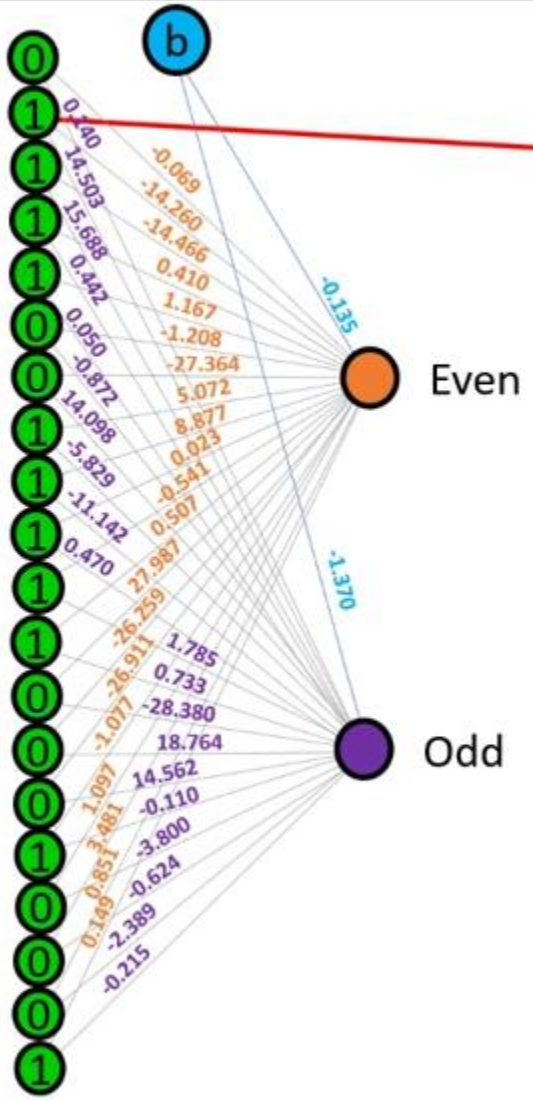


$$z = -0.135 + (-0.069) \cdot 0 + (-14.260) \cdot 1 +$$

Then we plug in the weight,

Predict

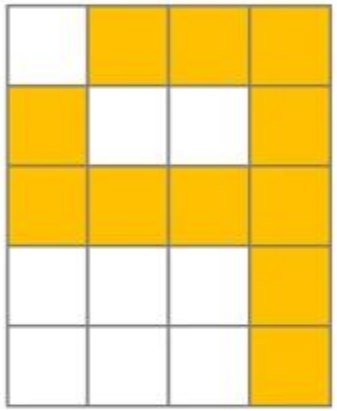


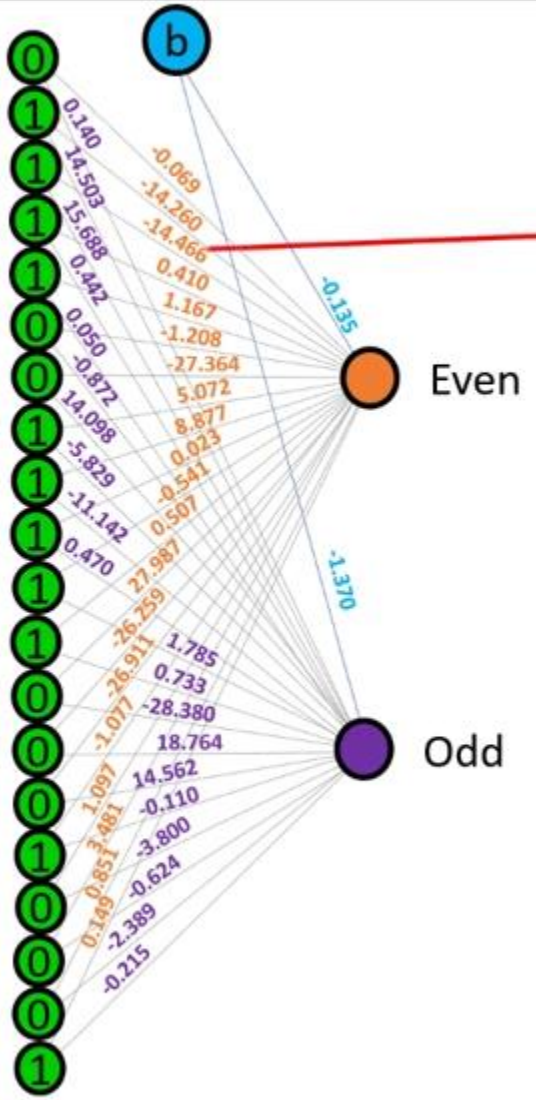


$$z = -0.135 + (-0.069) \cdot 0 + (-14.260) \boxed{1} +$$

and the input value of the second node,

Predict

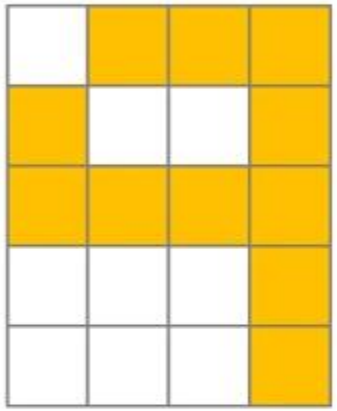


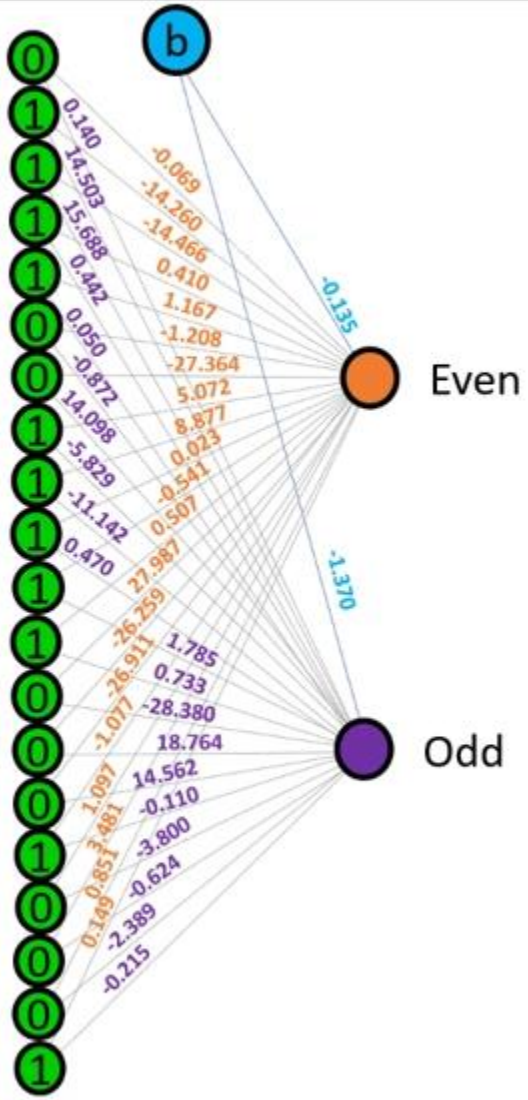


$$z = -0.135 + (-0.069) \cdot 0 + (-14.260) \cdot 1 + (-14.666) \cdot 1$$

then for the third node and so forth.

Predict

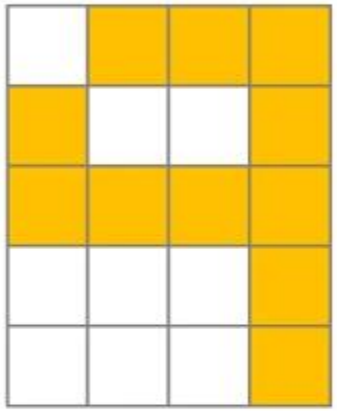


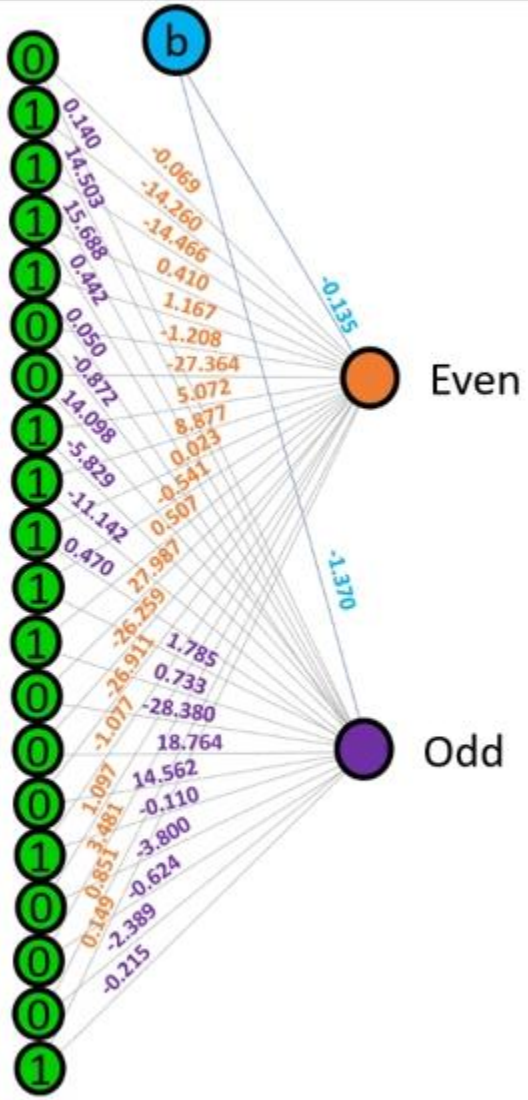


$$z = -0.135 + (-0.069) \cdot 0 + (-14.260) \cdot 1 + (-14.666) \cdot 1 + \cdots + 0.149 \cdot 1 = -14.275$$

If we sum all these terms,

Predict



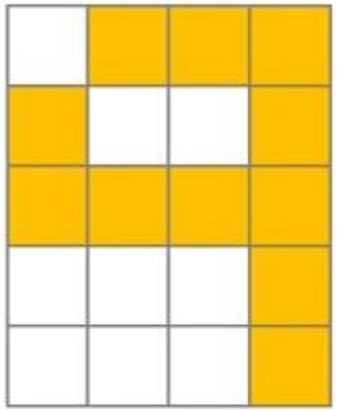


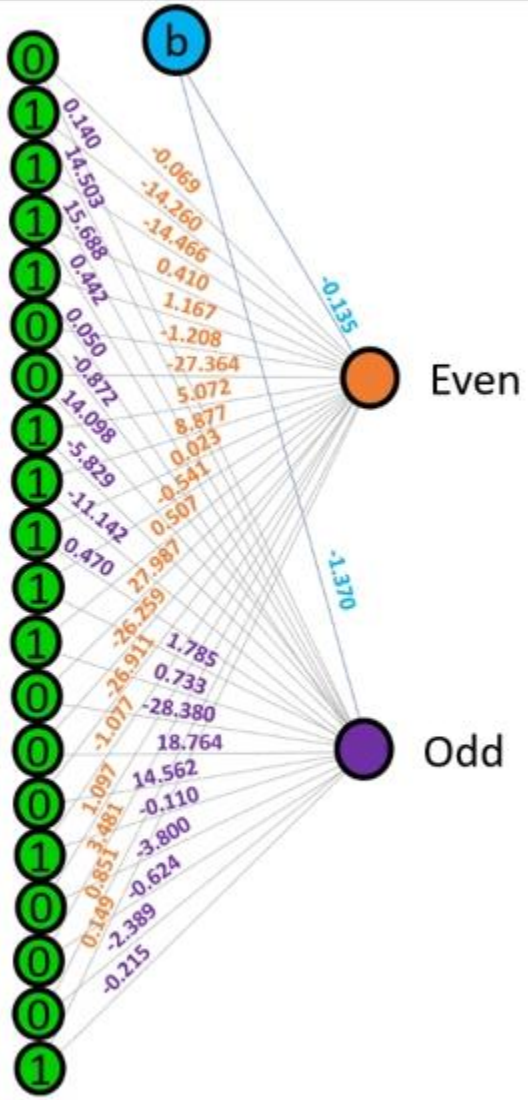
$$z = -0.135 + (-0.069) \cdot 0 + (-14.260) \cdot 1 + (-14.666) \cdot 1 + \cdots + 0.149 \cdot 1 = -14.275$$

$$f(z) = \frac{1}{1 + e^{-(-14.275)}}$$

and plug in this sum in the activation function, and do the math,

Predict



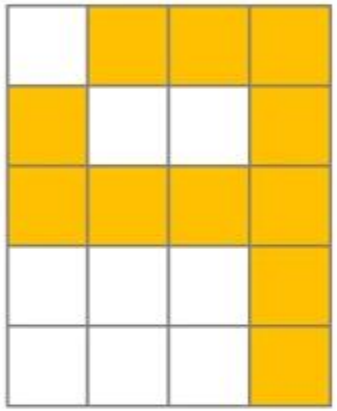


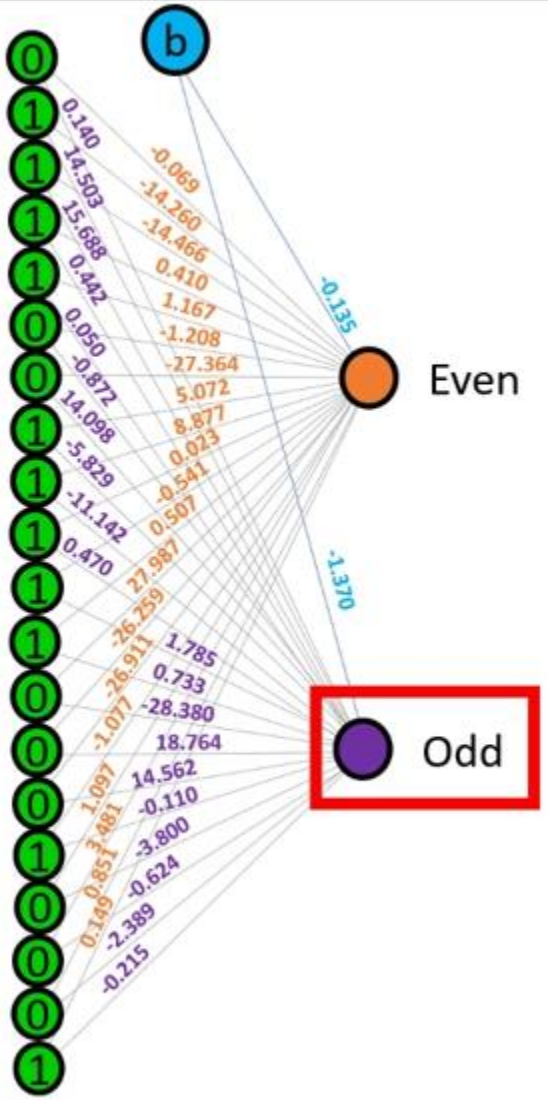
$$z = -0.135 + (-0.069) \cdot 0 + (-14.260) \cdot 1 + (-14.666) \cdot 1 + \cdots + 0.149 \cdot 1 = -14.275$$

$$f(z) = \frac{1}{1 + e^{-(-14.275)}} \approx \boxed{0}$$

we will get a value that is approximately equal to zero.

Predict



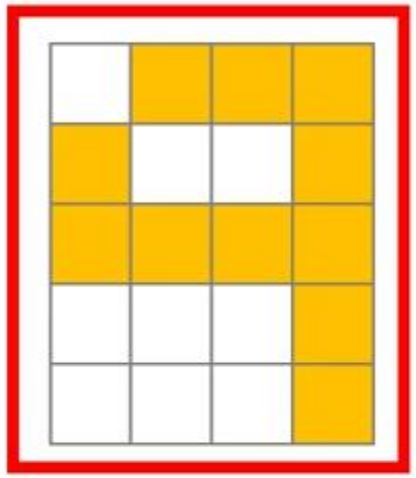


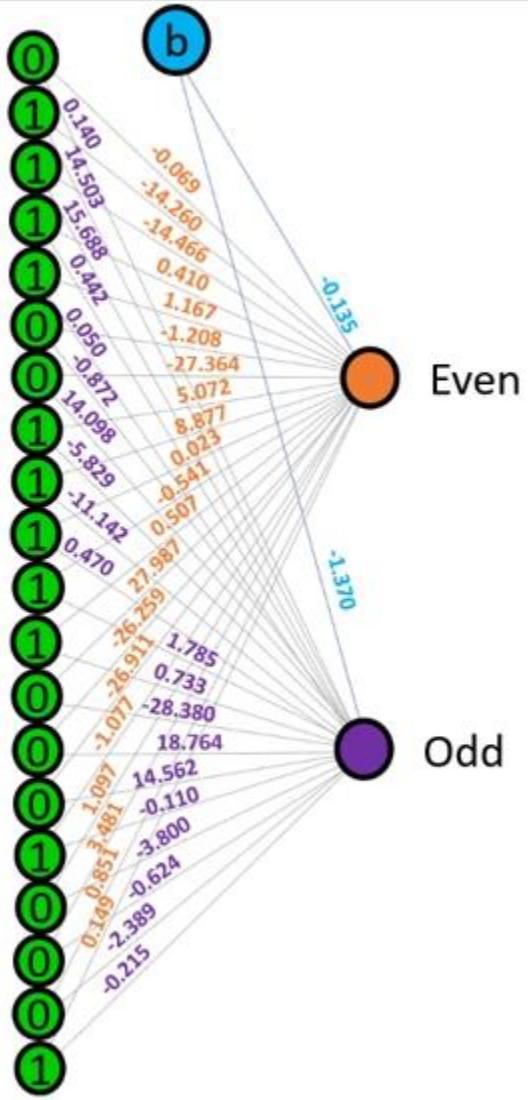
$$z = -1.370 + 0.140 \cdot 0 + 14.503 \cdot 1 + 15.688 \cdot 1 + \cdots + (-0.215) \cdot 1 = 15.005$$

$$f(z) = \frac{1}{1 + e^{-(15.005)}} \approx 1$$

If we instead calculate the corresponding output value for an odd number, we see that the value is approximately equal to one.

Predict





$$z = -1.370 + 0.140 \cdot 0 + 14.503 \cdot 1 + 15.688 \cdot 1 + \cdots + (-0.215) \cdot 1 = 15.005$$

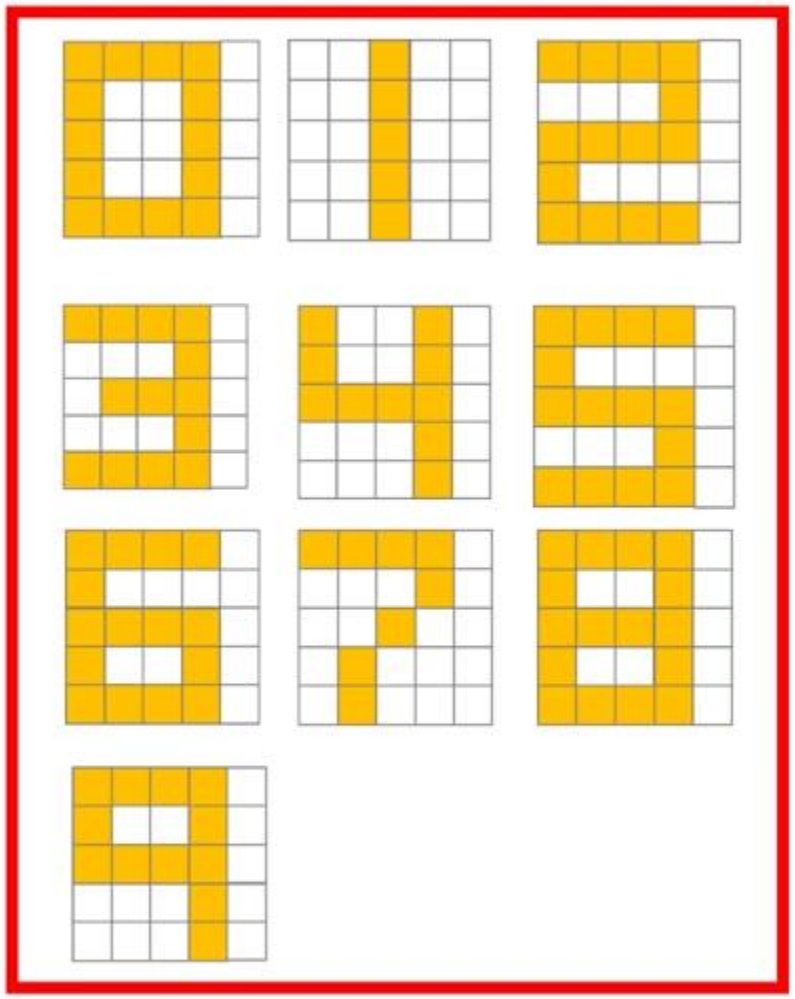
$$f(z) = \frac{1}{1 + e^{-(15.005)}} \approx 1$$

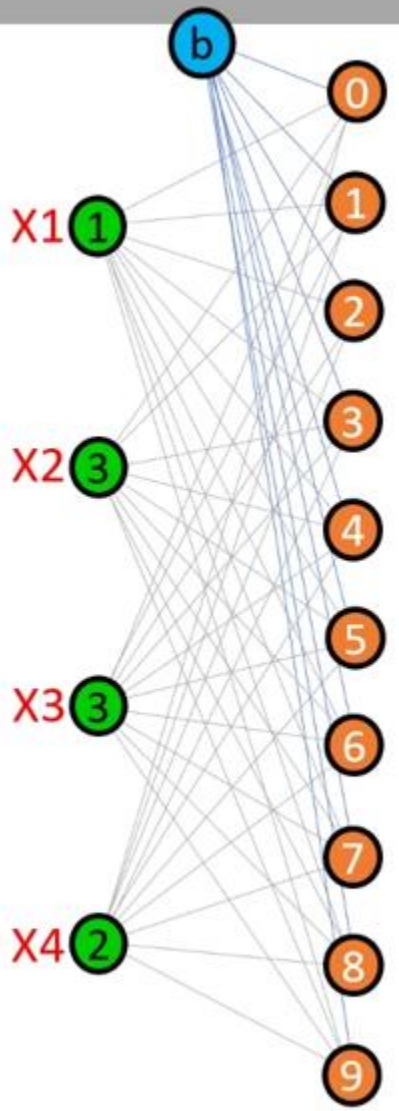
Since the output value for an odd digit is greater than the one for an even digit, the network will predict that the corresponding image contains an odd digit.

Convolutional Neural Networks

We will now have a look at convolutional neural networks, which are special neural networks that have been developed to improve image recognition.

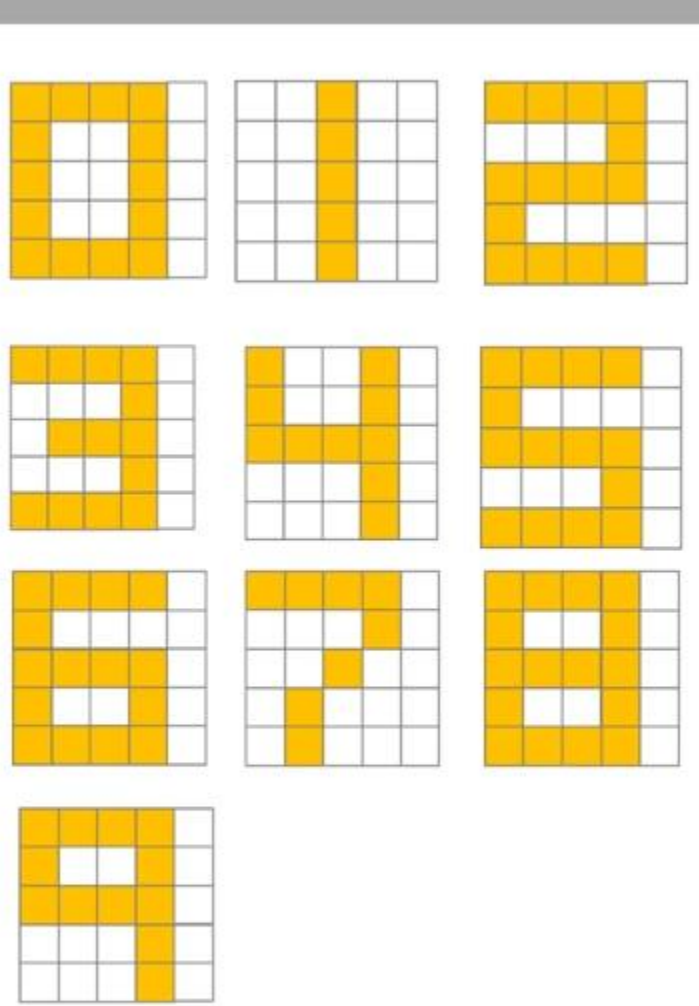
CNN

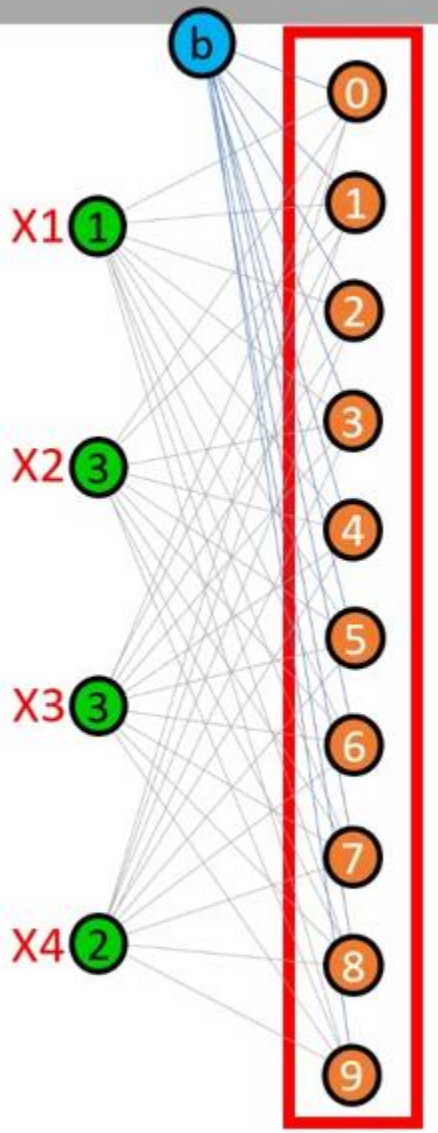




To illustrate how a CNN works, we will use the following 10 images where each image represents a digit and has 25 pixels.

CNN





We will here try to predict which digit that is included in the image.

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

To explain the basics of a CNN, let’s have a closer look at the image that represents a zero.

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Filter =

1	0
0	2

Filter = Kernel

In CNN one uses a so-called filter, also called a kernel.

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Filter =

1	0
0	2

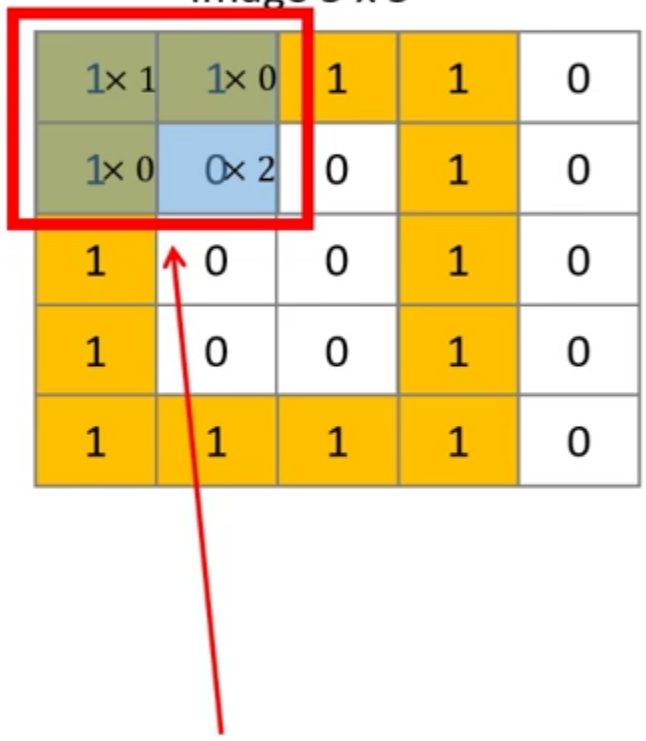
Filter = Kernel

In this example, the filter is a 2 by 2 matrix with some numbers. The numbers used in this filter have been optimized during the training process. The following numbers are therefore optimal for this simple example.

CNN

Image 5 x 5

1× 1	1× 0	1	1	0
1× 0	0× 2	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0



Filter =

1	0
0	2

The filter is then placed on top of the image where we perform a dot product,

CNN

Image 5 x 5

1× 1	1× 0	1	1	0
1× 0	0× 2	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$1 \cdot \mathbf{1} +$

Filter =

1	0
0	2

which means that we multiply the values in the overlapping elements, like this.

CNN

Image 5 x 5

1× 1	1× 0	1	1	0
1× 0	0× 2	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} +$

Filter =

1	0
0	2

CNN

Image 5 x 5

1× 1	1× 0	1	1	0
1× 0	0× 2	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$$1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 1 \cdot \mathbf{0} +$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1× 1	1× 0	1	1	0
1× 0	0× 2	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$$1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{2}$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1× 1	1× 0	1	1	0
1× 0	0× 2	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1			

$$1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$$

Filter =

1	0
0	2

Then we sum these products and add the value to a so-called feature map.

CNN

Image 5 x 5

1	1× 1	1× 0	1	0
1	0× 0	0× 2	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1			

Filter =

1	0
0	2

Next, we slide the filter one step to the right.

CNN

Image 5 x 5

1	1× 1	1× 0	1	0
1	0× 0	0× 2	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1			

Stride = 1

Filter =

1	0
0	2

How far we move the filter is referred to as the stride length. Since we here move the filter one pixel to the right, the stride length is equal to one.

CNN

Image 5 x 5

1	1× 1	1× 0	1	0
1	0× 0	0× 2	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1		

$$1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$$

Filter =

1	0
0	2

Then, we do the same calculations again,

CNN

Image 5 x 5

1	1	1× 1	1× 0	0
1	0	0× 0	1× 2	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	

$$1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{2} = 3$$

Filter =

1	0
0	2

and continue until we have moved the filter over the entire image.

CNN

Image 5 x 5

1	1	1	1× 1	0× 0
1	0	0	1× 0	0× 2
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1

$$1 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1	1	1	1	0
1× 1	0× 0	0	1	0
1× 0	0× 2	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1			

$$1 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1	1	1	1	0
1	0× 1	0× 0	1	0
1	0× 0	0× 2	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0		

$$0 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 0$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1	1	1	1	0
1	0	0× 1	1× 0	0
1	0	0× 0	1× 2	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	

$$0 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{2} = 2$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1× 1	0× 0
1	0	0	1× 0	0× 2
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1

$$1 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1× 1	0× 0
1	0	0	1× 0	0× 2
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1

$$1 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1× 1	0× 0	0	1	0
1× 0	0× 2	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1			

$$1 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0× 1	0× 0	1	0
1	0× 0	0× 2	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0		

$$0 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 0$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0× 1	1× 0	0
1	0	0× 0	1× 2	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	

$$0 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{2} = 2$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1× 1	0× 0
1	0	0	1× 0	0× 2
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1

$$1 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1× 1	0× 0	0	1	0
1× 0	1× 2	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3			

$$1 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 1 \cdot \mathbf{2} = 3$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0× 1	0× 0	1	0
1	1× 0	1× 2	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2		

$$0 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 1 \cdot \mathbf{2} = 2$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0× 1	1× 0	0
1	1	1× 0	1× 2	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	

$$0 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 1 \cdot \mathbf{2} = 2$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1× 1	0× 0
1	1	1	1× 0	0× 2

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

$$1 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$$

Filter =

1	0
0	2

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Convolution

Filter =

1	0
0	2

This procedure is called convolution, which is used to detect local features in the image.

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

ReLU

$$f(x) = \max(0, x)$$

Sigmoid

$$f(x) = \frac{1}{1 + e^{-x}}$$

Filter =

1	0
0	2

Then we apply some activation functions on the feature map, such as the ReLU function or the sigmoid function.

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

ReLU

$$f(x) = \max(0, x)$$

Filter =

1	0
0	2

For simplicity, I will here use the ReLU function.

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

ReLU

$$f(x) = \max(0, x)$$

Filter =

1	0
0	2

The ReLU function takes a value,

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

ReLU

$$f(x) = \max(0, x)$$

Filter =

1	0
0	2

and returns the maximum value between zero and the value that we plug in.

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

ReLU

$$f(1) = \max(0, 1)$$

Filter =

1	0
0	2

Suppose that we plug in this value here.

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

ReLU

$$f(x) = \max(0, x)$$

Filter =

1	0
0	2

The ReLU function basically sets all negative values to zero and leaves positive values unchanged. So, for any positive input value, the output of the ReLU function is the same as the input value.

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

ReLU

$$f(x) = \max(0, x)$$

Filter =

1	0
0	2

Since we only have positive values in our feature map, the ReLU function does not change any of the values.

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map after ReLU

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Filter =

1	0
0	2

Next, we may perform pooling. In this example, we use a 2 by 2 window and use max pooling which means that we take the maximum value of the four numbers in the window,

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map after ReLU

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	

Filter =

1	0
0	2

and place this number here.

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map after ReLU

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3

Filter =

1	0
0	2

Then we move the window to these four elements in the feature map and place the corresponding maximum number in the pooled feature map.

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map after ReLU

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3

Filter =

1	0
0	2

Stride = 2

Since we now move the pooling window two steps to the right, the stride length is here equal to two. Per default, the stride length is usually set to the size of the pooling window.

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map after ReLU

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	

Filter =

1	0
0	2

Then we move the pooling window two steps down,

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map after ReLU

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Filter =

1	0
0	2

and then two steps to the right.

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map after ReLU

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3

Filter =

1	0
0	2

Finally, we flatten the pooled feature map into a 1D array,

CNN

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map after ReLU

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2

Filter =

1	0
0	2

like this.

CNN

1	3	3	2

Image 5 x 5

0	0	1	0	0
0	0	1	0	0
0	0	1	0	0
0	0	1	0	0
0	0	1	0	0

Filter =

1	0
0	2

Now, suppose that we would have the following image that represents a one.

CNN

1	3	3	2

Image 5 x 5

0	0	1	0	0
0	0	1	0	0
0	0	1	0	0
0	0	1	0	0
0	0	1	0	0

Feature map

0	2	1	0
0	2	1	0
0	2	1	0
0	2	1	0

Filter =

1	0
0	2

After performing convolution of the image, that would result in the following feature map,

CNN

1	3	3	2

Image 5 x 5

0	0	1	0	0
0	0	1	0	0
0	0	1	0	0
0	0	1	0	0
0	0	1	0	0

Feature map after ReLU

0	2	1	0
0	2	1	0
0	2	1	0
0	2	1	0

Pooled feature map

2	1
2	1

Flatten

2	1	2	1

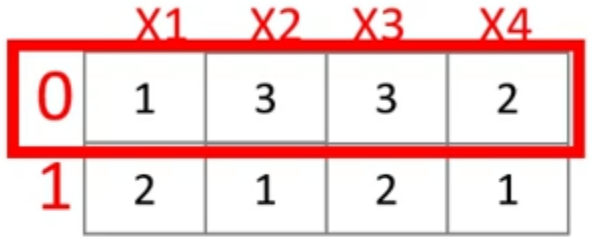
Filter =

1	0
0	2

and the following pooled feature map.

CNN

Input data



This is the pooled feature map of the image that contains a zero,

CNN

Input data

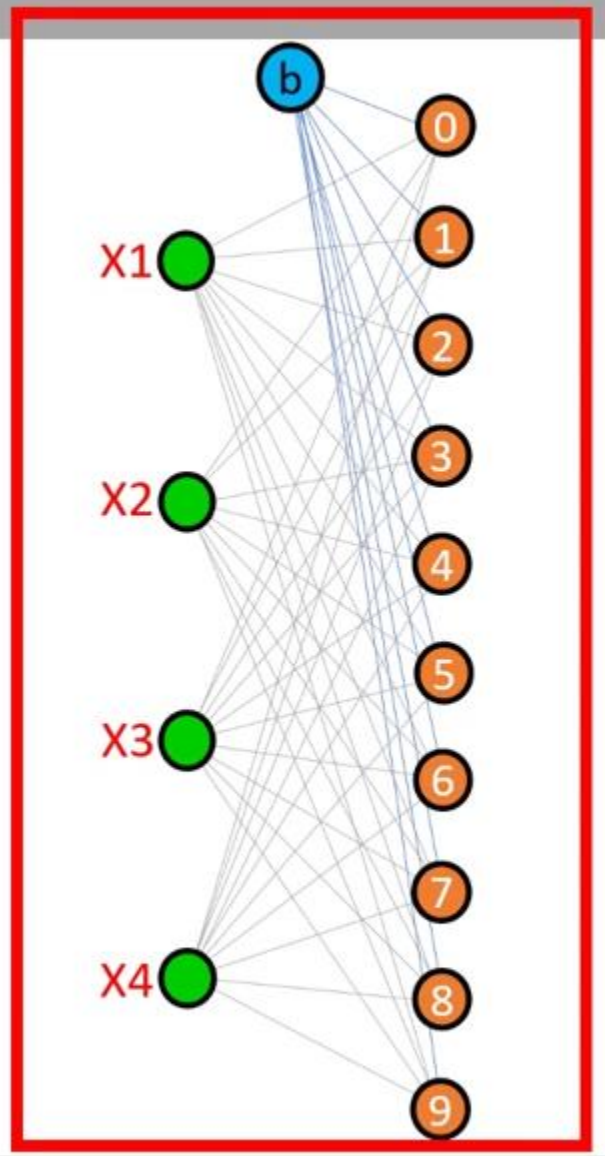
	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3

If we would compute the same thing for the other eight images, we will have the following data.

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3

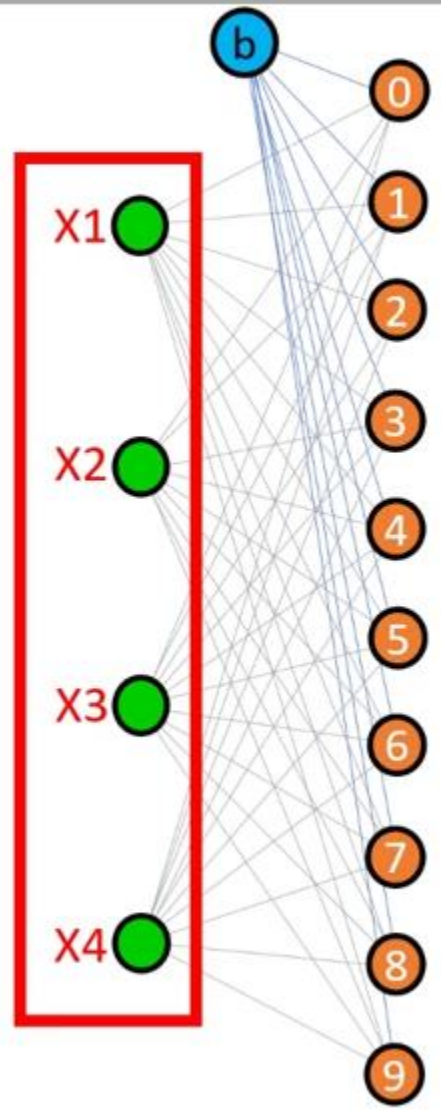


Next, we create a fully connected network. To simplify the illustrations and calculations, I have not included any hidden layers in this network.

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3

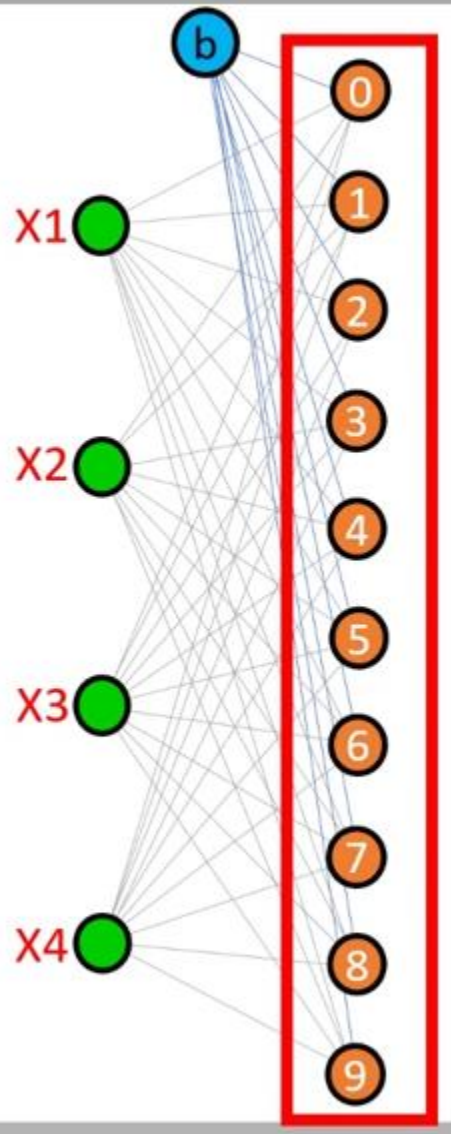


Since our pooled feature maps have four values for each image, the network will have only four input nodes. In comparison, using a simple artificial neural network directly on the image would require 25 input nodes, one for each pixel.

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3

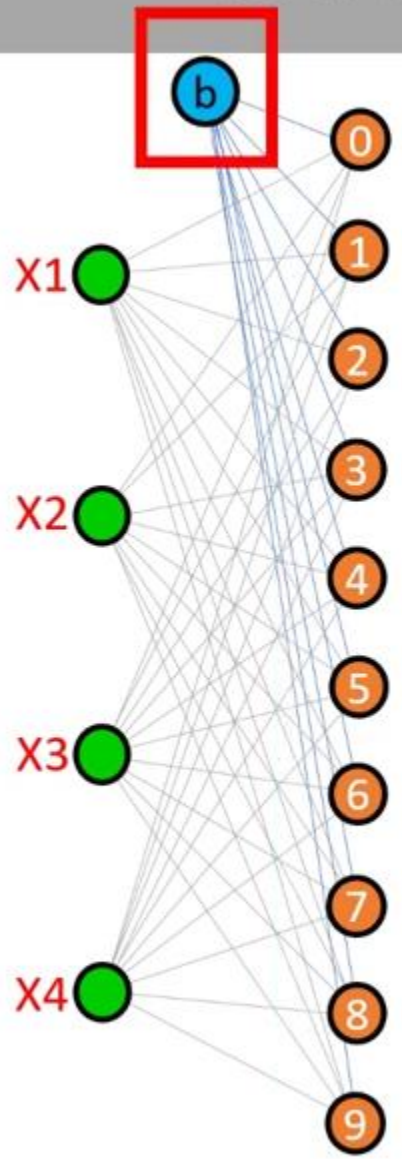


Since we like to predict 10 different digits, the network will, in this example, have 10 output nodes.

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3

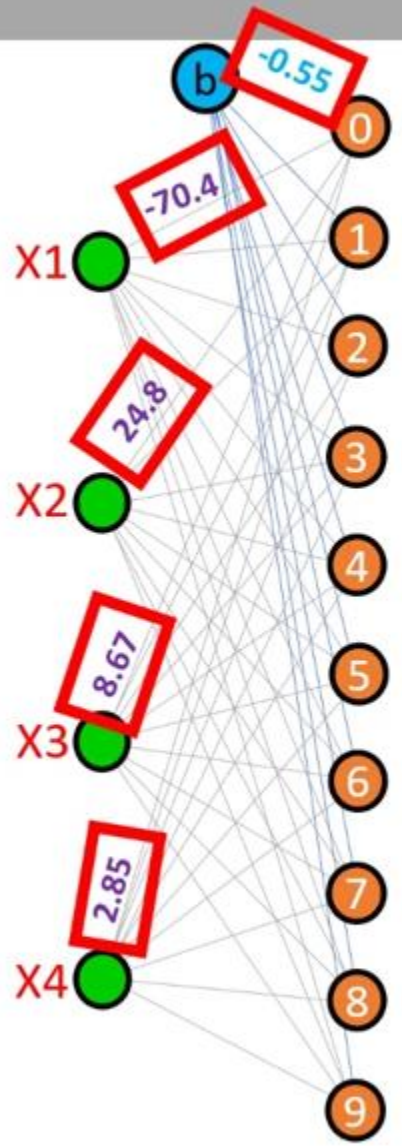


Remember that this is the bias.

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3

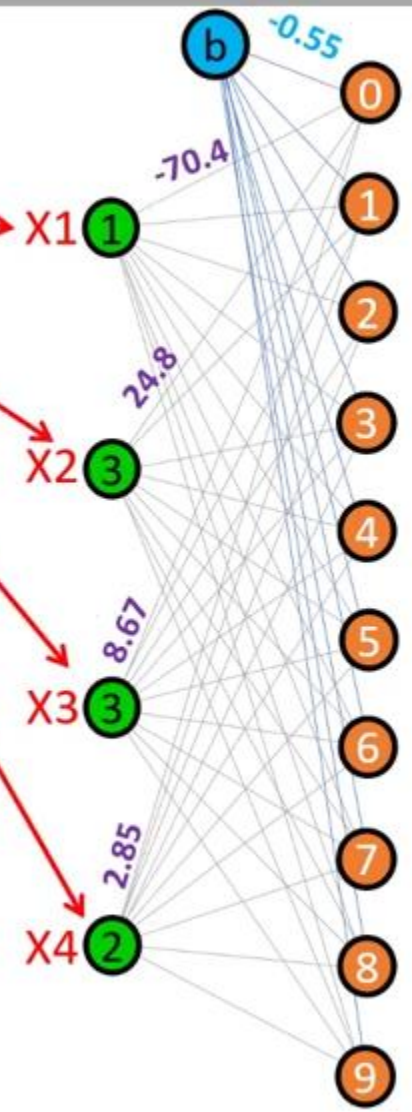


For example, these are the optimal values of the weights to predict if the image contains a zero.

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3

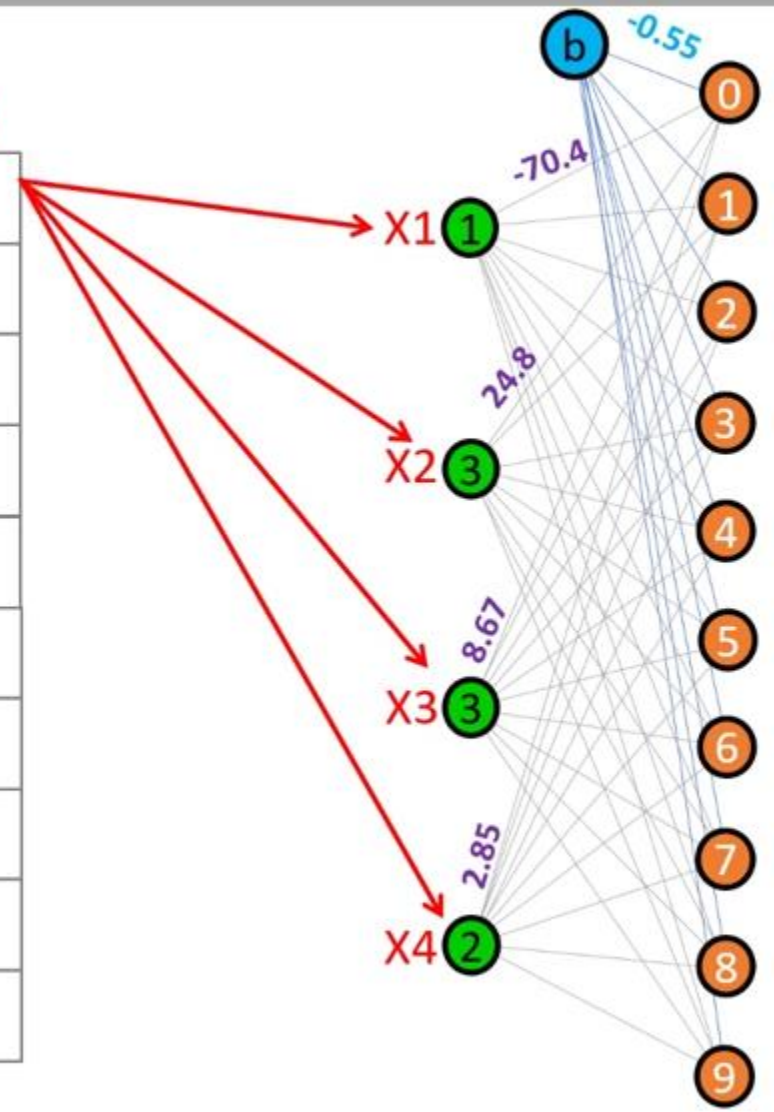


Let's see if the first image is correctly classified as a zero. We plug in the corresponding values from the pooled feature map in the input nodes,

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3



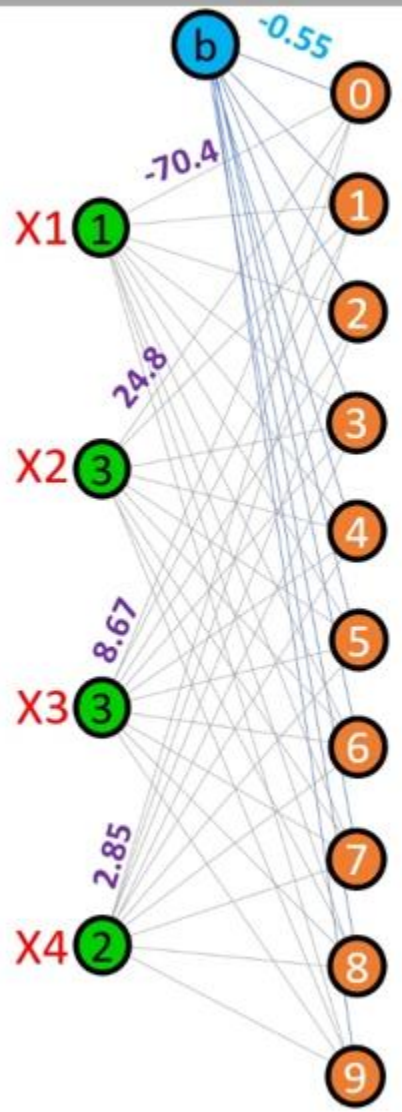
$$z = -0.55 + (-70.4) \cdot 1 + 24.8 \cdot 3 + 8.67 \cdot 3 + 2.85 \cdot 2 = 35.16$$

and do the same calculations that we did in our previous example.

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3



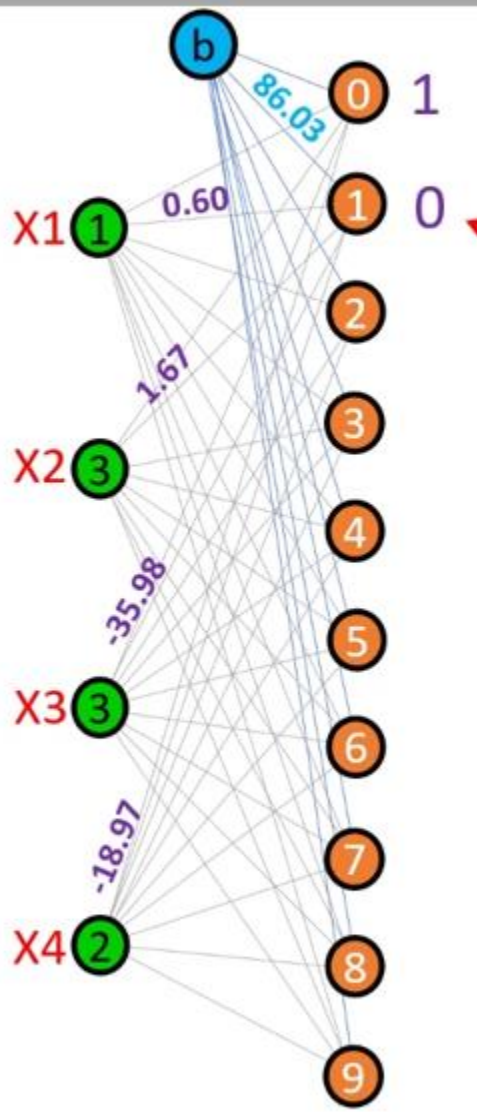
$$z = -0.55 + (-70.4) \cdot 1 + 24.8 \cdot 3 + 8.67 \cdot 3 + 2.85 \cdot 2 = 35.16$$

$$f(z) = \frac{1}{1 + e^{-(35.16)}} = 1$$

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3



$$z = 86.03 + 0.60 \cdot 1 + 1.67 \cdot 3 + (-35.98) \cdot 3 + (-18.97) \cdot 2 = -54.24$$

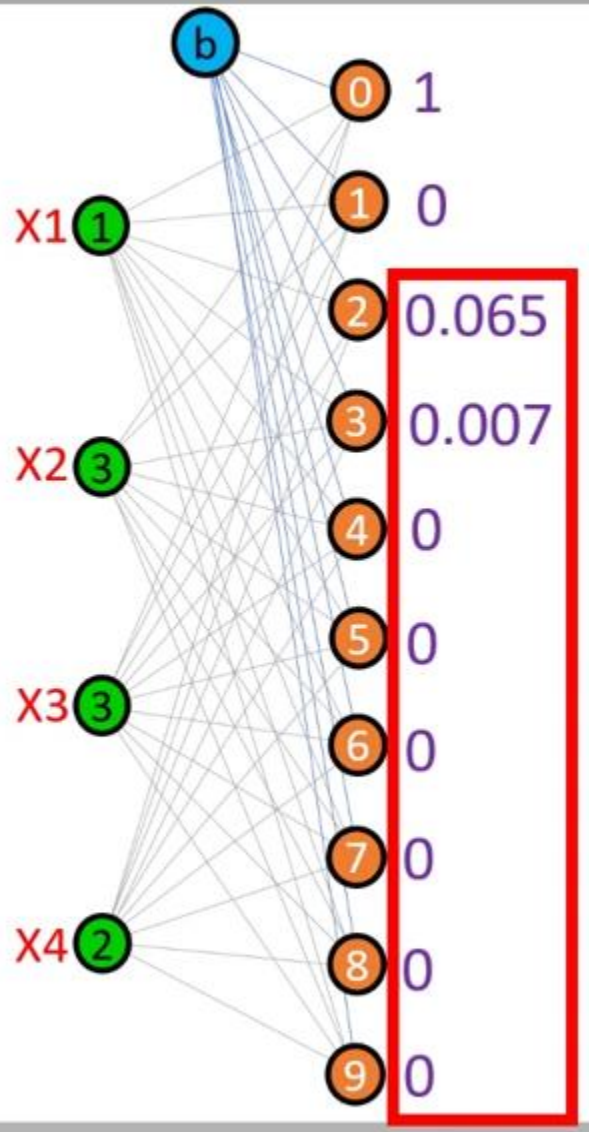
$$f(z) = \frac{1}{1 + e^{-(-54.24)}} = 0$$

Then we compute the output value that the image contains a one,

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3

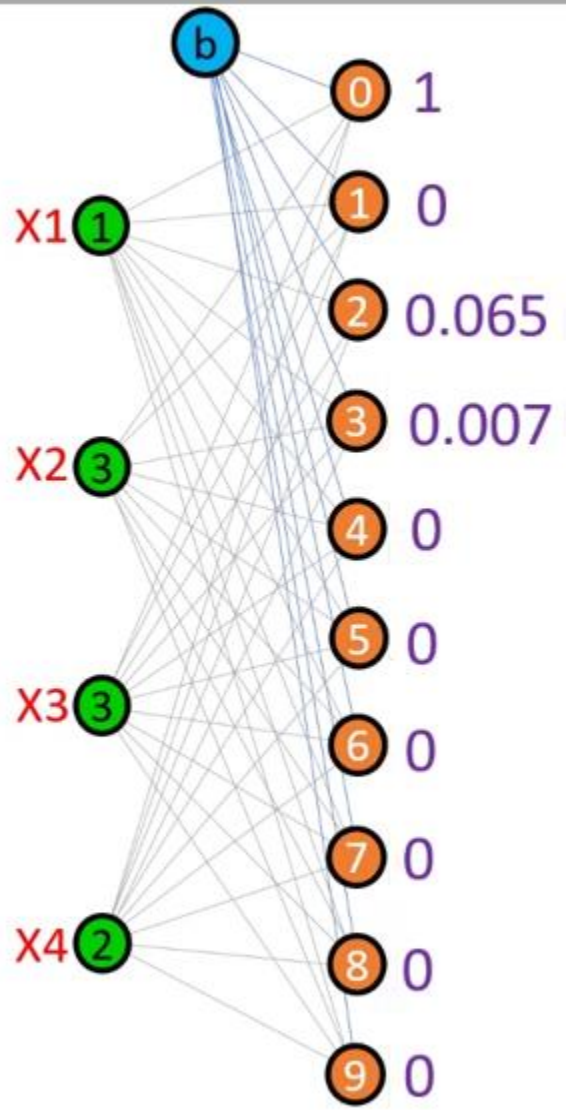


and for the eight other digits.

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3



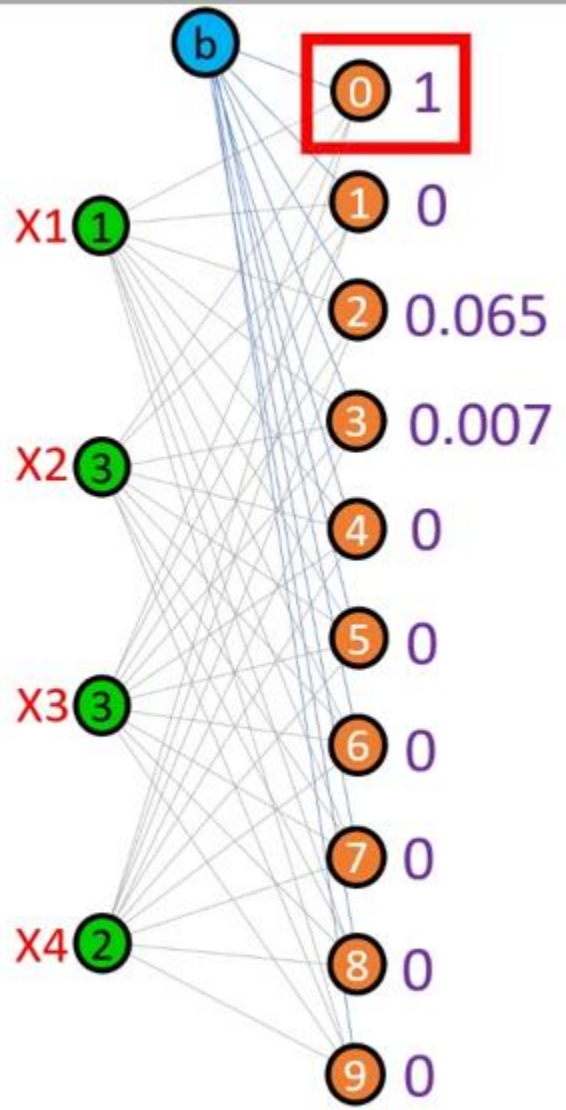
arg max

We can then use the argmax function to classify the image,

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3



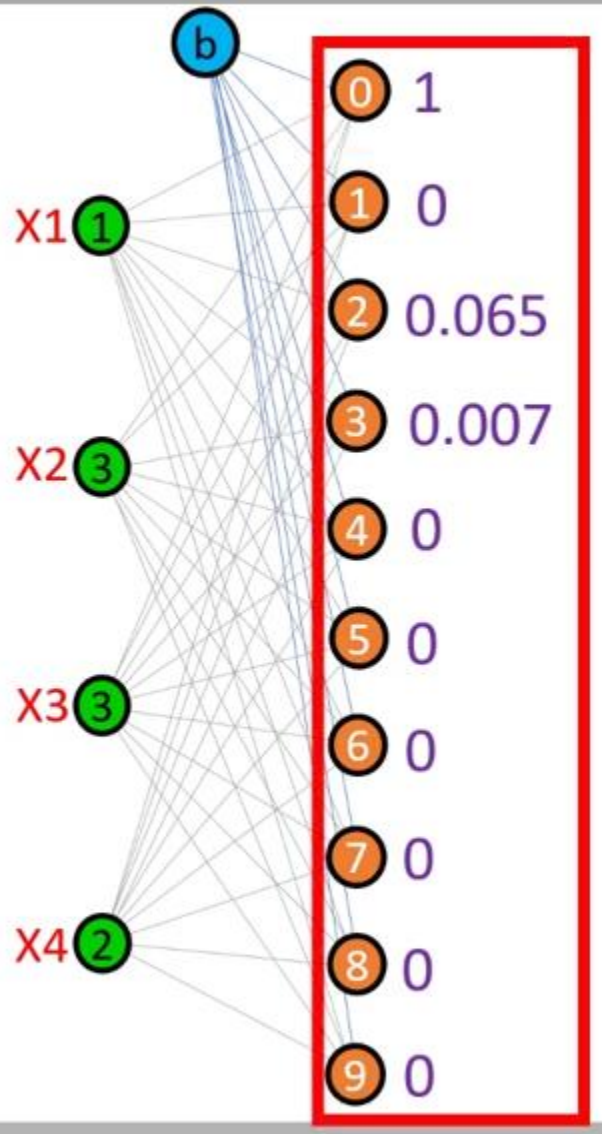
arg max

which means that the network predicts that the image includes a zero because the output node for this category has the largest value.

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3

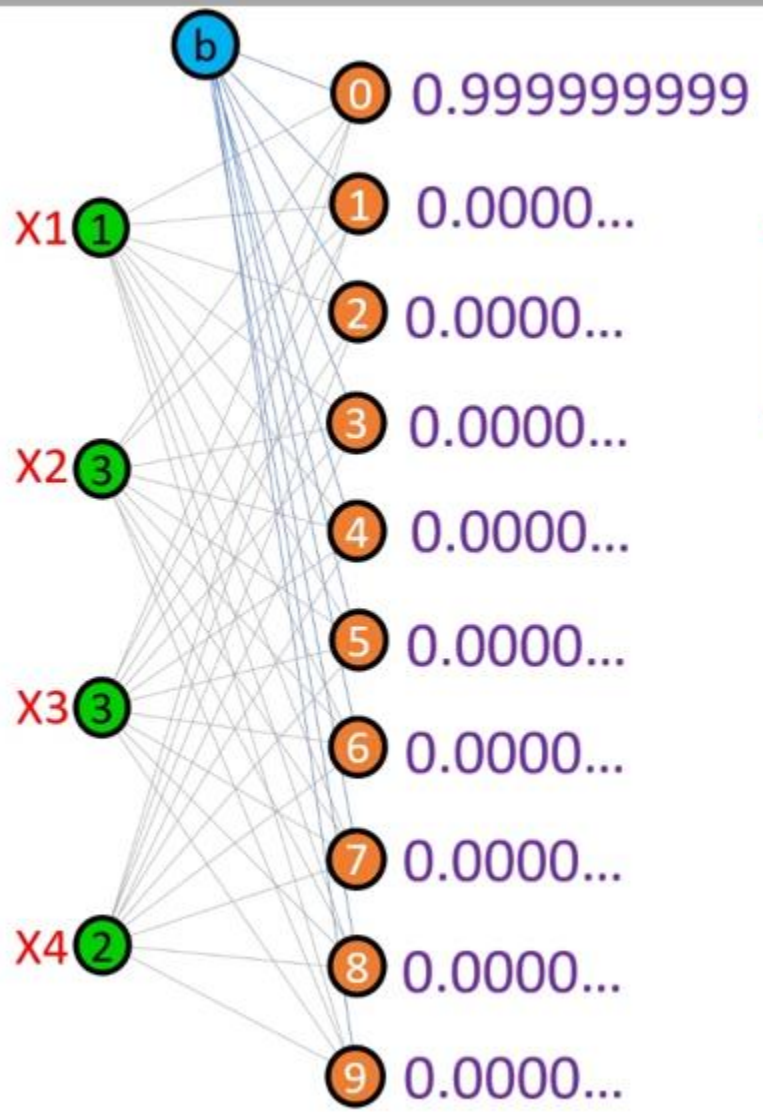


Note that these values do not sum up to one, which means that they cannot be interpreted as probabilities.

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3



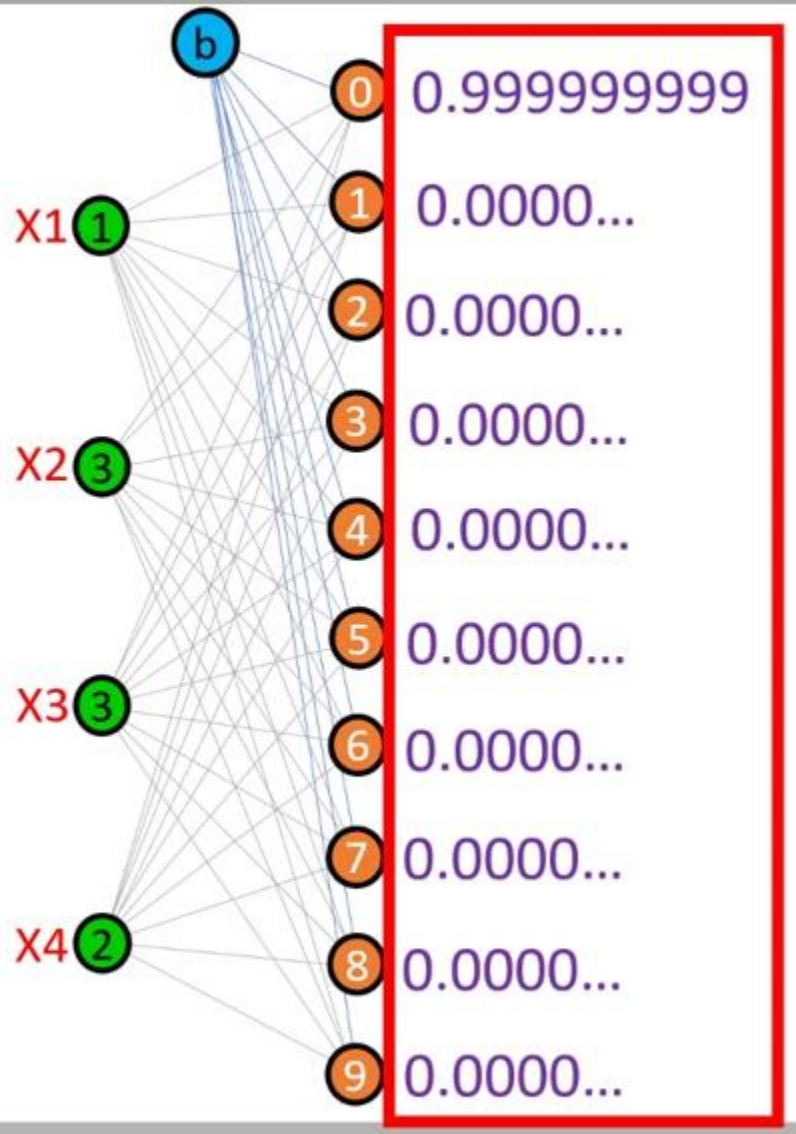
$$s(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$ **softmax**

If we instead use the softmax function, the output values will instead represent probabilities,

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3



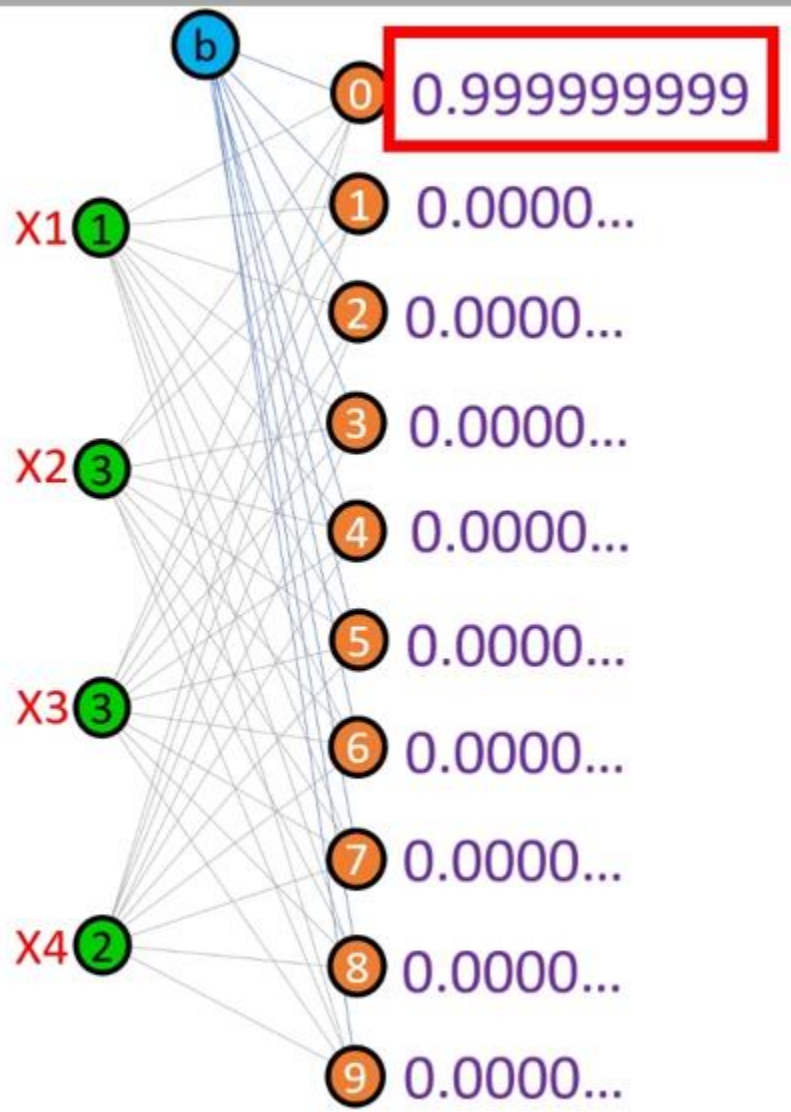
$$s(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$ **softmax**

which should sum up to one.

CNN

Input data

	X1	X2	X3	X4
0	1	3	3	2
1	2	1	2	1
2	2	3	3	2
3	2	3	2	3
4	3	2	1	3
5	3	2	2	3
6	3	2	3	3
7	2	3	2	1
8	3	3	3	3
9	3	3	1	3



$$s(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$ **softmax**

We can now see that the network predicts that the image includes a zero with approximately 100% probability.

More than one filter

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

→

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

→

Pooled feature map

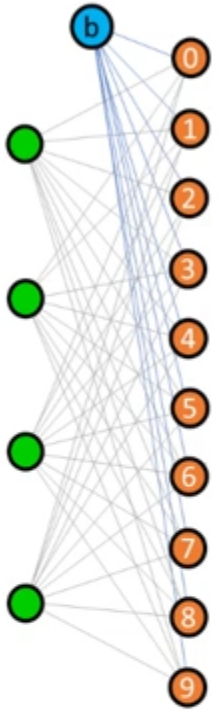
1	3
3	2

→

Flatten

1	3	3	2

→



Filter =

1	0
0	2

Remember that we used the following filter,

More than one filter

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

→

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

→

Pooled feature map

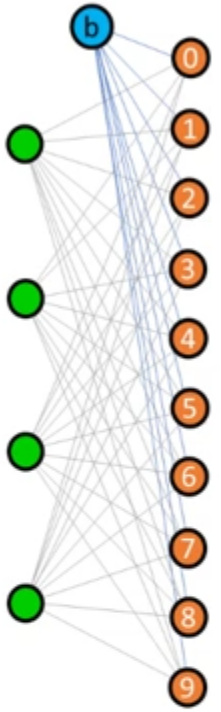
1	3
3	2

→

Flatten

1	3	3	2

→



Filter1 =

1	0
0	2

Filter2 =

1	0
0	0

It is, however, common to use several filters when we work with CNNs.

More than one filter

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

→

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

→

Pooled feature map

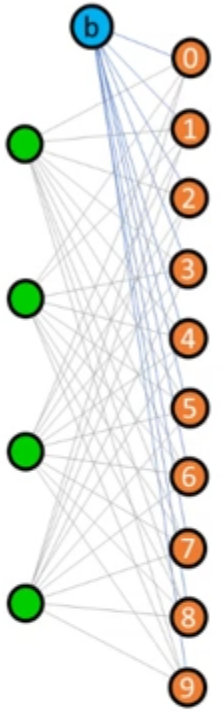
1	3
3	2

→

Flatten

1	3	3	2

→



Filter1 =

1	0
0	2

Filter2 =

1	0
0	0

Suppose that we would use also a second filter,

More than one filter

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

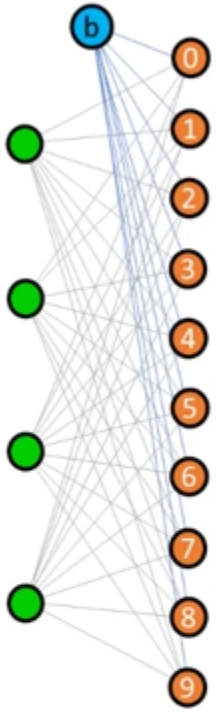
1	1	1	1
1	0	0	1
1	0	0	1
1	0	0	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2



Filter1 =

1	0
0	2

Filter2 =

1	0
0	0

using this second filter will result in the following feature map,

More than one filter

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

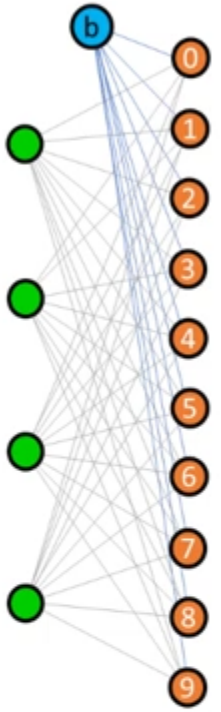
1	1	1	1
1	0	0	1
1	0	0	1
1	0	0	1

Pooled feature map

1	1
1	1

Flatten

1	1	1	1



Filter1 =

1	0
0	2

Filter2 =

1	0
0	0

and the following pooled feature map.

More than one filter

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

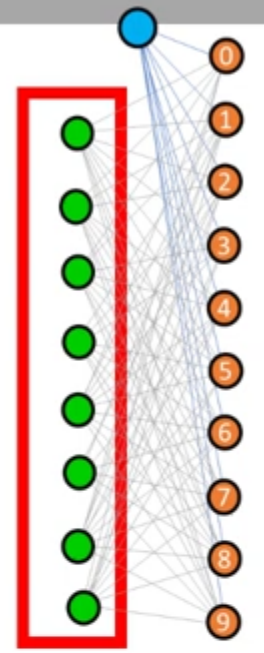
1	1	1	1
1	0	0	1
1	0	0	1
1	0	0	1

Pooled feature map

1	1
1	1

Flatten

1	1	1	1



Filter1 =

1	0
0	2

Filter2 =

1	0
0	0

We will then train a fully connected network with eight input nodes instead of just four.

CNN architecture

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map after ReLU

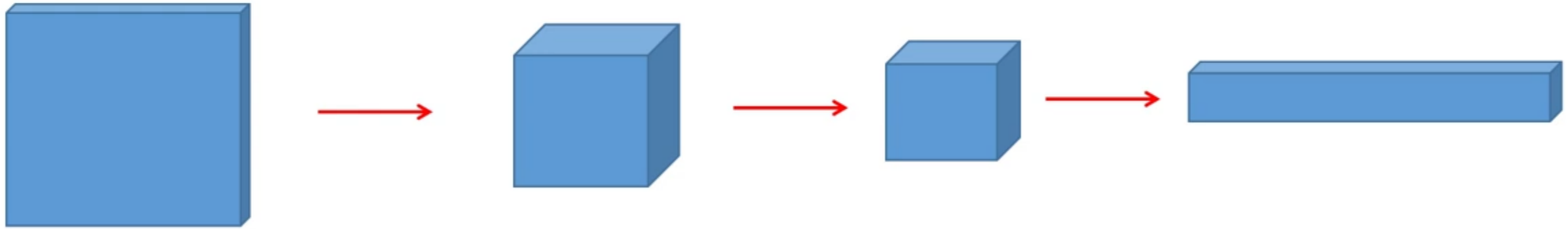
1	1	1	1
1	0	0	1
1	0	0	1
1	0	0	1

Pooled feature map

1	1
1	1

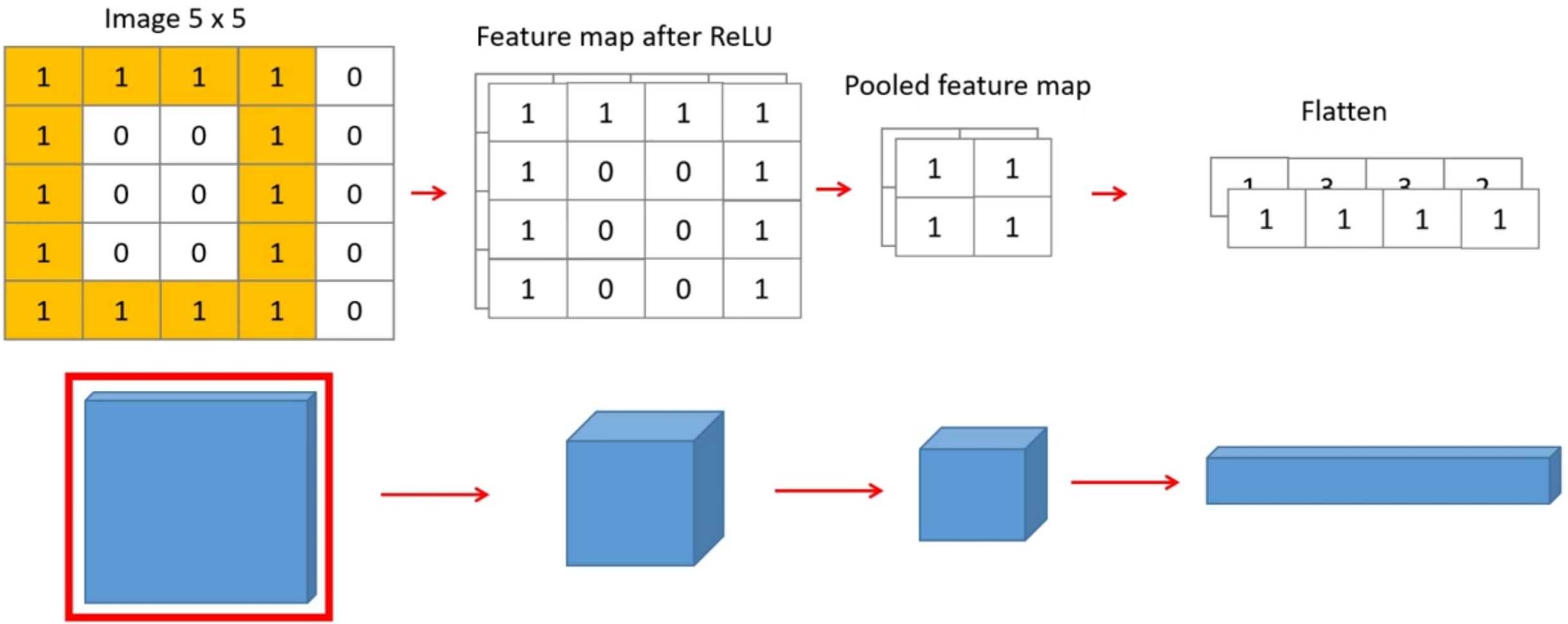
Flatten

1	1	1	1



If you read more about CNNs, you will probably see these kinds of illustrations,

CNN architecture



which means that we start with one image, which corresponds to one matrix or three matrices if we instead use a color image.

CNN architecture

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map after ReLU

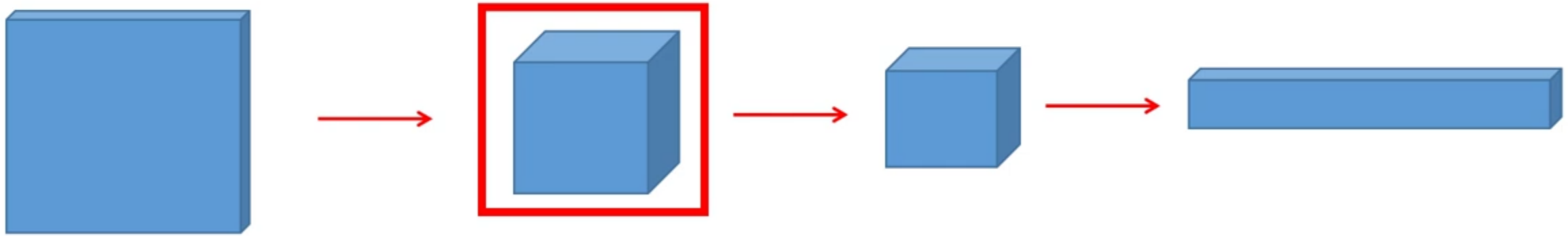
1	1	1	1
1	0	0	1
1	0	0	1
1	0	0	1

Pooled feature map

1	1
1	1

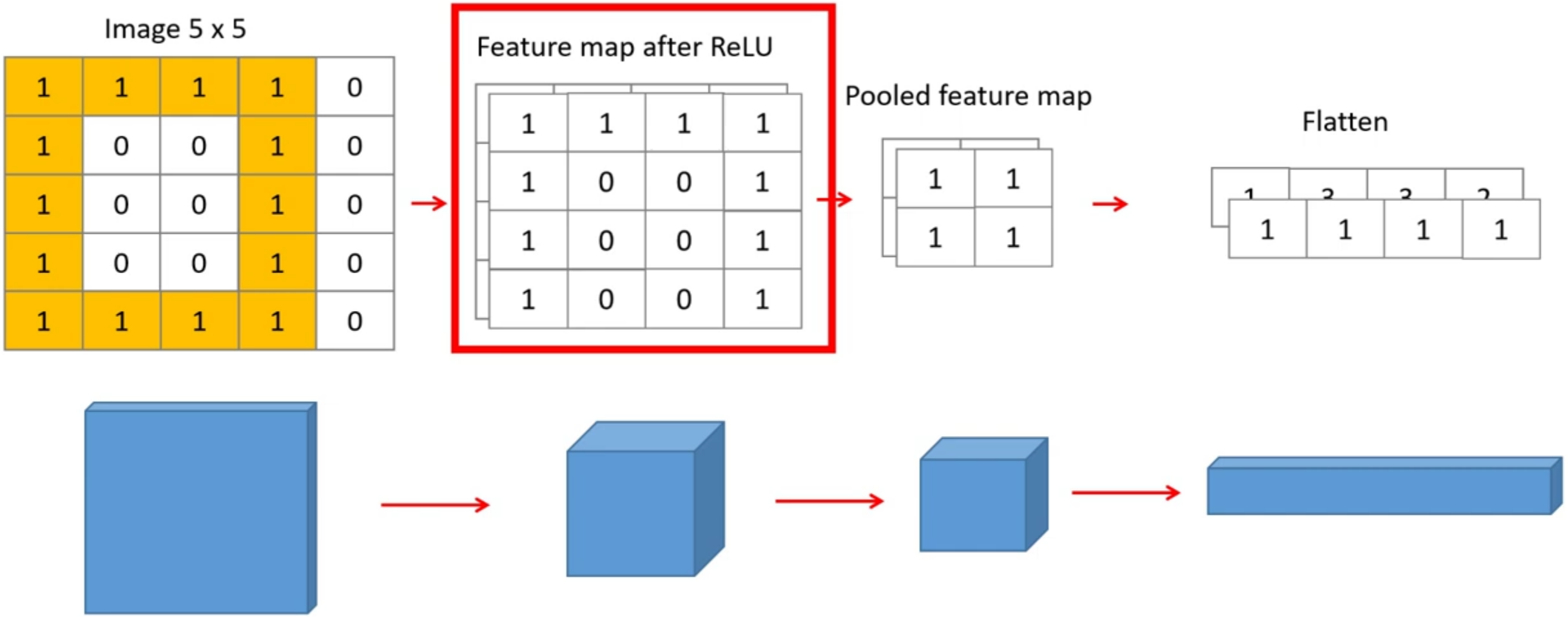
Flatten

1	1	1	1



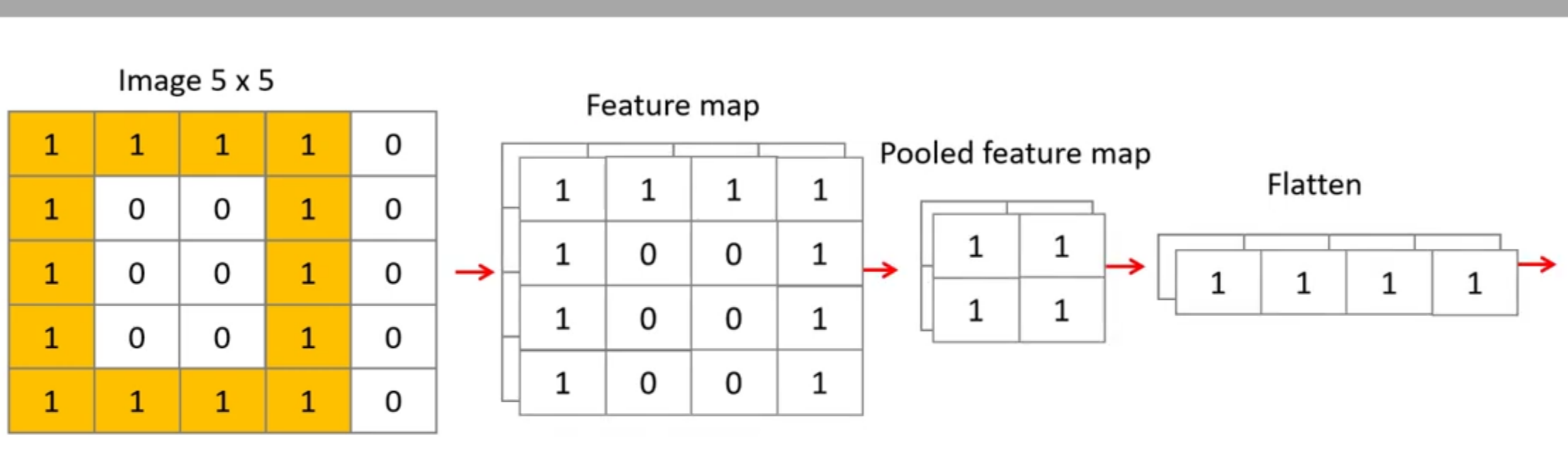
During the convolution, we usually reduce the size of the matrix and if we use several filters, we will get more matrices.

CNN architecture



We can also apply the filter on the feature map so that we perform several rounds of convolutions.

The filter



Filter1 = | 1 | 0 |
| --- | --- |
| 0 | 2 |

Filter2 = | 1 | 0 |
| --- | --- |
| 0 | 0 |

You might wonder what numbers you should use in your filters. Well, you do not need to worry about that,

The filter

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

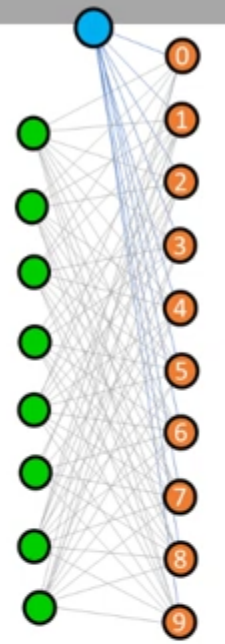
1	1	1	1
1	0	0	1
1	0	0	1
1	0	0	1

Pooled feature map

1	1
1	1

Flatten

1	1	1	1



Filter1 = | $W_{1,1}$ | $W_{2,1}$ |
|---|---|
| $W_{3,1}$ | $W_{4,1}$ |

Filter2 = | $W_{1,2}$ | $W_{2,2}$ |
|---|---|
| $W_{3,2}$ | $W_{4,2}$ |

the filters have weights that will be optimized during the training process.

The filter

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

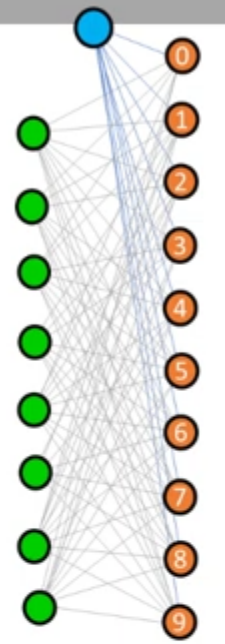
1	1	1	1
1	0	0	1
1	0	0	1
1	0	0	1

Pooled feature map

1	1
1	1

Flatten

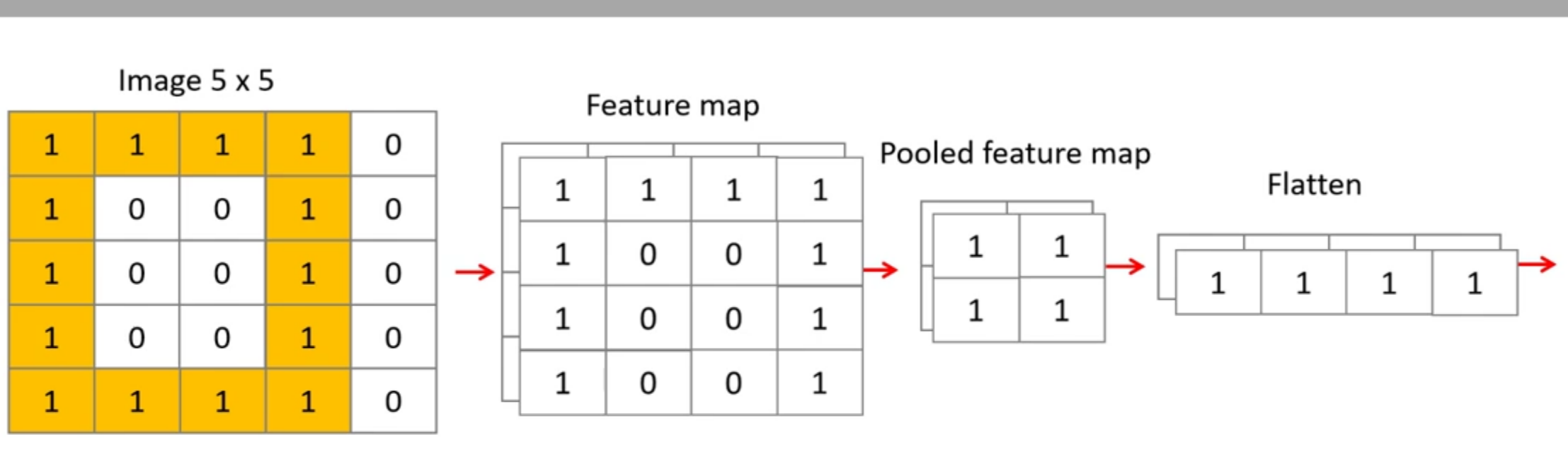
1	1	1	1



Filter1 = $\begin{bmatrix} W_{1,1} & W_{2,1} \\ W_{3,1} & W_{4,1} \end{bmatrix}$ Filter2 = $\begin{bmatrix} W_{1,2} & W_{2,2} \\ W_{3,2} & W_{4,2} \end{bmatrix}$

The weights are initialized with some random numbers and during each round of training iteration, the weights are updated,

The weights



Filter1 = $W_{1,1}$ $W_{2,1}$ $W_{3,1}$ $W_{4,1}$

Filter2 = $W_{1,2}$ $W_{2,2}$ $W_{3,2}$ $W_{4,2}$

So, in total, we here have eight weights for the two filters,

The weights

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

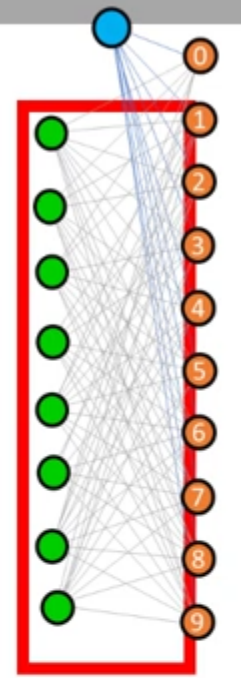
1	1	1	1
1	0	0	1
1	0	0	1
1	0	0	1

Pooled feature map

1	1
1	1

Flatten

1	1	1	1



Filter1 =

$W_{1,1}$	$W_{2,1}$
$W_{3,1}$	$W_{4,1}$

Filter2 =

$W_{1,2}$	$W_{2,2}$
$W_{3,2}$	$W_{4,2}$

and 80 weights that are associated with the edges between the eight input nodes and the 10 output nodes,

The weights

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

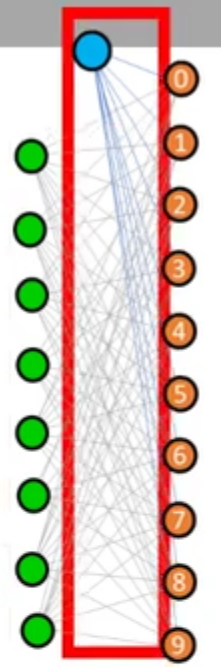
1	1	1	1
1	0	0	1
1	0	0	1
1	0	0	1

Pooled feature map

1	1
1	1

Flatten

1	1	1	1



Filter1 =

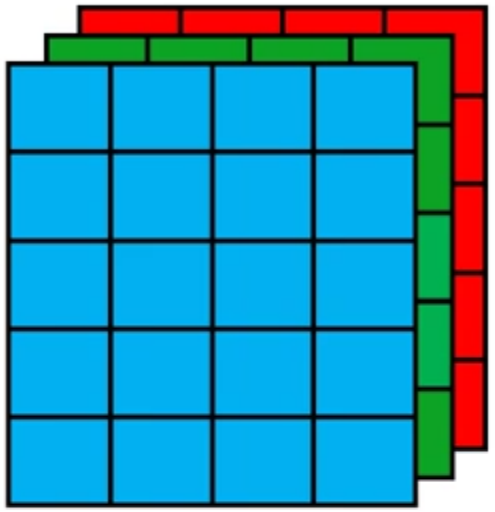
$w_{1,1}$	$w_{2,1}$
$w_{3,1}$	$w_{4,1}$

Filter2 =

$w_{1,2}$	$w_{2,2}$
$w_{3,2}$	$w_{4,2}$

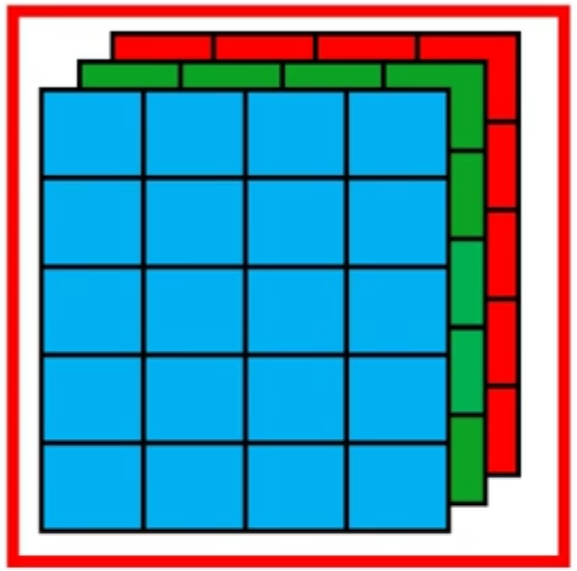
as well as 10 bias weights, which gives the network in total 98 weights to tune on during the training process.

Channels



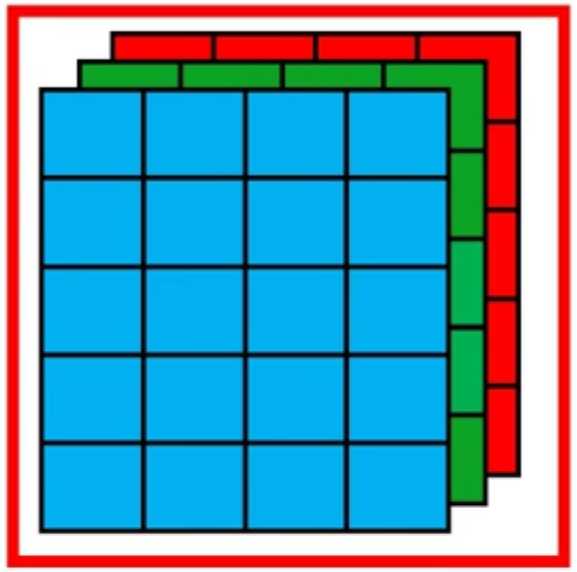
We will now have a look at how the network deals with color images.

Channels



A color image is represented by three layers, or grids, which usually represent the brightness of red, green and blue colors. The combination of the numbers in the three pixels then gives a certain color.

Channels



In CNN, we say that we have three channels, one for red, green and blue.

Channels

1	1	1	1
1	0	0	1
1	0	0	1
1	0	0	1
1	1	1	1

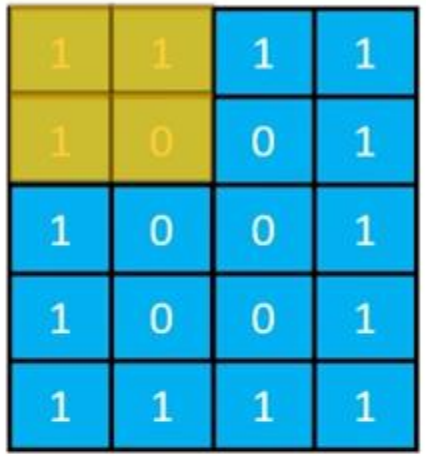
1	1	1	1
1	0	0	1
1	0	0	1
1	0	0	1
1	1	1	1

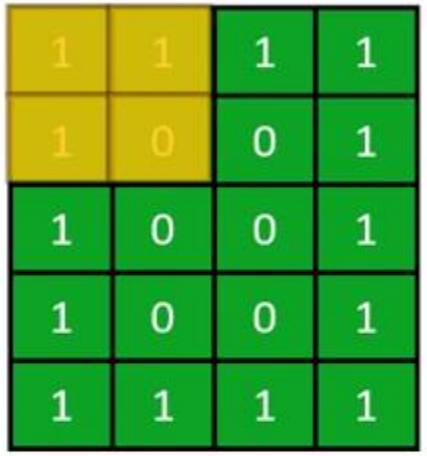
0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

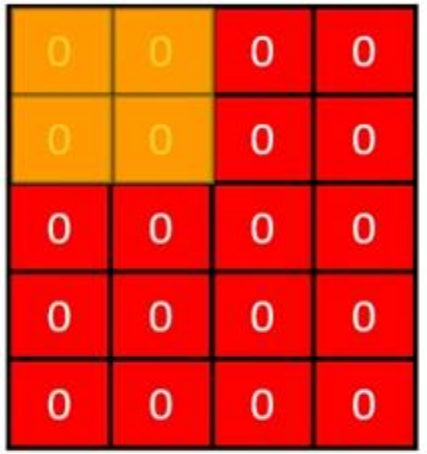
1	0
0	2

Suppose that we would use the following filter,

Channels



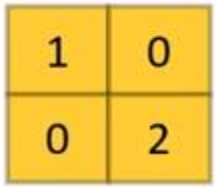




$1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$

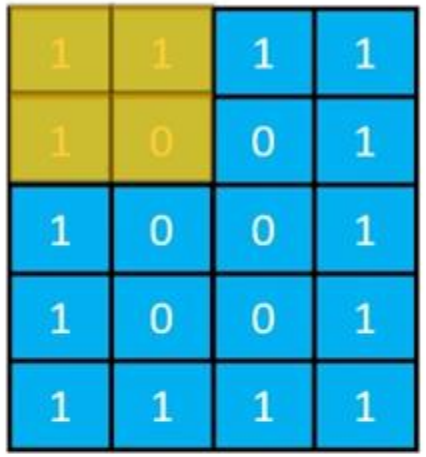
$1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$

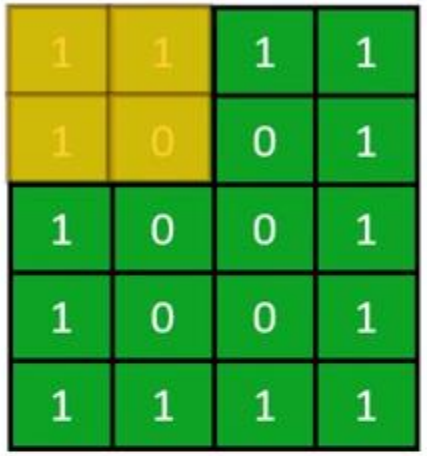
$0 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 0$

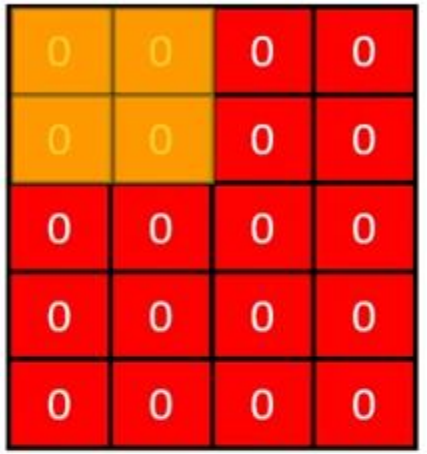


that we apply to all three channels and do the same calculations as we did before.

Channels







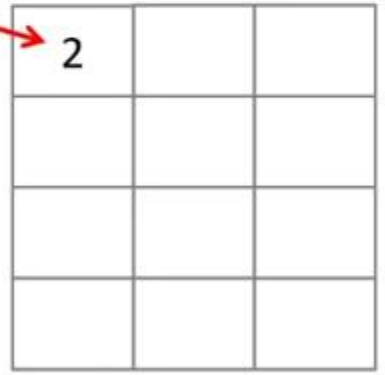
$1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$

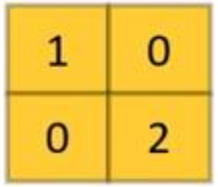
$1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$

$0 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 0$

$1 + 1 + 0 = 2$

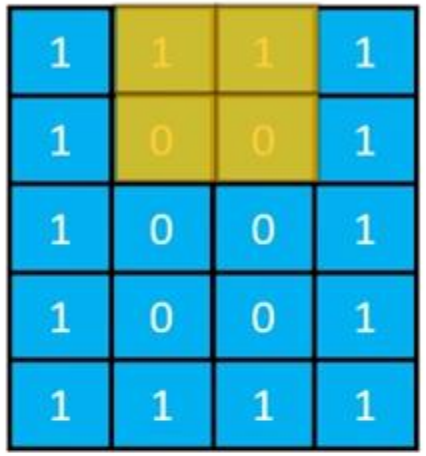
Feature map

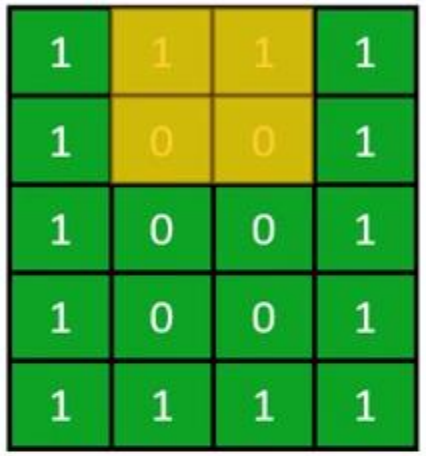


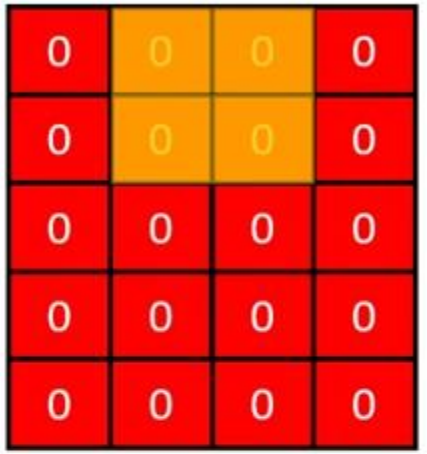


Then we sum the calculations from the three channels and plug in the sum in a feature map.

Channels







$1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$ $1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 1$ $0 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 0$

$$1 + 1 + 0 = 2$$

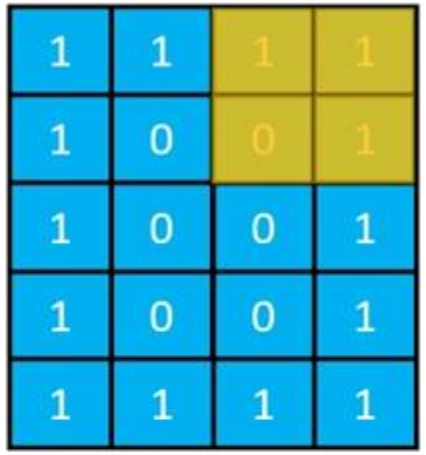
1	0
0	2

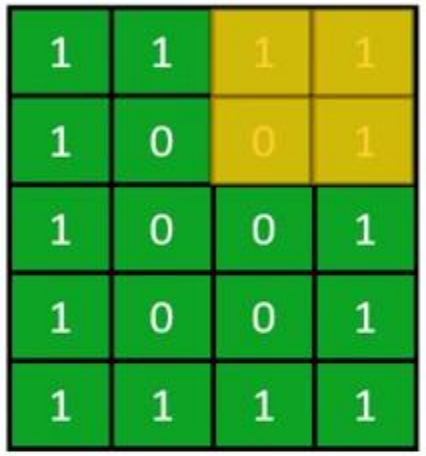
Feature map

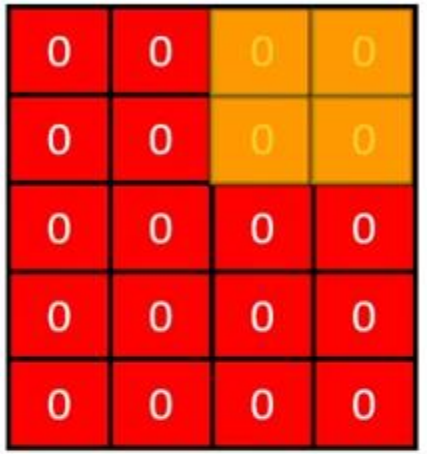
2	2	

Next, we move the filter one step to the right,

Channels







$1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{2} = 3$ $1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{2} = 3$ $0 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 0$

$3 + 3 + 0 = 6$

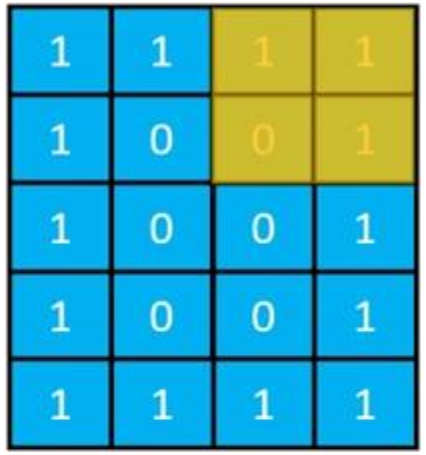
Feature map

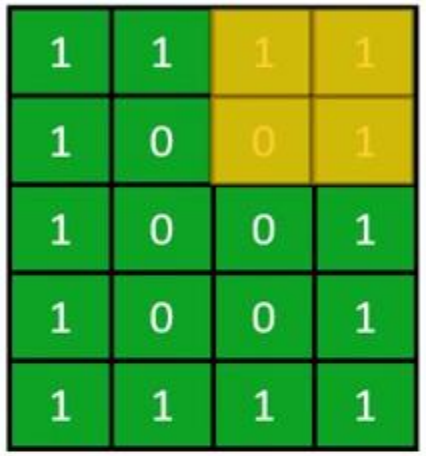
2	2	6

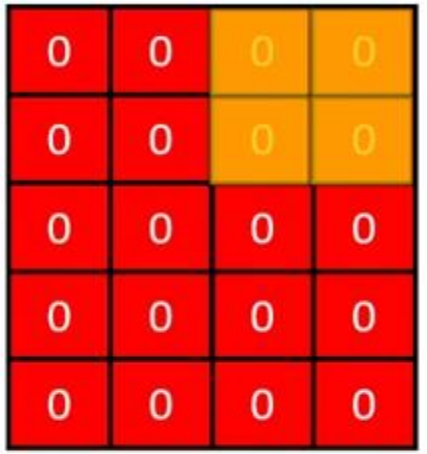
1	0
0	2

and so forth.

Channels







$1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{2} = 3$ $1 \cdot \mathbf{1} + 1 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 1 \cdot \mathbf{2} = 3$ $0 \cdot \mathbf{1} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{0} + 0 \cdot \mathbf{2} = 0$

$$3 + 3 + 0 + \boxed{bias} = 6$$

1	0
0	2

Feature map

2	2	6

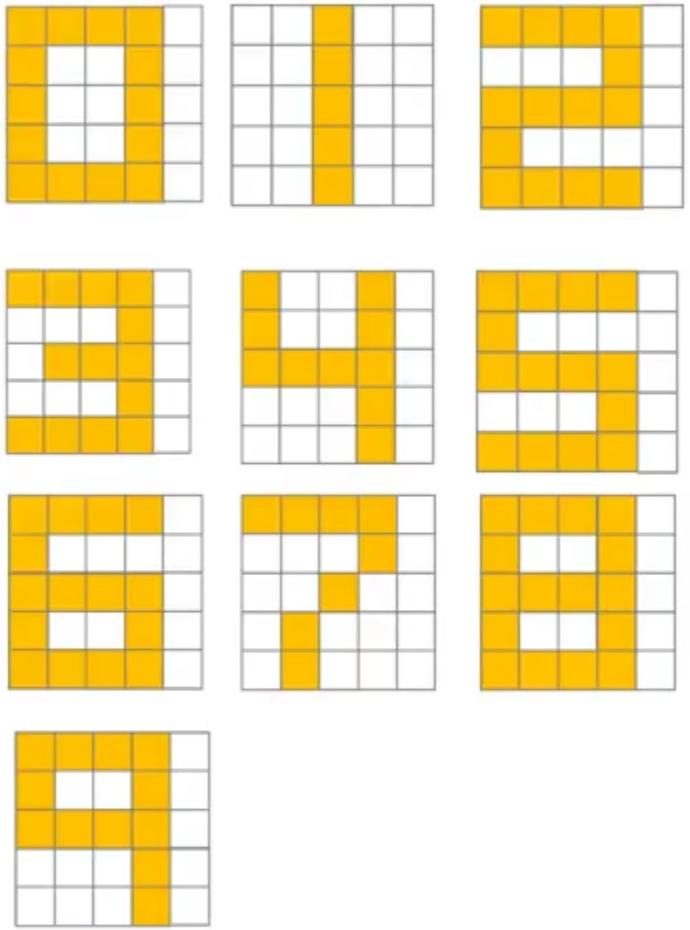
Note that one can also include a bias weight in this calculation so that the network has one additional weight to tune on.

How the filters identify local features

How do the filters identify local features in the image?

So far, I have just shown how the calculations are done with the filters but how do the filters identify local features in the image?

How the filters identify local features



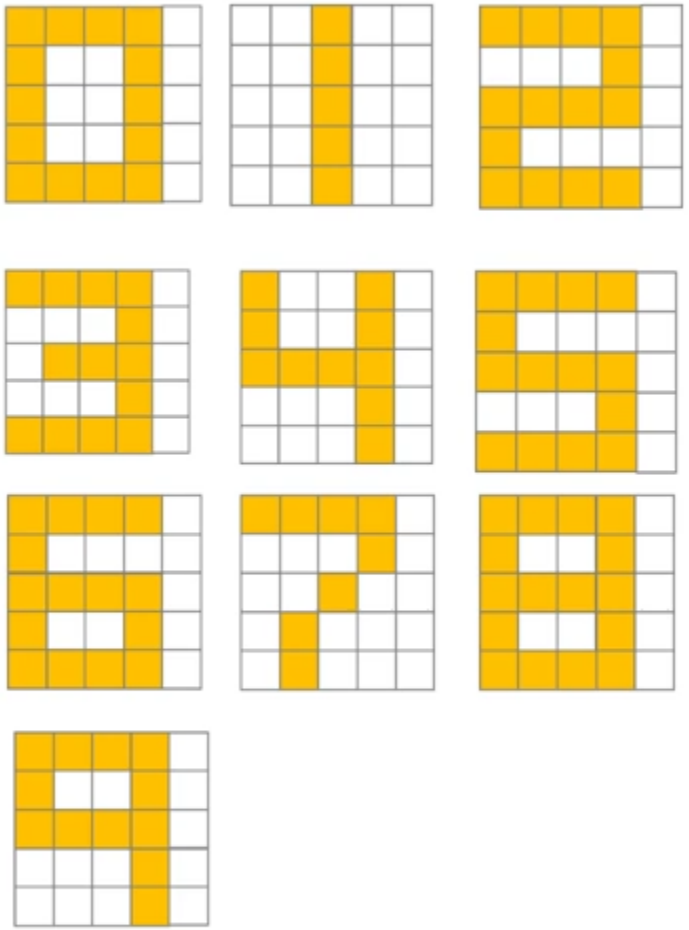
Filter1 =

1	1
1	-1

= 3

Well, suppose that we would apply the following filter on the 10 images and that it results in a three, which we will plug-in in the feature map,

How the filters identify local features



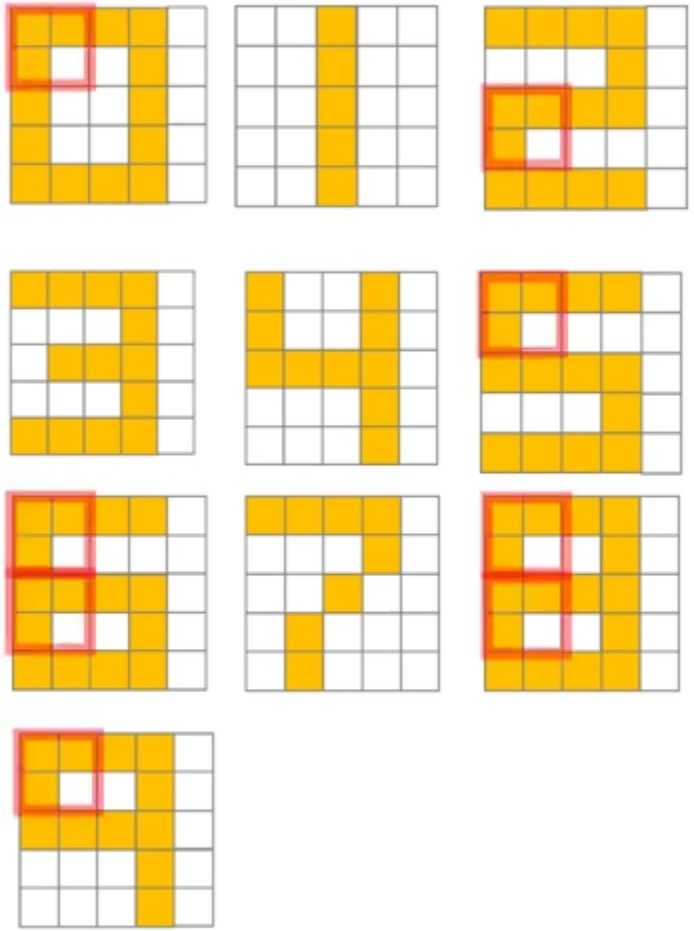
Filter1 =

1	1
1	-1

if = 3 →

that would mean that the filter has identified the following corner,

How the filters identify local features



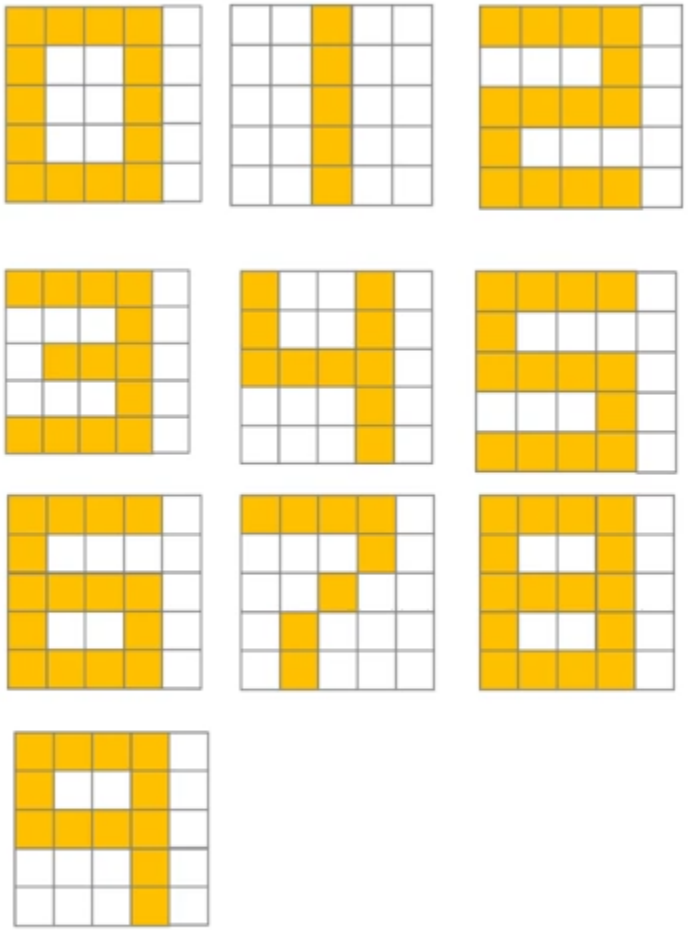
Filter1 =

1	1
1	-1

if = 3 →

which is present in the following images.

How the filters identify local features



Filter1 =

1	1
1	-1

if = 3 →

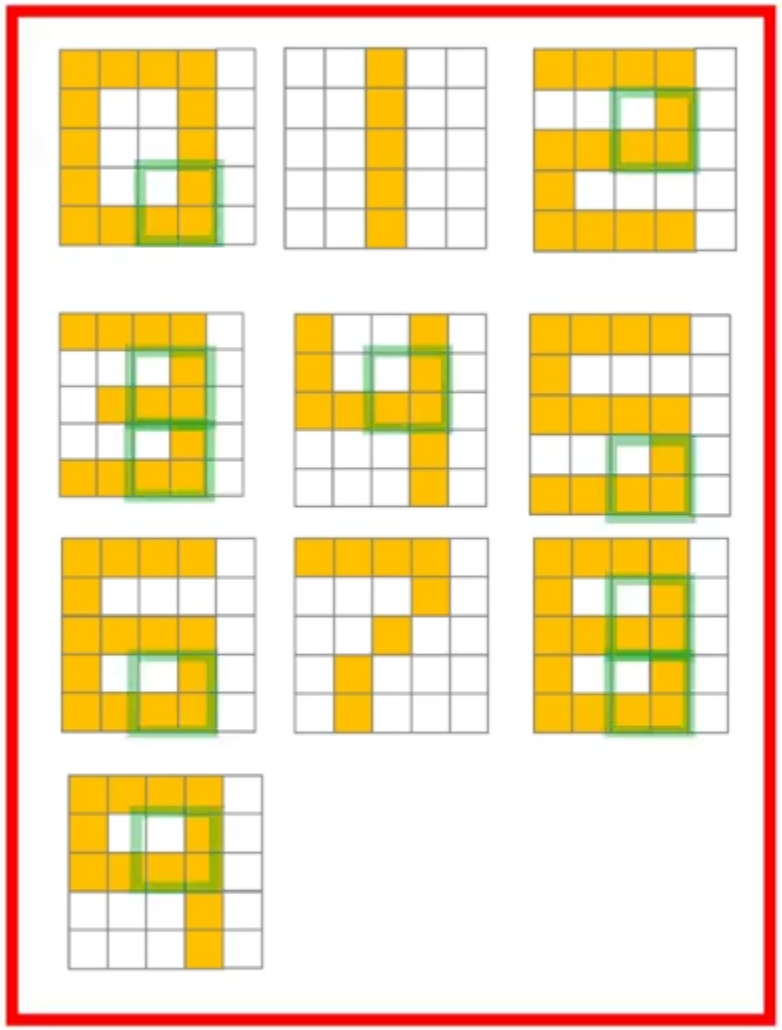
Filter2 =

-1	1
1	1

if = 3 →

By using also this filter, we can identify this kind of corner,

How the filters identify local features



Filter1 =

1	1
1	-1

if = 3 →

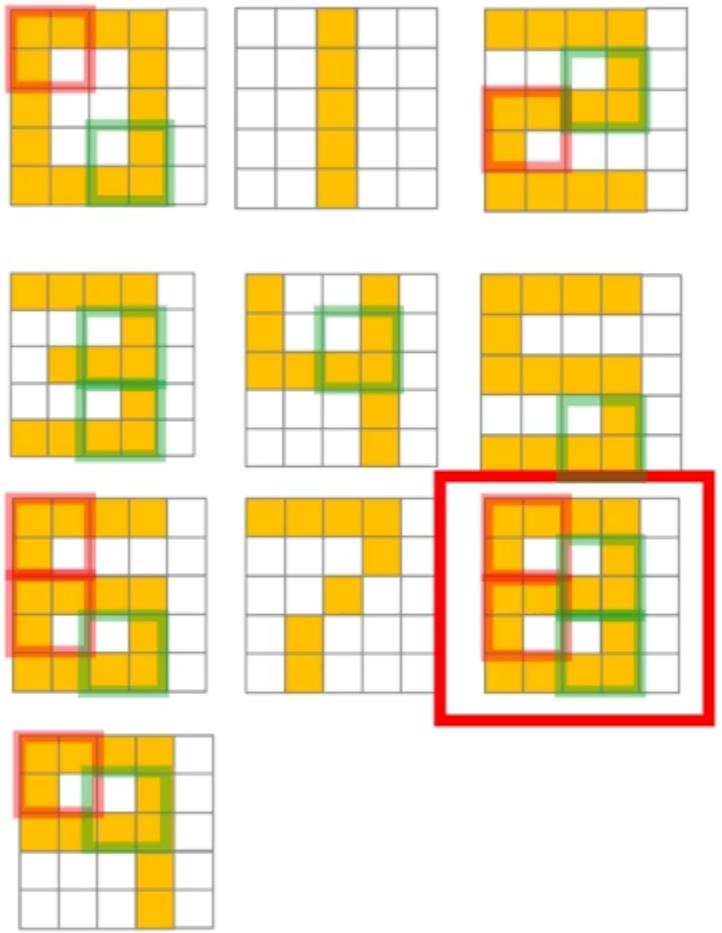
Filter2 =

-1	1
1	1

if = 3 →

which is present in the following images.

How the filters identify local features



Filter1 =

1	1
1	-1

if = 3 →

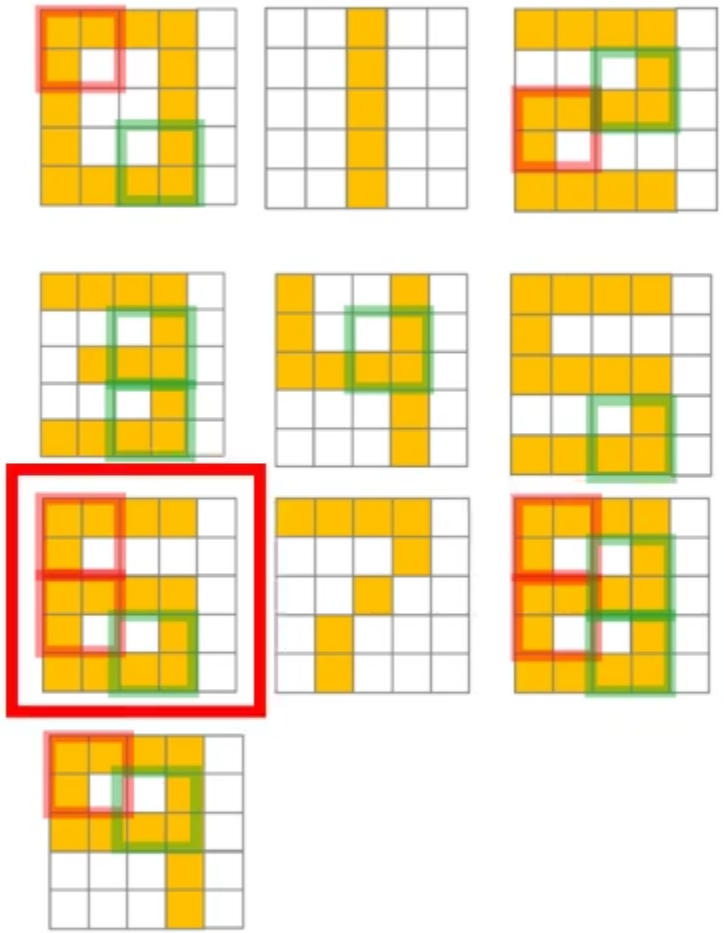
Filter2 =

-1	1
1	1

if = 3 →

So, this image is the only image that includes two of each of these corners,

How the filters identify local features



Filter1 =

1	1
1	-1

if = 3 →

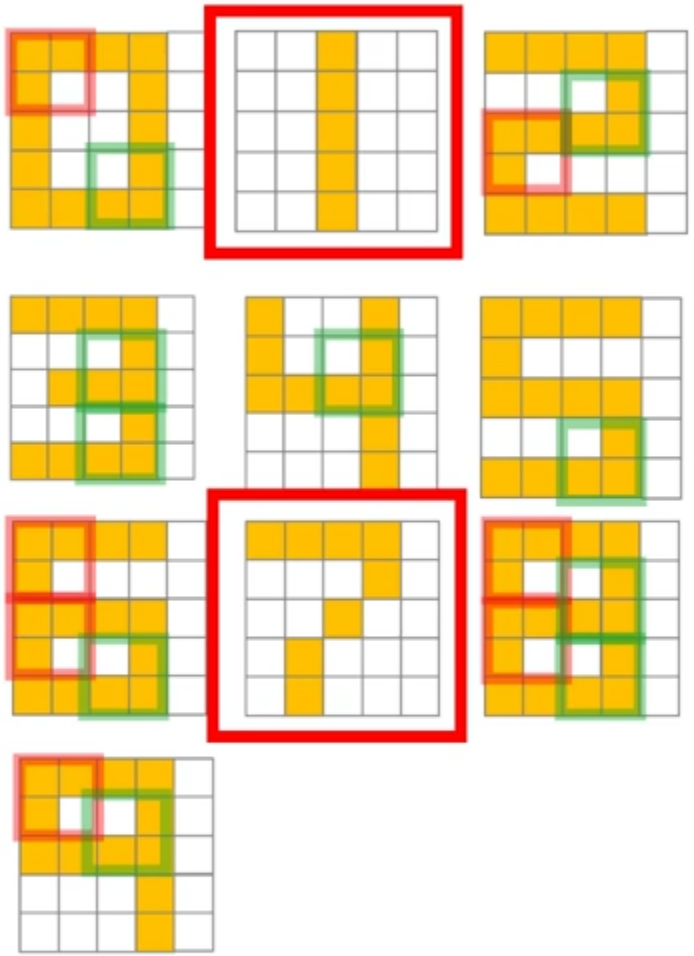
Filter2 =

-1	1
1	1

if = 3 →

whereas this image is the only image that includes two top-left corners and one bottom-right corner.

How the filters identify local features



Filter1 =

1	1
1	-1

if = 3 →

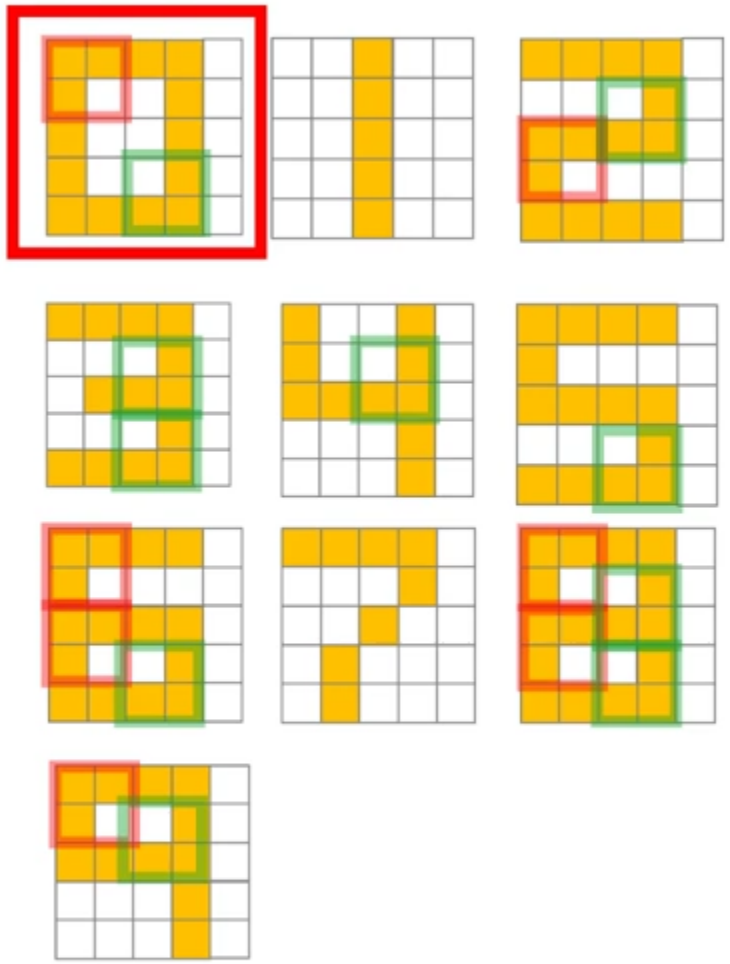
Filter2 =

-1	1
1	1

if = 3 →

These two images do not include any of these corners. The image classification will therefore rely on different local features that are identified in the image, and not on individual pixels.

How the filters identify local features



Filter1 =

1	1
1	-1

if = 3 →

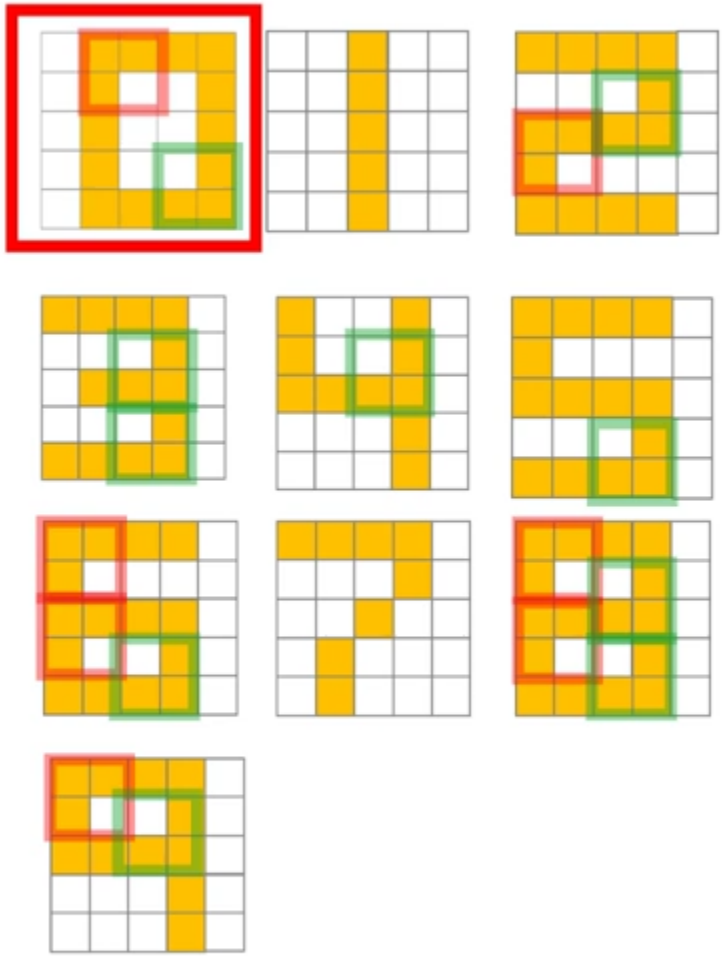
Filter2 =

-1	1
1	1

if = 3 →

If we would shift the digit in this image one step to the right,

How the filters identify local features



Filter1 =

1	1
1	-1

if = 3 →

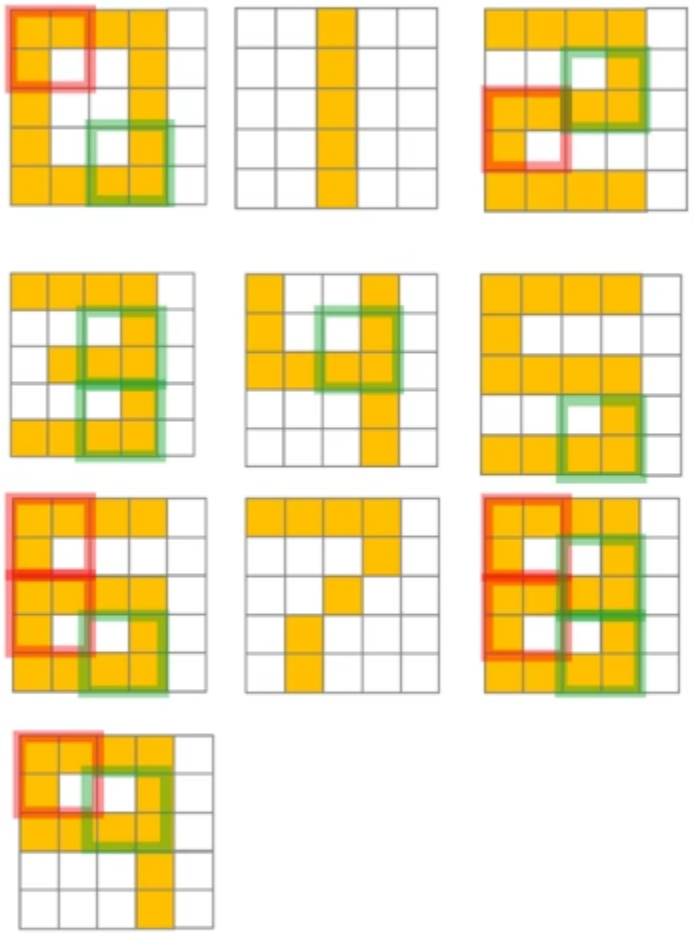
Filter2 =

-1	1
1	1

if = 3 →

that should not affect the classification of this image, because the identified local features are still the same. In contrast, a simple ANN will be highly affected by such a shift because its classification is based on the exact location of the pixel values.

How the filters identify local features



Filter1 =

1	1
1	-1

if = 3 →

Filter2 =

-1	1
1	1

if = 3 →

Filter3 =

1	-1
1	1

if = 3 →

Filter4 =

1	1
-1	1

if = 3 →

By using a range of different filters, we can identify a large variety of local features in the images. This is one reason why we use several filters, instead of just one, because different filters will learn to identify different features.

Padding

We will now discuss the meaning of padding.

Padding

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Filter =

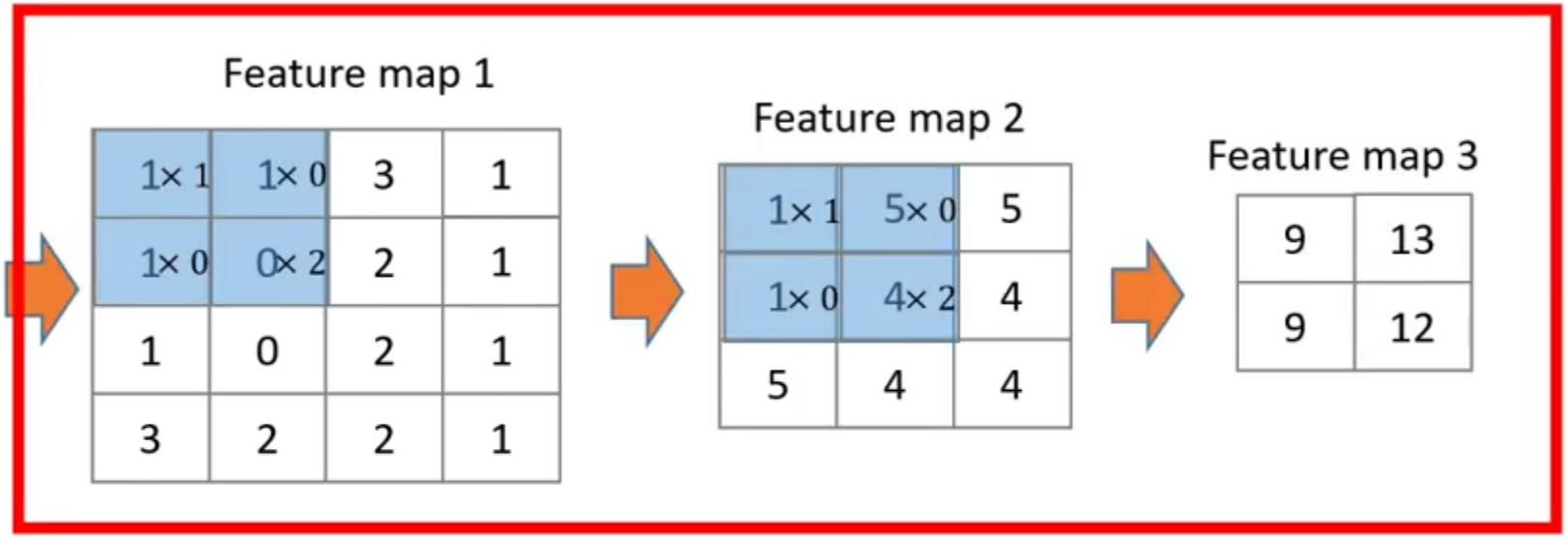
1	0
0	2

Remember that when we applied the filter to the image we got the following feature map, which is a 4 by 4 matrix,

Padding

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0



Filter =

1	0
0	2

If we apply many convolutional steps, the feature map will become smaller and smaller and might therefore be too small to be useful.

Padding

Image 5 x 5

1× 1	1× 0	1	1	0
1× 0	0× 2	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Filter =

1	0
0	2

Another problem is that the pixels in the edges will have less influence when we build the feature map. For example, the pixel in the top left corner is only used once,

Padding

Image 5 x 5

1× 1	1× 0	1	1	0
1× 0	0× 2	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Filter =

1	0
0	2

whereas this pixel is included four times when the feature map is created.

Padding

Image 5 x 5

1	1	1	1	0
1	0× 1	0× 0	1	0
1	0× 0	0× 2	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Filter =

1	0
0	2

Padding

Image 5 x 5

0	0	0	0	0	0	0
0	1	1	1	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Filter =

1	0
0	2

One way to solve these problems is to use padding, which means that we add one or several additional columns and rows to the image. The values in these extra elements are usually set to zero.

Padding

Image 5 x 5

0× 1	0× 0	0	0	0	0	0
0× 0	1× 2	1	1	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2					

Filter =

1	0
0	2

If we now apply the filter to perform convolution,

Padding

Image 5 x 5

0	0	0×1	0×0	0	0	0
0	1	1×0	1×2	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2	2	2			

Filter =

1	0
0	2

Padding

Image 5 x 5

0	0	0	0× 1	0× 0	0	0
0	1	1	1× 0	1× 2	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2	2	2	2		

Filter =

1	0
0	2

Padding

Image 5 x 5

0	0	0×1	0×0	0	0	0
0	1	1×0	1×2	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2	2	2			

Filter =

1	0
0	2

Padding

Image 5 x 5

0	0	0	0	0× 1	0× 0	0
0	1	1	1	1× 0	0× 2	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2	2	2	2	0	

Filter =

1	0
0	2

Padding

Image 5 x 5

0	0	0	0	0	0× 1	0× 0
0	1	1	1	1	0× 0	0× 2
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2	2	2	2	0	0

Filter =

1	0
0	2

Padding

Image 5 x 5

0	0	0	0	0	0	0
0×	1× 0	1	1	1	0	0
0× 0	1× 2	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2	2	2	2	0	0
2					

Filter =

1	0
0	2

we see that the pixel in the top left corner will now be involved four times to create the feature map.

Padding

Image 5 x 5

0	0	0	0	0	0	0
0	1	1	1	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2	2	2	2	0	0
2	1	1	3	1	0
2	1	0	2	1	0
2	1	0	2	1	0
2	3	2	2	1	0
0	1	1	1	1	0

Filter =

1	0
0	2

We also see that the feature map will now be bigger than the original image because the feature map is now a 6 by 6 matrix.

Padding

Image 5 x 5

0	0	0	0	0	0	0
0	1	1	1	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2	2	2	2	0	0
2	1	1	3	1	0
2	1	0	2	1	0
2	1	0	2	1	0
2	3	2	2	1	0
0	1	1	1	1	0

n: The width (or height) of the input image.
f: The width (or height) of the kernel (filter).
p: The number of padding layers
s: The number of steps the kernel moves each time it slides (stride).

Filter =

1	0
0	2

$$Output\ size = \left[\frac{n + 2p - f}{s}\right] + 1 = \left[\frac{5 + 2 \cdot 1 - 2}{1}\right] + 1 = 6$$

We can use the following formula to calculate the output size of the feature map.

Padding

Image 5 x 5

0	0	0	0	0	0	0
0	1	1	1	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2	2	2	2	0	0
2	1	1	3	1	0
2	1	0	2	1	0
2	1	0	2	1	0
2	3	2	2	1	0
0	1	1	1	1	0

n: The width (or height) of the input image.
f: The width (or height) of the kernel (filter).
p: The number of padding layers
s: The number of steps the kernel moves each time it slides (stride).

Filter =

1	0
0	2

$$Output\ size = \left[\frac{n + 2p - f}{s}\right] + 1 = \left[\frac{5 + 2 \cdot 1 - 2}{1}\right] + 1 = 6$$

In our example, the size of the image is five,

Padding

Image 5 x 5

0	0	0	0	0	0	0
0	1	1	1	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2	2	2	2	0	0
2	1	1	3	1	0
2	1	0	2	1	0
2	1	0	2	1	0
2	3	2	2	1	0
0	1	1	1	1	0

n: The width (or height) of the input image.
f: The width (or height) of the kernel (filter).
p: The number of padding layers
s: The number of steps the kernel moves each time it slides (stride).

Filter =

1	0
0	2

$$Output\ size = \left[\frac{n + 2p - f}{s}\right] + 1 = \left[\frac{5 + 2 \cdot 1 - 2}{1}\right] + 1 = 6$$

and we added one padding layer around the image,

Padding

Image 5 x 5

0	0	0	0	0	0	0
0	1	1	1	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2	2	2	2	0	0
2	1	1	3	1	0
2	1	0	2	1	0
2	1	0	2	1	0
2	3	2	2	1	0
0	1	1	1	1	0

n: The width (or height) of the input image.
f: The width (or height) of the kernel (filter).
p: The number of padding layers
s: The number of steps the kernel moves each time it slides (stride).

Filter =

1	0
0	2

$$Output\ size = \left[\frac{n + 2p - f}{s}\right] + 1 = \left[\frac{5 + 2 \cdot 1 - 2}{1}\right] + 1 = 6$$

and used a 2 by 2 filter,

Padding

Image 5 x 5

0	0	0	0	0	0	0
0	1	1	1	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2	2	2	2	0	0
2	1	1	3	1	0
2	1	0	2	1	0
2	1	0	2	1	0
2	3	2	2	1	0
0	1	1	1	1	0

n: The width (or height) of the input image.
f: The width (or height) of the kernel (filter).
p: The number of padding layers
s: The number of steps the kernel moves each time it slides (stride).

Filter =

1	0
0	2

$$Output\ size = \left[\frac{n + 2p - f}{s}\right] + 1 = \left[\frac{5 + 2 \cdot 1 - 2}{1}\right] + 1 = 6$$

with a stride length that was equal to one.

Padding

Image 5 x 5

0	0	0	0	0	0	0
0	1	1	1	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2	2	2	2	0	0
2	1	1	3	1	0
2	1	0	2	1	0
2	1	0	2	1	0
2	3	2	2	1	0
0	1	1	1	1	0

n: The width (or height) of the input image.
f: The width (or height) of the kernel (filter).
p: The number of padding layers
s: The number of steps the kernel moves each time it slides (stride).

Filter =

1	0	0
0	2	2
0	2	2

$$Output\ size = \left[\frac{n + 2p - f}{s}\right] + 1 = \left[\frac{5 + 2 \cdot 1 - 3}{1}\right] + 1 = 5$$

In order to get the same size of the feature map as the image, we can simply use a larger filter,

Padding

Image 5 x 5

0	0	0	0	0	0	0
0	1	1	1	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	0	0	1	0	0
0	1	1	1	1	0	0
0	0	0	0	0	0	0

Feature map 6 x 6

2	2	2	2	0	0
2	1	1	3	1	0
2	1	0	2	1	0
2	1	0	2	1	0
2	3	2	2	1	0
0	1	1	1	1	0

n: The width (or height) of the input image.
f: The width (or height) of the kernel (filter).
p: The number of padding layers
s: The number of steps the kernel moves each time it slides (stride).

Filter =

1	0	0
0	2	2
0	2	2

$$Output\ size = \left[\frac{n + 2p - f}{s}\right] + 1 = \left[\frac{5 + 2 \cdot 1 - 3}{1}\right] + 1 = 5$$

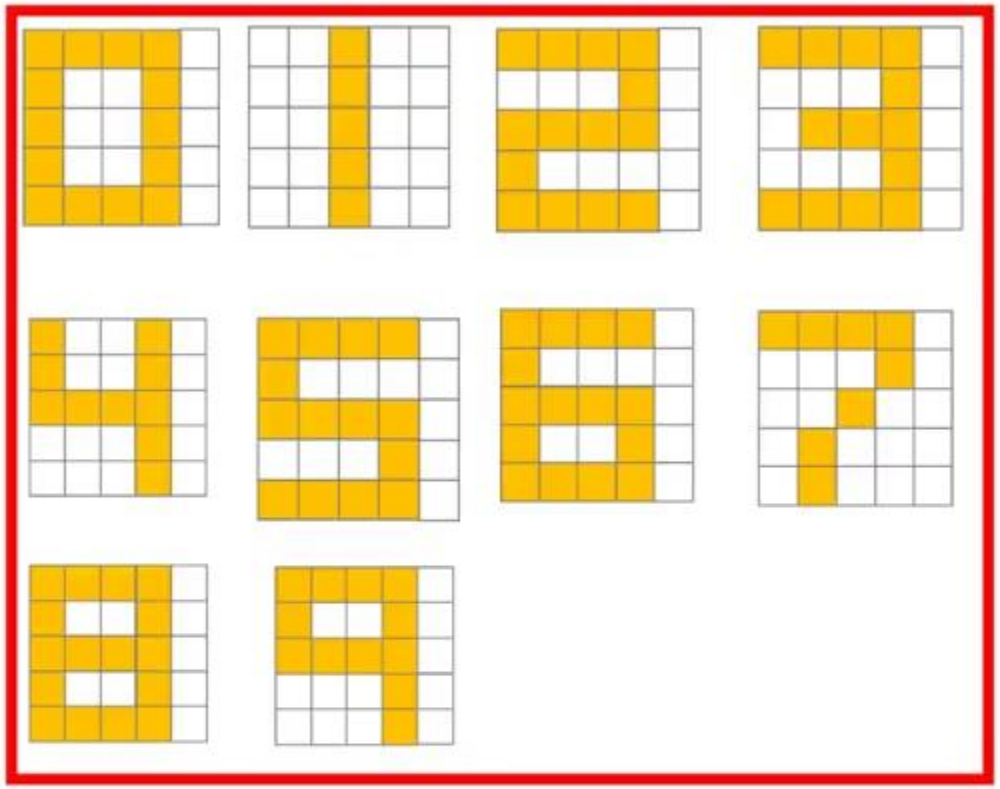
so that the feature map becomes a 5 by 5 matrix.

Python code

Python code

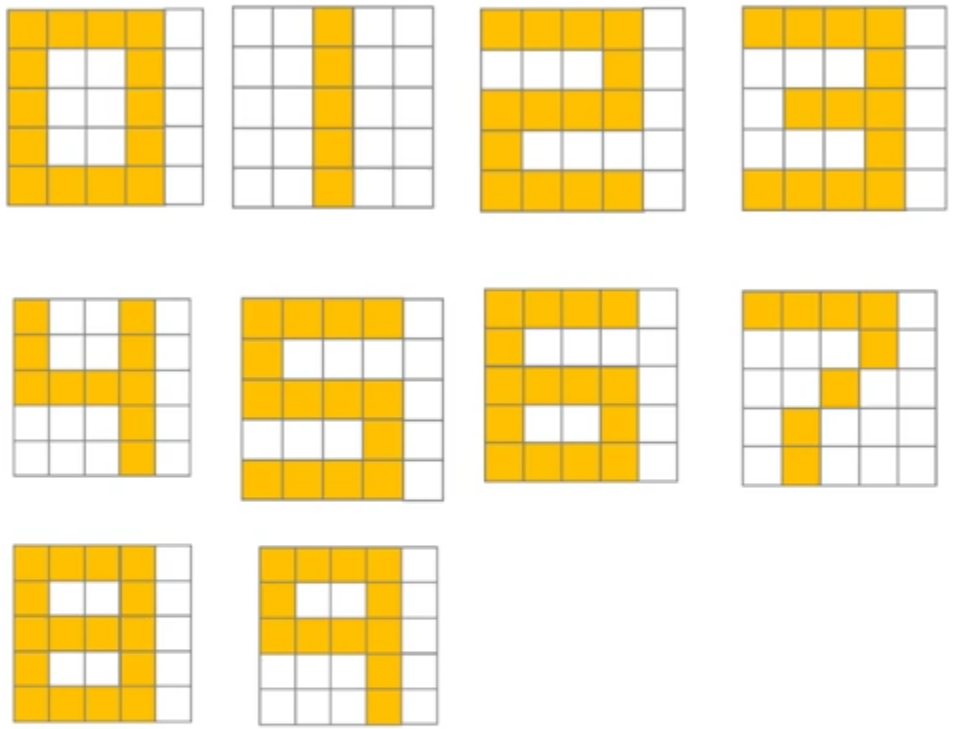
We will now have a look at some python code if you like to reproduce some of the results shown in this video. I will also show some code based on the classical MNIST data set.

Python code



We will first set up a code for a convolutional neural network to classify the same images that were used in our previous example.

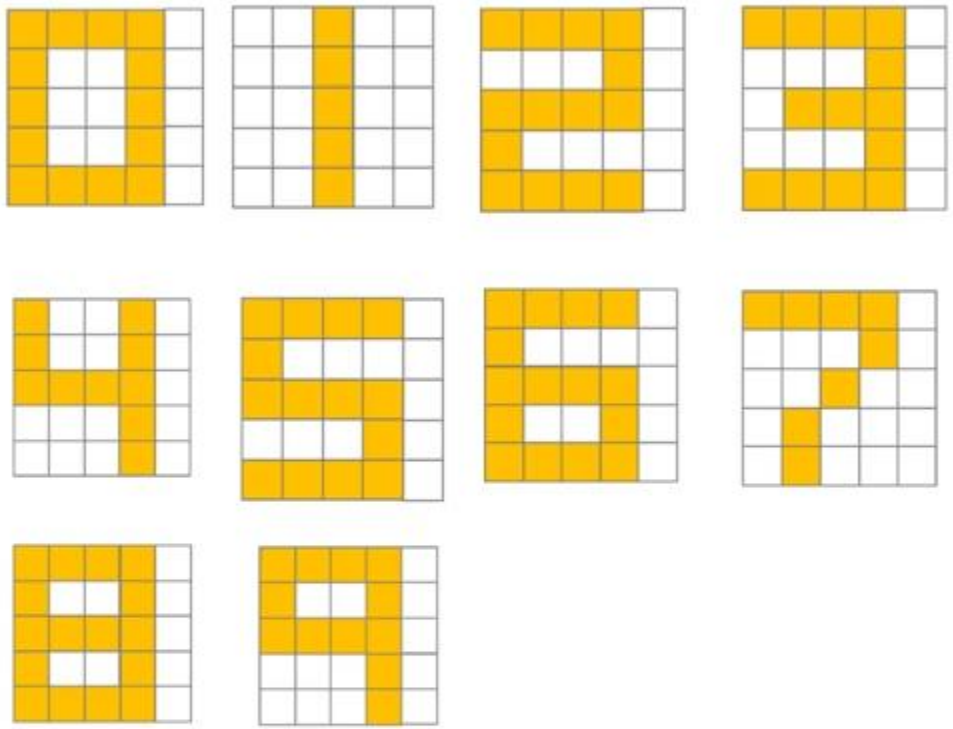
Python code



```
import numpy as np
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import random
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
```

We first import the following libraries,

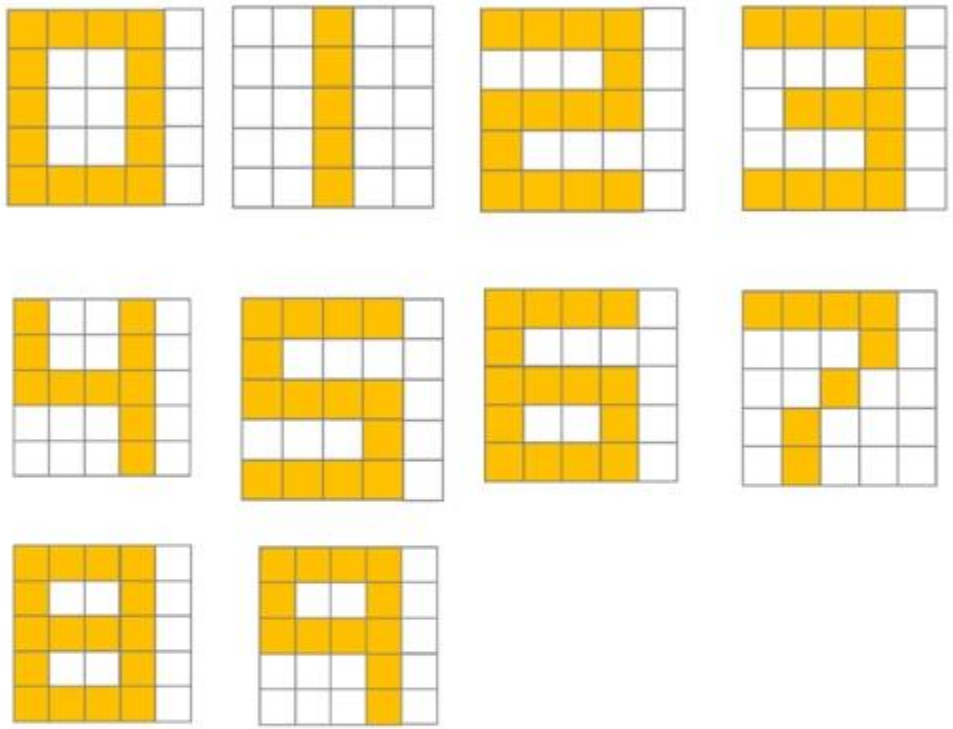
Python code



```
import numpy as np
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import random
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

zero = [1,1,1,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0]
one  = [0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0]
two  = [1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0]
three  = [1,1,1,1,0, 0,0,0,1,0, 0,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
four =  [1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]
five  = [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
six  =  [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
seven  =  [1,1,1,1,0, 0,0,0,1,0, 0,0,1,0,0, 0,1,0,0,0, 0,1,0,0,0]
eight= [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
nine  =  [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]

X = np.array([zero, one, two, three, four, five, six, seven, eight, nine])
X = X.reshape(10 ,5, 5)
```

and create a 3D array with the 10 images.

Python code



```
import numpy as np
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import random
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

zero = [1,1,1,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0]
one  = [0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0]
two  = [1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0]
three  = [1,1,1,1,0, 0,0,0,1,0, 0,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
four =  [1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]
five  = [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
six  =  [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
seven  =  [1,1,1,1,0, 0,0,0,1,0, 0,0,1,0,0, 0,1,0,0,0, 0,1,0,0,0]
eight= [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
nine  =  [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]

X = np.array([zero, one, two, three, four, five, six, seven, eight, nine])
X = X.reshape(10 ,5, 5)

y = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Then we create the following vector that tells which digit that is included in the image.

Python code

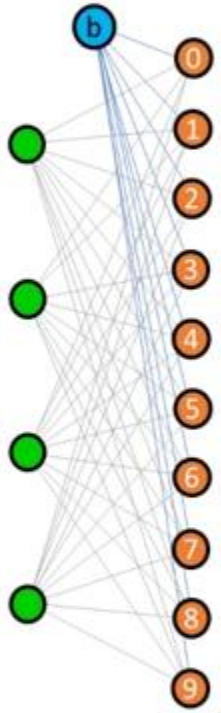
```
array([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 1., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 1., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 0., 1.]],
```

```
import numpy as np
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import random
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

zero = [1,1,1,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0]
one  = [0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0]
two  = [1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0]
three  = [1,1,1,1,0, 0,0,0,1,0, 0,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
four =  [1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]
five  = [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
six  =  [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
seven  =  [1,1,1,1,0, 0,0,0,1,0, 0,0,1,0,0, 0,1,0,0,0, 0,1,0,0,0]
eight= [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
nine  =  [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]

X = np.array([zero, one, two, three, four, five, six, seven, eight, nine])
X = X.reshape(10 ,5, 5)

y = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
y = to_categorical(y, 10)
```

Next, we use the following function to convert the integers in vector y into a binary 2D array.

Python code

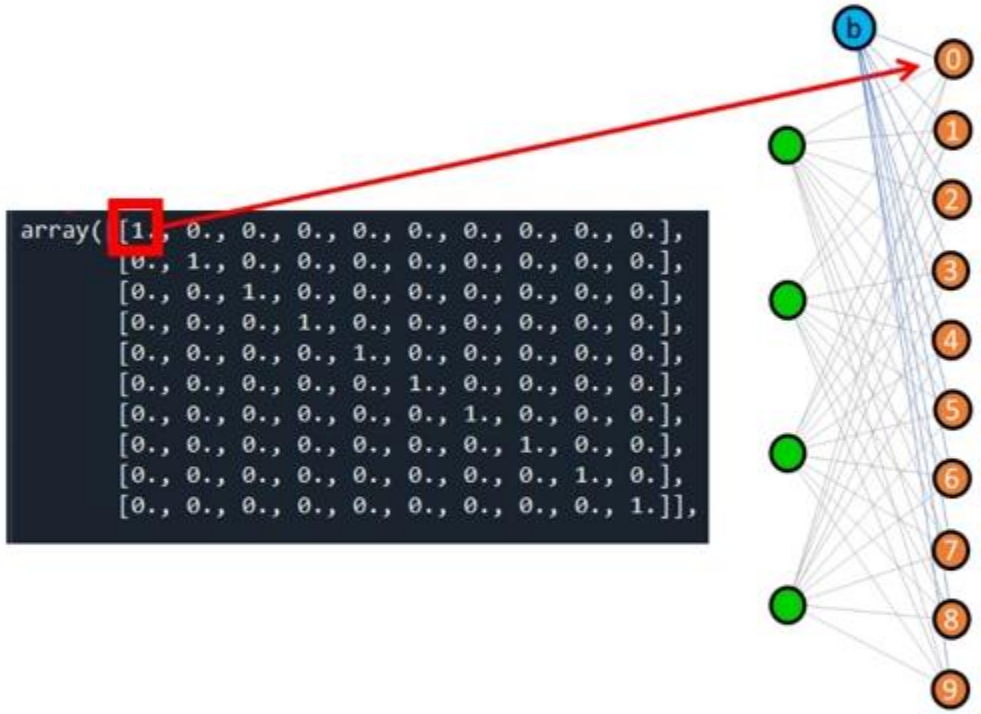
```
array([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 1., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 1., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 0., 1.]],
```




```
import numpy as np
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import random
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

zero = [1,1,1,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0]
one  = [0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0]
two  = [1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0]
three  = [1,1,1,1,0, 0,0,0,1,0, 0,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
four =  [1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]
five  = [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
six  =  [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
seven  =  [1,1,1,1,0, 0,0,0,1,0, 0,0,1,0,0, 0,1,0,0,0, 0,1,0,0,0]
eight= [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
nine  =  [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]

X = np.array([zero, one, two, three, four, five, six, seven, eight, nine])
X = X.reshape(10 ,5, 5)

y = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
y = to_categorical(y, 10)
```

For example, every row in this array will have 10 values,

Python code

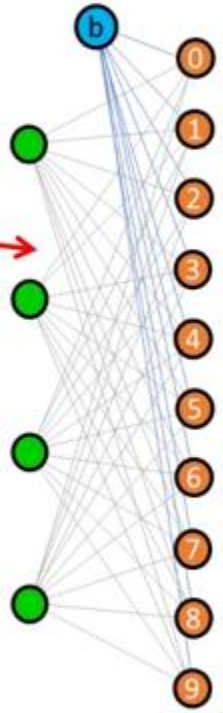



```
import numpy as np
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import random
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

zero = [1,1,1,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0]
one  = [0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0]
two  = [1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0]
three  = [1,1,1,1,0, 0,0,0,1,0, 0,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
four =  [1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]
five  = [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
six  =  [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
seven  =  [1,1,1,1,0, 0,0,0,1,0, 0,0,1,0,0, 0,1,0,0,0, 0,1,0,0,0]
eight= [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
nine  =  [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]

X = np.array([zero, one, two, three, four, five, six, seven, eight, nine])
X = X.reshape(10 ,5, 5)

y = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
y = to_categorical(y, 10)
```

The target probability value of the first output node should be equal to one because the first image contains a zero,

Python code

```
array([[1.  0., 0., 0., 0., 0., 0., 0., 0.,
        [0., 1., 0., 0., 0., 0., 0., 0., 0.],
        [0., 0., 1., 0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 1., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 1., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 1., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0., 1., 0., 0.],
        [0., 0., 0., 0., 0., 0., 0., 1., 0.],
        [0., 0., 0., 0., 0., 0., 0., 0., 1.]],
```

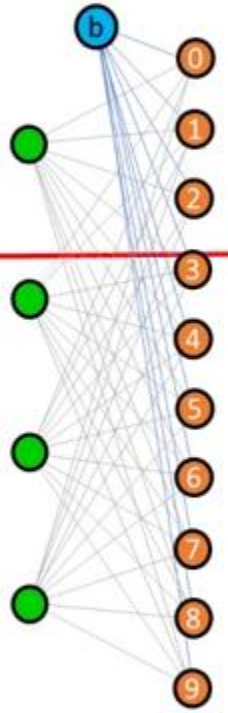



```
import numpy as np
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import random
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

zero = [1,1,1,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0]
one  = [0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0]
two  = [1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0]
three  = [1,1,1,1,0, 0,0,0,1,0, 0,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
four =  [1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]
five  = [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
six  =  [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
seven  =  [1,1,1,1,0, 0,0,0,1,0, 0,0,1,0,0, 0,1,0,0,0, 0,1,0,0,0]
eight= [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
nine  =  [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]

X = np.array([zero, one, two, three, four, five, six, seven, eight, nine])
X = X.reshape(10 ,5, 5)

y = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
y = to_categorical(y, 10)
```

whereas the other output nodes should have target probability values of zero.

Python code

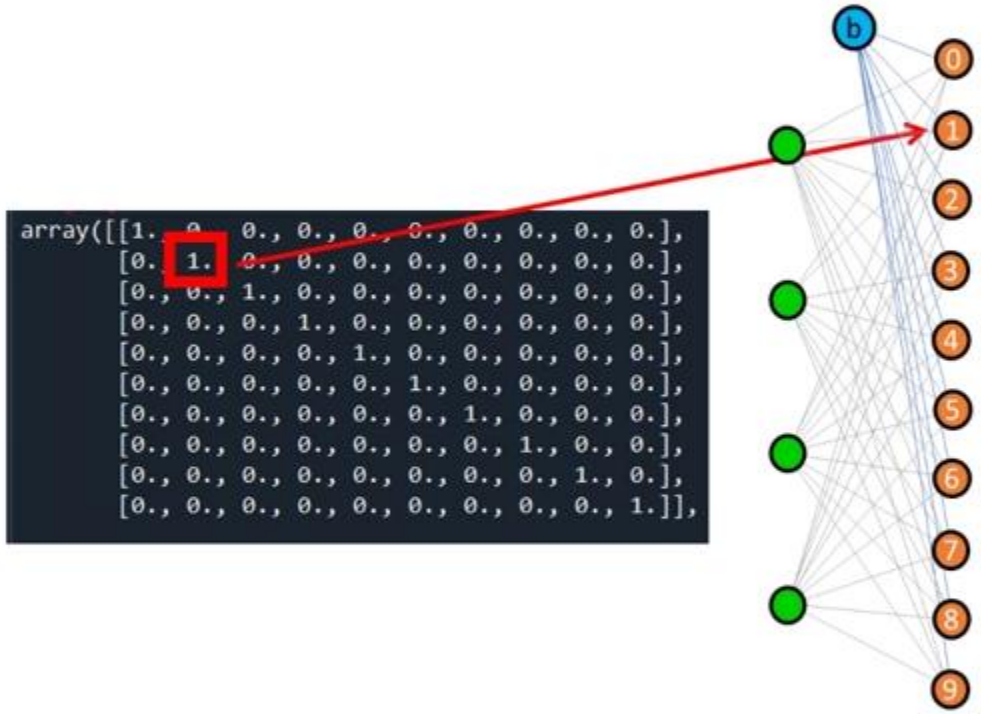
```
array([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0.]
       [0., 1., 0., 0., 0., 0., 0., 0., 0., 0.]
       [0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 1., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 1., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 0., 1.]],
```




```
import numpy as np
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import random
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

zero = [1,1,1,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0]
one  = [0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0]
two  = [1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0]
three  = [1,1,1,1,0, 0,0,0,1,0, 0,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
four =  [1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]
five  = [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
six  =  [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
seven  =  [1,1,1,1,0, 0,0,0,1,0, 0,0,1,0,0, 0,1,0,0,0, 0,1,0,0,0]
eight= [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
nine  =  [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]

X = np.array([zero, one, two, three, four, five, six, seven, eight, nine])
X = X.reshape(10 ,5, 5)

y = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
y = to_categorical(y, 10)
```

Since the second image represents a one,

Python code

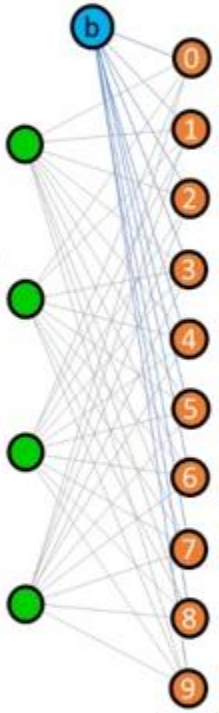



```
import numpy as np
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import random
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

zero = [1,1,1,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0]
one  = [0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0]
two  = [1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0]
three  = [1,1,1,1,0, 0,0,0,1,0, 0,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
four =  [1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]
five  = [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
six  =  [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
seven  =  [1,1,1,1,0, 0,0,0,1,0, 0,0,1,0,0, 0,1,0,0,0, 0,1,0,0,0]
eight= [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
nine  =  [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]

X = np.array([zero, one, two, three, four, five, six, seven, eight, nine])
X = X.reshape(10 ,5, 5)

y = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
y = to_categorical(y, 10)
```

the second target probability value should be equal to one,

Python code

```
array([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 1., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 1., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 0., 1.]],
```



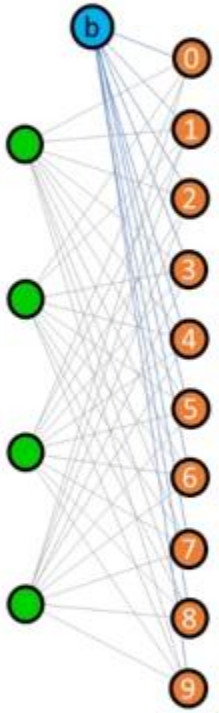

```
import numpy as np
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import random
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

zero = [1,1,1,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0]
one  = [0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0]
two  = [1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0]
three  = [1,1,1,1,0, 0,0,0,1,0, 0,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
four =  [1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]
five  = [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
six  =  [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
seven  =  [1,1,1,1,0, 0,0,0,1,0, 0,0,1,0,0, 0,1,0,0,0, 0,1,0,0,0]
eight= [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
nine  =  [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]

X = np.array([zero, one, two, three, four, five, six, seven, eight, nine])
X = X.reshape(10 ,5, 5)

y = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
y = to_categorical(y, 10)
```

whereas the rest should be equal to zero.

Python code

One-hot encoding

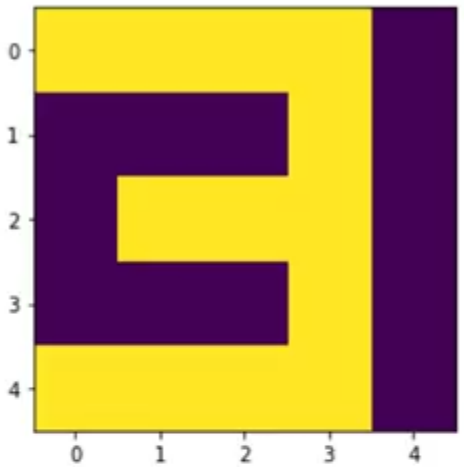
```
array([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 1., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 0., 1.]],
```




```
import numpy as np
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import random
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

zero = [1,1,1,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0]
one  = [0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0, 0,0,1,0,0]
two  = [1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0]
three  = [1,1,1,1,0, 0,0,0,1,0, 0,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
four =  [1,0,0,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]
five  = [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 0,0,0,1,0, 1,1,1,1,0]
six  =  [1,1,1,1,0, 1,0,0,0,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
seven  =  [1,1,1,1,0, 0,0,0,1,0, 0,0,1,0,0, 0,1,0,0,0, 0,1,0,0,0]
eight= [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0]
nine  =  [1,1,1,1,0, 1,0,0,1,0, 1,1,1,1,0, 0,0,0,1,0, 0,0,0,1,0]

X = np.array([zero, one, two, three, four, five, six, seven, eight, nine])
X = X.reshape(10 ,5, 5)

y = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
y = to_categorical(y, 10)
```

By the way, this type of encoding is called one-hot encoding.

Python code




```
plt.imshow(X[3])
```

Next, we could for example plot the fourth image in the data set, which is an image that represents a three.

Python code

```
plt.imshow(X[3])

random.seed(1)
```

To get a similar output that I will show later on, you need to set the following seed.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

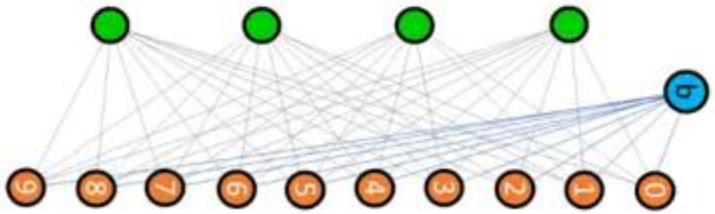
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
        kernel_size=(2, 2),
        strides=(1, 1),
        padding='valid',
        input_shape=(5, 5, 1),
        use_bias=False,
        activation='relu'),
  MaxPooling2D(pool_size=(2, 2)),
  Flatten(),
  Dense(10, activation='softmax')
  ])
```

Then we build the following network,

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

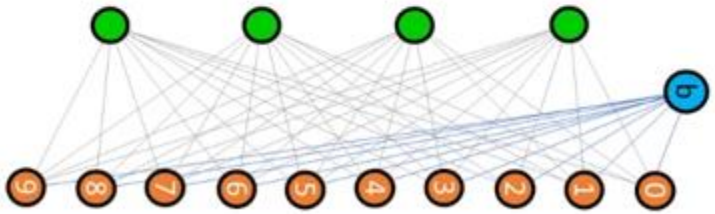
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
       kernel_size=(2, 2)
       strides=(1, 1),
       padding='valid',
       input_shape=(5, 5, 1),
       use_bias=False,
       activation='relu'),
  MaxPooling2D(pool_size=(2, 2)),
  Flatten(),
  Dense(10, activation='softmax')
  ])
```

with a size of two rows and two columns.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

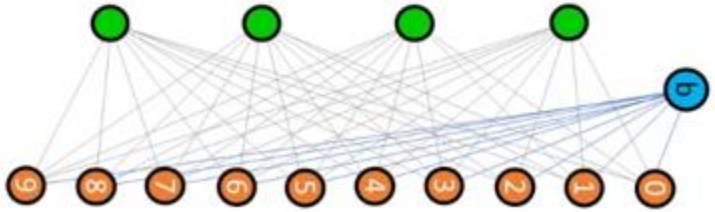
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
       kernel_size=(2, 2),
       strides=(1, 1),
       padding='valid',
       input_shape=(5, 5, 1),
       use_bias=False,
       activation='relu'),
  MaxPooling2D(pool_size=(2, 2)),
  Flatten(),
  Dense(10, activation='softmax')
  ])
```

Stride is here set to one, which means that the filter moves only one step down when the filter is moved downwards, and one step to the right when the filter is moved to the right.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

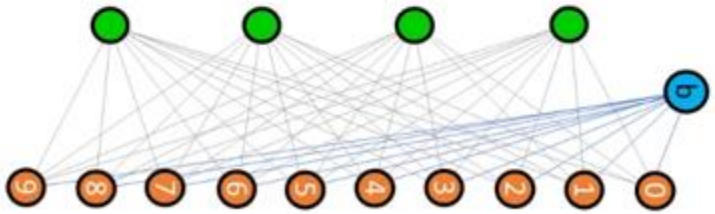
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
        kernel_size=(2, 2),
        strides=(1, 1),
        padding='valid',
        input_shape=(5, 5, 1),
        use_bias=False,
        activation='relu'),
  MaxPooling2D(pool_size=(2, 2)),
  Flatten(),
  Dense(10, activation='softmax')
  ])
```

In this example, we do not use any padding, which means that we set this argument to “valid”.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

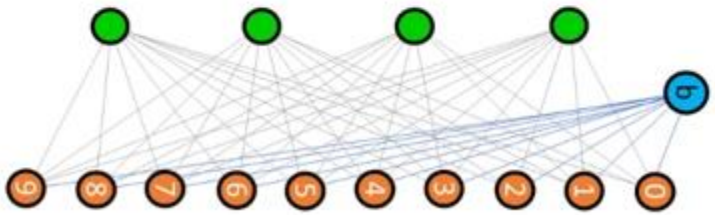
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
        kernel_size=(2, 2),
        strides=(1, 1),
        padding='same',
        input_shape=(5, 5, 1),
        use_bias=False,
        activation='relu'),
  MaxPooling2D(pool_size=(2, 2)),
  Flatten(),
  Dense(10, activation='softmax')
  ])
```

If you like to add padding, you can change this argument to “same”, which means that padding will be added in a way so that the feature map will have the same size as the original image.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

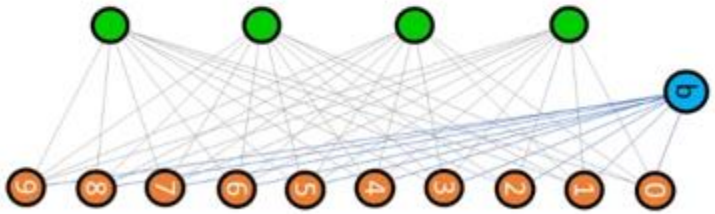
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
        kernel_size=(2, 2),
        strides=(1, 1),
        padding='valid',
        input_shape=(5, 5, 1),
        use_bias=False,
        activation='relu'),
  MaxPooling2D(pool_size=(2, 2)),
  Flatten(),
  Dense(10, activation='softmax')
  ])
```

We here define the size of the images, which is a 5 by 5 matrix, with only one channel because the image is represented by only one matrix.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

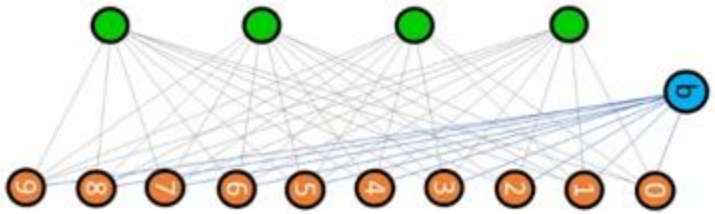
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
        kernel_size=(2, 2),
        strides=(1, 1),
        padding='valid',
        input_shape=(5, 5, 1),
        use_bias=False,
        activation='relu'),
  MaxPooling2D(pool_size=(2, 2)),
  Flatten(),
  Dense(10, activation='softmax')
  ])
```

During the convolution, we use the ReLU activation function and no bias weight.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

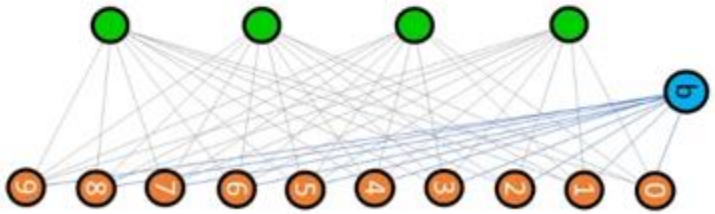
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
       kernel_size=(2, 2),
       strides=(1, 1),
       padding='valid',
       input_shape=(5, 5, 1),
       use_bias=False,
       activation='relu'),
  MaxPooling2D(pool_size=(2, 2)),
  Flatten(),
  Dense(10, activation='softmax')
  ])
```

Then we say that we like to use max pooling, with a 2 by 2 window.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

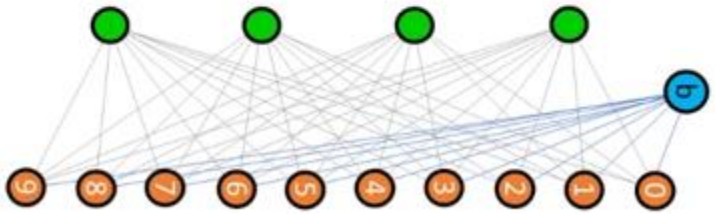
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
        kernel_size=(2, 2),
        strides=(1, 1),
        padding='valid',
        input_shape=(5, 5, 1),
        use_bias=False,
        activation='relu'),
  MaxPooling2D(pool_size=(2, 2)),
  Flatten(),
  Dense(10, activation='softmax')
  ])
```

Next, we flatten the pooled feature map,

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

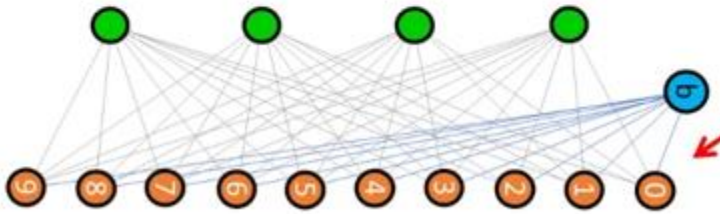
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
        kernel_size=(2, 2),
        strides=(1, 1),
        padding='valid',
        input_shape=(5, 5, 1),
        use_bias=False,
        activation='relu'),
   MaxPooling2D(pool_size=(2, 2)),
   Flatten(),
   Dense(10, activation='softmax')
   ])
```

and use a fully connected network with 10 output nodes,

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

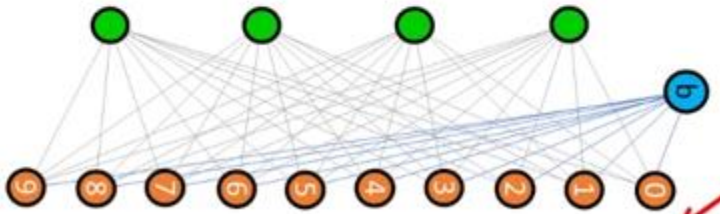
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
        kernel_size=(2, 2),
        strides=(1, 1),
        padding='valid',
        input_shape=(5, 5, 1),
        use_bias=False,
        activation='relu'),
  MaxPooling2D(pool_size=(2, 2)),
  Flatten(),
  Dense(10, activation='softmax')
  ])
```

where the predicted values are computed by the softmax function so that we get probabilities from the output nodes.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

Feature map

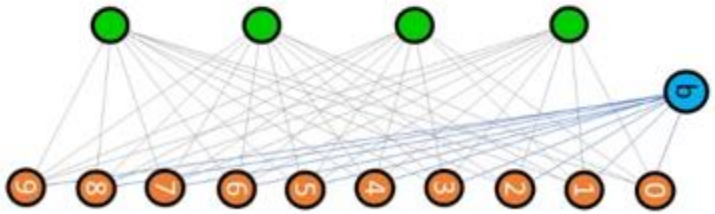
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
        kernel_size=(2, 2),
        strides=(1, 1),
        padding='valid',
        input_shape=(5, 5, 1),
        use_bias=False,
        activation='relu'),
   MaxPooling2D(pool_size=(2, 2)),
   Flatten(),
   Dense(10, activation='softmax')
   ])
model.summary()
```

If we run the model.summary function, we will see that 50 weights will be estimated in the fully connected network,

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

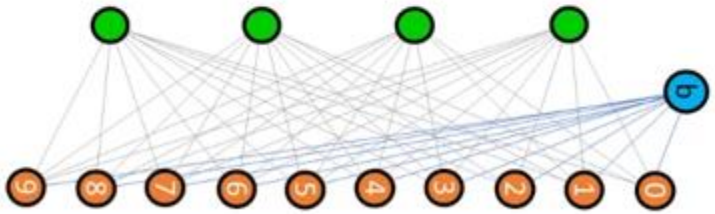
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
        kernel_size=(2, 2),
        strides=(1, 1),
        padding='valid',
        input_shape=(5, 5, 1),
        use_bias=False,
        activation='relu'),
   MaxPooling2D(pool_size=(2, 2)),
   Flatten(),
   Dense(10, activation='softmax')
   ])
model.summary()

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Then we compile the model, where we define that we like to use an optimizer called Adam,

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

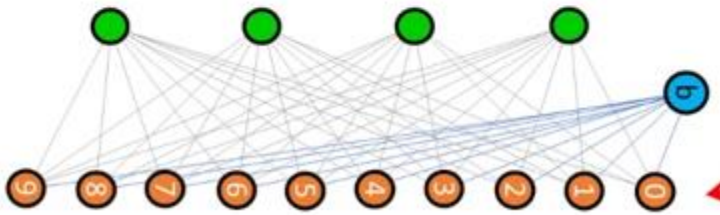
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
        kernel_size=(2, 2),
        strides=(1, 1),
        padding='valid',
        input_shape=(5, 5, 1),
        use_bias=False,
        activation='relu'),
   MaxPooling2D(pool_size=(2, 2)),
   Flatten(),
   Dense(10, activation='softmax')
   ])
model.summary()

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

and use the categorical cross-entropy loss function because we have more than 2 categories to predict.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

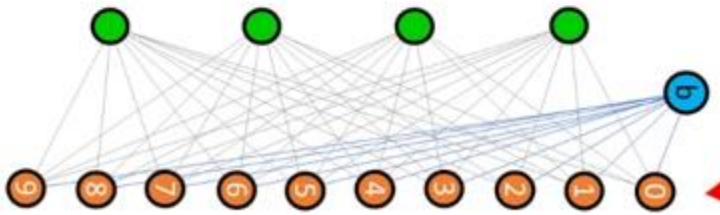
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
plt.imshow(X[3])

random.seed(1)

model = Sequential([
Conv2D(filters=1,
        kernel_size=(2, 2),
        strides=(1, 1),
        padding='valid',
        input_shape=(5, 5, 1),
        use_bias=False,
        activation='relu'),
   MaxPooling2D(pool_size=(2, 2)),
   Flatten(),
   Dense(10, activation='softmax')
   ])
model.summary()

model.compile(optimizer='adam', loss='categorical_crossentropy' metrics=['accuracy'])
```

Then we say that we like to monitor the accuracy during the training process, which means that we will see the proportion of correct predictions after each epoch during the training.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

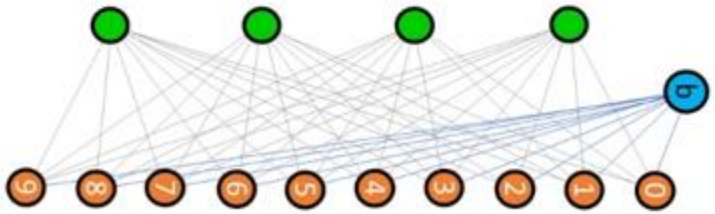
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
history=model.fit(X, y, epochs=1000, batch_size=1, verbose=1)
```

Then we train the network, by using 1000 epochs, which takes less than one minute on my computer.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

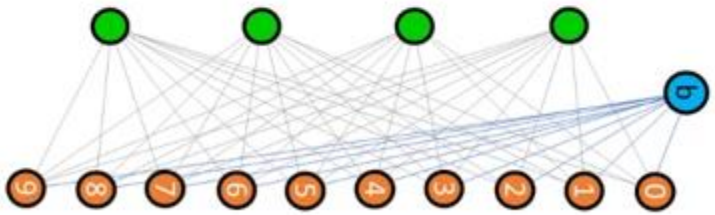
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
history=model.fit(X, y, epochs=1000, batch_size=1, verbose=1)
```

By setting the batch size to one, means that only one image is used in each iteration to update the weights.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

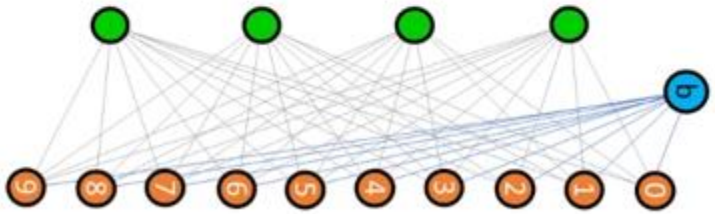
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

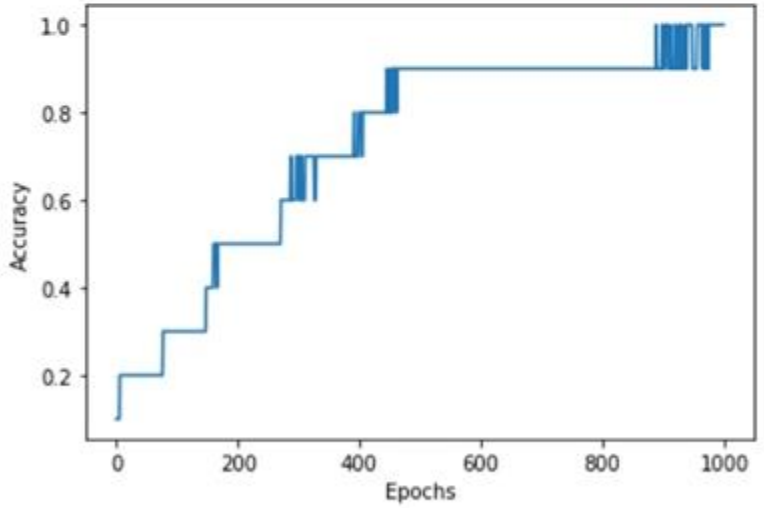
1	3
3	2

Flatten

1	3	3	2




```
history=model.fit(X, y, epochs=1000, batch_size=1, verbose=1)

plt.figure()
plt.plot(history.history['accuracy'])
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.show()
```



If we like, we can plot how the accuracy has changed during the training process.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

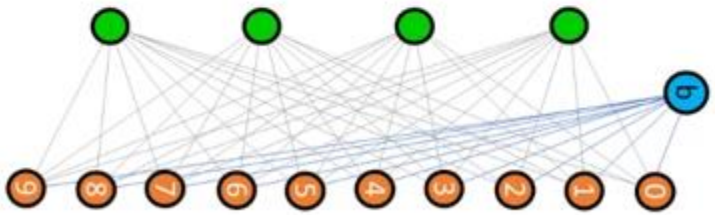
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

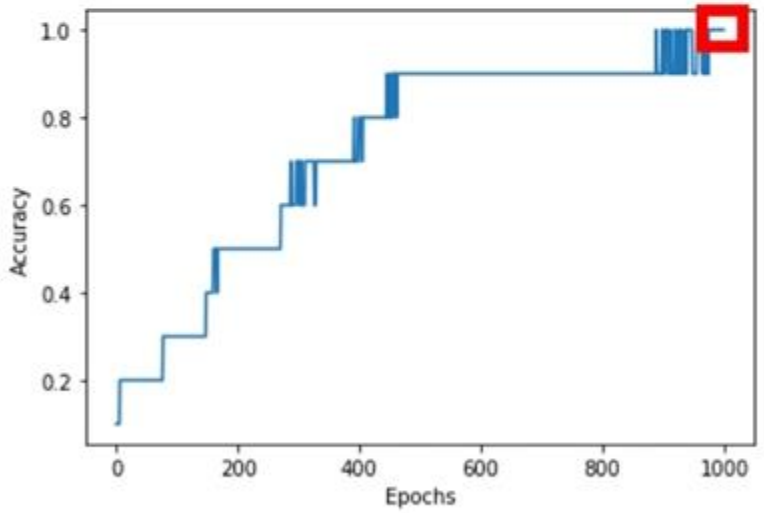
1	3
3	2

Flatten

1	3	3	2




```
history=model.fit(X, y, epochs=1000, batch_size=1, verbose=1)

plt.figure()
plt.plot(history.history['accuracy'])
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.show()
```



We see that the accuracy is 100% after 1000 epochs, which means that all 10 images are correctly classified by the network.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

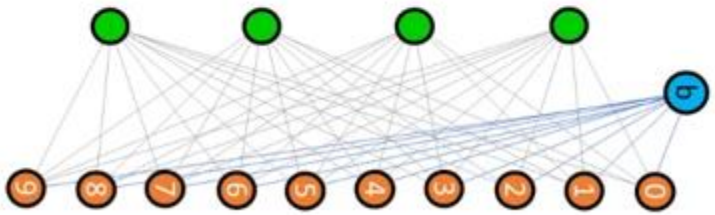
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
history=model.fit(X, y, epochs=1000, batch_size=1, verbose=1)

plt.figure()
plt.plot(history.history['accuracy'])
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.show()

model.predict(X[[0]])
```

We can now use the trained network, to see, for example, how it predicts the first image.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

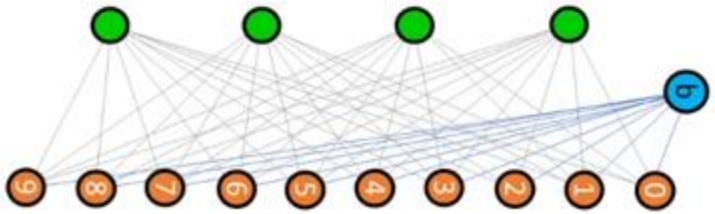
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
history=model.fit(X, y, epochs=1000, batch_size=1, verbose=1)

plt.figure()
plt.plot(history.history['accuracy'])
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.show()

model.predict(X[[0]])
```

```
array([[6.1261553e-01, 5.1606115e-08, 1.6133866e-03,
7.2489558e-03, 7.3408702e-04, 1.8421921e-01, 1.1932706e-01,
5.9056937e-10, 7.3687300e-02, 5.5441976e-04]], dtype=float32)
```

The output gives us the predicted probabilities for each of the ten digits.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

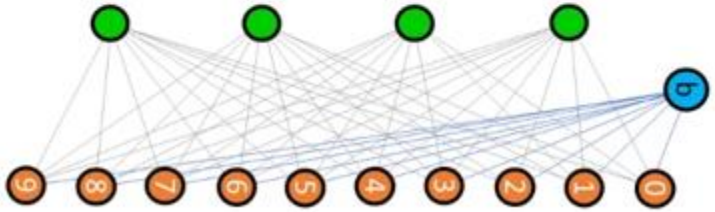
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
history=model.fit(X, y, epochs=1000, batch_size=1, verbose=1)

plt.figure()
plt.plot(history.history['accuracy'])
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.show()

model.predict(X[[0]])
```

```
        0                1                2
[[6.1261553e-01, 5.1606115e-08, 1.6133866e-03,
        3                4                5                6
 7.2489558e-03, 7.3408702e-04, 1.8421921e-01, 1.1932706e-01,
        7                8                9
 5.9056937e-10, 7.3687300e-02, 5.5441976e-04]]
```

We see that the network says that there is a 61.2% probability that the image contains a zero,

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

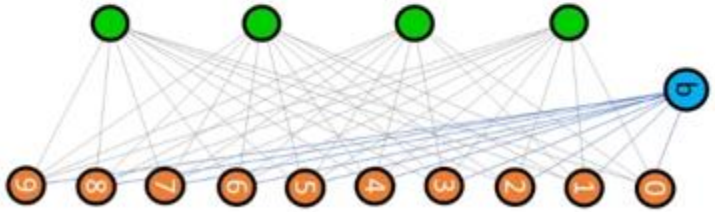
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
history=model.fit(X, y, epochs=1000, batch_size=1, verbose=1)

plt.figure()
plt.plot(history.history['accuracy'])
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.show()

model.predict(X[[0]])
```

```
        0             1             2
[[6.1261553e-01, 5.1606115e-08, 1.6133866e-03,
        3             4             5             6
 7.2489558e-03, 7.3408702e-04, 1.8421921e-01, 1.1932706e-01,
        7             8             9
 5.9056937e-10, 7.3687300e-02, 5.5441976e-04]]
```

whereas the probability that the image contains a one is almost zero.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

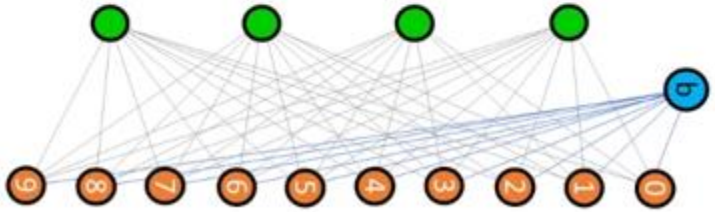
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
history=model.fit(X, y, epochs=1000, batch_size=1, verbose=1)

plt.figure()
plt.plot(history.history['accuracy'])
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.show()

model.predict(X[[0]])
```

```
        0             1             2
[[6.1261553e-01, 5.1606115e-08, 1.6133866e-03,
        3             4             5             6
 7.2489558e-03, 7.3408702e-04, 1.8421921e-01, 1.1932706e-01,
        7             8             9
 5.9056937e-10, 7.3687300e-02, 5.5441976e-04]]
```

The probability that the image contains a five, is about 18%.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

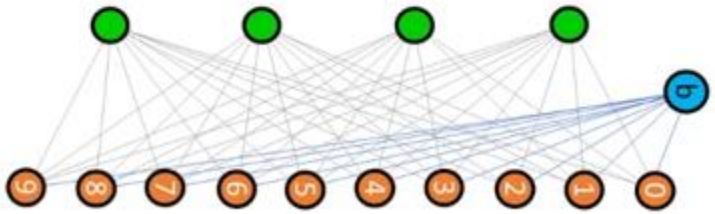
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
history=model.fit(X, y, epochs=1000, batch_size=1, verbose=1)

plt.figure()
plt.plot(history.history['accuracy'])
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.show()

model.predict(X[[0]])

np.argmax(model.predict(X[[0]]))
```

0

If we use the argmax function, we can extract the index of the output array that has the largest probability value.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

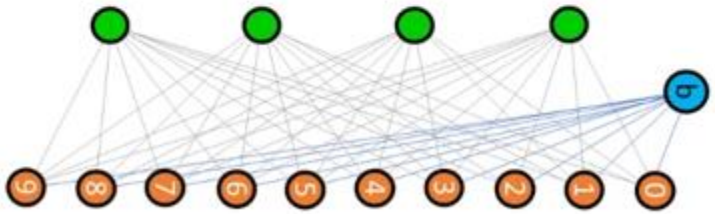
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
history=model.fit(X, y, epochs=1000, batch_size=1, verbose=1)

plt.figure()
plt.plot(history.history['accuracy'])
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.show()

model.predict(X[[0]])

np.argmax(model.predict(X[[0]]))
```

0

Since this index is equal to zero, the network predicts that the image contains a zero.

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

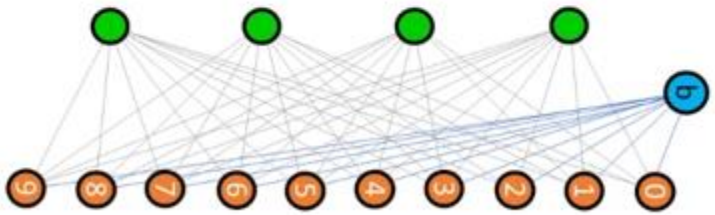
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
model.get_weights()
```

```
[array([[[[-1.1598074]],
         [[ 2.6410077]]],
        [[[ 5.1395645]],
         [[-1.866515 ]]]], dtype=float32),

array([[ 7.4895257e-01, -2.9651778e+00, -1.3097980e+00, -8.8790041e-01,
        -6.1686877e-03,  2.8675288e-01, -1.6478071e-01, -1.5060074e+00,
         1.9352853e-02,  1.1315626e+00],
       [-2.0203490e+00,  9.1947848e-01, -7.4016556e-02,  6.0655999e-01,
         7.9335028e-01, -2.2737984e-01, -6.8931669e-01,  6.6208267e-01,
        -2.7298284e-01,  5.7787257e-01],
       [-3.1617180e-01, -2.3955319e+00,  6.4576072e-01, -6.7359900e-01,
        -3.4072094e+00, -1.2921858e+00,  1.9157723e-02,  1.0847864e+00,
        -1.1094862e-03, -3.5361929e+00],
       [ 6.0592264e-01,  1.2454947e+00,  4.7832344e-02,  1.9928998e-01,
         4.3769455e-01,  4.8894987e-01,  3.4555271e-01, -3.0668852e+00,
        -2.2425653e-01, -5.9803623e-01]], dtype=float32),

array([ 0.19714221,  1.2909998 , -0.38536796, -0.02965794,  0.56840634,
       -0.5038543 , -0.4809876 , -0.16292042, -0.38849473,  0.25129244],
      dtype=float32)]
```

By using the following line of code, we can extract the optimized weights,

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$$\begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \end{bmatrix}$$

Feature map

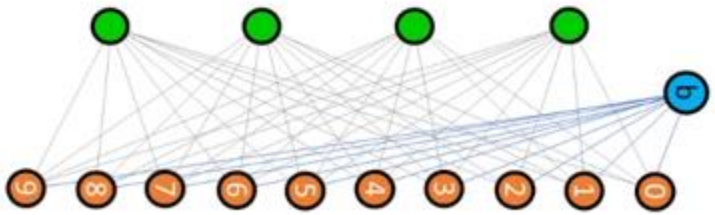
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
model.get_weights()
```

```
[array([[[[-1.1598074]],
         [[ 2.6410077]]],
        [[[ 5.1395645]],
         [[-1.866515 ]]]], dtype=float32),
```

```
array([[[ 7.4895257e-01, -2.9651778e+00, -1.3097980e+00, -8.8790041e-01,
          -6.1686877e-03,  2.8675288e-01, -1.6478071e-01, -1.5060074e+00,
           1.9352853e-02,  1.1315626e+00],
         [-2.0203490e+00,  9.1947848e-01, -7.4016556e-02,  6.0655999e-01,
           7.9335028e-01, -2.2737984e-01, -6.8931669e-01,  6.6208267e-01,
          -2.7298284e-01,  5.7787257e-01],
         [-3.1617180e-01, -2.3955319e+00,  6.4576072e-01, -6.7359900e-01,
          -3.4072094e+00, -1.2921858e+00,  1.9157723e-02,  1.0847864e+00,
          -1.1094862e-03, -3.5361929e+00],
         [ 6.0592264e-01,  1.2454947e+00,  4.7832344e-02,  1.9928998e-01,
           4.3769455e-01,  4.8894987e-01,  3.4555271e-01, -3.0668852e+00,
          -2.2425653e-01, -5.9803623e-01]], dtype=float32),

array([ 0.19714221,  1.2909998 , -0.38536796, -0.02965794,  0.56840634,
        -0.5038543 , -0.4809876 , -0.16292042, -0.38849473,  0.25129244],
       dtype=float32)]
```

where these weights are the ones that are used in the filter,

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

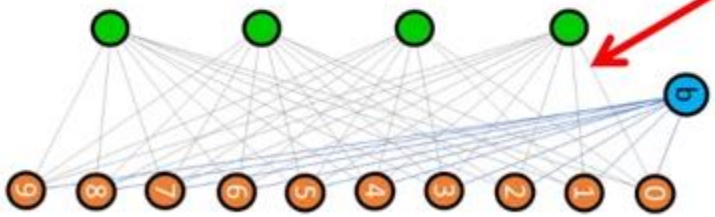
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2




```
model.get_weights()
```

```
[array([[[[-1.1598074]],
         [[ 2.6410077]]],
        [[[ 5.1395645]],
         [[-1.866515 ]]]], dtype=float32),
 array([[ 7.4895257e-01, -2.9651778e+00, -1.3097980e+00, -8.8790041e-01,
         -6.1686877e-03,  2.8675288e-01, -1.6478071e-01, -1.5060074e+00,
          1.9352853e-02,  1.1315626e+00],
        [-2.0203490e+00,  9.1947848e-01, -7.4016556e-02,  6.0655999e-01,
          7.9335028e-01, -2.2737984e-01, -6.8931669e-01,  6.6208267e-01,
         -2.7298284e-01,  5.7787257e-01],
        [-3.1617180e-01, -2.3955319e+00,  6.4576072e-01, -6.7359900e-01,
         -3.4072094e+00, -1.2921858e+00,  1.9157723e-02,  1.0847864e+00,
         -1.1094862e-03, -3.5361929e+00],
        [ 6.0592264e-01,  1.2454947e+00,  4.7832344e-02,  1.9928998e-01,
          4.3769455e-01,  4.8894987e-01,  3.4555271e-01, -3.0668852e+00,
         -2.2425653e-01, -5.9803623e-01]], dtype=float32),
 array([ 0.19714221,  1.2909998 , -0.38536796, -0.02965794,  0.56840634,
         -0.5038543 , -0.4809876 , -0.16292042, -0.38849473,  0.25129244],
       dtype=float32)]
```

and these weights are the ones associated with the input nodes,

Python code

Image 5 x 5

1	1	1	1	0
1	0	0	1	0
1	0	0	1	0
1	0	0	1	0
1	1	1	1	0

$w_{1,1}$	$w_{1,2}$
$w_{2,1}$	$w_{2,2}$

Feature map

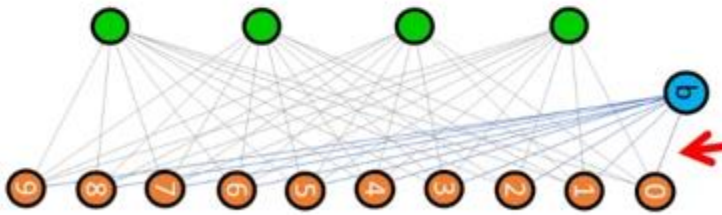
1	1	3	1
1	0	2	1
1	0	2	1
3	2	2	1

Pooled feature map

1	3
3	2

Flatten

1	3	3	2

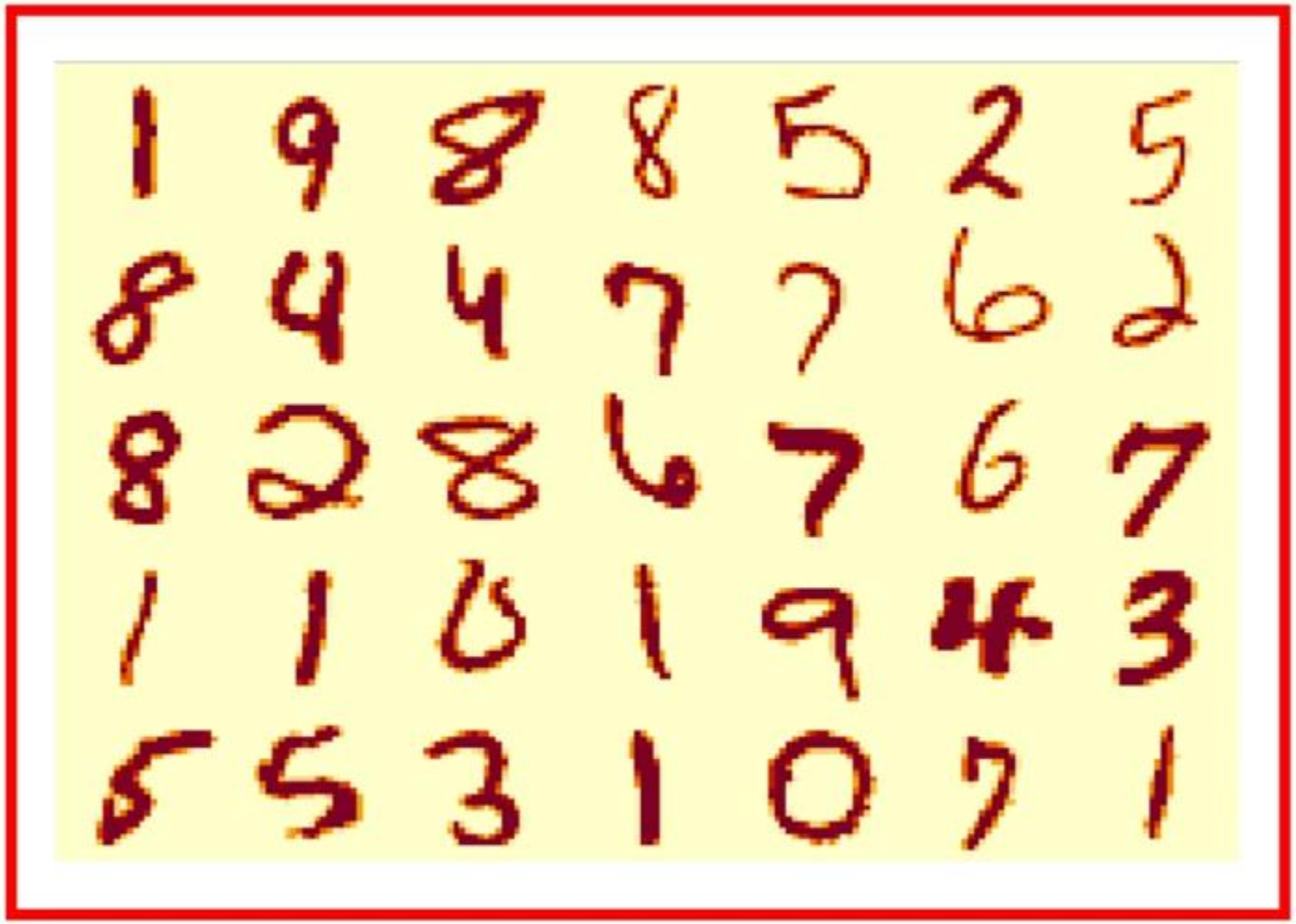



```
model.get_weights()
```

```
[array([[[[-1.1598074]],
         [[ 2.6410077]]],
        [[[ 5.1395645]],
         [[-1.866515 ]]]], dtype=float32),

array([[ 7.4895257e-01, -2.9651778e+00, -1.3097980e+00, -8.8790041e-01,
        -6.1686877e-03,  2.8675288e-01, -1.6478071e-01, -1.5060074e+00,
         1.9352853e-02,  1.1315626e+00],
       [-2.0203490e+00,  9.1947848e-01, -7.4016556e-02,  6.0655999e-01,
         7.9335028e-01, -2.2737984e-01, -6.8931669e-01,  6.6208267e-01,
        -2.7298284e-01,  5.7787257e-01],
       [-3.1617180e-01, -2.3955319e+00,  6.4576072e-01, -6.7359900e-01,
        -3.4072094e+00, -1.2921858e+00,  1.9157723e-02,  1.0847864e+00,
        -1.1094862e-03, -3.5361929e+00],
       [ 6.0592264e-01,  1.2454947e+00,  4.7832344e-02,  1.9928998e-01,
         4.3769455e-01,  4.8894987e-01,  3.4555271e-01, -3.0668852e+00,
        -2.2425653e-01, -5.9803623e-01]], dtype=float32),

array([ 0.19714221,  1.2909998 , -0.38536796, -0.02965794,  0.56840634,
       -0.5038543 , -0.4809876 , -0.16292042, -0.38849473,  0.25129244],
      dtype=float32)]
```

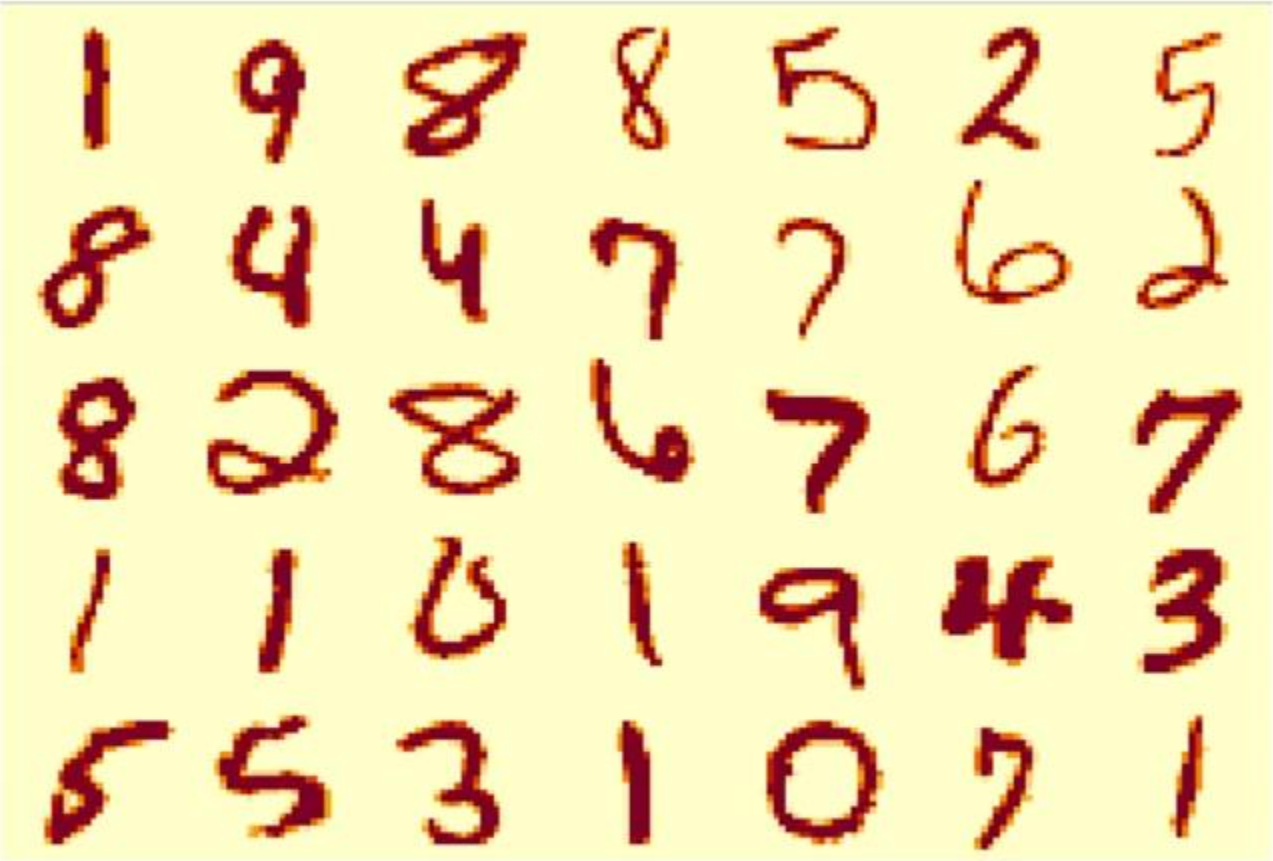
whereas these are the bias weights.

MNIST



Finally, we will see how to train a CNN based on the classical MNIST data set, which contains 70.000 images of handwritten digits.

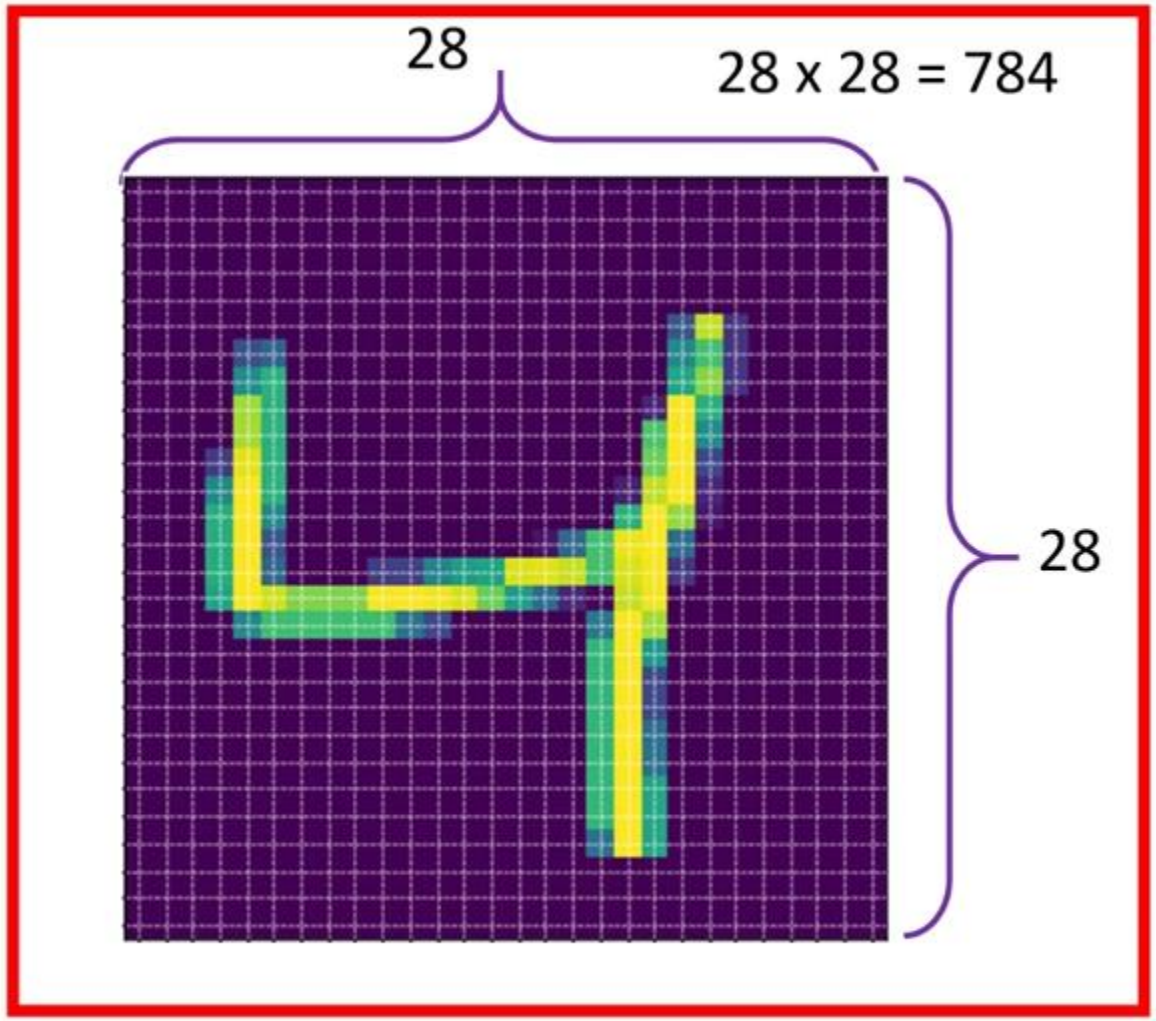
MNIST



Training data: 60.000 images
Test data: 10.000 images

60.000 of these images are dedicated for training and 10.000 for testing.

MNIST



Each image has a height and width of 28 pixels, which means that every image consists of 784 pixels.

Python code

```
from tensorflow.keras.utils import to_categorical
from keras.models import Sequential
from keras.layers import Dense, Flatten
from keras.layers import Conv2D, MaxPooling2D
import matplotlib.pyplot as plt
from keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

To train a CNN based on the MNIST data set, we load the following libraries.

Python code

```
from tensorflow.keras.utils import to_categorical
from keras.models import Sequential
from keras.layers import Dense, Flatten
from keras.layers import Conv2D, MaxPooling2D
import matplotlib.pyplot as plt
from keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.reshape(60000, 28, 28, 1) # Samples, height, width, channels
x_test = x_test.reshape(10000, 28, 28, 1) # Samples, height, width, channels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

x_train = x_train/255.0
x_test = x_test/255.0

plt.imshow(x_train[2])
```

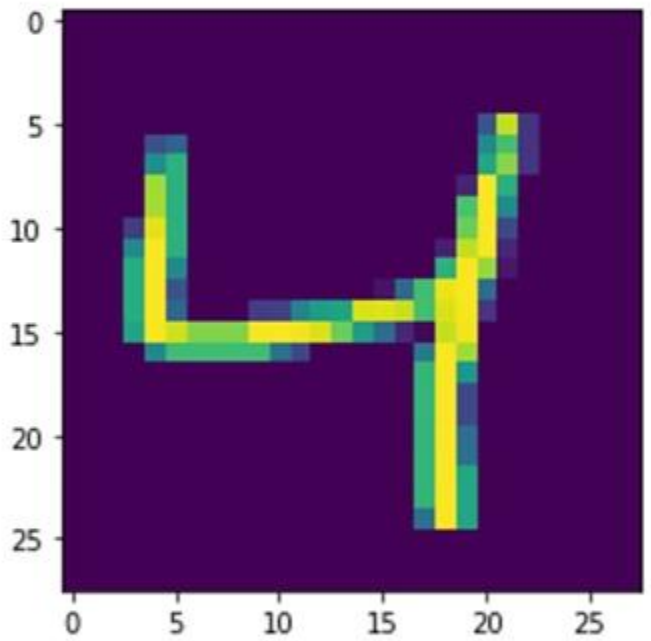
Then we set up the training data, with 60.000 images, and the test data with 10.000 images.

Python code

```
from tensorflow.keras.utils import to_categorical
from keras.models import Sequential
from keras.layers import Dense, Flatten
from keras.layers import Conv2D, MaxPooling2D
import matplotlib.pyplot as plt
from keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.reshape(60000, 28, 28, 1) # Samples, height, width, channels
x_test = x_test.reshape(10000, 28, 28, 1) # Samples, height, width, channels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

x_train = x_train/255.0
x_test = x_test/255.0

plt.imshow(x_train[2])
```

Next, we normalize the data so that all pixels have values between zero and one.

Python code




```
from tensorflow.keras.utils import to_categorical
from keras.models import Sequential
from keras.layers import Dense, Flatten
from keras.layers import Conv2D, MaxPooling2D
import matplotlib.pyplot as plt
from keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.reshape(60000, 28, 28, 1) # Samples, height, width, channels
x_test = x_test.reshape(10000, 28, 28, 1) # Samples, height, width, channels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

x_train = x_train/255.0
x_test = x_test/255.0

plt.imshow(x_train[2])
```

If we like, we can plot, for example, the third image in the training data, which shows a hand-written four.

Python code

```
model = Sequential([
Conv2D(32, kernel_size=(3, 3),activation='relu',input_shape=(28,28,1)),
Conv2D(64, kernel_size=(3, 3), activation='relu'),
MaxPooling2D(pool_size=(2, 2)),
Flatten(),
Dense(100, activation='sigmoid'),
Dense(10, activation='softmax')])
```

Then we build the following network,

Python code

```
model = Sequential([
Conv2D(32, kernel_size=(3, 3),activation='relu',input_shape=(28,28,1)),
Conv2D(64, kernel_size=(3, 3), activation='relu'),
MaxPooling2D(pool_size=(2, 2)),
Flatten(),
Dense(100, activation='sigmoid'),
Dense(10, activation='softmax')])
```

and 64 filters in the second convolution.

Python code

```
model = Sequential([
Conv2D(32, kernel_size=(3, 3),activation='relu',input_shape=(28,28,1)),
Conv2D(64, kernel_size=(3, 3), activation='relu'),
MaxPooling2D(pool_size=(2, 2)),
Flatten(),
Dense(100, activation='sigmoid'),
Dense(10, activation='softmax')])
```

The size of the filters corresponds to a 3 by 3 matrix.

Python code

```
model = Sequential([
Conv2D(32, kernel_size=(3, 3),activation='relu',input_shape=(28,28,1)),
Conv2D(64, kernel_size=(3, 3), activation='relu'),
MaxPooling2D(pool_size=(2, 2)),
Flatten(),
Dense(100, activation='sigmoid'),
Dense(10, activation='softmax')])
```

We here also use a hidden layer with 100 nodes in the fully connected network.

Python code

```
model = Sequential([
Conv2D(32, kernel_size=(3, 3),activation='relu',input_shape=(28,28,1)),
Conv2D(64, kernel_size=(3, 3), activation='relu'),
MaxPooling2D(pool_size=(2, 2)),
Flatten(),
Dense(100, activation='sigmoid'),
Dense(10, activation='softmax')])

model.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accuracy'])
model.fit(x_train, y_train, batch_size=100, epochs=10,verbose=1)
```

I here use a batch size of 100, which means that 100 images are used in each iteration to update the weights during the training. This means that 600 iterations are used in each epoch because we have 60.000 images to train on.

Python code

```
model = Sequential([
Conv2D(32, kernel_size=(3, 3),activation='relu',input_shape=(28,28,1)),
Conv2D(64, kernel_size=(3, 3), activation='relu'),
MaxPooling2D(pool_size=(2, 2)),
Flatten(),
Dense(100, activation='sigmoid'),
Dense(10, activation='softmax')])

model.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accuracy'])
model.fit(x_train, y_train,batch_size=100,epochs=10,verbose=1)
```

Training this network, by using 10 epochs, takes about 5 minutes on my computer.

Python code

```
model = Sequential([
Conv2D(32, kernel_size=(3, 3),activation='relu',input_shape=(28,28,1)),
Conv2D(64, kernel_size=(3, 3), activation='relu'),
MaxPooling2D(pool_size=(2, 2)),
Flatten(),
Dense(100, activation='sigmoid'),
Dense(10, activation='softmax')])

model.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accuracy'])
model.fit(x_train, y_train,batch_size=100,epochs=10,verbose=1)
model.evaluate(x_test, y_test)
```

~99%

The network predicts the correct digit in about 99% of the 10.000 images in the test data set. However, every time you run this code, you will likely get slightly different accuracies due to the random processes during the training.

Python code

```
model = Sequential([
Conv2D(32, kernel_size=(3, 3),activation='relu',input_shape=(28,28,1)),
Conv2D(64, kernel_size=(3, 3), activation='relu'),
MaxPooling2D(pool_size=(2, 2)),
Flatten(),
Dense(100, activation='sigmoid'),
Dense(10, activation='softmax')])

model.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accuracy'])
model.fit(x_train, y_train,batch_size=100,epochs=10,verbose=1)
model.evaluate(x_test, y_test)
```

~99%

Note that you can get a higher accuracy by tuning on the hyperparameters and by using dropout layers to prevent overfitting.

Python code

```
import numpy as np
from sklearn.metrics import confusion_matrix
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = np.argmax(y_test, axis=1)
print(confusion_matrix(y_true_classes, y_pred_classes))
```

```
array([[ 974,    0,    2,    1,    0,    0,    1,    0,    2,    0],
       [   0, 1128,    1,    2,    1,    0,    2,    0,    1,    0],
       [   1,    1, 1021,    2,    1,    0,    0,    6,    0,    0],
       [   0,    0,    0, 1007,    0,    2,    0,    0,    1,    0],
       [   1,    0,    1,    0,  974,    0,    3,    0,    0,    3],
       [   1,    0,    0,    5,    0,  884,    2,    0,    0,    0],
       [   2,    2,    0,    2,    1,    7,  943,    0,    1,    0],
       [   0,    3,    6,    2,    1,    1,    0, 1013,    1,    1],
       [   3,    0,    2,    3,    1,    5,    0,    3,  953,    4],
       [   0,    1,    1,    2,    9,    6,    0,    3,    1,  986]],
```

If we like, we can generate a confusion matrix,

Python code

```
import numpy as np
from sklearn.metrics import confusion_matrix
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = np.argmax(y_test, axis=1)
print(confusion_matrix(y_true_classes, y_pred_classes))
```

```
array([[ 974,    0,    2,    1,    0,    0,    1,    0,    2,    0],
       [   0, 1128,    1,    2,    1,    0,    2,    0,    1,    0],
       [   1,    1, 1021,    2,    1,    0,    0,    6,    0,    0],
       [   0,    0,    0, 1007,    0,    2,    0,    0,    1,    0],
       [   1,    0,    1,    0,  974,    0,    3,    0,    0,    3],
       [   1,    0,    0,    5,    0,  884,    2,    0,    0,    0],
       [   2,    2,    0,    2,    1,    7,  943,    0,    1,    0],
       [   0,    3,    6,    2,    1,    1,    0, 1013,    1,    1],
       [   3,    0,    2,    3,    1,    5,    0,    3,  953,    4],
       [   0,    1,    1,    2,    9,    6,    0,    3,    1,  986]],
```

that for example shows that 974 out of the 980 images that include a zero are correctly classified.

Python code

```
import numpy as np
from sklearn.metrics import confusion_matrix
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = np.argmax(y_test, axis=1)
print(confusion_matrix(y_true_classes, y_pred_classes))
```

```
array([[ 974,    0,    2,    1,    0,    0,    1,    0,    2,    0],
       [   0, 1128,    1,    2,    1,    0,    2,    0,    1,    0],
       [   1,    1, 1021,    2,    1,    0,    0,    6,    0,    0],
       [   0,    0,    0, 1007,    0,    2,    0,    0,    1,    0],
       [   1,    0,    1,    0,  974,    0,    3,    0,    0,    3],
       [   1,    0,    0,    5,    0,  884,    2,    0,    0,    0],
       [   2,    2,    0,    2,    1,    7,  943,    0,    1,    0],
       [   0,    3,    6,    2,    1,    1,    0, 1013,    1,    1],
       [   3,    0,    2,    3,    1,    5,    0,    3,  953,    4],
       [   0,    1,    1,    2,    9,    6,    0,    3,    1,  986]],
```

Two images were misclassified as eights.

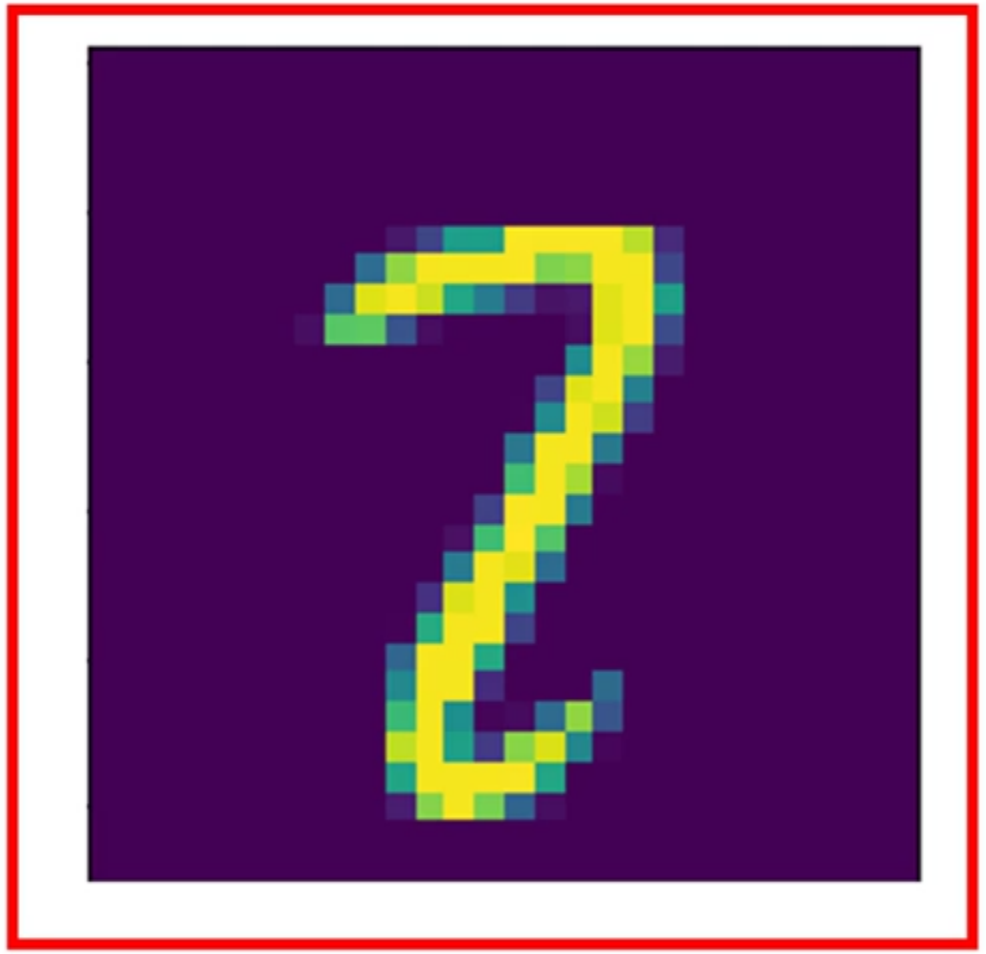
Python code

```
import numpy as np
from sklearn.metrics import confusion_matrix
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = np.argmax(y_test, axis=1)
print(confusion_matrix(y_true_classes, y_pred_classes))
```

```
array([[ 974,    0,    2,    1,    0,    0,    1,    0,    2,    0],
       [   0, 1128,    1,    2,    1,    0,    2,    0,    1,    0],
       [   1,    1, 1021,    2,    1,    0,    0,    6,    0,    0],
       [   0,    0,    0, 1007,    0,    2,    0,    0,    1,    0],
       [   1,    0,    1,    0,  974,    0,    3,    0,    0,    3],
       [   1,    0,    0,    5,    0,  884,    2,    0,    0,    0],
       [   2,    2,    0,    2,    1,    7,  943,    0,    1,    0],
       [   0,    3,    6,    2,    1,    1,    0, 1013,    1,    1],
       [   3,    0,    2,    3,    1,    5,    0,    3,  953,    4],
       [   0,    1,    1,    2,    9,    6,    0,    3,    1,  986]],
```

1128 out of the 1135 images that includ a one are correctly classified.

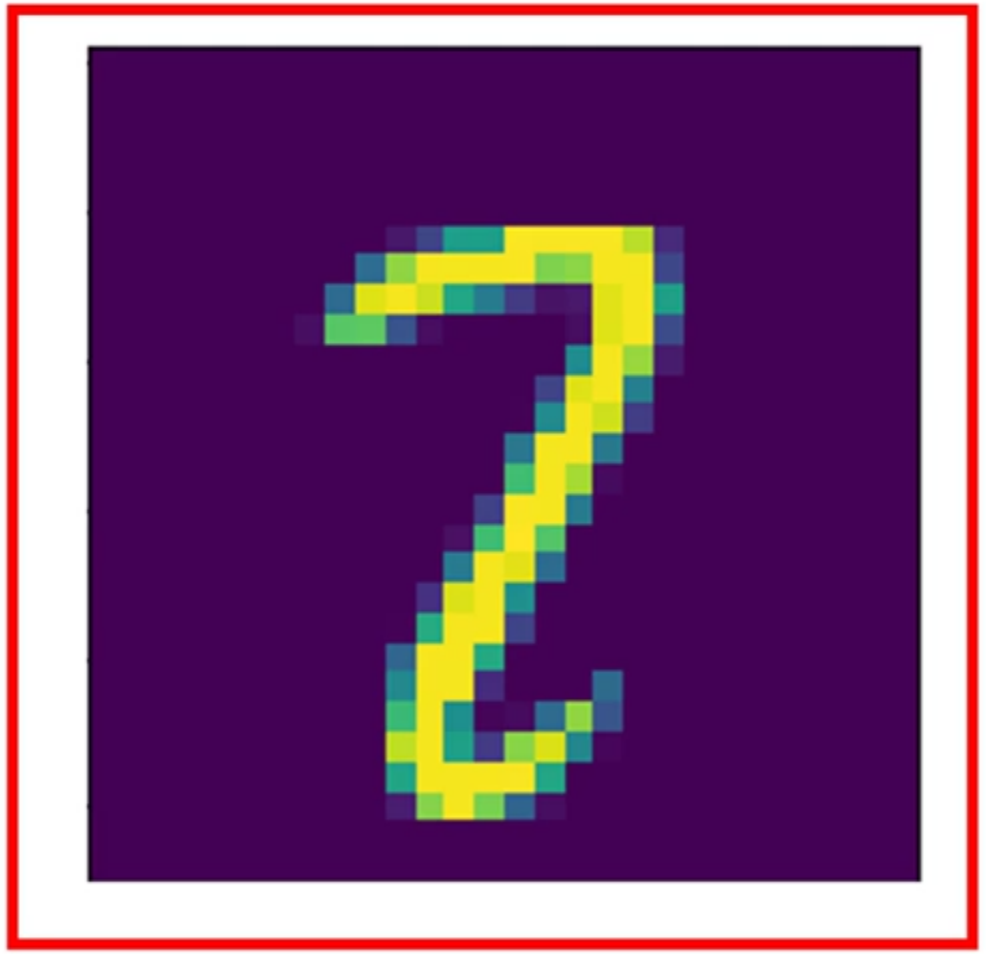
Python code



```
np.where(y_pred_classes != y_true_classes)
```

This is one of the images that were incorrectly predicted by the network. The network thinks that it is a seven but the true digit is actually a two.

Python code



```
np.where(y_pred_classes != y_true_classes)
```

This is one of the images that were incorrectly predicted by the network. The network thinks that it is a seven but the true digit is actually a two.